Digital Design with VHDL

Digital Design with VHDL

From a gate network on paper to a design running on a real FPGA

Erik Pihl

QRTECH ACADEMY
12 September 2026

Digital Design with VHDL
Copyright © 2026 QRTECH Academy.

The text and figures of this book, and of the course it is typeset from, are licensed under the Creative Commons Attribution-NonCommercial-ShareAlike 4.0 International License (CC BY-NC-SA 4.0). You may copy and redistribute them, and adapt them, for any non-commercial purpose, provided you give appropriate credit and share what you make under the same license. To view a copy of the license, visit `https://creativecommons.org/licenses/by-nc-sa/4.0/`.

The code in this book may also be used under the MIT License, which is the license of all the code in the companion repository. The repository holds the lecture material, the example designs, the testbenches and the self-assessment papers:

`https://github.com/qrtech-academy/digital-design-vhdl`

The VHDL in this book is VHDL-93 throughout, simulated with GHDL and synthesized with Quartus Prime Lite for the Terasic DE0-CV board and its Cyclone V FPGA. Quartus, Cyclone, DE0-CV and CircuitVerse are trademarks of their respective owners.

Set in TeX Gyre Pagella and DejaVu Sans Mono with LuaLaTeX. The figures are drawn from code, in the repository's `diagrams/`, except the circuits exported from CircuitVerse.

Typeset from the eight lectures of the course, their exercises, and the two self-assessment papers. This edition: 12 September 2026.

Contents

Preface **ix**

1 Combinational Logic and First VHDL **1**
1.1 Digital Signals and Logic Gates . . . 2
1.2 Boolean Algebra Notation . . . 2
1.3 Building and Simulating a Circuit in CircuitVerse . . . 4
1.4 Enough VHDL to Read and Write a Gate Circuit . . . 4
1.5 The Complete Module: `or_gate` . . . 7
1.6 The Worked Circuit: A Braking Assistant . . . 8
1.7 Review . . . 10
1.8 Exercises . . . 11

2 Larger Networks, Multiplexers and Submodules **19**
2.1 Karnaugh Maps: From Truth Table to Minimized Gate Network . . . 20
2.2 Translating the Karnaugh-Map Network into VHDL . . . 23
2.3 `std_logic_vector`: More Than One Wire . . . 25
2.4 Multiplexers . . . 26
2.5 Multiplexers in VHDL: `process` and `case` . . . 27
2.6 Building a Design from Submodules . . . 30
2.7 Checking Your Work . . . 32
2.8 Verifying Your VHDL with a Testbench . . . 33
2.9 Review . . . 36
2.10 Exercises . . . 37

3 Sequential Logic **49**
3.1 From Combinational to Sequential Logic . . . 50
3.2 The D Latch . . . 50
3.3 The D Flip-Flop . . . 51
3.4 Clocking . . . 54
3.5 Registers: Flip-Flops in Parallel . . . 54

3.6 The Synchronous Process Template in VHDL . . . 55
3.7 Edge Detection . . . 57
3.8 Worked Example: A D Flip-Flop in VHDL . . . 57
3.9 Worked Example: Edge-Detected LED Toggle . . . 59
3.10 Review . . . 61
3.11 Exercises . . . 62

4 Metastability and Synchronization **71**
4.1 The Problem: A Button Isn't a Testbench Signal . . . 72
4.2 Metastability: What Actually Happens Inside the Flip-Flop . . . 72
4.3 The Double-Flop Synchronizer . . . 73
4.4 Timing Intuition: Why "Almost Certainly" Is Good Enough . . . 75
4.5 Synchronizing a Reset Signal: Assert Async, Release Sync. . . . 76
4.6 Reusing the Chain for Edge Detection — and, Incidentally, Debouncing 76
4.7 What Real Hardware Debouncing Adds . . . 77
4.8 Generics: One Module, Several Sizes . . . 78
4.9 Worked Example: `led_toggle_sync` . . . 79
4.10 Review . . . 83
4.11 Exercises . . . 84

5 Variables and the Hardware Underneath **97**
5.1 What This Chapter Is About . . . 98
5.2 `variable`: Process-Local Storage with an Immediate Update . . . 98
5.3 What the FPGA Actually Builds . . . 102
5.4 Looking Ahead . . . 104
5.5 Review . . . 105
5.6 Exercises . . . 106

6 Counters and Shift Registers **115**
6.1 From Registers to Counters . . . 116
6.2 Shift Registers . . . 118
6.3 Serial-In/Parallel-Out (SIPO) . . . 119
6.4 Parallel-In/Serial-Out (PISO) . . . 120
6.5 Worked Example: An 8-Bit Serial Receiver . . . 121
6.6 FPGA Demonstration: Watching a Counter Wrap . . . 124
6.7 FPGA Demonstration: Shifting a Byte In by Hand . . . 125
6.8 Review . . . 126
6.9 Exercises . . . 127

7 Timers **137**
7.1 Timers: A Counter with a Target . . . 138
7.2 Building a Timer by Hand in CircuitVerse . . . 138
7.3 The `timer` Module in VHDL . . . 140
7.4 Composing a Full Timer Circuit . . . 142
7.5 FPGA Implementation: A Walking-LED Shift Register . . . 144
7.6 Review . . . 146
7.7 Exercises . . . 147

8 State Machines **159**
8.1 What Is a Finite State Machine . . . 160
8.2 Hand-Designing a Moore Machine . . . 161
8.3 From Gate Network to CircuitVerse . . . 164
8.4 The Same Machine in VHDL . . . 164
8.5 Mealy Machines: The One-Cycle Difference . . . 170
8.6 FPGA Demonstration . . . 174
8.7 Review . . . 174
8.8 Exercises . . . 176

A Short Answers **193**

B Quartus Prime Lite and the DE0-CV **211**
B.1 What You Need . . . 211
B.2 Installing Quartus Prime Lite . . . 212
B.3 Creating a New Project . . . 213
B.4 Adding Your VHDL Source . . . 214
B.5 Assigning Pins with the Pin Planner . . . 214
B.6 Compiling the Design . . . 215
B.7 Programming the DE0-CV Board . . . 215
B.8 Troubleshooting . . . 216

C The Self-Assessment Papers **217**

D Self-Assessment Paper A **221**

E Self-Assessment Paper B **231**

F Model Answers to Paper A **241**
Question 1: A table, a network, and a module . . . 241
Question 2: The map, the multiplexer, and the module used twice . . . 243
Question 3: What a clock edge does, and when an assignment happens . . . 246
Question 4: The input you do not control . . . 248
Question 5: The accumulator that isn't, and the latch nobody asked for . . . 252

Question 6: Counting, shifting, and knowing when a byte has arrived 255
Question 7: The counter that watches for a number 257
Question 8: A machine designed by hand, then written down 258

G Model Answers to Paper B **263**
Question 1: A requirement, and the shape of the network it asks for 263
Question 2: The map, the vector, and the process that lies 266
Question 3: Storage, and the two ways a value updates 269
Question 4: One module, several widths 271
Question 5: Two kinds of update, and a speed limit 274
Question 6: Counting to something that is not a power of two 277
Question 7: Turning ticks into time 280
Question 8: The output that arrives a cycle early 282

Preface

This book is for software and embedded engineers who know classical digital design and want to write VHDL. It takes you from a gate network on paper to a design running on a real FPGA (a *field-programmable gate array*: a chip full of logic you configure into the circuit you want, rather than one whose function was fixed when it was manufactured).

The subject is VHDL. Gates, truth tables, Boolean algebra and Karnaugh maps are refreshed rather than taught: they appear where the language needs them, and in Chapter 8 where the state machine is designed by hand before it is written.

The book covers writing synthesizable VHDL for combinational and sequential hardware: metastability and synchronization, counters, timers and finite state machines, each reasoned about as a circuit first and then expressed in the language, running on a real FPGA. Topics include:

- Combinational logic: gates, truth tables, Boolean algebra, and VHDL from the first chapter.
- Larger networks: Karnaugh maps, multiplexers, `process` and `case`.
- Sequential logic: registers, D flip-flops, clocking, and edge detection.
- Metastability and synchronization: asynchronous inputs, synchronizers, debouncing.
- The VHDL language: signals, variables, subcomponents, and generics.
- Counters, shift registers, and timers.
- Finite state machines: designed by hand, then written in VHDL, Moore and Mealy.

Wherever a circuit can usefully be drawn, it is drawn first: reasoned about as gates, then built and simulated by hand in CircuitVerse (`https://circuitverse.org/simulator`) before it is ever written in VHDL. Every chapter that introduces a new kind of circuit draws one that way first. The exception is Chapter 5, which is about the language and has no new circuit in it; and once a kind of circuit has been drawn, later designs of the same kind go straight into VHDL.

Verification works in three layers:

- **By hand, in CircuitVerse**, before any VHDL exists. You derive what the circuit should do, build it, and then simulate it yourself against that prediction, exhaustively, since the circuits are small enough for that to be cheap. Nothing automated checks this step,

deliberately: predicting a behaviour and then confirming it is the skill the rest of the book is built on.

- **In the exercises**, you write the VHDL and check it with a provided self-checking testbench (`<module>_tb.vhd`, runnable with GHDL). Nearly every VHDL exercise ships one, so nearly every exercise can be completed and verified on a laptop; the handful that do not say so, and say how to check by hand instead.
- **On the board**: the designs are synthesized in Quartus Prime Lite and run on a real Terasic DE0-CV board. In the course the book comes from, that is demonstrated from the front, and nobody needs a board of their own; Appendix B is the workflow, for a reader who has one.

Writing testbenches is not a topic here: this book only shows you how to run the ones provided.

Along the way you will build circuits, not just read about them. You will:

- Realize a logic function by hand as a gate network, and simulate it in CircuitVerse.
- Translate that same gate network into synthesizable VHDL.
- Build sequential circuits from D flip-flops: registers, counters, shift registers, and timers.
- Guard every asynchronous input with a double-flop synchronizer before it reaches synchronous logic.
- Design a Moore state machine by hand (diagram, table, Karnaugh maps, gate network), then write the same kind of machine in VHDL, and learn to recognize a Mealy machine and what it costs.
- Follow each design through Quartus Prime Lite and onto a Terasic DE0-CV FPGA board.
- Verify your own VHDL after each chapter by running the provided self-checking testbenches.

The examples and exercises are small, complete circuits, such as:

- A gate network realizing a Boolean function, in CircuitVerse and in VHDL.
- A synchronized, edge-detected push button driving an LED.
- A free-running timer and a shift-register-driven LED pattern.
- A three-state Moore machine cycling an LED off, blinking, on, composed from a synchronizer and a timer.
- A two-state Mealy sequence detector, contrasted against the Moore machine it would take three states to write.

The book then closes on two exercises that are not small: a serial receiver and a serial transmitter, each composed from modules you wrote in earlier chapters around a state machine you design yourself, and each verified by a provided testbench.

What you should be able to do at the end

Having worked through the book, you should be able to:

- Realize a Boolean function as a gate network, and simulate it by hand in CircuitVerse.
- Read and write combinational and sequential VHDL: entities, architectures, signals, variables, and processes.
- Explain metastability and apply the double-flop synchronizer to any asynchronous input.
- Design and implement counters, shift registers, and timers in VHDL.
- Design a Moore state machine from a specification and implement it in VHDL, and recognize a Mealy machine on sight, knowing what its earlier output costs.
- Compose a design out of previously built modules (a reset synchronizer, a button synchronizer, a timer) rather than rewriting their logic.
- Explain how a VHDL design reaches an FPGA: synthesis, pin assignment, and programming the board in Quartus Prime Lite.
- Verify a VHDL design by running a self-checking testbench under GHDL.

What you are assumed to know

The subject of this book is **VHDL**, not digital design. You are expected to arrive already comfortable with:

- Logic gates and truth tables, and reading a sum-of-products equation off a table.
- Boolean algebra: the basic identities and De Morgan's laws.
- Karnaugh maps, for functions of three or four variables.
- Registers, D flip-flops, and what a clock edge does.
- State machines as a concept: states, transitions, and the difference between Moore and Mealy.

None of that is taught from scratch, and this book is the poorer place to meet it first. The junior-track course *Digital Electronics* (`https://github.com/qrtech-academy/digital-electronics`) teaches all five points above from the ground up, derived by hand and simulated in CircuitVerse; this book is written to start where that course ends, so read it first if any of the list is unfamiliar.

What is here instead is refreshed where the language needs it, briefly: Chapter 1 restates gate and Boolean notation so the book has one convention, Chapter 2 works a Karnaugh map end to end because minimization is what motivates the VHDL that follows, and Chapter 8 designs one state machine by hand before writing it, because seeing the state table become a `case` statement is the point of the chapter.

What the book does teach from nothing is the language and the toolchain: `entity` and `architecture`, signals and variables, processes, generics, subcomponents, synthesis in Quartus, and simulation with GHDL.

No programming language beyond general software fluency is assumed. Familiarity with C or a similar language helps, since several explanations contrast VHDL's semantics with it.

How the book is organised

The book is typeset from an eight-lecture course, and each chapter is one lecture. Its sections are the parts of the lecture's appendix, in the same order and with the same subsections, so a chapter here and a lecture directory in the repository can be read side by side:

Chapter	Lecture	Topic
1	`L01`	Combinational logic and first VHDL
2	`L02`	Larger networks, multiplexers and submodules
3	`L03`	Sequential logic
4	`L04`	Metastability and synchronization
5	`L05`	Variables and the hardware underneath
6	`L06`	Counters and shift registers
7	`L07`	Timers
8	`L08`	State machines

Every chapter opens with what is in it, works through the material, closes with a review of what you should now be able to explain, and ends with its exercises. The appendices hold the short answers to the exercises, the Quartus workflow, and the two written papers described below.

What the exercises look like

The exercises are grouped into **sets** by topic, and each is labelled with the kinds of work it asks for:

Kind	What it asks of you
PAPER	Work it by hand: a truth table, a derivation, a Karnaugh map, a trace through the clock edges, a calculation, or a question answered in a sentence or two.
CIRCUITVERSE	Build the circuit in CircuitVerse and simulate it, exhaustively, against the behaviour you predicted first.
VHDL	Write a module, with the entity name and port order the exercise gives, and check it with the testbench it ships.

The answers to the paper work are in Appendix A, short, and by exercise number, so that an answer worked on paper can be checked on paper. **The VHDL is checked by its testbench**, which is the point of shipping one, and reference solutions are published in the companion repository, under `lectures/LNN/solutions`. Try each exercise before you read its answer.

Two written papers, and what they are not

Nothing in the course this book comes from is marked. Assessment is the exercises after every chapter, the self-checking testbench nearly every one of them ships, and the circuit you predict and then simulate by hand in CircuitVerse before any VHDL exists.

Appendices C to G hold two three-hour papers with worked solutions, and they check something else: **your own skills and knowledge, on paper, with nothing in front of you.** Eight questions each, one per chapter, mixing theory with VHDL you write out by hand and with designs that compile cleanly and are wrong anyway.

They exist purely so that you can test yourself after working through the book. They gate nothing, they are not a qualification, and no part of the book requires them.

Take one after the last chapter, and after the serial receiver and transmitter that close it. Both papers draw on all eight chapters, so sitting one partway through examines material you have not read yet.

Setting up

Everything in this book runs on an ordinary laptop, with no hardware of any kind. You need:

- **CircuitVerse**, the free, browser-based logic simulator used to build and simulate every circuit by hand before it is written in VHDL, at `https://circuitverse.org/simulator`. Nothing to install.
- **GHDL**, the free, open-source VHDL analyzer and simulator, used to run the self-checking testbenches that verify the exercises. Running testbenches is covered in §2.8.
- **Git** and **make**, to fetch the companion repository and run its checks.

On Linux, or on Windows through WSL with Ubuntu:

```
sudo apt -y update
sudo apt -y install git make ghdl
ghdl --version           # 3.x or newer; the repository's CI builds against 4.1.0
```

Then clone the companion repository:

```
git clone https://github.com/qrtech-academy/digital-design-vhdl.git
```

Every lecture directory holds its appendices, its worked examples, each with a reference testbench, and an `exercises` directory with the testbench for every VHDL exercise, in a directory named after the module you write. You write your module beside its testbench and run the three GHDL commands §2.8 gives; the testbench prints its "all checks passed" note when your module is right. At the repository's root:

```
make help                # List every target.
make build               # Build and simulate every example; check every exercise ↩
    ↪ testbench.
make build MODULE=timer  # Build only the examples whose path contains "timer".
```

Quartus Prime Lite and a DE0-CV board are needed for nothing in the book. If you have them, Appendix B takes a design from VHDL onto the board.

CHAPTER 1

Combinational Logic and First VHDL

From a truth table to a working gate network, in CircuitVerse and in VHDL.

IN THIS CHAPTER

- Logic gates and truth tables, and why digital logic tolerates noise that analog cannot.
- Boolean algebra: reading a sum-of-products equation straight off a truth table.
- Building and simulating a gate network by hand in CircuitVerse.
- VHDL from the first chapter: `entity`, `architecture`, `std_logic`, and the concurrent assignment.
- A braking assistant, from a requirement and its truth table to a gate network, and on to switches and an LED on a DE0-CV FPGA board.
- `or_gate`, taken from truth table to complete module in §1.4 and §1.5.

This chapter takes one circuit from a requirement to running hardware, and then does it again. The first time is `or_gate`, the smallest complete VHDL design there is, which §1.4 and §1.5 take end to end in writing. The second is a braking assistant: four gates, derived from a requirement and its truth table, simulated against it in CircuitVerse, written as an `entity` and an `architecture` with an internal `signal` and two concurrent assignments, and finally taken through Quartus onto a DE0-CV, with its four inputs on switches and its brake output on an LED. Flick the switches and watch the car decide to brake.

In the course this book comes from, the braking assistant is built live, and §1.6 gives you its specification without its answer so that you can derive it yourself first, the way the people in the room are asked to. Do that before you read on: it is the habit the rest of the book is built on. You then write its VHDL in Exercise 1.7, and a testbench decides whether your reading of the requirement matches the truth table.

You need nothing installed to read this chapter. The exercises need GHDL, which §2.8 installs and explains how to run; the three commands in §1.5 are enough to get started. The board work is shown in Appendix B for anyone who has a DE0-CV; no exercise needs one.

1.1 Digital Signals and Logic Gates

A digital signal takes one of two values, `0` or `1`, rather than the continuous range an analog signal uses. Anything close enough to `0` or `1` is read as exactly `0` or `1`, so small amounts of electrical noise change nothing. That noise immunity is why almost all modern computation, from microcontrollers to FPGAs, is digital.

A **logic gate** has one or more single-bit inputs, one single-bit output, and a fixed Boolean function between them.

Combinational logic, this chapter's topic, produces outputs from the current inputs alone, with no memory. **Sequential logic** (Chapter 3) depends on the current inputs *and* on state stored in flip-flops.

Every basic 2-input gate, plus the unary `NOT`:

A	B	AND	OR	NAND	NOR	XOR	XNOR
0	0	0	0	1	1	0	1
0	1	0	1	1	0	1	0
1	0	0	1	1	0	1	0
1	1	1	1	0	0	0	1

A	NOT A
0	1
1	0

Worth internalizing straight from the tables:

- `NAND`, `NOR` and `XNOR` are the inverses of `AND`, `OR` and `XOR`.
- `XOR` is `1` when the inputs differ. Beyond two inputs this generalizes to **odd parity**: `1` when an odd number of inputs are `1` (Exercise 1.3).
- Any Boolean function can be built from `AND`, `OR` and `NOT`, and also from `NAND` alone or `NOR` alone.

1.2 Boolean Algebra Notation

Three operators, mapping directly onto the gates above:

Gate	Boolean algebra notation
AND	multiplication: `A` **AND** `B` is written `AB`
OR	addition: `A` **OR** `B` is written `A + B`
NOT	prime: **NOT** `A` is written `A'`

A function written as an OR of AND-terms, such as `X = AB' + CD`, is in **sum-of-products** form.

Every truth table reads off into sum-of-products directly:

1. Find every row where the output is `1`.
2. For each, write one AND-term containing every input: directly if it is `1` in that row, primed if it is `0`.
3. OR those terms together.

For example:

A	B	X
0	0	0
0	1	1
1	0	1
1	1	0

Row `AB = 01` contributes `A'B` and row `AB = 10` contributes `AB'`, giving `X = A'B + AB'`. Check it against the table: that is `XOR`, so the same network is one `XOR` gate rather than two `AND` gates, two `NOT` gates and an `OR`.

1.2.1 The laws, as a reminder

You have met these. They are here so the book has one notation for them and so the later chapters can cite them, not because they need teaching:

Law	With OR	With AND
Identity	`A + 0 = A`	`A · 1 = A`
Null	`A + 1 = 1`	`A · 0 = 0`
Idempotent	`A + A = A`	`A · A = A`
Complement	`A + A' = 1`	`A · A' = 0`
Absorption	`A + AB = A`	`A(A + B) = A`
De Morgan	`(A + B)' = A'B'`	`(AB)' = A' + B'`

De Morgan is the one this book leans on, less for minimization than as a rule about moving an inversion through a gate: an `OR` with both inputs inverted *is* a `NAND`, and an `AND` with both inputs inverted *is* a `NOR`. It is why a `NAND` gate can build anything, which Exercise 1.6 has you write in VHDL, and it comes back every time an active-low signal meets logic written for active-high. In a book where every reset and every button is active-low, that is often.

That gap between a straightforward sum-of-products reading and the smallest network is what a **Karnaugh map** closes, and Chapter 2 works one end to end. Not because the technique is new to you, but because the minimized equation is what gets written in VHDL, and Chapter 8 derives its state machine's next-state logic exactly this way.

1.3 Building and Simulating a Circuit in CircuitVerse

CircuitVerse (`https://circuitverse.org/simulator`) is a free, browser-based logic simulator. Every gate network in this book is built and tested there by hand before it is written in VHDL:

1. Open the simulator and start a new project.
2. Place an **input** element per input signal and an **output** element per output, renamed to match your variables.
3. Place the gates your equation calls for, from the *Gates* panel.
4. Wire it up one gate at a time, following your derived equation. You should already know exactly which gates you need before you open the simulator, which is why §1.2 comes first.
5. Toggle each input and confirm the output matches your truth table for **every** row.
6. Save via `Project -> Save Online`, or `Project -> Download as image/data` for a re-importable `.cv` file.

Step 5 is not optional, and nothing else does it for you. Three rules make it a check rather than a formality:

- **Exhaustive, not spot-checked.** Two inputs is four rows, three is eight. At these sizes there is no excuse for sampling. For the sequential circuits later on this becomes "step the clock and check at every edge", the same discipline applied to time.
- **Derive first, simulate second.** Draw the circuit and *then* decide what it should do, and you will read your expectations off the simulation, which proves nothing.
- **A mismatch is information.** It says the drawing and the equation disagree, and one of them is wrong.

Every hardware bug you will chase is found by predicting a behaviour and then checking it. Doing it by hand on a four-gate circuit is how you learn to do it on one you cannot see all at once.

Once the design is VHDL, a self-checking testbench takes over that job. Every worked example and every exercise ships one; you run them rather than write them, as covered in §2.8.

1.4 Enough VHDL to Read and Write a Gate Circuit

VHDL describes hardware, not a sequence of instructions. Every statement below runs concurrently and continuously, exactly like the gates it describes. The language arrives as it is needed; this section is enough for a simple combinational circuit.

Every VHDL module has two parts:

- An **entity**, the black-box view: the ports, and nothing about what happens inside.
- An **architecture**, the implementation behind it.

That split is a contract, not ceremony. The entity is everything another design needs in order to use this module, which is what lets one design instantiate another without reading it, and lets you rewrite an implementation without touching what depends on it. The shape is a header beside a source file, or an interface beside its class; the difference is that VHDL enforces it for every module, with no way to skip the interface or leak the implementation through it.

Take `or_gate`: inputs `a`, `b`, output `x`, and `x = a or b`.

```
entity or_gate is
    port(a, b: in std_logic;
         x   : out std_logic);
end entity;
```

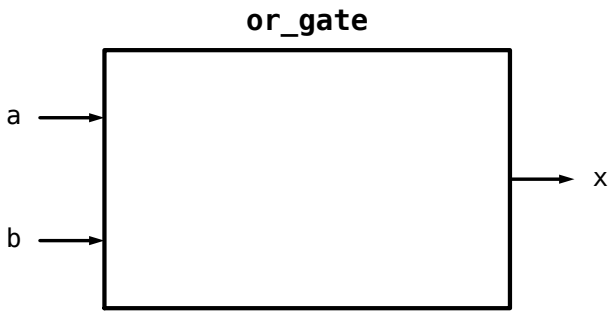


Figure 1.1 The entity `or_gate`: two inputs and an output, and nothing about what happens inside.

`std_logic` is the single-bit signal type. It is not built into VHDL but comes from the `std_logic_1164` package, which is why every file in this book opens with:

```
library ieee;
use ieee.std_logic_1164.all;
```

It has nine values rather than two, because real hardware needs more than true and false:

- `'0'` and `'1'`: driven low and driven high. The only two you will write in this book.
- `'Z'`: high-impedance, nothing driving the line at all.
- `'U'`: uninitialized, what a signal holds before anything assigns it.
- `'X'`: unknown, what a simulator shows when two things drive one signal and disagree.
- `'-'`, `'W'`, `'L'`, `'H'`: don't-care and weak-drive values, rarely written by hand.

Recognizing the seven you will not write still matters: they are how a simulator tells you something is wrong. A signal sitting at `'U'` or `'X'` is a bug, not a value.

The architecture fills in the box:

```
architecture behaviour of or_gate is
begin
    x <= a or b;
end architecture;
```

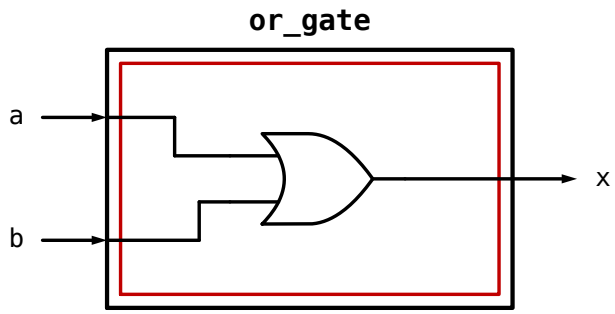


Figure 1.2 The architecture of `or_gate`: the gate that fills the box.

Together they form the complete module:

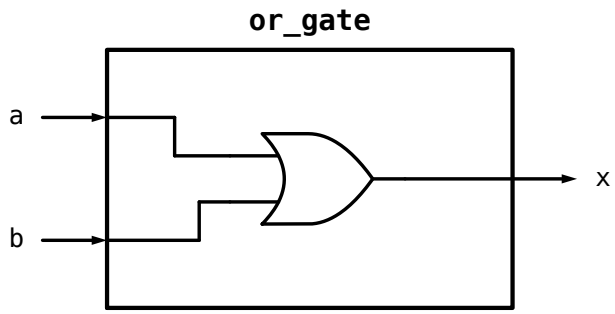


Figure 1.3 The module `or_gate`: the entity and the architecture together.

`x <= a or b;` is §1.2's Boolean algebra almost verbatim: VHDL's `and`, `or`, `not` and `xor` work directly on `std_logic`, so any equation you derived by hand becomes VHDL with barely any translation.

The assignment is **concurrent**. It is not executed once; it holds continuously, like a wire between real gates.

The keyword `signal` declares an internal wire inside an architecture, as opposed to an externally visible port. You use one in Exercise 1.7 to name an intermediate result rather than nesting it inline, and Chapter 2 leans on the same idea throughout.

1.5 The Complete Module: or_gate

§1.4's pieces together make a complete, synthesizable file. This is the whole of `lectures/L01/or_gate/or_gate.vhd`, and it is a real design that synthesizes and runs on the DE0-CV:

lectures/L01/or_gate/or_gate.vhd

```
--------------------------------------------------------------------------------
-- Two-input OR gate: x = a or b.
--
-- Inputs:
--     - a, b: OR inputs.
-- Outputs:
--     - x: OR output.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity or_gate is
    port(a, b: in std_logic;
         x   : out std_logic);
end entity;

architecture behaviour of or_gate is
begin
    x <= a or b;
end architecture;
```

- The `library`/`use` clauses pull in `std_logic`.
- `entity` declares the outside: two inputs, one output.
- `architecture` implements the inside, as one concurrent assignment.

That is the smallest complete VHDL design there is, and the shape never changes. Every module in this book, up to the state machines in Chapter 8, is this same `library`/`entity`/`architecture` skeleton. What grows is the architecture body, never the frame.

Checking your work. `or_gate` ships `or_gate_tb.vhd`, which drives all four input combinations and checks the output:

```
cd lectures/L01/or_gate
ghdl -a --std=93 or_gate.vhd or_gate_tb.vhd
ghdl -e --std=93 or_gate_tb
ghdl -r --std=93 or_gate_tb --assert-level=error --stop-time=10ms
```

No output beyond a final note means every check passed. Testbenches are covered properly in §2.8; for now, treat these three commands as how you confirm a module works.

1.6 The Worked Circuit: A Braking Assistant

The course builds a different circuit from `or_gate`, live, so you see the same path walked twice: once here in writing, and once on something you have not already read the answer to.

The requirement: **the car brakes if the driver brakes, or if the assistance system decides to.** The assistance system decides to when either detector sees something, unless it is reporting a fault, in which case it is not trusted to brake at all.

Port	Direction	Meaning
`driver_brake`	in	The driver is on the brake pedal.
`sensor`	in	The proximity sensor sees an obstacle.
`radar`	in	The radar sees an obstacle.
`error`	in	The assistance system is reporting a fault.
`engine_brake`	out	Apply the brakes.

Three clauses decide the whole structure between them:

- either detector is enough on its own, and neither is trusted more than the other.
- a fault does not raise an alarm; it *suppresses* the assistance system's ability to brake.
- the driver can always brake, including when the assistance system is faulty and doing nothing.

That last one is a safety requirement, and it is the one to hold on to when you derive the network: the pedal is not an input to the assistance logic, it bypasses it. Work out what changes if the pedal is fed through the fault logic instead, and you have found the bug the structure exists to prevent.

1.6.1 The truth table

Sixteen rows, in the order a four-bit counter would produce them. `adas_brake` is not a port; it is the assistance system's own decision, shown as an intermediate column so you can see where the structure comes from.

driver_brake	sensor	radar	error	adas_brake	engine_brake
0	0	0	0	0	0
0	0	0	1	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	1	0	0	1	1
0	1	0	1	0	0
0	1	1	0	1	1
0	1	1	1	0	0
1	0	0	0	0	1
1	0	0	1	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	1	0	1	1
1	1	1	1	0	1

The bottom half is entirely `1`: once `driver_brake` is set nothing else changes the answer. That is what "overrides everything" looks like in a table.

1.6.2 Before you read on

Everything above is the specification. The derivation, the gate network, and the VHDL are what the course builds live, and it is worth arriving at them with your own answer to compare against rather than one you have already read. So, from the table above:

- Read a sum-of-products equation for `engine_brake` straight off it, the way §1.2 did.
- Simplify it by hand. The requirement is three clauses long, so the minimal network is a great deal smaller than the row-by-row reading gives you. Work out how much smaller before you are told.
- Count the gates your simplified equation needs, and write the number down.
- Sketch the network, and decide which single gate the safety argument above rests on.

Write those four answers down. Being wrong about any of them is worth more than not having guessed, because you will know exactly which step to re-examine when you check them. That is §1.3's "derive first, simulate second" applied to a circuit you have not built yet, and it is the habit the rest of the book is built on. Afterwards you write the VHDL yourself, in Exercise 1.7, and a testbench decides whether your equations and this truth table agree.

1.6.3 Why there is a testbench for it

Sixteen combinations is few enough to check by hand once. It is not few enough to check every time somebody edits the file, which is the situation every module in this book is in from here on.

A **testbench** is a second VHDL file that does the checking. It is not part of the design and never reaches the FPGA: it drives the inputs through every case, works out what the output should have been, compares, and reports. For `adas`, the braking assistant's module, that is a loop over the sixteen combinations restating the braking rule independently.

Independently is the important word. A testbench that read its expected answer out of the design would agree with it by construction, including when the design is wrong. Restating the requirement separately is what turns it into a check.

That is why almost every directory in the companion repository has a `_tb.vhd` beside the module: it is how you can be handed a module, change it, and know within a second whether you broke it. §2.8 covers running them; you are not asked to write one anywhere in this book.

1.7 Review

What you should be able to do now

After this chapter, you should be able to:

- Derive a sum-of-products equation from a truth table, then build and simulate the network in CircuitVerse.
- Apply De Morgan to move an inversion through a gate, and use it to realize a function in a gate set that does not contain the operator you started with.
- Read and write a combinational VHDL module, an `entity` with `std_logic` ports and an `architecture` driving its output, and say why VHDL separates the two.
- Explain why a concurrent assignment describes a wire rather than a statement that runs once, and why a signal sitting at `'U'` or `'X'` is a bug rather than a value.
- Run a provided testbench and tell whether your module passed.

Questions to test yourself

You should be able to explain:

1. What is the difference between an `entity` and an `architecture`, and why does VHDL separate them?
2. Why is `x <= a or b;` not "a statement that runs once"? What does it describe instead?
3. In the braking assistant, the driver's pedal is ORed in last rather than passing through the fault logic. What breaks if you wire it the other way?

Looking ahead

Chapter 2 is about larger networks:

- Karnaugh maps: finding the *smallest* gate network, not just a correct one.
- `std_logic_vector`, `process` and `case`.
- Submodules, demonstrated on a two-digit hexadecimal display driven by eight switches.
- Self-checking testbenches: verifying your own VHDL without a board.

1.8 Exercises

> **How to work these.** Exercise 1.1 is a short pen-and-paper warm-up on notation this book uses throughout; it needs no VHDL, GHDL, or board. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
> **How to check your work.** From Exercise 1.2 on, every exercise that asks you to write a VHDL module ships a self-checking testbench under `lectures/L01/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
> No FPGA board is needed for any exercise. Your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L01/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

Truth Tables and Gates

Exercise 1.1 A function that ignores an input

PAPER CIRCUITVERSE

A function of two inputs `A`, `B` and one output `X` has the truth table below:

A	B	X
0	0	1
0	1	0
1	0	1
1	1	0

a) Derive the sum-of-products equation for `X` directly from the truth table (§1.2). Two rows have `X = 1`, so you get two AND-terms.

b) Simplify it algebraically. Factor out what the two terms share, and apply `A + A' = 1` and `1 · B' = B'` from §1.2. You should end up with a single literal.

c) Your answer to **b**) says something about `A`. What? Which single gate from §1.1's table computes the function, and what happens to the `A` input when you draw it?

d) Build that single-gate version in CircuitVerse and confirm it reproduces every row of the truth table above. Leave the `A` input unconnected, or simply leave it out of the drawing; **b**) is the justification for doing so.

EXERCISE SET 2

Entities and Architectures

Exercise 1.2 AND and NAND

VHDL

Write two small gates as complete VHDL modules, `and_gate` and `nand_gate`.

Neither gate is the point, and you should not expect either to be difficult. The point is that this is where an entity and an architecture stop being something you read in §1.4 and become something you write, and that the two halves are worth writing in that order.

Both entities have the same ports:

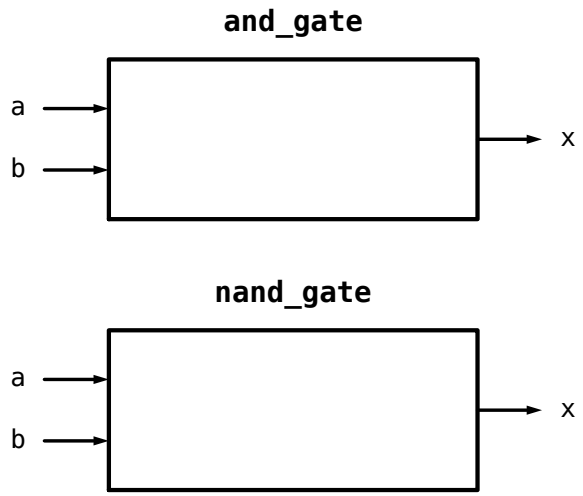
Port	Direction	Type
`a`	in	`std_logic`
`b`	in	`std_logic`
`x`	out	`std_logic`

a) Write the `entity` declaration for `and_gate`, and nothing else. In VHDL a module's interface is a thing you can write down and reason about before any implementation exists at all, which is the habit this part exists to build. Identify the required ports, the direction of each, and the type of each.

b) Add an `architecture` implementing the AND gate with a single concurrent signal assignment.

c) Now write `nand_gate`: the same interface with an inverted output, using VHDL's `nand` operator directly. Do **not** write `not (a and b)`.

The two are equivalent as Boolean algebra, and on an FPGA they make no difference at all, because both end up in the same lookup table. It matters on the kind of hardware Exercise 1.6 is about, where NAND is the gate you physically have and everything else has to be built from it. Writing what you mean, and letting the tools decide what it costs, is the habit; reaching past an operator the language already gives you is the thing to avoid.



Self-check: name your entities `and_gate` and `nand_gate`, each with ports `a`, `b` (in) and `x` (out), declared in that order; their testbenches are in `exercises/and_gate` and `exercises/nand_gate`, and each checks all four rows of the corresponding truth table.

Exercise 1.3 Three-input XOR

PAPER VHDL CIRCUITVERSE

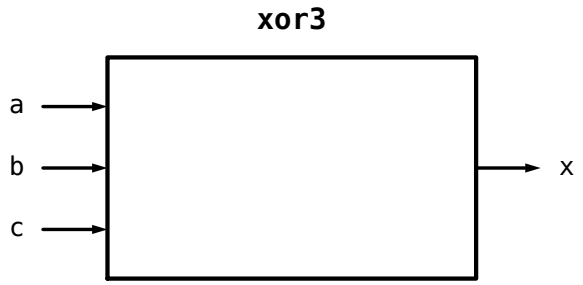
An XOR gate has three inputs, `a`, `b`, `c`, and one output, `x`. The output is `1` whenever an odd number of the inputs are `1` (the 3-input XOR, or parity, function).

a) Write the truth table for `x` over all eight combinations of `a`, `b`, `c`, and confirm it matches the "odd number of `1`s" description.

b) Implement the gate in VHDL, as a module of your own with an entity whose ports are three `std_logic` inputs and one `std_logic` output, and an architecture that drives `x` with a single concurrent assignment (§1.4 shows the entity/architecture pattern and notes that VHDL's `xor` operator works directly on `std_logic`; §1.5 shows a complete module end to end).

c) Build the same gate in CircuitVerse and confirm it reproduces every row of your truth table.

Tip: `xor` chains left-to-right, so the whole architecture body is a single line, `x <= a xor b xor c;`, with no internal `signal` or intermediate gates needed.



Self-check: name your VHDL entity `xor3`, with ports `a`, `b`, `c` (in) and `x` (out), declared in that order; its testbench is in `exercises/xor3`, and §1.5 lists the three commands that run it.

Exercise 1.4 Majority gate

PAPER VHDL CIRCUITVERSE

A **majority gate** has three inputs, `a`, `b`, `c`, and one output, `x`. The output is `1` whenever *at least two* of the three inputs are `1`.

This is the circuit behind a triple-redundant vote: three sensors report the same measurement, and the system acts on whatever answer two of them agree on, so a single failed sensor cannot decide anything on its own.

a) Write the truth table for `x` over all eight combinations of `a`, `b`, `c`.

b) Derive the sum-of-products equation for `x` directly from the truth table (§1.2):

- Four rows have `x = 1`, so you get four AND-terms.
- Each term includes all three inputs, primed where that input is `0` in its row.

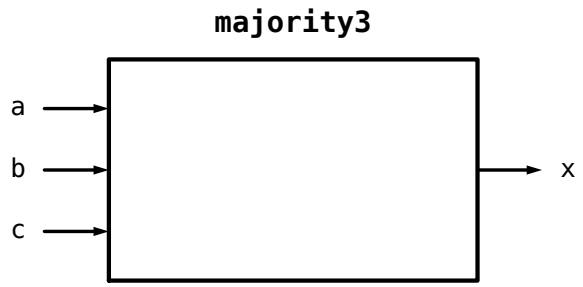
c) Implement the gate in VHDL, as a module of your own:

- An entity with three `std_logic` inputs and one `std_logic` output.
- An architecture driving `x` with a single concurrent assignment, translating your four AND-terms directly (§1.4).

d) Build the same network in CircuitVerse and confirm it reproduces every row of your truth table.

Tip: Write the equation out on paper before you open either the editor or CircuitVerse. Four three-input AND-terms plus one four-input OR gate is a lot of wiring to correct after the fact.

Looking ahead: count the gates your equation needs, then convince yourself by inspection that `x = ab + ac + bc` computes the same function with three two-input AND gates instead of four three-input ones. Sum-of-products gave you a *correct* network, not the *smallest* one; finding the smaller one systematically, rather than by staring at it, is what Chapter 2 is for.



Self-check: name your VHDL entity `majority3`, with ports `a`, `b`, `c` (in) and `x` (out), declared in that order; its testbench is in `exercises/majority3` and checks all eight rows. Either equation passes it: the testbench checks the function, not which network you chose.

Exercise 1.5 Half adder

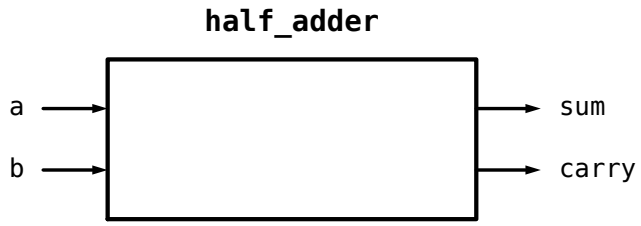
VHDL CIRCUITVERSE

Write a **half adder**: `sum` and `carry` from two single bits. You know the circuit; the reason it is here is that it is the first module in the book with **two outputs**, and that is a VHDL question rather than a logic one.

a) Implement it, with two concurrent assignments and no internal `signal`.
Two outputs does not mean two entities, two architectures, or a process. Each output port is driven by its own concurrent assignment, and the two are not steps that happen in an order: they describe two separate pieces of wire, both live all the time. If you find yourself reaching for a process to "return" two values, that instinct is the software one, and §1.4's "a concurrent assignment describes a wire" is the correction.

b) Build it in CircuitVerse and confirm both outputs, so you have seen the two-output shape as a drawing before you meet it as an entity with several `out` ports in Chapter 2.

Looking ahead: this is the cell Chapter 6's counter is built from. The 4-bit adder there is four of these chained carry-to-carry, and the counter is that adder with its output fed back to its input, which is why the wraparound needs no logic of its own.



Self-check: name your entity `half_adder`, with ports `a`, `b` (in) and `sum`, `carry` (out), declared in that order; its testbench is in `exercises/half_adder` and checks `sum` and `carry` separately, so a design that has one right and the other wrong is told which is which.

EXERCISE SET 3

Universal Gates

Exercise 1.6 OR from NAND

PAPER CIRCUITVERSE VHDL

§1.1 states that any Boolean function can be built using only `NAND` gates. This exercise makes you prove it to yourself for the three operators you have been using.

a) Derive the `OR` case on paper before you build anything, because De Morgan (§1.2.1) gives it to you directly. Start from `(A'B')'` and apply the law; you should arrive at `A + B`. Now read that expression back as a circuit: what is `A'B'` with an inversion on its output, and how many `NAND` gates is the whole thing?

b) In CircuitVerse, build each of the following from `NAND` gates and nothing else, confirming each against its truth table in §1.1 as you go:

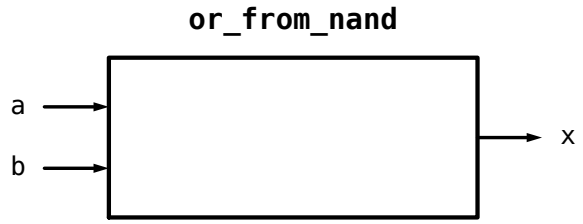
- `NOT a`: one NAND gate is enough, if you feed the same signal to both of its inputs.
- `a AND b`: one NAND gate, followed by the `NOT` you just built.
- `a OR b`: the network you derived in **a**). Check your gate count against what you predicted.

c) Write the `OR` version in VHDL, as a module of your own:

- An entity with two `std_logic` inputs and one `std_logic` output.
- An architecture driving `x` with a single concurrent assignment.
- Use only VHDL's `nand` operator:
  - no `or`, no `and`, no `not` anywhere in the expression.

d) Explain why this matters in practice. Consider that a chip fabrication process has to physically implement every gate type it offers, and that `NAND` is the cheapest gate to build in CMOS.

Tip: VHDL's `nand` operator does not chain the way `and` and `or` do: `a nand b nand c` is not legal. Parenthesize every NAND explicitly, which is what you want here anyway, since each pair of parentheses is one physical gate in your CircuitVerse drawing.



Self-check: name your VHDL entity `or_from_nand`, with ports `a`, `b` (in) and `x` (out), declared in that order; its testbench is in `exercises/or_from_nand` and checks it against all four rows of the `OR` truth table.

EXERCISE SET 4

Putting It Together

Exercise 1.7 The braking assistant

VHDL PAPER

Write the braking assistant from §1.6 in VHDL.

§1.6 had you derive its equations, count its gates and sketch its network *before* reading on. This is where you write it yourself, and where a testbench decides whether your reading of the requirement matches the one in §1.6's truth table.

The entity must have:

Port	Direction	Type	Description
`driver_brake`	in	`std_logic`	The driver is on the brake pedal.
`sensor`	in	`std_logic`	The proximity sensor sees an obstacle.
`radar`	in	`std_logic`	The radar sees an obstacle.
`error`	in	`std_logic`	The assistance system is reporting a fault.
`engine_brake`	out	`std_logic`	Apply the brakes.

a) Implement it from **your own** equations rather than from anyone else's. If your equations and the truth table disagree, that disagreement is the most useful thing this exercise can give you, and the testbench will tell you which way.
Use two concurrent assignments and one internal `signal`:

- The signal holds the assistance system's own decision, before the driver's pedal is considered.

- This is the first exercise in the book with an internal signal, and it is the reason the keyword exists: a name for an intermediate result, so each line still reads like the clause of the requirement it came from.

b) Now break it deliberately, in the specific way §1.6 warns about. Feed `driver_brake` *through* the fault logic instead of letting it bypass:

```
engine_brake <= (driver_brake or sensor or radar) and (not error);
```

That is one gate simpler, and it looks reasonable. Run the testbench against it and answer:

- Which input combination does it fail on first, and what is physically happening in the car at that moment?
- How many of the sixteen rows does it get wrong, and what do they all have in common?
- The failing rows are exactly the ones the safety requirement was written for. What does that tell you about the value of a truth table you derived from a requirement, as opposed to one you read off a circuit you had already built?



Self-check: name your entity `adas`, with inputs `driver_brake`, `sensor`, `radar`, `error` and output `engine_brake`, declared in that order; its testbench is in `exercises/adas` and checks all sixteen rows of §1.6's truth table.

CHAPTER 2

Larger Networks, Multiplexers and Submodules

Minimizing a network, the first constructs that need a process, and the first design built from more than one entity.

IN THIS CHAPTER

- `process` and `case`: sequential statements inside otherwise parallel hardware.
- `std_logic_vector`: bundling several wires under one name, indexing and slicing.
- Multiplexers: a 4-to-1 mux, the smallest thing worth writing a `case` for.
- Karnaugh maps: why reading sum-of-products off a truth table rarely gives the smallest network.
- Submodules: writing a block once and instantiating it more than once.
- `hex_display` on a DE0-CV: eight switches in, two hexadecimal digits out.
- Multiplexers at other sizes, in §2.4 and §2.5, and the 4-to-1 re-derived by hand in Exercises 2.4 to 2.6.
- Running a self-checking testbench with GHDL, the guide every later chapter points back to, in §2.8.

The course this book comes from builds this chapter live, in a particular order. First a 4-to-1 multiplexer in VHDL: four inputs, a two-bit selector, four branches, the smallest thing worth writing a `case` for, and the place where `process`, `case` and vector indexing all arrive. Then a Karnaugh map, worked end to end in CircuitVerse: §2.1's three-variable example, five `1` rows off the truth table, two groups on the map, and `X = AB + C` built as two gates. It is small enough to derive and check by hand, and Exercise 2.9 scales the same technique up to a segment decoder. Then `display`, a 7-segment decoder, which is the multiplexer's `case` with sixteen branches instead of four, the only new thing being the segment table; and `hex_display`, two instances of `display` with eight switches sliced into two halves. Finally the design goes onto the board: set the switches, read the hex digits.

Multiplexers at other sizes are for reading rather than presenting, in §2.4 and §2.5. Exercise 2.5 builds the 2-to-1, and Exercises 2.4 and 2.6 have you re-derive the 4-to-1, by hand and then in VHDL, rather than copy it.

Read the sections that follow before you start the exercises, and install GHDL, following §2.8, if you have not already: Chapter 1's exercises needed it too, and every exercise from here on that asks you to write a VHDL module does. The board work is shown in Appendix B for anyone who has a DE0-CV; no exercise needs one.

2.1 Karnaugh Maps: From Truth Table to Minimized Gate Network

Reading a sum-of-products equation straight off a truth table, the way §1.2 did, always works but rarely gives the smallest network: every `1` row becomes its own AND-term, even when several rows share most of their inputs and could be combined.

A **Karnaugh map** (K-map) makes those shared patterns visible by hand, for functions of 2 to 4 variables, with no formal Boolean-algebra manipulation. One worked example follows; further practice is in §2.10, and classical minimization theory (don't-cares, Quine–McCluskey) is out of scope.

The idea is to lay the truth table out as a grid rather than a list, ordering the header labels so every cell differs from each horizontal and vertical neighbour in exactly one input value, including around the left/right and top/bottom edges. That ordering is **Gray code**: for two bits, `00, 01, 11, 10`, where only one bit changes at each step, including from the last entry back to the first.

Then:

1. Write a `1` in every cell where the output is `1`, and leave the rest blank.
2. Draw rectangular groups around the `1`s:
   - Every group must be a rectangle whose width and height are each a power of 2 (1, 2 or 4), so its size is a power of 2 as well.
     - a group of 4 is therefore either a 1×4 line or a 2×2 block, and both are allowed.
     - cells that touch only at a corner are not a rectangle, so they cannot be grouped.
   - Groups may overlap, and may wrap around an edge of the grid.
3. Make every group as large as possible, and cover every `1` at least once. A group of 1 keeps all its inputs in the term, a group of 2 drops one, a group of 4 drops two, and so on.
   - Then use as few groups as will do it: drop any group whose cells are every one of them already covered by another. Nothing above forbids a redundant group, and a redundant group is a redundant AND-term in the answer, correct but not minimal.

4. For each group, keep only the input(s) that hold the same value in every cell of that group, dropping the rest, and write that as one AND-term:
   - write a kept input directly if that constant value is `1`, and primed if it is `0`.
   - so a group covering only cells where `B = 0` contributes the term `B'`.
5. `X` is the OR of all the group terms.

2.1.1 Worked example

Three inputs `A`, `B`, `C`, one output `X`:

A	B	C	X
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

Lay it out with `AB` in Gray code down the rows and `C` across the columns, filling in a `1` wherever `X = 1` (Figure 2.1).

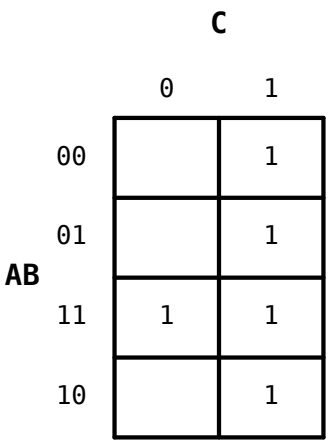


Figure 2.1 The Karnaugh map for `X`, filled in from the truth table: `AB` in Gray code down the rows, `C` across the columns.

The entire `C = 1` column is four `1`s, a group of 4. Every cell shares `C = 1` and nothing else is constant (`A` and `B` both vary), so it reduces to the single-variable term `C`:

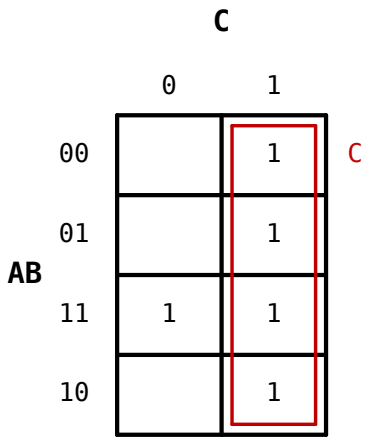


Figure 2.2 The same map with the four-cell `C = 1` group marked, which reduces to the term `C`.

One `1` is still uncovered, on the `AB = 11` row. Group it with its neighbour to the right, which the `C` group already covers. Every cell in the pair shares `A = 1` and `B = 1`, and `C` varies, so it reduces to `AB`:

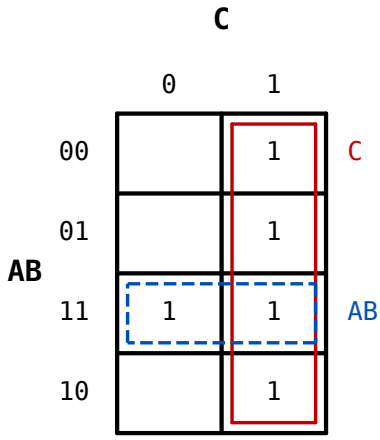


Figure 2.3 The same map with the two-cell `AB = 11` group added, overlapping the `C` group in one cell.

Every `1` is now covered; the `AB = 11, C = 1` cell is in both groups, and overlap is fine. Summing gives:

```
X = AB + C
```

Reading the truth table directly, as Chapter 1 did, would have given five AND-terms, one per 1 row. The K-map found an equivalent two-term equation, which is the entire value of doing this by hand.

2.1.2 Realizing the network

`X = AB + C` is two gates: an `AND` computing `AB`, feeding an `OR` whose other input is `C`.

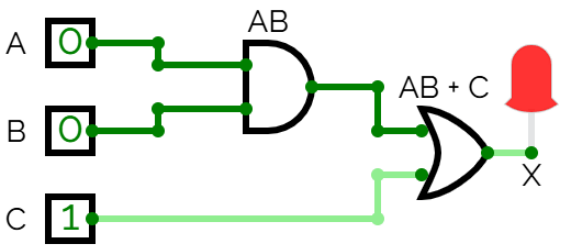


Figure 2.4 The gate network for the minimized expression `X = AB + C`: an `AND` gate feeding an `OR`, built in CircuitVerse.

The course builds this network by hand in CircuitVerse, live; build it yourself exactly as you built `or_gate`'s in Chapter 1. §2.2 translates it into VHDL.

2.2 Translating the Karnaugh-Map Network into VHDL

`X = AB + C` translates straight into VHDL. Three inputs, one output:

```
library ieee;
use ieee.std_logic_1164.all;

entity gate_network is
    port(a, b, c: in std_logic;
         x      : out std_logic);
end entity;
```

The architecture uses an internal signal `ab` to hold the `AND` gate's output, mirroring the two-gate network one-for-one:

```
architecture behaviour of gate_network is
signal ab: std_logic;
begin
    -- AND gate: AB.
    ab <= a and b;

    -- OR gate: AB + C.
    x <= ab or c;
end architecture;
```

Without the intermediate signal this collapses to `x <= (a and b) or c;`. Both are equally valid and synthesize to the same hardware; use whichever is more readable for the network at hand.

The buildable version, `lectures/L02/gate_network/gate_network.vhd`, uses the collapsed form, since at two gates there is nothing to gain from naming the intermediate. It is printed in full below; set it beside the version above and satisfy yourself they describe the same circuit. That is the claim this section is making, and it costs nothing to check.

lectures/L02/gate_network/gate_network.vhd

```
--------------------------------------------------------------------------------
-- Minimized gate network for the Appendix A Karnaugh example: x = ab + c.
--
-- Inputs:
--     - a, b, c: The three function inputs.
-- Outputs:
--     - x: Network output.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity gate_network is
    port(a, b, c: in std_logic;
         x      : out std_logic);
end entity;

architecture behaviour of gate_network is
begin
    x <= (a and b) or c;
end architecture;
```

To run it on hardware, assign the ports to switches and an LED with the DE0-CV's pin planner (Appendix B). Nothing here depends on specific pin numbers.

2.3 std_logic_vector: More Than One Wire

Everything so far has been `std_logic`, one wire carrying one bit. Bundling several wires under one name is what `std_logic_vector` is for.

```
signal count: std_logic_vector(3 downto 0);
```

That declares four wires: `count(3)` down to `count(0)`. The range is `3 downto 0` rather than `0 to 3` so the leftmost bit is the most significant, the way you would write the number on paper. This book uses `downto` everywhere, and so does almost all the VHDL you will meet.

A vector is a port type like any other:

```
entity display is
    port(number: in  std_logic_vector(3 downto 0);
         hex   : out std_logic_vector(6 downto 0));
end entity;
```

2.3.1 Values

A single bit takes single quotes, a vector double quotes:

```
x     <= '1';               -- std_logic: one bit
count <= "1010";            -- std_logic_vector(3 downto 0): four bits
count <= (others => '0');   -- every bit '0', whatever the width
```

The last is an *aggregate*: `(others => '0')` means "all the remaining bits are `'0'`", and since none were named individually, that is all of them. It keeps working when the width changes, and it appears throughout the rest of the book.

2.3.2 Indexing and slicing

One bit out of a vector is a `std_logic`; a contiguous run is a *slice*, itself a vector:

```
carry     <= count(3);
high_half <= input(7 downto 4);
```

Slicing is how one wide port feeds several narrower ones, which is what §2.6's two-digit display does with its eight switch inputs.

One detail is not what a software engineer expects. `input(7 downto 4)` has the index range `7 downto 4`, not `3 downto 0`. Connected to a port declared `std_logic_vector(3 downto 0)`, the two are matched **by position, left to right**, not by index number: `input(7)` drives bit 3, `input(6)` drives bit 2, and so on. The lengths have to match. The numbering does not.

2.3.3 A vector is a bundle of bits, not a number

`count <= "1010"` does not make `count` equal ten. `std_logic_vector` says nothing about what the bits *mean*, so it offers only the operations that make sense for a bundle of wires: the logical operators applied bit by bit (`a and b`, `not a`, `a xor b`), comparison (`count = "1010"`), and concatenation (`"10" & "11"` is `"1011"`).

Arithmetic is not on that list. `count + 1` will not compile, because nothing has said whether those four wires are an unsigned number, a signed one, or four unrelated signals.

This book does not work around that with a numeric vector type. Anything counted is a plain integer with a range:

```
signal count: natural range 0 to 9;
```

which you can add to and compare directly, and which synthesis turns into exactly the flip-flops the range needs. Vectors stay what they are: bundles of wires, for things that really are bundles of wires, like eight switches or seven display segments.

The two worlds meet only at a boundary, and the conversions appear where they are needed:

```
count_v <= std_logic_vector(to_unsigned(count, 4));   -- integer to vector
count   <= to_integer(unsigned(count_v));             -- vector to integer
```

Both come from `ieee.numeric_std`, which is why some files add a third `use` clause.

2.4 Multiplexers

A **multiplexer** ("mux") routes exactly one of several data inputs to a single output, chosen by a separate set of **selector** inputs. An `N`-to-1 mux has `N` data inputs and `ceil(log2(N))` selector bits.

The simplest case, a 2-to-1 mux with inputs `A`, `B`, selector `S` and output `X`:

S	X
0	B
1	A

As a Boolean equation, `X = S'B + SA`: for every input combination exactly one AND-term is active, and that term's data input passes through.

Larger muxes are the same idea with more selector bits, though *which* data input a given selector value names is a convention that changes from part to part, and has to be read off the specification rather than assumed. The 8-to-1 below is the opposite way round from the 2-to-1 above: there `0` names the second input, here `000` names the first. An 8-to-1 mux with data inputs `A`–`H` and selector `S[2:0]`:

S[2:0]	X
000	A
001	B
010	C
011	D
100	E
101	F
110	G
111	H

Read into sum-of-products one term per row, the `000` row contributes `A*S2'S1'S0'` and the rest follow the same shape, each gating its data input with the selector combination that selects it:

```
X = AS2'S1'S0' + BS2'S1'S0 + ... + HS2S1S0
```

Worth writing out once by hand, not worth memorizing. The pattern is the point, and it is the same at any size.

As gates: one 4-input `AND` per data input (the data bit plus all three selector bits), inverting selector bits as needed, with all eight `AND` outputs summed by a tree of `OR` gates (Figure 2.5). This is a real building block. The ATmega328P (the chip on an Arduino Uno/Nano) uses one internally so its analog-capable pins, plus an internal bandgap reference and a temperature sensor, can share a single analog-to-digital converter, selected by the four `MUX` bits of its `ADMUX` register.

2.5 Multiplexers in VHDL: `process` and `case`

Every statement so far has been **concurrent**: a standing description of a wire, all active at once. A multiplexer is the first circuit that is awkward to write that way, and needs two new constructs:

- a **`process`**, a block whose contents run sequentially, top to bottom, like ordinary code. It re-runs whenever any signal in its **sensitivity list** changes, and from the architecture's point of view it is still just one more concurrent statement. Every process in this book has this shape; Chapter 3 reuses it for clocked logic, with the clock in the sensitivity list instead of the inputs being decoded.
- a **`case` statement**, VHDL's `switch`. Like `if` and `for` it is a *sequential* statement, legal only inside a process (or a subprogram, which this book does not use), and every value the selector can take must be covered, which `when others` guarantees.

"Pick one of several inputs based on a selector" *is* a `case` statement, almost word for word. The 2-to-1 mux from §2.4:

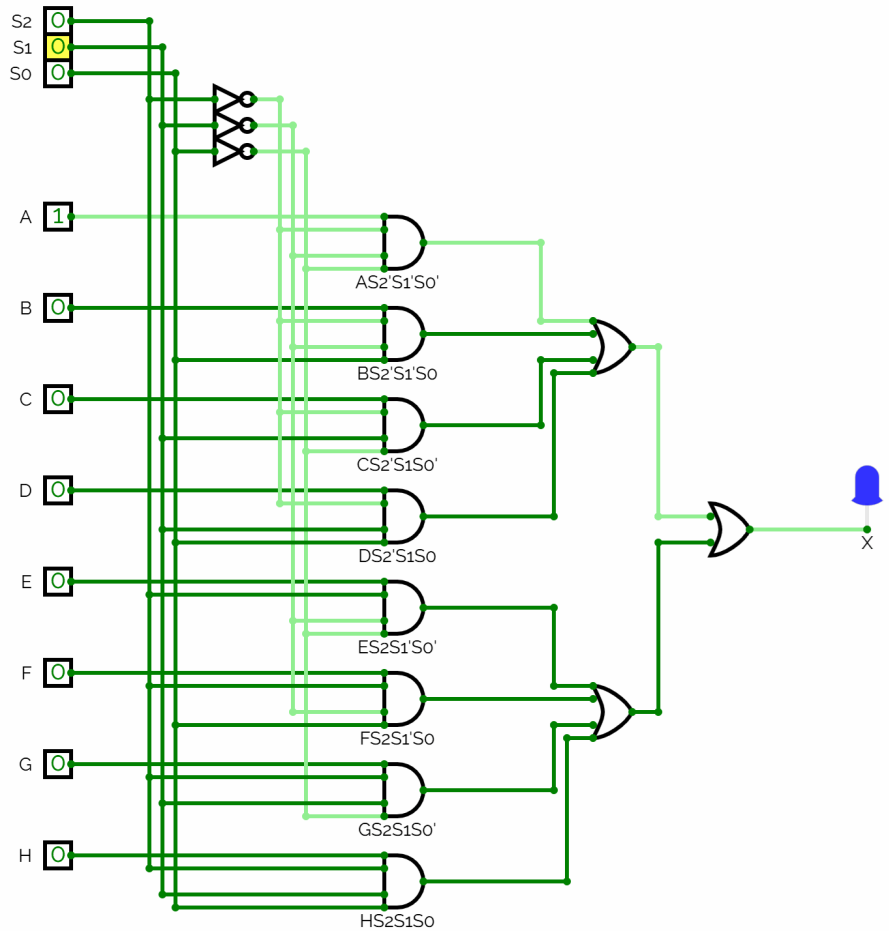


Figure 2.5 The 8-to-1 multiplexer as gates: eight 4-input `AND` gates, each gating one data input with the selector combination that selects it, feeding a tree of `OR` gates.

```
MUX_PROCESS: process(a, b, sel) is
begin
    case (sel) is
        when '0'    => x <= b;
        when '1'    => x <= a;
        when others => x <= '0';
    end case;
end process;
```

Two points from that:

- Single-bit values take single quotes, `'0'`/`'1'`, as opposed to the double quotes used for vectors like `"000"`.
- `when others` is required rather than defensive. VHDL insists a `case` cover every value of its selector's type, and `std_logic` has nine (§1.4), so `'0'` and `'1'` alone never make one exhaustive. This is a language rule, not a claim about the wire: a synthesized net carries a `0` or a `1` and nothing else, and the other seven values are the simulator's, where a signal sitting at `'U'` or `'X'` is a bug rather than a level.

2.5.1 Worked example: the 8-to-1 mux

Same shape with a 3-bit selector and eight branches, one per row of §2.4's truth table. The data inputs `A`–`H` become the vector `inputs(7 downto 0)`, with `A` as `inputs(7)` down to `H` as `inputs(0)`, so selector `"000"` routes `inputs(7)` and `"111"` routes `inputs(0)`.

The buildable module is `lectures/L02/mux_8to1/mux_8to1.vhd`:

lectures/L02/mux_8to1/mux_8to1.vhd

```
--------------------------------------------------------------------------------
-- 8-to-1 multiplexer: routes one of inputs(7:0) to x, selected by sel.
--
-- Inputs:
--     - inputs: Data inputs; sel "000" routes inputs(7), "111" routes inputs(0).
--     - sel   : Selector.
-- Outputs:
--     - x: Selected input.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity mux_8to1 is
    port(inputs: in std_logic_vector(7 downto 0);
         sel   : in std_logic_vector(2 downto 0);
         x     : out std_logic);
end entity;
```

```
architecture behaviour of mux_8to1 is
begin
    MUX_PROCESS: process(inputs, sel) is
    begin
        -- Check the selector combination, update the output accordingly.
        -- Safety default: drive output low if an invalid selector occurs.
        case (sel) is
            when "000" =>
                x <= inputs(7);
            when "001" =>
                x <= inputs(6);
            when "010" =>
                x <= inputs(5);
            when "011" =>
                x <= inputs(4);
            when "100" =>
                x <= inputs(3);
            when "101" =>
                x <= inputs(2);
            when "110" =>
                x <= inputs(1);
            when "111" =>
                x <= inputs(0);
            when others =>
                x <= '0';
        end case;
    end process;
end architecture;
```

Compare it against §2.4's eight-term equation for the same circuit. The equation describes the gates; the `case` describes the *intent* and lets the synthesis tool produce the gates. Both are correct and both synthesize to the same hardware. That gap, between describing structure and describing behaviour, is most of what a hardware description language buys you.

2.6 Building a Design from Submodules

Everything so far has been one entity with one architecture. That stops being enough the moment a design needs the same block twice.

The DE0-CV has six 7-segment displays, and you want to drive two, each showing one hexadecimal digit. The logic turning a 4-bit number into seven segment signals is the same for both. You could write it twice inside one architecture and nothing would stop you, but

the second copy is a liability: every fix has to be made in both, and the day they drift apart is the day you lose an evening working out which one is wrong.

Write the block once, as its own entity, and *instantiate* it twice.

2.6.1 The submodule

An ordinary module. Nothing about it says "I am a submodule":

```
entity display is
    port(number: in  std_logic_vector(3 downto 0);
         hex   : out std_logic_vector(6 downto 0));
end entity;
```

Whether it ends up as a whole design or one piece of a larger one is decided by whoever instantiates it, not by the module.

2.6.2 Instantiating it

With `entity work.<name>`, inside another architecture:

```
entity hex_display is
    port(input     : in  std_logic_vector(7 downto 0);
         hex1, hex0: out std_logic_vector(6 downto 0));
end entity;

architecture structure of hex_display is
begin
    display1: entity work.display
        port map(input(7 downto 4), hex1);

    display0: entity work.display
        port map(input(3 downto 0), hex0);
end architecture;
```

- `display1` and `display0` are **instance labels**, naming the two copies. They must be unique within the architecture, and they are not the module's name: both instances are `display`.
- `entity work.display` selects the module. `work` is the library your analyzed files land in by default, in GHDL and Quartus alike.
- `port map(...)` connects the instance's ports to signals in the enclosing architecture, **in the order that entity declares them**. `display` declares `number` then `hex`, so the first item drives `number` and the second is driven by `hex`.
- `input(7 downto 4)` is a **slice**, which the instance sees as a `std_logic_vector(3 downto 0)`.

Note what `hex_display`'s architecture does *not* contain: any logic. It is wiring and nothing else. Every gate in the finished design comes from the two `display` instances.

2.6.3 Positional and named association

The `port map` above is **positional**, matching ports by order. VHDL also allows **named** association, which you will meet in other people's code:

```
display1: entity work.display
    port map(number => input(7 downto 4), hex => hex1);
```

The two are equivalent. This book uses positional everywhere, which means an entity's port declaration order is something you have to get right. That is not a formality: it is exactly what the testbenches you are given rely on, which is why every exercise's self-check line spells out its ports *in order*.

2.6.4 What this costs in hardware

Nothing surprising. Instantiating a module twice puts two copies of its gates on the FPGA, exactly as writing the logic twice would. The saving is in the source, not the silicon: one description, one place to fix, and a design you can read.

2.7 Checking Your Work

Every VHDL module you write for an exercise comes with a **self-checking testbench**: a small VHDL file that drives your inputs, checks your outputs, and tells you which case failed. Chapter 1's Exercise 1.2 already shipped one, and §1.5 gave you the three commands that run it.

You do not write testbenches in this book. Running them is enough to verify every exercise on your own laptop, with no FPGA board.

§2.8 is the full guide: what a testbench is and why its entity has no ports, the analyze → elaborate → run sequence, how to read a failed assertion, and how your module's **port order** has to match what the testbench expects.

Run one now, against this chapter's worked example:

```
cd lectures/L02/gate_network
ghdl -a --std=93 gate_network.vhd gate_network_tb.vhd
ghdl -e --std=93 gate_network_tb
ghdl -r --std=93 gate_network_tb --assert-level=error --stop-time=10ms
```

2.8 Verifying Your VHDL with a Testbench

This section is the book's guide to **running** testbenches, and every later chapter's exercises point back here. You do not write testbenches in this book, but you run one after every VHDL exercise, because with no FPGA board of your own that is what verifies your VHDL. Every worked example and every exercise ships a ready-made `<module>_tb.vhd`.

2.8.1 What you need

GHDL, the open-source VHDL analyzer and simulator. Check with `ghdl --version`; install on WSL/Ubuntu with `sudo apt -y install ghdl`.

Every command below uses `--std=93`, the VHDL standard this book targets.

2.8.2 What a testbench is

A testbench is itself a VHDL module, but a special one:

- Its entity has **no ports**: nothing connects to the outside, because it is the top of the simulation.
- Its architecture **instantiates your design**, the "device under test" labelled `dut`, and drives its inputs.
- It **checks** each output with an `assert`. If an output is wrong, the assertion fires with a message and the simulation stops.

So instead of you reading a waveform to decide whether the output is right, the testbench decides, and only speaks up when something is wrong.

2.8.3 Running a testbench: analyze → elaborate → run

Three steps. Pass `--std=93` to all three, and give the **testbench entity name**, not a filename, to the last two:

```
ghdl -a --std=93 <module>.vhd <module>_tb.vhd  # analyze both files (module first)
ghdl -e --std=93 <module>_tb                   # elaborate the testbench
ghdl -r --std=93 <module>_tb --assert-level=error --stop-time=10ms   # run it
```

- `--assert-level=error` makes a failed check stop the run and return a **non-zero exit code**, a real pass/fail you can check with `echo $?`. The testbenches here raise failures at `severity failure`, which stops the run on its own, so you get a real pass/fail even without the flag. Keep it on anyway: it is the habit that also works with testbenches reporting at a lower severity.
- `--stop-time=10ms` caps how long the simulation may run. A testbench often waits for your module to do something, with a line like `wait until tx = '0';`. If your module never does, that wait never finishes and without a stop time the simulation hangs: no

output, no error, just a terminal sitting there. 10 ms of *simulated* time is far more than any testbench here needs, and takes well under a second of real time.

- A passing run prints its "all checks passed" note and exits 0. Every testbench in this book ends with that note, so if you do not see it, the run did not finish: it either failed a check or hit the stop time.

2.8.4 Checking an exercise

Each VHDL exercise ships its testbench in its own directory. For example, this chapter's Exercise 2.3 has `lectures/L02/exercises/xyz_logic/xyz_logic_tb.vhd`. Write your module in **that same directory**, named exactly as the exercise asks, then run the three commands from there:

```
cd lectures/L02/exercises/xyz_logic
ghdl -a --std=93 xyz_logic.vhd xyz_logic_tb.vhd
ghdl -e --std=93 xyz_logic_tb
ghdl -r --std=93 xyz_logic_tb --assert-level=error --stop-time=10ms
```

Each testbench header lists these exact commands.

The testbench connects to your module **by position**, not by name, so your entity must declare its ports in the **same order** the exercise lists them. The names are yours to choose, but use the ones specified anyway, so your module and the testbench's error messages talk about the same signals.

A port declared in the wrong order fails in one of two ways, neither of which says "wrong order":

- if the misplaced ports have **different** types or widths, analysis stops with `can't associate "sel" with port "sel"`, pointing at the testbench's `port map` line.
- if they have the **same** type, several `std_logic` ports say, it analyzes and elaborates quite happily, then fails at run time as an ordinary assertion failure, which reads exactly like a logic bug in your design.

So when a testbench fails on the very first case it checks, re-read your port order before hunting through your architecture. The same applies to **generics**: a module that carries them must declare exactly the ones the exercise specifies, in order.

2.8.5 When the design needs more than one file

Some exercises build a module out of another one, so the `ghdl -a` line names several files instead of two. The elaborate and run steps are unchanged: they still take the testbench **entity name** only.

```
ghdl -a --std=93 <subcomponent>.vhd <module>.vhd <module>_tb.vhd
ghdl -e --std=93 <module>_tb
ghdl -r --std=93 <module>_tb --assert-level=error --stop-time=10ms
```

- **Order matters on the analyze line**: name a module before anything that instantiates it, so GHDL has already seen the entity when it reaches the `entity work.<name>` that refers to it. Subcomponents first, your top level next, the testbench last.
- No exercise directory ships a subcomponent. Every module a design instantiates is one you wrote in an earlier exercise and copy in yourself; the exercise says which, and where from.
- The exercises that need this are this chapter's `hex_display` (Exercise 2.8, which instantiates your `display`), Chapter 4's `led_toggle_sync2`, Chapter 7's `blinker` (`reset_sync` and `timer`) and `walking_led` (those two plus `button_sync`), and all of Chapter 8's, where the receiver names five files and the transmitter four.
- Every testbench header lists the exact command for its own exercise.

2.8.6 Reading the result

A failed check looks like this:

```
xyz_logic_tb.vhd:37:13:@10ns:(assertion failure): xyz_logic: wrong X with a='0' b
    ↪ ='0' c='0' d='0', expected '0' but got '1'!
ghdl:error: assertion failed
```

Read it in order: `file:line` is where the assert is, `@time` is when it fired, then the message. It is the same shape as a GHDL compile error, so it maps onto what you already know. No failure means every case matched.

2.8.7 A complete example

A combinational testbench in the shape every testbench in this book uses, here for Chapter 1's `or_gate`:

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity or_gate_tb is
end entity;

architecture behaviour of or_gate_tb is
signal a, b, x: std_logic;
begin
```

```
    dut: entity work.or_gate
        port map(a, b, x);   -- positional: same order as or_gate's port clause.

    SIM_PROCESS: process is
    begin
        for i in 0 to 3 loop
            (a, b) <= std_logic_vector(to_unsigned(i, 2)); -- apply inputs
            wait for 10 ns;                                -- let them settle
            assert x = (a or b)                            -- check AFTER the wait
                report "or_gate: wrong output with a=" & std_logic'image(a)
                     & " b=" & std_logic'image(b)
                     & ", expected " & std_logic'image(a or b)
                     & " but got " & std_logic'image(x) & "!"
                severity failure;
        end loop;
        report "or_gate: all checks passed!" severity note;
        wait;                                              -- end the simulation
    end process;
end architecture;
```

Two rules keep a combinational testbench correct:

- **Apply → wait → check, in that order.** A signal assignment only *schedules* a value, so the inputs and output do not update until the `wait`. Checking before it tests the previous iteration's values.
- **End with `wait;`** so the stimulus process suspends and the simulation stops on its own.

Clocked designs need a little more, a clock-generator process and a way to stop it, which is exactly the machinery the provided testbenches already contain. For the exercises you only ever write the module.

2.9 Review

What you should be able to do now

After this chapter, you should be able to:

- Derive a minimized equation from a Karnaugh map and turn it into a VHDL architecture.
- Read and write `std_logic_vector` ports and signals.
- Write a `process` with a correct sensitivity list and a `case` covering every input value.
- Build a design out of more than one entity with a positional `port map`, and say why instantiating a module twice puts two copies of its gates on the FPGA.
- Verify a module by running its testbench, and read a failed assertion.

Questions to test yourself

You should be able to explain:

1. Given a truth table, can you derive its equation both directly and via a Karnaugh map? What is different about the two results?
2. Why can a `case` statement not appear directly in an architecture body?
3. What does `when others` protect you against, given that a selector "can only" be `0` or `1`?
4. In `display1: entity work.display`, what does `display1` name, and what does `work.display` name? Could a second instance reuse the same label?
5. `hex_display`'s architecture contains no logic at all. Where do the gates in the finished design come from?
6. You instantiated `display` twice rather than writing its `case` twice. Does the FPGA end up with one copy of that logic or two? So what did you actually gain?
7. This book always writes `port map` positionally. What breaks if an entity's ports are declared in a different order from the one its testbench expects, and when would you find out?

Looking ahead

Chapter 3 is about sequential logic, circuits that remember:

- The D latch, and the D flip-flop built from a pair of them.
- Clocking, and when a signal assignment actually takes effect.
- Edge detection: a button press that toggles an LED.

2.10 Exercises

How to work these. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
How to check your work. Every exercise that asks you to write a VHDL module ships a self-checking testbench under `lectures/L02/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
No FPGA board is needed for any exercise. Your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L02/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

Karnaugh Maps

Exercise 2.1 A three-variable map

PAPER CIRCUITVERSE

Derive a minimized equation for `X` from the Karnaugh map below, then realize the corresponding gate network and simulate it in CircuitVerse:

ABC	X
000	1
001	1
010	0
011	0
100	1
101	1
110	0
111	0

Tip: Lay `AB` down the rows in Gray-code order (`00, 01, 11, 10`) and `C` across the columns, exactly as in §2.1's worked example, before you start grouping `1`s.

Exercise 2.2 A four-variable map

PAPER CIRCUITVERSE

Derive a minimized equation for `X` from the Karnaugh map below, then realize the corresponding gate network and simulate it in CircuitVerse:

ABCD	X
0000	0
0001	1
0010	0
0011	1
0100	0
0101	0
0110	0
0111	0
1000	0
1001	1
1010	0
1011	1
1100	1
1101	0
1110	1
1111	0

Tip: With four variables, put two of them (in Gray code) on each axis, so you get a 4×4 grid. Groups can still wrap around every edge of the grid, not just left-right; remember to check the top/bottom wraparound too.

EXERCISE SET 2

Circuits in VHDL

Exercise 2.3 A three-output network

PAPER CIRCUITVERSE VHDL

A gate network has:

- Four inputs, `ABCD`.
- Three outputs, `XYZ`.

It is given by the truth table below:

ABCD	XYZ
0000	001
0001	010
0010	001
0011	011
0100	100
0101	110
0110	100
0111	111
1000	101
1001	110
1010	101
1011	111
1100	000
1101	010
1110	000
1111	011

a) Treat `X`, `Y`, and `Z` as three separate single-output functions of `ABCD`, and derive a minimized equation for each via a Karnaugh map.

b) Realize the resulting gate network and simulate it in CircuitVerse; confirm all three outputs against the truth table above.

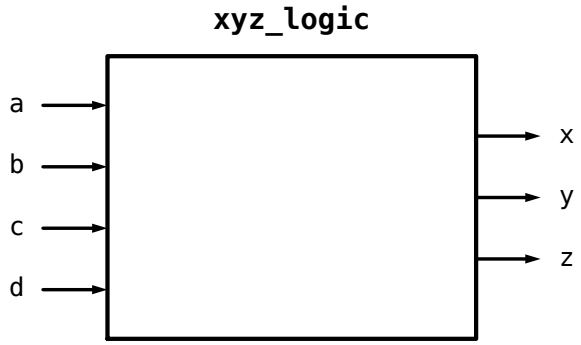
c) Implement the design in VHDL, in a module of your own with four `std_logic` inputs and three `std_logic` outputs (§1.4 and §2.2 show the pattern for turning a derived equation into concurrent VHDL assignments).

d) Run the testbench (see the aside at the start of these exercises) and confirm all 16 rows pass.

If a row fails, the assertion message names the input combination:

- Go back to the Karnaugh map for whichever of `X`, `Y`, `Z` is wrong.
- A single wrong output on a single row is nearly always one mis-grouped `1` in one map.

Tip: Two of the three outputs here turn out to depend on only one or two of the four inputs once minimized. Don't assume every output needs all four.



Self-check: name your VHDL entity `xyz_logic`, with ports `a`, `b`, `c`, `d` (in) and `x`, `y`, `z` (out), declared in that order; its testbench is in `exercises/xyz_logic` and checks all 16 rows of the truth table above.

EXERCISE SET 3

Multiplexers

Exercise 2.4 The 4-to-1 mux by hand

PAPER CIRCUITVERSE

A 4-to-1 multiplexer has:

- Four data inputs, `A`–`D`.
- Two selector bits, `S[1:0]`.
- One output, `X`.

Take the selector to name its data input directly: `S[1:0] = "00"` selects `A`, `"01"` selects `B`, `"10"` selects `C`, and `"11"` selects `D`. That is the mapping Exercise 2.6 implements.

a) Write out the multiplexer's truth table (four rows, one per selector combination).

b) Derive the sum-of-products equation for `X` in terms of `A`–`D` and `S[1:0]` (§2.4 works the same way for the 8-to-1 case; halve it here).

c) Build and simulate the network in CircuitVerse, and confirm `X` follows each data input in turn as you change `S[1:0]`.

Keep your answer: Exercise 2.6 below implements this exact circuit in VHDL, once §2.5 has given you the `process` and `case` statements it needs.

Exercise 2.5 A 2-to-1 mux with `if` VHDL

§2.5 writes the 2:1 multiplexer using a `case` statement. Write the same circuit again with an `if/else` instead, and confirm for yourself that the two describe identical hardware.

Write an entity named `mux2` with:

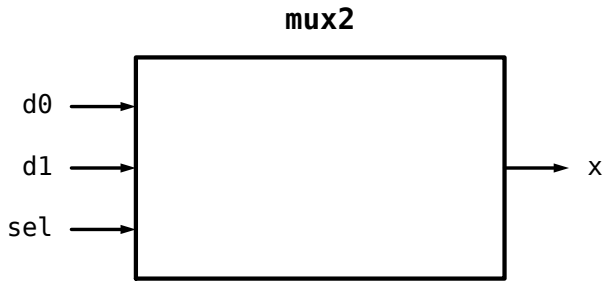
Port	Direction	Type	Description
`d0`	in	`std_logic`	Data input.
`d1`	in	`std_logic`	Data input.
`sel`	in	`std_logic`	Selector.
`x`	out	`std_logic`	The selected data input.

The selector routes the data input whose number it names:

- `sel = '0'` selects `d0`.
- `sel = '1'` selects `d1`.
- Note this is the opposite way round from §2.5's `case` example, which selects its *second* input `b` on `'0'`. Which input a selector value names is a convention, not a rule, so read it off the specification every time rather than assuming.

Implement the multiplexer using a process-based architecture:

- Include `d0`, `d1`, and `sel` in the sensitivity list.
- Use an `if/else` statement on `sel`.
- Assign `x` in every branch.
- Do not use:
  - A conditional concurrent assignment (`x <= d1 when sel = '1' else d0;`).
  - A `with...select` statement:
    - a concurrent form of `case` this book doesn't cover.
    - mentioned only so you don't reach for it if you've met it elsewhere.



Self-check: name your entity `mux2`, with ports `d0`, `d1`, `sel` (in) and `x` (out), declared in that order; its testbench is in `exercises/mux2` and checks all eight combinations of the three inputs.

Exercise 2.6 The 4-to-1 mux in VHDL

VHDL

Extend Exercise 2.5 into a 4:1 multiplexer named `mux4`.

This is the circuit you derived, drew, and simulated by hand in Exercise 2.4 above. Re-derive the `case` branches yourself for four inputs and two selector bits rather than copying the 8-to-1 example from §2.5.

The entity must have:

Port	Direction	Type	Description
`d0`	in	`std_logic`	Data input.
`d1`	in	`std_logic`	Data input.
`d2`	in	`std_logic`	Data input.
`d3`	in	`std_logic`	Data input.
`sel`	in	`std_logic_vector(1 downto 0)`	Selector.
`x`	out	`std_logic`	The selected data input.

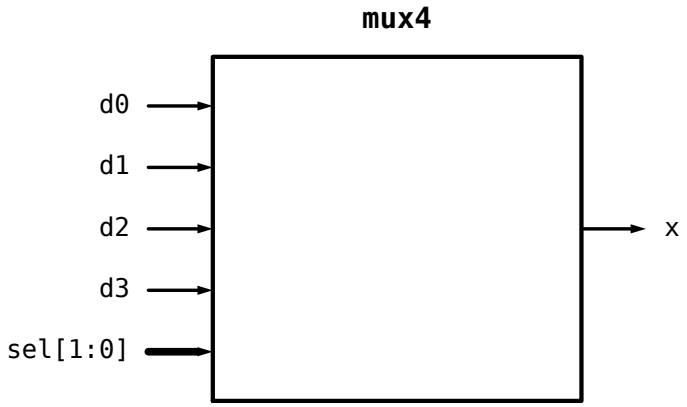
As in Exercise 2.5, the selector names its data input directly:

- `"00"` selects `d0`.
- `"01"` selects `d1`.
- `"10"` selects `d2`.
- `"11"` selects `d3`.
- Again, that is the opposite mapping from §2.5's 8-to-1 worked example, where `"000"` routes `inputs(7)`. Derive your `case` branches from the four lines above, not from that listing.

Implement the multiplexer using a process:

- Include all data inputs and `sel` in the sensitivity list.
- Use a `case` statement on `sel`, one branch per selector value above.
- Include a `when others` branch.

  - Choose a safe output value for select inputs containing values such as `'X'`, `'U'`, or `'Z'`.
- Do not replace the `case` statement with an `if/elsif` chain.



Self-check: name your entity `mux4`, with ports `d0`, `d1`, `d2`, `d3`, `sel` (`std_logic_vector(1 downto 0)`) and `x` (out), declared in that order; its testbench is in `exercises/mux4` and sweeps all four selector values against all sixteen data-input combinations.

EXERCISE SET 4

Submodules and 7-Segment Displays

Exercises 2.7 and 2.8 build one design in two halves: a module that drives a single 7-segment display, and a top level that uses two copies of it to show a hexadecimal digit on each. Do them in order: Exercise 2.8 instantiates what you write in Exercise 2.7.

A 7-segment display is seven LEDs, named `a` through `g`, arranged around a figure-eight. Light the right subset and you get a digit. This is the layout, and it is the one you need in order to work out the codes below:

```
 aaaa
f    b
f    b
 gggg
e    c
e    c
 dddd
```

So `1` lights `b` and `c` (the two on the right), and `7` lights `a`, `b` and `c`. Comparing the codes given for those two digits is a quick way to confirm you have the layout the right way round before you start deriving `A` to `F`.

On the DE0-CV the displays are wired **active-low**: driving a segment to `'0'` lights it, and `'1'` turns it off. That is why the code for `8`, which lights every segment, is `"0000000"`, and why a blank display is `"1111111"`.

Throughout, `hex(6 downto 0)` maps to segments `g`, `f`, `e`, `d`, `c`, `b`, `a`: bit 6 is `g` and bit 0 is `a`.

Exercise 2.7 One 7-segment digit

VHDL PAPER

Write a module named `display` that drives one 7-segment display with a hexadecimal digit.

The entity must have:

Port	Direction	Type	Description
`number`	in	`std_logic_vector(3 downto 0)`	The value to show, `0000` through `1111`.
`hex`	out	`std_logic_vector(6 downto 0)`	The segment code for that value.

So feeding `number` the value `0111` must put the code for `7` on `hex`, and feeding it `1110` must put the code for `E` on `hex`.

Implement it using a process:

- Put `number` in the sensitivity list.
- Use a `case` statement on `number`, one branch per value.
- Include a `when others` branch, and blank the display in it.

You are given the codes for `0` to `9`. Work out `A` to `F` yourself, from the segment layout above:

```
constant DISPLAY_0  : std_logic_vector(6 downto 0) := "1000000";
constant DISPLAY_1  : std_logic_vector(6 downto 0) := "1111001";
constant DISPLAY_2  : std_logic_vector(6 downto 0) := "0100100";
constant DISPLAY_3  : std_logic_vector(6 downto 0) := "0110000";
constant DISPLAY_4  : std_logic_vector(6 downto 0) := "0011001";
constant DISPLAY_5  : std_logic_vector(6 downto 0) := "0010010";
constant DISPLAY_6  : std_logic_vector(6 downto 0) := "0000010";
constant DISPLAY_7  : std_logic_vector(6 downto 0) := "1111000";
constant DISPLAY_8  : std_logic_vector(6 downto 0) := "0000000";
constant DISPLAY_9  : std_logic_vector(6 downto 0) := "0010000";
-- Todo: add DISPLAY_A through DISPLAY_F here.
constant DISPLAY_OFF: std_logic_vector(6 downto 0) := "1111111";
```

Two of the six are lower-case on a 7-segment display, because their upper-case forms would be indistinguishable from a digit: `b` would read as `8` and `d` would read as `0`. `A`, `C`, `E` and `F` are upper-case.

All sixteen codes are also sitting in `display_tb.vhd`, since a self-checking testbench cannot check an output without knowing what it should be. So this is not a puzzle you can be locked out of. Derive the six anyway: it takes a couple of minutes with the segment diagram, and reading them off the testbench teaches you nothing the `case` statement does not. The work of this exercise is the sixteen-branch `case`, not the lookup table.



Self-check: name your entity `display`, with ports `number` (`std_logic_vector(3 downto 0)`) and `hex` (out, `std_logic_vector(6 downto 0)`), declared in that order; its testbench is in `exercises/display` and checks all sixteen values against the full code table, naming the digit and both codes when one is wrong.

Exercise 2.8 Two digits from submodules

VHDL

Write a module named `hex_display` that shows a two-digit hexadecimal number on two displays, using two instances of your `display` from Exercise 2.7.

The entity must have:

Port	Direction	Type	Description
`input`	in	`std_logic_vector(7 downto 0)`	Eight slide switches: the top four are the left digit, the bottom four the right.
`hex1`	out	`std_logic_vector(6 downto 0)`	Segment code for the left digit.
`hex0`	out	`std_logic_vector(6 downto 0)`	Segment code for the right digit.

When it works:

- the number on `input(7 downto 4)` appears on `hex1`.
- the number on `input(3 downto 0)` appears on `hex0`.

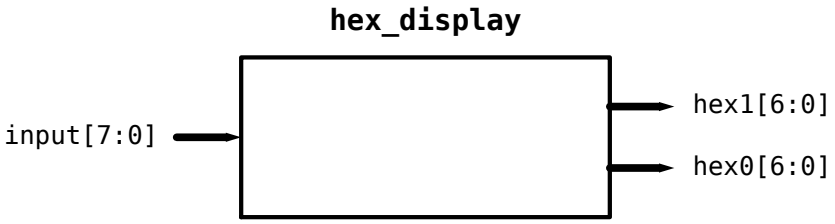
Build it with submodules, following §2.6:

- Create two instances of `display`, labelled `display1` and `display0`.
- Connect `input(7 downto 4)` and `hex1` to `display1`.
- Connect `input(3 downto 0)` and `hex0` to `display0`.
- Use positional `port map`, as everywhere else in this book.
- Write no logic of your own in `hex_display`. If you find yourself writing a `case` statement here, you are duplicating Exercise 2.7 rather than reusing it.

Copy your `display.vhd` from Exercise 2.7 into the exercise directory, since `hex_display` cannot be analyzed without it. This is the first design in the book built from more than one file, so name `display.vhd` first, ahead of the file that instantiates it:

```
ghdl -a --std=93 display.vhd hex_display.vhd hex_display_tb.vhd
ghdl -e --std=93 hex_display_tb
ghdl -r --std=93 hex_display_tb --assert-level=error --stop-time=10ms
```

§2.8.5 explains why the order matters.



Self-check: name your entity `hex_display`, with ports `input` (`std_logic_vector(7 downto 0)`), `hex1` (out) and `hex0` (out, both `std_logic_vector(6 downto 0)`), declared in that order; its testbench is in `exercises/hex_display` and sweeps all 256 input values, checking each half independently so a design that swaps the two digits is caught rather than passing on the values where both halves happen to match.

Exercise 2.9 A digit as gates

PAPER CIRCUITVERSE

If you have time. Realize one digit as a gate network, the way Chapter 1 and §2.1 had you do it:

- Take the four inputs and one segment, say `a`, and derive the Boolean equation for it from the 16-row truth table. A Karnaugh map is worth using here: most segments minimize substantially.
- Repeat for as many of the remaining six segments as you have patience for, and build the result in CircuitVerse.
- Then copy the whole gate network to drive a second display.

That copy is the point of the exercise. Duplicating a block of gates is exactly what instantiating `display` a second time did in Exercise 2.8, and it is worth having done once by hand to see what the one line of VHDL stands for.

EXERCISE SET 5

Naming an Intermediate Result

Exercise 2.10 Naming a sub-expression

VHDL PAPER

Write an entity named `combo_logic` that implements the Boolean function:

$$x = ab + c'd$$

§2.2 made the case that naming a sub-expression with an internal signal costs nothing and buys readability. This exercise is where you confirm that rather than take it on trust: you write the function twice, once as a single expression and once with the `c'd` term pulled out into an internal signal named `y`.

A signal declared inside an architecture is a wire. It is not a variable, it does not store anything, and naming a sub-expression with one does not add a gate, a delay, or a flip-flop to the circuit. It only gives a piece of the network a name, so that you, reading it later, can see what that piece is for. Both architectures below synthesize to the same gates, and the provided testbench passes against either.

The entity must have:

Port	Direction	Type
`a`	in	`std_logic`
`b`	in	`std_logic`
`c`	in	`std_logic`
`d`	in	`std_logic`
`x`	out	`std_logic`

a) Implement the entire function using a single concurrent signal assignment:

- Do not declare any internal signals.
- Translate each Boolean operation into its corresponding VHDL operator.

b) Rewrite the architecture using an internal signal named `y`:

- Use `y` to represent the term:

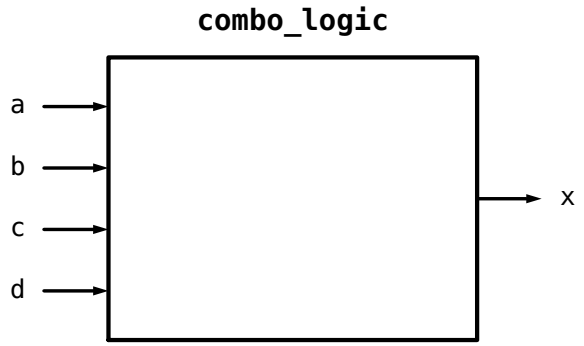
$$c'd$$

- Assign the final expression to `x`.
- This is the same shape as `gate_network` in §2.2, which names its `AND` term `ab`.

Confirm that the two architectures describe the same circuit.

Tip: Two descriptions represent the same combinational circuit when they produce the same output for every possible input combination. With four inputs:

- There are 16 possible input combinations.
- Write the complete truth table.
- Verify that both architectures produce the same value of `x` in every row.



Self-check: name your entity `combo_logic`, with ports `a`, `b`, `c`, `d` (in) and `x` (out), declared in that order; its testbench is in `exercises/combo_logic` and checks all 16 rows. It passes against either architecture, which is the point of the comparison above.

CHAPTER 3

Sequential Logic

Storing state: registers, D flip-flops, and clocking.

In this chapter

- Why combinational logic alone cannot remember anything, and what "state" means in a circuit.
- The D latch and the D flip-flop, and why synchronous designs are built from the flip-flop.
- Clocking: period, frequency, and rising versus falling edges.
- The synchronous process template in VHDL, and when a signal assignment actually takes effect.
- Edge detection: turning a level into a one-cycle pulse.
- `led_toggle` on a DE0-CV: a button press that toggles an LED and holds it.
- Registers as flip-flops in parallel, in §3.5, and `d_flip_flop` on its own, in §3.8.

The course this book comes from builds this chapter live, one circuit growing out of the last. It starts with a D latch in CircuitVerse, cross-coupled gates that are transparent while enabled, and makes a D flip-flop from a pair of them: the same circuit, now changing only on an edge. Then comes an edge-detected LED toggle, also in CircuitVerse, with one button: one flip-flop remembers the previous button state, a second holds the LED, and a falling edge on the button toggles it. The same circuit is then written in VHDL for two buttons at once, the synchronous process template twice over, with the single wires of the drawing widened into 2-bit vectors so that one process serves both buttons. That widening is the whole difference between the drawing and the module. Finally `led_toggle` goes through Quartus onto the DE0-CV: press the button, and watch the LED hold its new state until the next press. §3.9 writes up that last part.

Two predictions are worth making before each simulation runs:

- for the latch, what `Q` does when `enable` goes low while `D` is still changing.
- for the flip-flop, what `Q` does when `D` changes *between* two clock edges.

Exercises 3.1 and 3.2 are the latch and the flip-flop, which is where those two predictions are worth confirming rather than taking on trust. The module `d_flip_flop` is also written up on its own, in §3.8, with a reference testbench. It is for reading rather than presenting: the

course builds the flip-flop as part of the toggle circuit instead, which is the same module arriving with something to do.

Read the sections that follow before you start the exercises. This is the first chapter whose exercises need a testbench for a clocked design, so §2.8 is worth rereading before you start `register4` in Exercise 3.3. The board work is shown in Appendix B for anyone who has a DE0-CV; no exercise needs one.

3.1 From Combinational to Sequential Logic

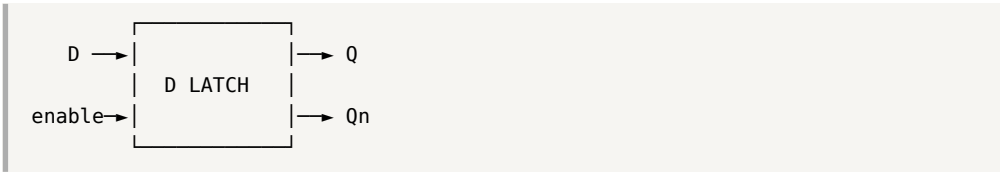
Every circuit in Chapters 1 and 2 was **combinational**: its output is a pure function of its present inputs, so the same inputs always give the same output, and there is no notion of "before" or "after" inside the circuit.

Most useful systems need more. A counter must remember its count. A register must hold a value after the signal that produced it has changed. An LED toggled by a button must remember whether it is on, since that is not encoded in the button's momentary state at all.

All of these need **memory**: an output that depends on the circuit's own *previous* output. Sequential logic is combinational logic with part of the output fed back into the input, and that **feedback loop** is what turns a stateless gate network into a circuit with state. This chapter builds the primitive that makes controlled feedback practical, the D flip-flop, then the two things you build directly out of it: registers and edge detectors.

3.2 The D Latch

The simplest storage element is the **D latch**: two cross-coupled gates, gated by an `enable` input, storing a single bit.



`D` is the value to store, `enable` controls whether the latch is *open* (transparent) or *closed* (holding), `Q` is the stored value and `Qn` is always its inverse.

Each output feeds back into the other's equation, which is exactly §3.1's feedback loop:

$$Q = (D' \cdot enable + Qn)'$$

$$Qn = (D \cdot enable + Q)'$$

`enable`	Behaviour	Result
`1`	latch is **open** (transparent)	`Q = D`, `Qn = D'`; output follows the input immediately
`0`	latch is **closed** (locked)	`Q` and `Qn` hold whatever they last had

Build it in CircuitVerse from one NOT gate, two AND gates and two NOR gates (a NOR is an OR followed by a NOT). Cross-couple the feedback so the NOR producing `Q` feeds the NOR on the `Qn` side and vice versa. Drive `D` and `enable` from input switches and watch `Q` and `Qn` on LEDs.

Then test it:

- With `enable = 1`, change `D` and confirm `Q` follows immediately. The latch is **transparent**.
- With `enable = 0`, confirm `Q` is unchanged whatever `D` does. The latch is **locked**.
- Cycle `D` a few times while toggling `enable`, and confirm the latch always freezes whatever `Q` held at the instant `enable` fell. That is what **level-sensitive** means: transparent for the entire time `enable` is high, not just at one instant.

That last point is also the latch's fundamental weakness. Because it is transparent for the whole time `enable = 1`, any glitch on `D` during that window passes straight through to `Q`. In a large synchronous system, guaranteeing stable data throughout an entire clock phase is far harder than guaranteeing it around a single instant. That is the problem the D flip-flop solves.

3.3 The D Flip-Flop

A **D flip-flop** stores a bit the same way a latch does, but is **edge-triggered** rather than level-sensitive: it samples `D` only at the *instant* the clock transitions, not for the whole time the clock is high or low.

Feed the same `D` and the same clock to both and the difference is immediate (Figure 3.1).

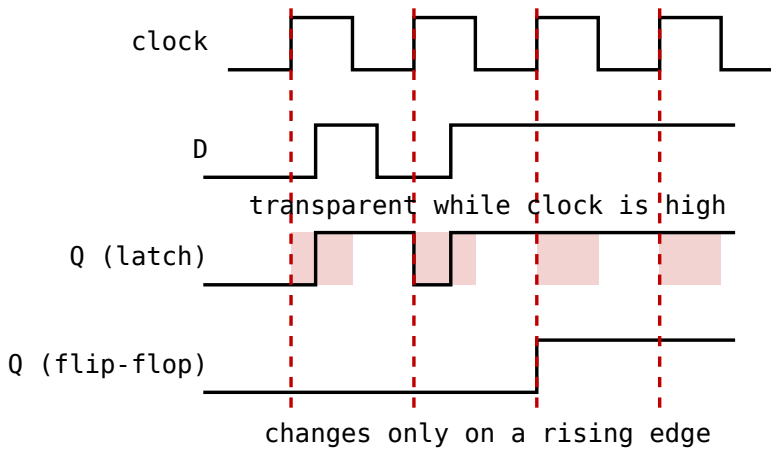


Figure 3.1 A latch and a flip-flop fed the same `D` and clock: the latch follows `D` throughout every clock-high window, while the flip-flop changes only on the rising edges.

The latch changes three times: twice because `D` moved while the clock was high, and once because the clock went high onto a `D` that had already changed. The flip-flop changes once, because only one of the four rising edges found `D` different from the value `Q` was already holding. Both are behaving correctly; they are answering different questions.

```
          ┌─────────────────┐
    D ──►│                 │──► Q
clock ──►│  D FLIP-FLOP    │
enable──►│ (rising-edge)   │──► Qn
reset_n►│                 │
          └─────────────────┘
```

`clock` is the timing reference (§3.4). `enable` gates whether the flip-flop accepts a new value *at the triggering edge*. `reset_n` is an asynchronous, active-low reset: whenever it is `0`, `Q` is forced to `0` immediately, regardless of the clock.

Condition	Result
`reset_n = 0`	`Q = 0`, `Qn = 1`, immediately (asynchronous, ignores the clock entirely)
`reset_n = 1`, rising edge on `clock`, `enable = 1`	`Q = D`, `Qn = D'`, sampled at that instant
`reset_n = 1`, rising edge on `clock`, `enable = 0`	`Q` holds its previous value
`reset_n = 1`, no rising edge on `clock`	`Q` holds its previous value, no matter what `D` does

That last row is worth seeing rather than reading (Figure 3.2).

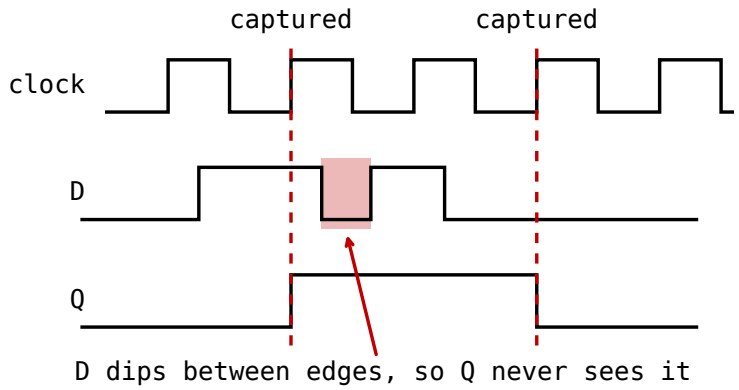


Figure 3.2 A flip-flop between edges: `D` rises mid-cycle and is captured at the next rising edge, then dips and recovers entirely between two edges without `Q` ever moving.

`D` dips low and comes back inside a single clock period, and `Q` never moves, because no edge happened while it was low. A latch in the same position would have followed it down and back up.

Internally, an edge-triggered flip-flop is two D latches in series, a **master-slave** pair. One is transparent while the clock is low and captures `D`; the other is transparent while the clock is high and passes the captured value to `Q`. The net effect is that `Q` changes only at the instant the clock goes low to high, never while it sits at either level. That external behaviour is what VHDL's `rising_edge()` models (§3.6).

Build it in CircuitVerse by extending your §3.2 latch:

- Duplicate the latch so two sit in series: the *master* feeds the *slave*, whose `Q` is the flip-flop's output.
- Add `clock`, `reset_n`, `D` and `enable` as inputs.
- Drive the two latches' transparency from the clock and its inverse. To keep the two meanings apart, call that internal per-latch control `gate`: the master's `gate` is `clock'` and the slave's is `clock`. The flip-flop's external `enable` is a different signal and is wired separately, below.
- Wire the external `enable` as a 2-to-1 mux (§2.4) on the master's data input: when `enable = 1` the mux passes `D`, and when `enable = 0` it passes the flip-flop's own `Q` back in, so the edge re-captures the value already stored. Gate the *data*, never the clock.
- Add the reset to **both** latches. Forcing `Q` low takes two changes in each latch, not one:
  - feed an active-high `reset` (`reset_n` through a NOT) into the NOR producing `Q`, forcing `Q` to `0`.
  - AND `reset_n` into that same latch's `D AND gate` term, which frees `Qn` to go high.
- Set the clock period to something slow enough to watch, e.g. `1000 ms`.

With the reset wired in, each latch's equations become:

$$Q = (D' \cdot gate + Qn + reset)'$$

$$Qn = (D \cdot gate \cdot reset_n + Q)'$$

A tempting shortcut is to leave the `Qn` equation alone and break the loop instead, ANDing `reset_n` onto the `Qn` wire feeding `Q`'s NOR. That is not enough. With `gate = 1` and `D = 1` it leaves `Qn` at `0`, which is what holds `Q` at `1`, so the reset silently does nothing in exactly the case you are most likely to test it in. Both changes, in both latches, or neither.

Confirm that `Q` updates only at the instant the clock rises, holding steady between edges even if `D` changes, and that pulling `reset_n` low forces `Q` to `0` immediately, with releasing it changing nothing until the next rising edge.

3.4 Clocking

The **clock** is a periodic square wave, read by every flip-flop in a synchronous circuit from the same source. Two numbers describe it: the **period** `T`, the time for one full cycle, and the **frequency** `f = 1/T` in Hz.

Each period has exactly two **edges**: a **rising edge** from `0` to `1`, and a **falling edge** from `1` to `0`. A synchronous design picks one of them, almost always the rising edge, and updates every flip-flop on that edge and only that edge. Using a single edge everywhere is what makes a large circuit's timing analyzable: every flip-flop changes at the same well-defined instants, so you can reason one clock cycle at a time rather than tracking every signal continuously.

Simulated and real clocks run on wildly different timescales, and the logic is identical either way:

- In CircuitVerse, set a period you can watch, e.g. `1000 ms`.
- On the DE0-CV the system clock is a fixed `50 MHz`, a period of `20 ns`. Nobody perceives individual edges at that rate, so you reason in terms of "on the next rising edge", never in wall-clock time.

3.5 Registers: Flip-Flops in Parallel

A **register** stores more than one bit: N D flip-flops side by side, one per bit, all sharing the same `clock`, `reset_n` and `enable`.

```
D(0) ──►[D FF]──► Q(0)
D(1) ──►[D FF]──► Q(1)
D(2) ──►[D FF]──► Q(2)
   ⋮        ⋮
D(N-1)──►[D FF]──► Q(N-1)

  (clock, reset_n, enable shared by every flip-flop above)
```

That is the entire idea: an 8-bit register is eight D flip-flops updating on the same clock edge.

In VHDL you rarely instantiate N flip-flops explicitly. Declare one `std_logic_vector` and assign the whole thing inside a single synchronous process (§3.6): every bit synthesizes to its own flip-flop, all sharing the process's clock and reset.

```
signal q: std_logic_vector(3 downto 0);
...
process (clock, reset_n) is
begin
    if (reset_n = '0') then
        q <= "0000";
    elsif (rising_edge(clock)) then
        if (enable = '1') then
            q <= d; -- d is a std_logic_vector(3 downto 0) input
        end if;
    end if;
end process;
```

Four bits, one process, four flip-flops in hardware: no different in principle from four `d_flip_flop` instances wired in parallel.

3.6 The Synchronous Process Template in VHDL

Every synchronous element in this chapter, the flip-flop, the register and the edge detector, uses the same shape. Without a reset, a single D flip-flop is:

```
process (clock) is
begin
    if (rising_edge(clock)) then
        q <= d;
    end if;
end process;
```

`q` takes `d`'s value at every rising edge and holds it the rest of the time. That one line inside the `if` is enough for synthesis to infer a real D flip-flop.

3.6.1 Rules

- **`clock` is the only signal in the sensitivity list**, unless there is an asynchronous reset. The process should run only when the clock changes; it has no reason to react to anything else.
- A process reacts to *any* change on a sensitivity-list signal, not just rising ones, so `rising_edge()` filters for the rising edge specifically. All logic that updates synchronously goes inside that `if`.
- Every signal assigned inside such a process synthesizes to a flip-flop, one bit each.

3.6.2 With an asynchronous reset

An asynchronous reset must also appear in the sensitivity list, since it can be asserted independently of the clock, and it takes priority when active.

```
process (clock, reset_n) is
begin
    if (reset_n = '0') then
        q <= '0';
    elsif (rising_edge(clock)) then
        q <= d;
    end if;
end process;
```

Read it in priority order: check the reset first, and only if it is inactive check for a rising edge. If neither holds (a falling edge, or `reset_n` going from `0` back to `1`, which also re-triggers the process), nothing is assigned and `q` keeps its previous value.

3.6.3 When the assignment actually happens

One rule sits underneath all of the above, and it is the single place where <= behaves unlike assignment in any language you have written software in.

A signal assignment does not take effect when its line runs. It **schedules** a value, applied only once the process finishes this pass and suspends. Until then, every read of that signal returns the value it held *before the pass started*, however many assignments have already executed.

That sounds like a technicality. It is the reason the template above is a flip-flop rather than a wire:

```
process (clock) is
begin
    if (rising_edge(clock)) then
        b <= a;
        c <= b;
    end if;
end process;
```

Read as software, `b` and `c` both end up holding `a`. In hardware they do not. On each edge `b` is scheduled to take `a`'s value, and `c` is scheduled to take the value `b` held *coming into this edge*, not the one just scheduled onto it. The result is two flip-flops in a chain: `a` reaches `b` after one edge and `c` after two. Swap the lines and nothing changes, because neither assignment can observe the other's result.

Two consequences worth carrying forward:

- **Order does not matter** between assignments to *different* signals in the same clocked process. They all read the old values and all take effect together. That is the opposite of the sequential reading the word "process" invites, and it is why a chain of flip-flops can be written in any order.
- **Assign the same signal twice in one pass and only the last assignment survives.** The earlier ones are discarded without ever reaching the signal.

§3.7's edge detector is exactly the two-element chain above, and works for exactly this reason. Chapter 5 returns to this rule from the other side, with a `variable`, which updates immediately and so behaves the way the software reading expects.

3.7 Edge Detection

A flip-flop's other everyday job is detecting the *instant* a signal changes rather than reading its level: store the signal's previous value in a flip-flop, then compare it against the current value with a gate.

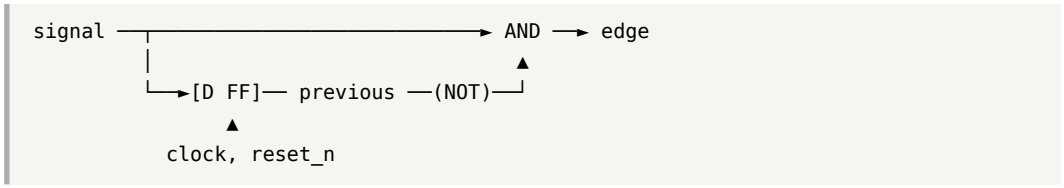
- **Rising-edge detection** (`0` to `1`):

$$edge = current \cdot previous'$$

- **Falling-edge detection** (`1` to `0`):

$$edge = current' \cdot previous$$

The rising-edge form as a circuit; swap which side is inverted for a falling edge, which is the form §3.9's active-low button needs:



`edge` is high for exactly one clock cycle, the one immediately after the transition, because `previous` only catches up on the *next* edge. This is the mechanism behind "do something once per button press" rather than "do something every clock cycle the button is held", which is exactly what §3.9 solves.

3.8 Worked Example: A D Flip-Flop in VHDL

`lectures/L03/d_flip_flop/d_flip_flop.vhd` implements §3.3 and §3.6 combined: an enable gating whether a rising edge captures `d`, and an asynchronous active-low reset.

```
entity d_flip_flop is
    port(clock, reset_n, d, enable: in std_logic;
         q, q_n                   : out std_logic);
end entity;
```

Its single process is the reset-plus-enable variant of §3.6's template: reset takes priority, then a rising edge captures `d` into an internal signal `q_s` if `enable = '1'`. `q_s` exists because an *output* port cannot be read back inside the same architecture; `q` and `q_n` are both driven combinationally from it, outside the process. This is the whole of `d_flip_flop.vhd`:

lectures/L03/d_flip_flop/d_flip_flop.vhd

```
--------------------------------------------------------------------------------
-- Rising-edge D flip-flop with asynchronous reset and enable.
--
-- Inputs:
--     - clock  : 50 MHz system clock.
--     - reset_n: Active-low asynchronous reset.
--     - d      : Data input.
--     - enable : Capture enable ('1' = load d on the edge, '0' = hold).
-- Outputs:
--     - q  : Flip-flop output.
--     - q_n: Inverse of q.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity d_flip_flop is
    port(clock, reset_n, d, enable: in std_logic;
         q, q_n                   : out std_logic);
end entity;

architecture behaviour of d_flip_flop is
-- Internal representation of q.
signal q_s: std_logic;
begin
    q   <= q_s;
    q_n <= not q_s;

    -- Implement D flip flop logic.
    DFF_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            q_s <= '0';
```

```
        elsif (rising_edge(clock)) then
            if (enable = '1') then
                q_s <= d;
            end if;
        end if;
    end process;
end architecture;
```

To try it on hardware, assign `clock` to the `50 MHz` system clock, `reset_n` to a pushbutton, `d` and `enable` to two slide switches, and `q`/`q_n` to two LEDs. Then flip `enable` on and change `d`, and the LED follows; flip `enable` off and change `d`, and it does not.

There is no clock edge you can perceive at `50 MHz`, so what you are really confirming is that the logic still holds once you can no longer see the individual edges. The flip-flop looks like it responds to your switch; in fact it responded to one of fifty million clock edges that second, and the switch only decided which value that edge captured.

3.9 Worked Example: Edge-Detected LED Toggle

`lectures/L03/led_toggle/led_toggle.vhd` combines §3.5, §3.6 and §3.7 into one small system: two independent push buttons, each toggling its own LED exactly once per press.

```
entity led_toggle is
    port(clock, reset_n: in std_logic;
         button_n      : in std_logic_vector(1 downto 0);
         led           : out std_logic_vector(1 downto 0));
end entity;
```

Three 2-bit signals, two held in clocked processes and one computed between them:

- `button_prev_n` is a 2-bit register (§3.5) holding last cycle's `button_n`, reset to `"11"` since the buttons are active-low.
- `button_edge <= (not button_n) and button_prev_n` is §3.7's falling-edge detector applied to both bits at once, high for one cycle exactly when a button goes from released to pressed.
- `led_s` is a second 2-bit register which, instead of capturing a new value each cycle, **toggles** bit `i` only when `button_edge(i) = '1'`. Without that gating it would toggle every clock cycle instead of once per press, which is the whole point of computing `button_edge`.

This is the whole of `led_toggle.vhd`:

```
lectures/L03/led_toggle/led_toggle.vhd

--------------------------------------------------------------------------------
-- Toggles led(i) on each falling edge of button_n(i); all LEDs off on reset.
```

```
--
-- Inputs:
--     - clock   : 50 MHz system clock.
--     - reset_n : Active-low reset.
--     - button_n: Active-low push buttons.
-- Outputs:
--     - led: LED toggled by the matching button.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity led_toggle is
    port(clock, reset_n: in std_logic;
         button_n      : in std_logic_vector(1 downto 0);
         led           : out std_logic_vector(1 downto 0));
end entity;

architecture behaviour of led_toggle is
-- Previous button values (for edge detection).
signal button_prev_n: std_logic_vector(1 downto 0);

-- Button pressdown indicators (falling edge).
signal button_edge: std_logic_vector(1 downto 0);

-- LED states (necessary for toggling, as output signals cannot be read).
signal led_s: std_logic_vector(1 downto 0);
begin
    -- Detect a falling edge of button_n.
    button_edge <= (not button_n) and button_prev_n;
    led <= led_s;

    -- Update button_prev_n on system reset or on rising edge of the system clock.
    BUTTON_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            button_prev_n <= "11";
        elsif (rising_edge(clock)) then
            button_prev_n <= button_n;
        end if;
    end process;

    -- Update led_s on system reset or on rising edge of the system clock.
    LED_PROCESS: process(clock, reset_n) is
    begin
```

```
        if (reset_n = '0') then
            led_s <= "00";
        elsif (rising_edge(clock)) then
            for i in 0 to 1 loop
                if (button_edge(i) = '1') then
                    led_s(i) <= not led_s(i);
                end if;
            end loop;
        end if;
    end process;
end architecture;
```

On the board, as the course demonstrates it: create the Quartus Prime Lite project targeting the DE0-CV's `5CEBA4F23C7N` exactly as in Chapter 1 (Appendix B), then assign `clock` to the `50 MHz` oscillator, `reset_n` and `button_n(1 downto 0)` to three push buttons, and `led(1 downto 0)` to two LEDs. Each press should toggle exactly one LED, once, no matter how long you hold it.

A caveat, deliberately left open: `button_n` is a raw signal straight off a physical pushbutton pin. In simulation that is a clean, instantaneous transition; on real hardware it is neither instantaneous (contacts bounce) nor synchronized to `clock`. This edge detector is logically correct but *unsafe* to wire directly to a real button on an FPGA without more work first, and that work is exactly what Chapter 4 covers.

3.10 Review

What you should be able to do now

After this chapter, you should be able to:

- Say why sequential logic exists, explain the functional difference between a D latch and a D flip-flop, and why synchronous designs are built almost entirely from the flip-flop.
- Read and write the synchronous process template, and recognize a register as several flip-flops sharing a clock, reset and enable.
- Say when a signal assignment takes effect: `<=` schedules rather than applies, so two assignments in one clocked process build a chain, and their order makes no difference.
- Build an edge detector from a flip-flop and a gate, and say exactly what signal it produces and for how long.

Questions to test yourself

You should be able to explain:

1. What is the functional difference between a D latch and a D flip-flop, and under what condition does each change its output?
2. Why do synchronous circuits update their state on a single clock edge instead of continuously reacting to their inputs?
3. Given a signal and a flip-flop holding its value from one cycle earlier, how do you combine them into a pulse that is high for exactly one clock cycle when the signal rises?
4. Why must `clock` be the only signal in a flip-flop process's sensitivity list, unless the design has an asynchronous reset?
5. A clocked process contains `b <= a;` and then `c <= b;`. After one clock edge, what does `c` hold, and why is it not `a`? What changes if you swap the two lines, and why?

Looking ahead

Chapter 4 is about metastability and synchronization:

- Why the raw button feeding this chapter's edge detector can drive a flip-flop into an undefined, unstable state, and why that is a different kind of problem from anything here.
- The double-flop synchronizer, and what it does and does not do about a bouncing button.
- The same LED toggle, rebuilt so it is actually safe on real hardware.

3.11 Exercises

How to work these. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
How to check your work. Every exercise that asks you to write a VHDL module ships a self-checking testbench under `lectures/L03/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
No FPGA board is needed for any exercise. Your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L03/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

D Latch

Exercise 3.1 A D latch

PAPER CIRCUITVERSE

Build the first circuit in this book that can *remember* something.

Everything you have built so far has been combinational: the outputs follow the inputs, and the moment the inputs change the outputs change with them. A latch is the smallest departure from that. It has a data input `D` and a control input `enable`, and it has two outputs, `Q` and its complement `Qn`. While `enable` is high the latch is **transparent**: `Q` simply follows `D`, and it looks like a wire. The interesting part is what happens when `enable` goes low. The latch **locks**, holding whatever `D` was at that instant, and from then on `D` can do what it likes without `Q` noticing. That held value is one bit of memory, and it is built out of nothing but the gates you already know, wired so that the output feeds back into the input.

`Qn` is not decoration. The feedback loop is what makes the circuit remember, and the two outputs are what feed each other: `Q` is computed from `Qn` and `Qn` from `Q`. Build it and you can see the loop; that is the point of drawing it by hand before you ever meet the VHDL.

Use the equations from §3.2:

$$Q = (D' \cdot enable + Qn)'$$

$$Qn = (D \cdot enable + Q)'$$

a) Realize the corresponding gate network:

- Draw the network by hand.
- Recreate and simulate it in CircuitVerse (`https://circuitverse.org/simulator`).

b) Test the transparent and locked states:

- Set `enable = 1`:
  - Change `D`.
  - Observe what happens to `Q` and `Qn`.
- Set `enable = 0`:
  - Change `D` again.
  - Observe what happens to `Q` and `Qn`.
- Explain the difference between the two cases.

c) Cycle `D` and `enable` through the following combinations twice:

- `00`
- `01`
- `10`
- `11`

Confirm that you observe both cases:

- The latch locks after being transparent with `D = 1`.
- The latch locks after being transparent with `D = 0`.

Explain how the value of `Q` depends on the value of `D` at the instant `enable` changes from `1` to `0`.

EXERCISE SET 2

D Flip-Flop

Exercise 3.2 A D flip-flop from two latches

PAPER CIRCUITVERSE

Extend the D latch from Exercise 3.1 into a D flip-flop, following §3.3.

The flip-flop must have:

- Inputs:
  - `clock`
  - `reset_n`
  - `D`
  - `enable`
- Outputs:
  - `Q`
  - `Qn`

The reset signal is:

- Asynchronous:
  - It takes effect without waiting for a clock edge.
- Active-low:
  - Setting `reset_n = 0` must force `Q` to `0` immediately.

Tip:

- The reset has to do two things in each of the two internal latches, not one:
  - Feed an active-high `reset` (`reset_n` through a NOT gate) into the NOR gate that produces `Q`. This forces `Q` low without waiting for a clock edge.
  - `AND` `reset_n` into the AND gate that produces that latch's `D` and `enable` product. This frees `Qn` to go high.
  - Gating only the `Q` feedback path is not enough: with `enable = 1` and `D = 1` it leaves `Qn` low, and a low `Qn` is exactly what holds `Q` high, so the reset would do nothing.
- Drive the enable inputs of the two internal latches using:
  - `clock`
  - The inverse of `clock`

- Follow §3.3 for how the external `enable` input controls whether the flip-flop accepts a new value.

a) Realize the corresponding gate network:

- Draw the network by hand.
- Recreate and simulate it in CircuitVerse.
- Set the clock period to `1000 ms`.
  - This is slow enough to observe the behaviour by eye.

b) Test the clocked behaviour:

- Change `D` during the high phase of the clock.
- Change `D` during the low phase of the clock.
- Observe `Q` and `Qn`.
- Confirm that the outputs change only on the active clock edge, never between clock edges.

Explain how the two internal latches work together to produce edge-triggered behaviour.

c) Test the asynchronous reset:

- First set `Q = 1`.
- Assert the reset by setting `reset_n = 0`.
- Observe whether `Q` resets:
  - Immediately.
  - At the next clock edge.

Explain the result by referring to the difference between an asynchronous signal and a synchronous signal.

EXERCISE SET 3

Registers

Exercise 3.3 A four-bit register

VHDL PAPER

Write a complete VHDL entity named `register4`: four flip-flops side by side, sharing one clock, one reset, and one enable.

This is the exercise where the flip-flop stops being a curiosity and becomes a building block. A single D flip-flop stores one bit, which is not much use on its own; what a design actually wants is somewhere to keep a *number*. Put four flip-flops in a row, wire the same clock to all of them, and you have a four-bit register. Nothing about the individual flip-flop changes. Only the width does.

The part worth paying attention to is how little the VHDL has to change to say that. You are writing the same synchronous process template from §3.6 that you already used for

one bit, with `std_logic_vector(3 downto 0)` in place of `std_logic`. There is no loop, no repetition, no four copies of anything. That is not a shortcut the language is offering you; it is a fact about the hardware, and part **c**) below asks you to say why.

The entity must have:

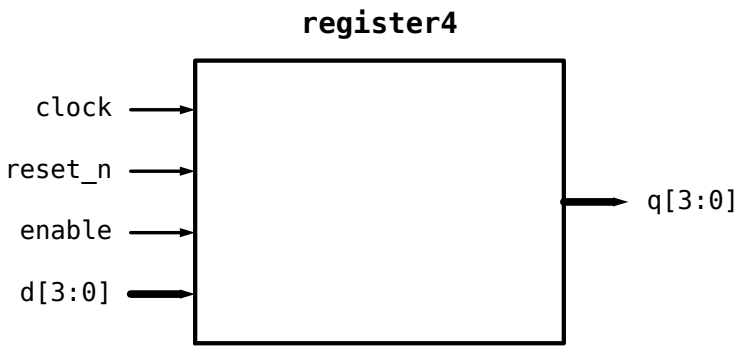
Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_n`	in	`std_logic`	Asynchronous, active-low. Clears the register to `"0000"`.
`enable`	in	`std_logic`	`'0'` retains the current value; `'1'` captures `d` on the next rising edge.
`d`	in	`std_logic_vector(3 downto 0)`	The value to capture.
`q`	out	`std_logic_vector(3 downto 0)`	The stored value.

Write the process out yourself rather than copying the worked D flip-flop example, `d_flip_flop.vhd` (§3.8). Typing it is how the template stops being something you recognize and starts being something you can produce from memory, and you will be writing it in every remaining chapter of this book.

a) Write the entity and the synchronous process, adapting the template from a single bit to a four-bit vector.

b) Check it with the testbench, as described at the start of these exercises.

c) Explain why the same template scales from one flip-flop to four without any structural change, while a four-bit *counter* could not simply be four one-bit counters side by side. What is different about the two cases?



Self-check: name your entity `register4`, with ports `clock`, `reset_n`, `enable`, `d` (`std_logic_vector(3 downto 0)`) and `q` (`std_logic_vector(3 downto 0)`), declared in that order; its testbench is in `exercises/register4`.

EXERCISE SET 4

Edge Detection

Exercise 3.4 An edge-detected toggle

PAPER CIRCUITVERSE

Construct a circuit that toggles an LED once for every sampled rising edge of a button signal.

Follow:

- §3.7
- §3.9

The circuit must have:

- Inputs:
  - `button`
  - `clock`
  - `reset`
- Output:
  - `led`

For this exercise, `reset` is active-high, unlike the reset used in the worked example. It is still **asynchronous**, like every reset in this book: assert it and `led` clears immediately, without waiting for a clock edge. In the template from §3.6 that means the reset branch stays outside the `rising_edge(clock)` test, and only the polarity of the test changes.

Tip: Use two flip-flops:

- One flip-flop stores the previous sampled value of `button`.
  - Use this value to detect a rising edge.
- One flip-flop stores the current state of `led`.
  - Toggle it only when a rising edge is detected.

a) Realize the corresponding gate network:

- Draw the network by hand.
- Recreate and simulate it in CircuitVerse.
- Set the clock period to `1000 ms`.

b) Test the edge detector:

- Hold `button` high for several consecutive clock cycles.
- Confirm that `led` toggles exactly once.
- Confirm that it does not toggle once per clock cycle.

Explain which stored signal prevents the circuit from repeatedly detecting the same high level as a new rising edge.

Exercise 3.5 The toggle in VHDL

VHDL

Implement the circuit from Exercise 3.4 in VHDL.

a) Create an entity named `led_toggle_single` with:

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset`	in	`std_logic`	Asynchronous, and **active-high**, unlike every other module in this book. Assert it and `led` clears without waiting for a clock edge, so the reset branch belongs outside the `rising_edge(clock)` test.
`button`	in	`std_logic`	Toggles `led` once per press.
`led`	out	`std_logic`	The LED state.

Note the single-bit types: the worked example uses `std_logic_vector` ports for two buttons and two LEDs, and this one drives a single pair.
Do not copy `led_toggle.vhd` (§3.9) directly. Instead:

- Re-derive the two clocked processes yourself:
  - One process stores the previous value of `button`.
  - One process stores and toggles the current value of `led`.
- Derive the edge-detection equation for:
  - A rising edge.
  - An active-high button.
- Use the rising-edge equation from §3.7:

$$edge = current \cdot previous'$$

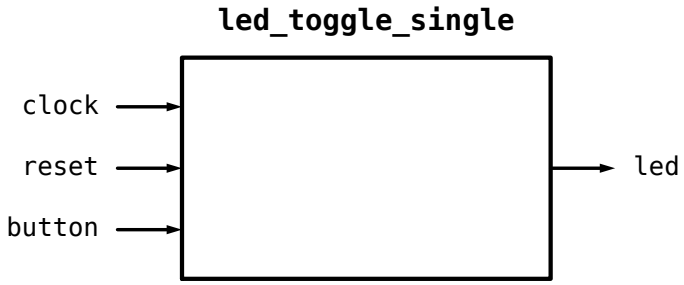
Do not use the falling-edge equation from the worked example, which uses an active-low button.

b) Verify the design with its testbench (see the aside at the start of these exercises).
The testbench drives the same sequence you would exercise by hand on a board:

- It asserts `reset` and checks `led` is cleared.
- It raises `button` and checks `led` toggles exactly once.
- It **holds `button` high** for several more clock cycles and checks, after every one of them, that `led` does *not* toggle again.
- It releases and presses again, and checks the second press toggles `led` once more: back off, since the first press had turned it on.
- It releases and presses a third time, so the LED is lit when the reset arrives.
- It asserts `reset` with `led` lit and **no clock edge following**, and checks `led` clears anyway. This is the one that fails if you put the reset inside the `rising_edge(clock)` branch.

That third check is the one that fails if your edge detector is wrong:

- An `led` that toggles on every clock cycle while the button is held means you drove the toggle from `button`'s **level** rather than from its edge.
- Re-read the equation above: `edge` is high only when `button` is high *now* and its stored previous value is low.



Self-check: name your entity `led_toggle_single`, with ports `clock`, `reset`, `button` (in) and `led` (out), declared in that order; its testbench is in `exercises/led_toggle_single`. Note `reset` is **active-high** here, unlike the worked example.

CHAPTER 4

Metastability and Synchronization

Making an asynchronous input safe to use in a synchronous system.

In this chapter

- Why an asynchronous input can break the clean edge detectors and registers built in Chapter 3.
- Metastability: what it is, and why it cannot simply be designed away.
- The double-flop synchronizer, and the assert-asynchronously, release-synchronously reset pattern.
- Reusing the same chain for edge detection, and what it does and does not do about bounce.
- Generics: one module serving several sizes.
- `led_toggle_sync`, and what it does on the DE0-CV when a real, bouncing push button drives it.
- Setup/hold timing detail (§4.4), and what a real debounce adds (§4.7).

This chapter takes Chapter 3's LED toggle and makes it safe for a real button. The course builds it live, in this order: first as a gate network in CircuitVerse, with two flip-flops in front of the button, two in front of the reset, and the edge detector behind them; then in VHDL, as `reset_sync`, which asserts asynchronously and releases synchronously, and `button_sync`, the three-flop chain with a `generic` so that one file serves a design with one button and a design with two; then `led_toggle_sync`, the two subcomponents wired together around the toggle logic, which is all that is left of the original circuit; and finally onto the board. Before the synchronizer goes in, predict what a real, asynchronous button does to Chapter 3's edge detector. That prediction is the reason for the whole chapter.

The chapter opens with metastability itself, and the argument worth holding on to is why two flip-flops make the failure *unlikely* rather than impossible: that is the part that does not survive being remembered as a rule of thumb. Two parts of the material are ones the course leaves to reading rather than presenting: the setup/hold timing detail in §4.4, beyond the fact that the window exists and that violating it is what causes the trouble; and §4.7, on what

a real debounce circuit adds. Both come back in the review questions and are drilled by Exercises 4.2 and 4.7, so they are not optional.

Before you read on, be comfortable with D flip-flops, clocking and edge detection from Chapter 3; this chapter assumes all of it. It also builds on §2.6: its worked example is composed from two subcomponents, and §4.8 adds the `generic` that lets one of them serve designs with different numbers of buttons. Afterwards, work through the exercises and read `lectures/L04/led_toggle_sync/led_toggle_sync.vhd`. Its `reset_sync` and `button_sync` subcomponents are Exercises 4.8 and 4.9 rather than files in the repository, so do those first, copy your own into `led_toggle_sync/`, and then run its testbench. If this chapter does not click on the first read, Erik Pihl's video walkthrough of metastability and the double-flop synchronizer (`https://www.youtube.com/watch?v=KrssJRgF13I`) and the metastability pages at nandland (`https://nandland.com/lesson-13-metastability/`) and VHDLwhiz (`https://vhdlwhiz.com/terminology/metastability/`) cover the same concept from a slightly different angle.

4.1 The Problem: A Button Isn't a Testbench Signal

In Chapter 3 you built edge detectors and reasoned about them against a clean clock, where every input changed neatly between edges, exactly the way a simulator draws it. A push button, an external reset line, or any signal from another clock domain changes whenever the outside world decides to, completely independent of your FPGA's clock.

Every D flip-flop has two timing requirements around the active clock edge:

- a **setup time**: how long `D` must be stable *before* the edge.
- a **hold time**: how long `D` must stay stable *after* it.

If an input changes inside that window the flip-flop is not guaranteed to capture a clean `0` or `1`, and you can never promise it will land outside, because you do not control when an external signal changes. The rest of this chapter solves that not by avoiding the problem but by containing it.

4.2 Metastability: What Actually Happens Inside the Flip-Flop

A D flip-flop is built from cross-coupled gates that snap to `0` or `1` the instant the clock samples the input. If `D` changes too close to the edge, the internal feedback loop can be left balanced between the two states, neither fully high nor fully low, for longer than expected. That is **metastability**.

Picture a ball balanced exactly on top of a hill. Given enough time it rolls down one side, but exactly when it commits, and to which side, is not predictable from the moment it was placed there, and while it is still balanced its position is not a valid digital value at all.

A metastable output behaves the same way:

- it almost always resolves to a valid `0` or `1` on its own, given enough time.
- "enough time" is not free, and is not the same on every clock cycle.
- if the unresolved voltage reaches downstream logic before it settles, different gates can legitimately read it differently: one sees `0`, another `1`, at the same instant.

That disagreement is where real systems misbehave: a counter that increments in one part of the circuit but not another, or a state machine in an impossible state. Everything in this chapter exists to make sure metastability resolves *before* a signal touches the rest of your design.

In time, the whole problem looks like this:

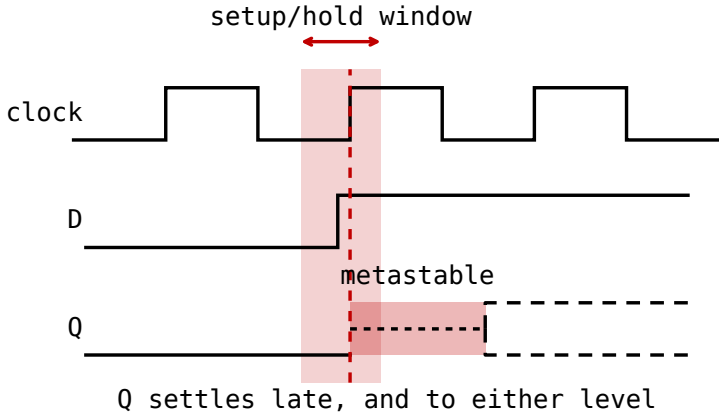


Figure 4.1 `D` changes inside the setup/hold window around a rising clock edge, leaving `Q` metastable before it settles, late, to either level.

Read it left to right. The shaded band is the window from §4.1, `t_su` before the edge and `t_h` after it. `D` changes inside it, which is exactly what an input you do not control is free to do. `Q` is then neither `0` nor `1` for a while, and when it does settle, *when* it settles and *which level* it settles to are both outside your control. Both dashed continuations are legitimate outcomes of the same input.

Note what the diagram does not show: a wrong answer. Sampling a signal mid-transition is allowed to give you either level, and that on its own is harmless, because the input genuinely was changing at that instant. The problem is the delay before you get an answer at all, and that is what the next section buys time for.

4.3 The Double-Flop Synchronizer

The standard fix is the **double-flop synchronizer**:

- Place two D flip-flops in series on every asynchronous input: every input that changes with no fixed relationship to your clock. A push button, a reset line, a signal arriving from another clock domain.
- An input that is already clocked by *this* clock is not asynchronous, and must **not** be synchronized. Doing so only delays it, and shifts its timing relative to everything around it. `seq_detect_mealy` in Chapter 8 is exactly that case: its `din` is synchronous serial data, so it goes straight into the machine while `reset_n` still gets a synchronizer.
- Only ever use the output of the *second* flip-flop elsewhere in the design.

```
async_in
   |
   ▼
 [FF1] ──────► [FF2] ──────► synchronized, safe to use
   (may go        (extremely likely
   metastable)     to be stable)
```

The same two flip-flops, in time:

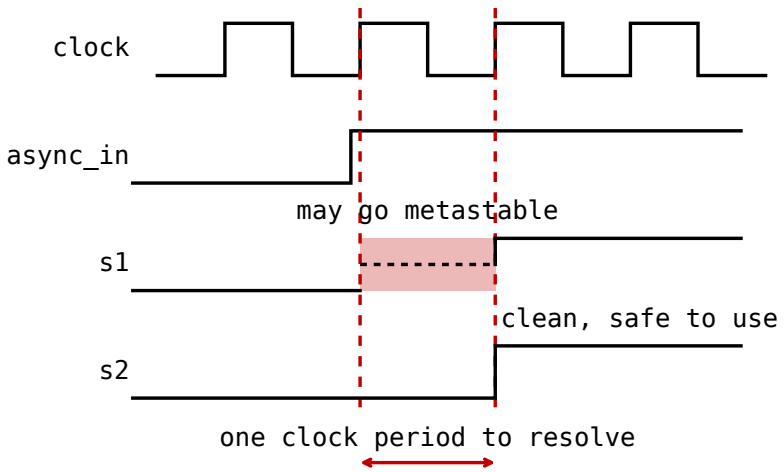


Figure 4.2 `async_in` changes just before a clock edge; `s1` may go metastable for a clock period, and `s2` comes out clean one edge later.

`FF1` is the one exposed to the asynchronous input, so it is the one that might go metastable. Its output only has to reach `FF2`, and it has an entire clock period to get there. The probability that a metastable node is still unresolved after a full period is small, and shrinks the longer it is given, so by the time `FF2` samples on the next edge, `FF1` has almost always settled. `FF2`'s output is what you build the rest of your synchronous logic on.

This is probabilistic, not absolute, and §4.4 says what that means. At the speeds used in this book, tens of MHz, two stages are the standard, industry-accepted answer.

One rule ties it together: **every asynchronous input gets its own double-flop chain before it is used anywhere else.** External reset buttons, push buttons, and later any **single-bit** signal crossing from another clock domain.

That last word is load-bearing, and it is the one limit worth learning before you over-apply the rule. A double-flop chain synchronizes **one bit**. Put a multi-bit bus through a chain per bit and each bit is individually safe but they are not safe *together*: each independently resolves to the old or new value on the same edge, so the receiver can sample a combination that was never on the bus. A counter stepping from `0111` to `1000` has every bit changing at once, and a per-bit synchronizer can hand you `1111` or `0000`, values the counter never held.

Buses crossing clock domains need a handshake, a FIFO, or Gray coding, which arranges that only one bit ever changes at a time. This book never crosses clock domains with a bus, so you will not need them here. Knowing that the two-flop answer stops at one bit is enough for now.

4.4 Timing Intuition: Why "Almost Certainly" Is Good Enough

A synchronizer does not buy "no metastability". It buys metastability that gets a full clock period to resolve before it can affect anything else, and that is a statistical argument rather than a proof. The correct posture is "reasonably certain, backed by an extremely low, well-characterized failure rate", not "provably safe under all conditions", which is why extra stages exist for extreme cases instead of two being declared sufficient by law of physics.

The relationship has a standard shape, worth seeing once even though you will not compute with it here:

```
MTBF = e^(t_r / tau) / (T_w * f_clock * f_data)
```

Read it for what it says rather than for numbers. `t_r` is the resolution time a stage gets, roughly one clock period; `tau` and `T_w` are both properties of the flip-flop, `tau` being how fast a metastable value decays and `T_w` the width of the window around the clock edge in which the input can provoke one at all. Adding a stage buys another `t_r` in the exponent, so **the numerator grows exponentially with every stage you add**, against one clock cycle of added latency. The denominator grows linearly with the clock rate and, just as importantly, with `f_data`: how often the asynchronous input actually changes. A button pressed once a second and a data line toggling at 10 MHz differ by seven orders of magnitude in that term alone, which is why the case for a third stage is a question about the input, not about the clock.

4.5 Synchronizing a Reset Signal: Assert Async, Release Sync

Resets deserve a specific pattern, which `reset_sync` demonstrates. There is no `reset_sync.vhd` in the repository to open: it is Exercise 4.8, and you write it. The listing below is the whole of its logic; the `library`/`use` clauses, the `entity`, the `architecture` frame and the declaration of the internal `reset_s1_n` are yours to add around it.

```
process(clock, reset_n) is
begin
    if (reset_n = '0') then
        reset_s1_n <= '0';
        reset_s2_n <= '0';
    elsif (rising_edge(clock)) then
        reset_s1_n <= '1';
        reset_s2_n <= reset_s1_n;
    end if;
end process;
```

- **Assert asynchronously.** When `reset_n` drops, both stages are forced to `0` immediately, with no dependency on the clock. A reset that waited for a clock edge would defeat its own purpose: it must take effect the instant it is needed, clock or no clock.
- **Release synchronously.** When `reset_n` returns to `1` the reset condition does not vanish everywhere at once. It shifts out through the two stages like any other asynchronous input, so `reset_s2_n`, the signal the design actually uses, releases cleanly on a clock edge together with everything else. Without this, different flip-flops could leave reset on different cycles and momentarily disagree about system state, the same class of bug metastability causes.

This is why `reset_n` is synchronized by its own chain before anything else depends on it, including the button synchronizer, whose reset input is `reset_s2_n` rather than the raw `reset_n`.

4.6 Reusing the Chain for Edge Detection — and, Incidentally, Debouncing

The button needs synchronizing, and from Chapter 3 you also need to detect *when* it is pressed rather than its level. `button_sync`, which is Exercise 4.9 and so is also yours to write rather than a file in the repository, gets both from a single chain of **three** flip-flops:

```
button_n
   │
   ▼
[FF1] → [FF2] → [FF3]
        │        │
        │        └─ previous value
        └─ current value
```

Flip-flops 1 and 2 are §4.3's synchronizer, making `button_s2_n` the stable current value. Flip-flop 3 stores last cycle's `button_s2_n` as `button_s3_n`, the previous value. Flip-flops 1 and 2 are worth noticing on their own: Chapter 5's Exercise 5.4 extracts that double-flop chain into a reusable `sync` module, leaving the third flip-flop here as the edge detector's own. Comparing stage 2 against stage 3 gives edge detection for free:

```
button_edge_s2 = (not button_s2_n) and button_s3_n
```

which is true for exactly one clock cycle: the one where the button reads pressed now and did not the cycle before.

Synchronization, on its own, only solves metastability. It says nothing about a mechanical button's tendency to bounce: to make and break contact several times over the first millisecond or so, each bounce looking like a separate press. What keeps this book's exercises from being swamped by bounce is the *combination* of the chain with a detection window that is slow relative to the bounce:

- In this chapter's CircuitVerse exercises the clock period is a full second. Bounce settles in far less than that, so by the next edge `button_s2_n` reads one clean level. The chain looks like it is debouncing, but it is really just sampling too infrequently to see bounce.
- On the FPGA the clock is `50 MHz`, one sample every `20 ns`, while bounce lasts from under a millisecond to a few tens of milliseconds, many thousands of cycles. At that speed `button_sync` faithfully detects *every* bounce transition as its own edge, and the LED could toggle several times for one physical press.

So precisely: the double-flop synchronizer reliably solves metastability at the speeds used here. Whether it also *looks* like debouncing depends entirely on the clock period being slow relative to the switch's bounce time, which is true in CircuitVerse and not automatically true on the FPGA.

4.7 What Real Hardware Debouncing Adds

This book has no dedicated debounce chapter, and the worked examples deliberately keep using §4.6's circuit, bounce and all. It is good enough for a demo LED. A production design adds one of the following *on top*:

- **An analog RC low-pass filter**, optionally with a Schmitt-trigger buffer, on the button input before it reaches an FPGA pin. The RC pair smooths the fast make/break transitions

into a single gradual edge, and the Schmitt trigger, with its two switching thresholds, turns that back into one clean digital transition. This solves bounce before the signal is even digital, but costs board components and does not help for inputs that are not mechanical switches.

- **A digital debounce counter.** Rather than acting on the next detected edge, require the input to hold a new level for some minimum duration, a few milliseconds' worth of clock cycles, before accepting it. That builds your own artificially slow detection window on-chip, the same effect the 1-second CircuitVerse clock gives for free, without slowing the system clock. You will meet counters in Chapter 6 and timers, which this technique needs, in Chapter 7.

Both sit on top of the synchronizer, not instead of it: even a perfectly debounced signal is still asynchronous to your clock and still needs a double-flop chain.

4.8 Generics: One Module, Several Sizes

`reset_sync` and `button_sync` are built from the same idea, but only one has a question to answer. A reset is one bit and always will be. How many *buttons* does a design have? This chapter's `led_toggle_sync` has one, Chapter 8's `fsm_led` has two. Writing the same three-flop chain out once per width is exactly the duplication that splitting them into modules set out to avoid.

A **generic** answers it: a value an entity declares alongside its ports, fixed when the design is built rather than changing while it runs.

```
entity button_sync is
    generic(COUNT: natural range 1 to 3 := 1);
    port(clock, reset_s2_n: in  std_logic;
         button_n         : in  std_logic_vector(COUNT-1 downto 0);
         button_edge_s2   : out std_logic_vector(COUNT-1 downto 0));
end entity;
```

- `COUNT` behaves like a constant inside the architecture. It cannot change while the circuit runs; it is decided before a single gate is placed.
- `:= 1` is its default, used by any instantiation that says nothing.
- The port widths are written *in terms of it*, and so is every internal signal that follows.

4.8.1 Overriding it

An instantiation passes generics with a `generic map`, before the `port map`:

```
button_sync1: entity work.button_sync
    generic map(2)
```

```
    port map(clock, reset_s2_n, button_n, button_edge_s2);
```

Generics are matched **by position**, in declaration order, exactly as ports are in §2.6. Leaving the `generic map` out applies the default; this book writes it explicitly even when it matches, because a reader should not have to open the module to find out how wide an instance is.

4.8.2 When a generic is worth having

`button_sync` has one and `reset_sync` has none, and that difference is the whole rule: **add a generic when something genuinely varies between instances, not on principle.**

`button_sync` is instantiated at `COUNT = 1` by §4.9's `led_toggle_sync` and at `COUNT = 2` by Chapter 8's `fsm_led`: two widths, one file. `reset_sync` is one bit everywhere, so a `WIDTH` generic on it would be a parameter nobody ever passes anything but `1` to, and one more thing for a reader to check. That second instantiation is what makes a generic worth having; a parameter with exactly one value in the entire book would just be indirection. `button_sync`'s own testbench instantiates it at *both* widths in one simulation, precisely to prove that one file really does serve both.

You will meet one more generic in Chapter 7: `timer`'s `TICK_COUNT`, which is a count rather than a width, and is what lets the same file measure a fiftieth of a second in one design and a whole second in another.

4.9 Worked Example: `led_toggle_sync`

This ties §4.1 to §4.6 together into Chapter 3's toggle circuit, made safe for a real, asynchronous button and reset.

Port	Direction	Description
`clock`	in	`50 MHz` system clock on the DE0-CV board.
`reset_n`	in	Active-low asynchronous reset, wired to a push button.
`button_n`	in	Active-low push button; toggles the LED on its falling edge.
`led`	out	LED, toggled on each synchronized, edge-detected button press.

Internal signals: `led_s` holds the LED state, `reset_s2_n` is the reset after the two-flop synchronizer (the `s2` suffix is this chapter's convention for "synchronized with two flip-flops"), and `button_edge_s2` is the one-cycle pulse marking a synchronized falling edge on `button_n`.

Like Chapter 2's two-digit display, this design is built from more than one entity, using the instantiation from §2.6 and the generic from §4.8. The difference is what the pieces do: the display instantiated one module twice to do the same job on two halves of its input, while this instantiates two *different* modules with one job each. Both are the same mechanism. Composing a design out of modules is not a special technique for repeated logic; it is how designs are built once they outgrow a single file.

The two chains live in two subcomponents:

- `reset_sync` is §4.5's two-flop reset synchronizer. It takes the raw `reset_n` and returns `reset_s2_n`.
- `button_sync` is §4.6's three-flop synchronizer-plus-edge-detector. It takes the already synchronized `reset_s2_n` and pulses `button_edge_s2` once per press.

Keeping them apart matters more than it looks. A design with no button of its own still needs its reset synchronized, and with two modules it simply instantiates the one it wants, as Chapter 8's `seq_detect_mealy` does. Welded into one module, that design would have to instantiate the whole thing and disable the half it does not use.

`button_sync` takes a *vector* of buttons so one file can serve a two-button design. This design has one, so `led_toggle_sync.vhd` declares a pair of one-element vectors to connect to it:

```
signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);
...
button_n_v(0)  <= button_n;
button_edge_s2 <= button_edge_s2_v(0);
```

That is plumbing, not logic: two wires renamed.

The top-level architecture contains nothing but the toggle process itself, and it only ever touches the already-synchronized `reset_s2_n` and `button_edge_s2`, never the raw inputs:

```
LED_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        led_s <= '0';
    elsif (rising_edge(clock)) then
        if (button_edge_s2 = '1') then
            led_s <= not led_s;
        end if;
    end if;
end process;
```

The full source of the top level is `lectures/L04/led_toggle_sync/led_toggle_sync.vhd`:

lectures/L04/led_toggle_sync/led_toggle_sync.vhd

```
--------------------------------------------------------------------------------
-- Toggle an LED on each synchronized, edge-detected press of button_n.
-- The LED is cleared on reset.
--
-- Inputs:
--     - clock: 50 MHz system clock.
--     - reset_n: Active-low asynchronous reset.
--     - button_n: Active-low button.
```

```
-- Outputs:
--     - led: Toggled on the falling edge of button_n.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity led_toggle_sync is
    port(clock, reset_n, button_n: in std_logic;
         led                     : out std_logic);
end entity;

architecture behaviour of led_toggle_sync is
-- Stable reset signal synchronized using a two-flip-flop synchronizer.
signal reset_s2_n: std_logic;

-- Pulse indicating a falling edge on button_n, synchronized using a two-flip-flop synchronizer.
signal button_edge_s2: std_logic;

-- One-element vectors adapting the scalar button signals above to button_sync,
-- whose button ports are COUNT-bit vectors.
signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);

-- Internal LED state.
signal led_s: std_logic;

begin
    led <= led_s;

    -- Adapt the scalar button signals to button_sync's vector ports.
    button_n_v(0)  <= button_n;
    button_edge_s2 <= button_edge_s2_v(0);

    -- Synchronize the asynchronous reset.
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    -- Synchronize button_n and pulse on each press (a single button, so COUNT = 1).
    button_sync1: entity work.button_sync
        generic map(1)
        port map(clock, reset_s2_n, button_n_v, button_edge_s2_v);

    -- Toggle the LED on button press event.
```

```
    LED_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            led_s <= '0';
        elsif (rising_edge(clock)) then
            if (button_edge_s2 = '1') then
                led_s <= not led_s;
            end if;
        end if;
    end process;
end architecture;
```

The two subcomponents are not in the repository, because writing them is Exercises 4.8 and 4.9. What this chapter gives you of each is: `reset_sync`'s process in §4.5 above, and `button_sync`'s chain as a diagram and an equation in §4.6 with its entity in §4.8. The surrounding entity and architecture in the first case, and the whole architecture body in the second, are the exercises' own work. Copy your own into `led_toggle_sync/` to build it.

By hand, in CircuitVerse first. Before touching VHDL, build it as a gate network: two D flip-flops in series for the reset synchronizer; two for `button_n` plus a third storing the previous synchronized value; an AND gate combining the inverted current value with the previous one to produce `button_edge_s2`; and a toggle flip-flop for `led_s`, its inverted output fed back to its input, enabled only when `button_edge_s2 = 1`. Set the clock period to `1000 ms` so you can watch each stage by eye.

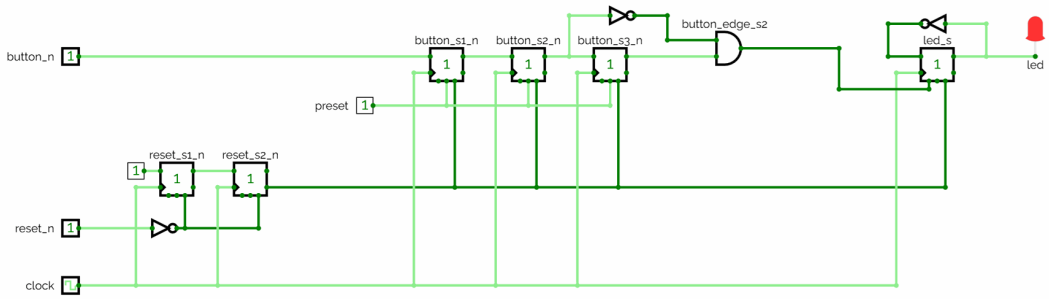


Figure 4.3 The synchronized LED toggle in CircuitVerse: the reset and button double-flop synchronizers, the edge-detecting AND gate, and a toggle flip-flop for `led_s`.

One thing in the drawing needs reading carefully: the flip-flops carry a `preset` input, and on the three button-synchronizer stages it is tied to `1`. That is not decoration. In CircuitVerse, `preset` is the value a flip-flop loads when its reset is asserted, so tying it to `1` is what makes `button_s1_n`, `button_s2_n` and `button_s3_n` come out of reset holding `1` rather than `0`.

That matches `button_sync`, which resets the same three stages to `(others => '1')`, and it matters for the same reason: the buttons are active-low, so `1` means "released". Reset the chain to `0` instead and it comes up claiming the button is already held down. The reset flip-flops leave `preset` alone, because there a reset to `0` really is all the circuit asks for.

Press the button in the simulation and watch it propagate. The LED does not toggle instantly: it waits for the synchronized, edge-detected pulse to reach the toggle flip-flop, which takes a small, fixed number of clock pulses, exactly the latency §4.3 called the price of safety.

On the FPGA: the course demonstrates the same design on the Terasic DE0-CV (device `5CEBA4F23C7N`), with `clock` on the `50 MHz` oscillator, `reset_n` and `button_n` on two push buttons, and `led` on an LED; Appendix B is the workflow, for a reader who has a board. The LED toggles on every press, and per §4.6 a real press may occasionally produce more than one toggle from contact bounce.

4.10 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what metastability is, why an asynchronous input can cause it, and why two flip-flops make it unlikely rather than impossible.
- Apply the double-flop synchronizer to any asynchronous input, including the "assert asynchronously, release synchronously" pattern used for resets.
- Say why that chain doubles as a practical but incomplete debounce, and what a real one would add.
- Give a module a `generic`, override it with a `generic map`, write a port's width in terms of it, and judge when a generic earns its place rather than adding one on principle.

Questions to test yourself

You should be able to explain:

1. What exactly happens when an asynchronous input to a flip-flop changes too close to an active clock edge?
2. How does the double-flop synchronizer address that, and why is the *second* flip-flop's output the one that is safe to use?
3. Why does the chain used here behave like a debounce circuit in CircuitVerse but not fully debounce a real button at `50 MHz`, and why is that a property of the clock period rather than of the circuit? What would you add to fix it?
4. `button_sync` has a generic and `reset_sync` has none. What is the rule, and what would be wrong with giving `reset_sync` a `WIDTH` generic "for symmetry"?

Looking ahead

Chapter 5 is about the one construct left, and what the toolchain builds:

- `variable`: the one construct you have not met yet, and the sharp distinction between how it updates and how a `signal` does.
- Why the same accumulation written with a `signal` and with a `variable` gives two different answers, and which of the two is the bug.
- What the toolchain actually builds: LUTs and flip-flops, what limits a clock's speed, and the latch you infer by accident.

4.11 Exercises

How to work these. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
How to check your work. Every exercise that asks you to write a VHDL module ships a self-checking testbench under `lectures/L04/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
No FPGA board is needed for any exercise. Your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L04/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

Metastability Concepts

Exercise 4.1 Metastability in your own words PAPER

Explain metastability in your own words. In your explanation, describe:

- What happens inside a D flip-flop when its input changes too close to the active clock edge.
- Why the flip-flop output may temporarily remain between a valid logic `0` and logic `1`.
- Why the output may take an unpredictable amount of time to settle.
- Why two downstream gates may interpret the unresolved output differently:
  - One gate may interpret it as `0`.
  - Another gate may interpret it as `1`.

Explain why this disagreement can cause different parts of a digital circuit to enter inconsistent states.

Exercise 4.2 The setup-and-hold window PAPER

Setup time and hold time define a window around the active clock edge during which the input of a flip-flop must remain stable.
Explain:

- Why a synchronously generated signal can normally be designed to satisfy these timing requirements.
- Why an asynchronous input, such as a mechanical pushbutton, has no fixed relationship to the system clock.
- Why an asynchronous input may therefore change during the setup-and-hold window regardless of how the surrounding synchronous logic is designed.

Conclude why the metastability risk from an asynchronous input must be managed rather than simply avoided.

EXERCISE SET 2

Synchronizer Design

Exercise 4.3 Drawing a double-flop synchronizer PAPER

Draw a double-flop synchronizer for one asynchronous input.
The circuit must contain:

- One asynchronous input.
- Two D flip-flops connected in series.
- A shared system clock.
- A synchronized output taken from the second flip-flop.

Explain in your own words:

- Why the first flip-flop is exposed directly to the asynchronous input.
- Why the first flip-flop may become metastable.
- How the interval between the first and second clock edges gives the first stage time to settle.
- Why the output of the second flip-flop has a much lower probability of being metastable.
- Why the second stage, rather than the first, should be used by the rest of the design.

A synchronizer does not make the probability of metastability exactly zero. Explain why it nevertheless reduces the risk to an acceptably low level in most designs.

Tip: Consider how much resolution time the first flip-flop receives before its value is sampled by the second flip-flop.

Exercise 4.4 A third synchronizer stage PAPER

A colleague suggests using three synchronizer flip-flops on every asynchronous input in a low-speed design clocked at a few megahertz, just to be safe.

A third stage buys one more clock period for a metastable value to settle, and costs one more clock cycle of latency on every synchronized input. At a few megahertz that extra clock period is an enormous amount of time in metastability terms, which is exactly why the proposal sounds prudent, and also why it may already be unnecessary.

a) Work the trade out in both directions. Under what circumstances does it come out in favour of the third stage, and under what circumstances is the second stage already enough? Your answer has to turn on something other than the clock frequency, because frequency alone does not settle it: name what it does turn on.

b) §4.4 got to MTBF from the resolution time available per stage. Without redoing that arithmetic, say which way each of these pushes the decision, and why:

- How often the asynchronous input actually changes.
- The metastability characteristics of the FPGA family you are targeting.
- What a synchronization failure would cost if one happened.

c) Give your recommendation for an ordinary low-speed design like the ones in this book. Then describe a design, real or imagined, where you would argue for the third stage instead, and say which of the factors in **b)** is doing the work in that case.

Check your answer covers the two things a third stage provides, an additional clock period of resolution time and a lower probability that an unresolved value reaches the functional logic, and the two it costs, an additional cycle of latency and additional flip-flops. If your answer to **a)** rests on the clock rate rather than on the required mean time between failures, read §4.4 again before moving on.

Exercise 4.5 Assert async, release sync PAPER

Explain the **assert asynchronously, release synchronously** reset pattern used in `reset_sync.vhd` (§4.5).
Describe the two parts separately:

- Asynchronous assertion:
  - The reset takes effect immediately.
  - It does not wait for a clock edge.
- Synchronous release:
  - The reset is removed only on an active clock edge.
  - All affected logic leaves reset in a controlled relationship with the clock.

Explain:

- Why immediate reset assertion can be important when:

  - The clock is stopped.
  - The clock is unstable.
  - The system must enter its reset state without delay.
- Why releasing an asynchronous reset close to a clock edge can violate the recovery or removal timing requirements of a flip-flop.
- Why synchronizing the release prevents different flip-flops from leaving reset on different clock cycles.

EXERCISE SET 3

Building the Circuit

Exercise 4.6 `led_toggle_sync` by hand

CIRCUITVERSE PAPER

Build the `led_toggle_sync` circuit from §4.9 by hand in CircuitVerse.

a) Add the external ports:

- Inputs:
  - `clock`
  - `reset_n`: asynchronous, active-low.
  - `button_n`: active-low.
- Output:
  - `led`

b) Build the two-flip-flop reset synchronizer:

- Connect the two flip-flops in series.
- Assert both stages asynchronously using the raw `reset_n` input.
- Allow the reset to propagate out of the chain synchronously.
- Use the synchronized reset output, `reset_s2_n`, throughout the rest of the circuit.
- Do not use the raw `reset_n` input directly outside the reset synchronizer.

c) Build the three-flip-flop button synchronizer and edge detector:

- Use the first two flip-flops as the double-flop synchronizer.
- Use the third flip-flop to store the previous synchronized button value.
- Compare the current and previous synchronized values.
- Detect a button press when:
  - The current synchronized active-low button value is `0`.
  - The previous synchronized value is `1`.
- Use an AND gate with the current value inverted to generate `button_edge_s2`.

The falling-edge relationship for the active-low button is:

$$button_edge_s2 = previous \cdot current'$$

d) Build the LED toggle logic:

- Add a flip-flop to store `led_s`.
- Reset `led_s` using `reset_s2_n`.
- Toggle `led_s` only when `button_edge_s2 = 1`.
- Connect the `led` output to `led_s`.

e) Simulate the complete design:

- Set the clock period to `1000 ms`.
- Assert and release `reset_n`.
- Press and release `button_n`.
- Observe the synchronized button stages.
- Observe `button_edge_s2`.
- Observe `led_s`.

Determine:

- Whether the LED toggles on the first clock edge after the physical button press.
- How many clock cycles are required for the button value to propagate through the synchronizer and edge detector.
- Whether the observed latency matches the behaviour predicted in §4.9.

EXERCISE SET 4

Debouncing and Timing

Exercise 4.7 Synchronized is not debounced

PAPER

You have now built and watched the circuit, so this is a question about what you saw.

The three-flip-flop chain in Exercise 4.6 does three jobs at once:

- Reduces the risk of metastability.
- Stores the previous synchronized button value.
- Detects a button press.

Explain why it does **not**, by itself, guarantee that the button is debounced.

a) Distinguish the three things it is easy to run together:

- Synchronization: converts an asynchronous input into a signal suitable for use in the clock domain.
- Edge detection: generates a pulse when the sampled signal changes.

- Debouncing: prevents several physical transitions from being interpreted as several separate presses.

Which two of these does your circuit actually do?

b) In your simulation, one press produced one clean toggle. That is a stronger claim than it looks: it holds only under a particular condition about where the bouncing fell relative to the clock edges. State that condition precisely, in terms of what your circuit sampled and what it therefore saw.
Then explain why that condition is an accident of your clock period rather than a property of your circuit. Be specific about which of the two, the bounce or the sampling, your design actually controls.

c) Your simulation ran at a `1000 ms` clock period; the FPGA runs at `50 MHz`. A push button's contacts typically bounce for something on the order of 1 ms. Work out the consequences yourself:

- How many times does a `1000 ms` clock sample the button during that 1 ms of bouncing?
- How many times does a `50 MHz` clock sample it?
- Each sampled transition looks like a fresh edge to your edge detector. Given those two numbers, predict what the LED does on the board for one physical press, and say why the identical circuit behaved impeccably in CircuitVerse.

d) Describe one fix of each kind:

- A hardware fix: for example, an RC low-pass filter followed by a Schmitt-trigger input.
- A purely digital fix:
  - for example, requiring the synchronized signal to hold a new level for some minimum number of clock cycles before accepting it.
  - or a counter that ignores further transitions for a fixed interval after a press, which is what Chapter 7's timer would give you.

e) Explain why the double-flop synchronizer must still be kept even once a debounce method is added. What problem does debouncing not solve?

EXERCISE SET 5

The Synchronizers as Modules of Their Own

The worked example puts each synchronizer in a module of its own, so every later design instantiates the same two files instead of re-deriving them. That is why Chapter 7's `walking_led` and Chapter 8's `fsm_led` both ask you to copy these two in, and why Chapter 8's `seq_detect_mealy`, which has no button, takes the reset synchronizer alone and leaves the other behind (§4.9).

The next two exercises are you writing those modules. §4.5 and §4.6 give you the logic; what they do not give you is the entity around it, and, for the second one, the step from a single button to a vector of them. Write each module yourself before opening the worked example's copy, then compare. Exercise 4.10 then composes both of them into a working design.

Exercise 4.8 The reset synchronizer

VHDL PAPER

Write the reset synchronizer as a module named `reset_sync`.
The entity has:

Port	Direction	Type	Description
`clock`	in	`std_logic`	50 MHz system clock.
`reset_n`	in	`std_logic`	Active-low asynchronous reset, straight from the outside world.
`reset_s2_n`	out	`std_logic`	Active-low synchronized reset, safe for the rest of the design to use. Goes low the instant `reset_n` does, with no clock edge involved, and returns high two rising edges after `reset_n` does.

a) Implement **assert asynchronously, release synchronously** (§4.5):

- One internal signal for the first stage, plus the output for the second.
- One process, sensitive to both `clock` and `reset_n`.
- On `reset_n = '0'`, drive both stages low, outside the clocked branch.
- On a rising clock edge, shift a constant `'1'` in through the two stages.

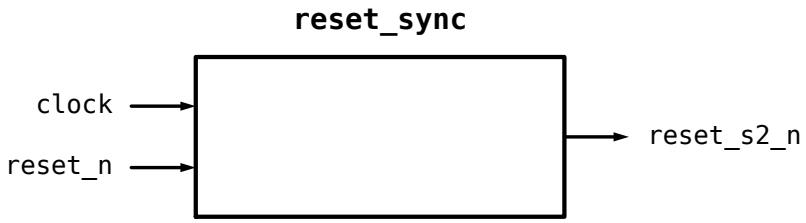
b) The value forced on reset and the value shifted in on the clock are opposites, `'0'` and `'1'`. Explain in one sentence why that is not a contradiction, in terms of what each branch is for.

c) Verify the design with its testbench (see the aside at the start of this section). It checks both halves of the pattern separately: that pulling `reset_n` low takes `reset_s2_n` low with no clock edge in between, and that releasing `reset_n` leaves `reset_s2_n` low until two rising edges have passed.

d) Answer, in prose:

- The output is named `reset_s2_n` rather than `reset_n_sync`. What does `s2` say, and what would have to change about this module for that name to become wrong?
- Every other module in this book takes `reset_s2_n` as its reset. This one takes the raw `reset_n`. Why must exactly one module in a design be the exception?

Tip: the release is the interesting half. On reset the process does not care about the clock at all, so both stages go low together; on the clock it does not care about `reset_n`, so the `'1'` takes one edge per stage to reach the output. Two edges of latency on release is the price of the whole pattern.



Self-check: name your entity `reset_sync`, with ports `clock`, `reset_n` (in) and `reset_s2_n` (out), declared in that order; its testbench is in `exercises/reset_sync`.

Exercise 4.9 The button synchronizer

VHDL PAPER

Write the button synchronizer and edge detector as a module named `button_sync`.

Exercise 4.10 handles two buttons by writing a two-bit synchronizer. This one is the same circuit with the width left open: one generic decides how many buttons it serves, so `led_toggle_sync` can use it for one button and `fsm_led` for two without either file being edited. See §4.8 for the syntax and for why this module has a generic where `reset_sync` has none.

The entity has one generic:

Generic	Type	Default	Description
`COUNT`	`natural range 1 to 3`	`1`	The number of buttons, and therefore the width of both vector ports.

and these ports:

Port	Direction	Type	Description
`clock`	in	`std_logic`	50 MHz system clock.
`reset_s2_n`	in	`std_logic`	Active-low, **already synchronized** reset. This module does not synchronize its own reset; Exercise 4.8's module did that.
`button_n`	in	`std_logic_vector(COUNT - 1 downto 0)`	Active-low asynchronous push buttons.
`button_edge_s2`	out	`std_logic_vector(COUNT - 1 downto 0)`	A one-cycle pulse on each button's falling edge, two clock edges after the press. Each bit is independent of every other.

a) Build the three-stage chain from §4.6, now over vectors rather than single bits:

- Three internal `std_logic_vector(COUNT - 1 downto 0)` signals, one per stage.
- Stages 1 and 2 are the double-flop synchronizer; stage 3 stores the previous value of stage 2.
- On reset, set all three stages to `(others => '1')`, not `(others => '0')`.
- Derive the output with a single concurrent assignment, and note that the same expression works unchanged on vectors as on bits.

Keep this one in mind. Those first two flip-flops are the whole of a synchronizer and nothing else, and Chapter 5's Exercise 5.4 has you pull them back out as a module of their own, generic in width and in the value they reset to. Writing them inline here first is what makes that exercise a consolidation rather than a new idea.

b) Explain why the reset value is all-ones. All-ones means "released", because the buttons are active-low, so an all-zeros reset starts the chain out claiming every button is already held down. Trace what that costs:

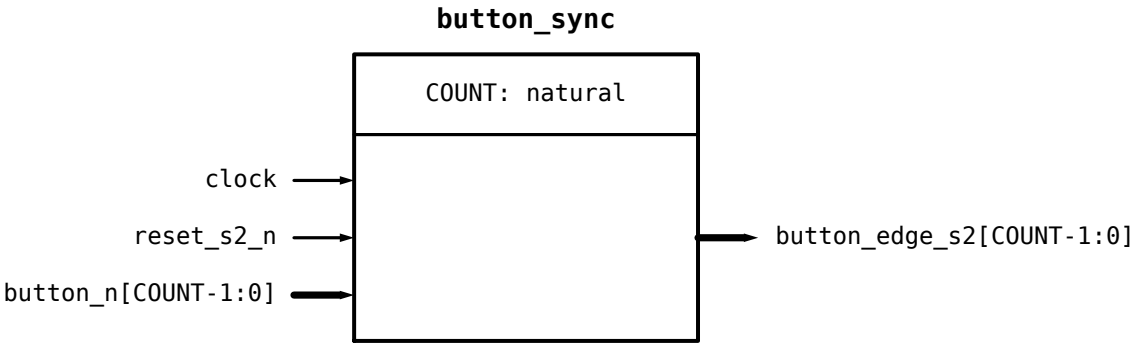
- With the buttons released and a reset to all-zeros, does the module emit a spurious pulse as the released state propagates through the three stages? Work it through the output expression edge by edge before you answer, rather than guessing.
- Now the case that actually breaks. A user is holding a button down at the moment the reset is released. What does the module report with an all-ones reset, and what does it report with all-zeros? Which of the two is the behaviour you want, and why?

c) Verify the design with its testbench. It instantiates your module **twice**, once at `COUNT = 1` and once at `COUNT = 2`, and checks that a press produces exactly one single-cycle pulse, that holding the button produces no further pulses, that a release produces none at all, that pressing one button of a pair never pulses the other, and — the case part **b**) asks you to reason about — that a button already held down when the reset releases produces exactly one pulse with an all-ones reset and none at all with all-zeros.

d) Answer, in prose:

- `COUNT` is constrained to `1 to 3` rather than left an unbounded `natural`. What does the constraint buy, given that nothing in the architecture depends on the upper bound?
- This module is deliberately not merged with Exercise 4.8's, even though a design that has a button always needs both. Give the design that justifies the split, and say what it would have had to do instead.

Tip: write the architecture for `COUNT = 1` in your head first, then check that every line you wrote is already correct for a vector. `and`, `not` and the aggregate `(others => '1')` all work element-wise, so the generalization should cost you no extra code at all. If it does, that is worth a second look.



Self-check: name your entity `button_sync`, with generic `COUNT` (`natural range 1 to 3`, default `1`), inputs `clock`, `reset_s2_n`, `button_n` (`std_logic_vector(COUNT - 1 downto 0)`) and output `button_edge_s2` (`std_logic_vector(COUNT - 1 downto 0)`), declared in that order; its testbench is in `exercises/button_sync`.

EXERCISE SET 6

Composing Them Into a Design

Exercise 4.10 Two synchronized LED toggles

VHDL PAPER

Now make Chapter 3's `led_toggle` metastability-safe, in VHDL, as a new module named `led_toggle_sync2`.

Chapter 3's `led_toggle` toggled two LEDs from two push buttons, but fed their raw, asynchronous signals straight into the edge detector: logically correct, but unsafe on real hardware (Chapter 3's own "Looking ahead" flagged exactly this). Here you rebuild it with every asynchronous input guarded by this chapter's double-flop synchronizer.
The entity has:

Port	Direction	Type	Description
`clock`	in	`std_logic`	50 MHz system clock.
`reset_n`	in	`std_logic`	Active-low asynchronous reset.
`button_n`	in	`std_logic_vector(1 downto 0)`	Two active-low asynchronous push buttons.
`led`	out	`std_logic_vector(1 downto 0)`	Two LEDs; `led(i)` toggles once per press of `button_n(i)`.

a) Synchronize every asynchronous input:

- Apply the two-flip-flop reset synchronizer from §4.5 (assert asynchronously, release synchronously).
- Give the buttons a double-flop synchronizer plus a third stage for edge detection, as in §4.6, now over a `std_logic_vector(1 downto 0)`, so both buttons are synchronized in parallel.
- Never let the raw `reset_n` or `button_n` reach the toggle logic; use only the synchronized reset and the per-button edge pulses.

Tip: you can inline the synchronizer, or split it out into a subcomponent taking a two-bit `button_n`, using the instantiation syntax from §2.6. Inlining is the shorter route, and then the build is a single file:

```
ghdl -a --std=93 led_toggle_sync2.vhd led_toggle_sync2_tb.vhd
```

If you split it out instead, copy in the `reset_sync.vhd` and `button_sync.vhd` you wrote for Exercises 4.8 and 4.9 (instantiating `button_sync` at `COUNT = 2`), and name them first on the `ghdl -a` line, ahead of your own module, so GHDL sees them before the file that instantiates them:

```
ghdl -a --std=93 reset_sync.vhd button_sync.vhd \
                 led_toggle_sync2.vhd led_toggle_sync2_tb.vhd
```

See §2.8.5. Neither module is shipped anywhere in the repository, because writing them is Exercises 4.8 and 4.9. If yours are not working yet, go back to §4.5, which prints `reset_sync`'s process in full, and §4.6, which gives `button_sync`'s three-flop chain and its edge equation.

b) Build the toggle logic:

- One LED-state flip-flop per LED.
- On the synchronized reset, clear both LEDs.
- Toggle `led(i)` only when button `i`'s synchronized falling-edge pulse fires.
- Confirm `led(i)` responds to `button_n(i)` alone, never to the other button.

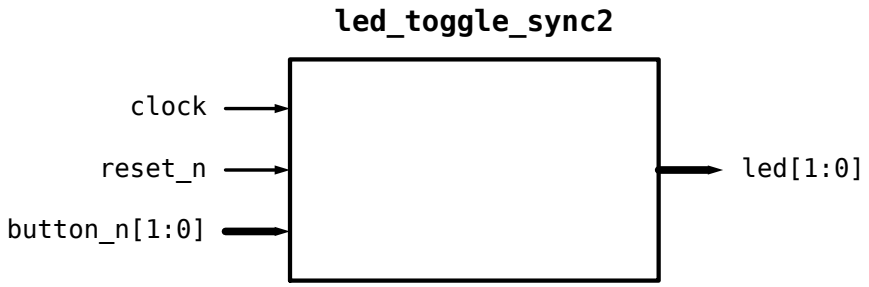
c) Verify the design with its testbench (see the aside at the start of this section). The testbench presses each button in turn and checks that:

- Reset clears both LEDs.
- No press means no toggle.
- Pressing button 0 toggles `led(0)` **only**, once the synchronizer latency has elapsed.
- Holding that button down does not toggle it again.
- Pressing button 1 then toggles `led(1)` only, leaving `led(0)` where it was.

Pay attention to the **latency**: the testbench checks the *exact* edge, not merely that the LED settles eventually. A press must reach the toggle logic through two synchronizer flip-flops, so `led` is still unchanged after the first and second rising edges and toggles on the third. Count the flip-flops a press travels through and satisfy yourself that the third edge is the right one, and that a defensive extra pipeline stage would now be caught rather than tolerated.

d) Answer, in prose, by comparing this design with your Chapter 3 `led_toggle`:

- What does the synchronizer change about pressing a real, bouncing button?
- Why was the raw Chapter 3 version unsafe to wire to a physical button at `50 MHz`?
- Your testbench drives clean, perfectly-timed transitions. Which of the two problems from this chapter, metastability or contact bounce, can a testbench therefore never demonstrate to you?



Self-check: name your entity `led_toggle_sync2`, with inputs `clock`, `reset_n`, `button_n` (`std_logic_vector(1 downto 0)`) and output `led` (`std_logic_vector(1 downto 0)`), declared in that order; its testbench is in `exercises/led_toggle_sync2`. It presses each button in turn and checks that `led(i)` toggles on `button_n(i)`'s falling edge, that it does so on the right clock edge rather than merely settling to the right value eventually, and that it never fires the wrong LED; see §2.8.

CHAPTER 5

Variables and the Hardware Underneath

The one construct left, and what the toolchain does with all of it.

In this chapter

- `variable`: process-local storage, updated immediately where a `signal` schedules.
- Why the same accumulation gives two different answers depending on which you reach for.
- What an FPGA is physically made of: look-up tables and flip-flops, and nothing that is a gate.
- The critical path and Fmax, and the latch you infer by accident.
- Chapter 4's double-flop synchronizer extracted as a generic module (Exercise 5.4), and what a generic costs in the hardware Quartus builds.
- `parity_gen` on the DE0-CV: the LED toggles whenever exactly one switch changes.

The course builds this chapter live, in this order. First `parity_gen` with a `signal` accumulator, written the way a programmer would write it, then traced by hand, one loop iteration at a time, to the wrong answer. Then `parity_gen` with a `variable`: the same loop, traced the same way, to the right one. Then what Quartus made of it: what it actually built, how fast it says the design will run, and an inferred-latch warning produced on purpose. And finally onto the board, `parity_gen` on the DE0-CV: eight switches in, one LED out, toggling every time exactly one switch changes.

The first step is the point of the chapter. The code looks correct, simulates without complaint, and is wrong, and the only way to see why is to trace it rather than read it.

Nothing new is assumed beyond Chapters 1 to 4: every construct except `variable` has already appeared in a design you have written. The exercises that close the chapter include two written exercises about what the toolchain does to your VHDL, and a code review (Exercise 5.7) of a design that compiles cleanly and is wrong anyway. Appendix B is the toolchain reference behind the chapter's demonstrations; no exercise requires you to have Quartus or a board.

5.1 What This Chapter Is About

Every construct you need in order to describe hardware, you have already used: `entity` and `architecture` in Chapter 1, `std_logic_vector`, `process` and `case` in Chapter 2, the clocked process and the scheduling rule in Chapter 3, subcomponents and generics in Chapters 2 and 4.

Two things are left, and they are the two hardest to work out on your own:

- The first is `variable`, the one construct the book has not needed until now:
  - It looks like a signal and behaves nothing like one, and the difference is exactly the kind that produces a design which simulates plausibly and is quietly wrong.
  - §5.2 works through the case where it bites.
- The second is what any of this actually becomes:
  - Since Chapter 1 you have taken it on trust that the synthesis tool builds the circuit you described. It does, but not out of anything you would recognize.
  - §5.3 opens the box. An FPGA turns out to contain no gates at all, which is why Chapter 2's Karnaugh maps bought you less than they seemed to. Your clock has a speed limit, and something specific sets it. And a `case` with a branch missing will quietly hand you a memory element nobody asked for.

Neither section answers "is my logic right?" You have been answering that since Chapter 1. They answer the two questions after it: is this what I meant, and will it fit and run.

5.2 `variable`: Process-Local Storage with an Immediate Update

A `variable` is declared inside a `process`, between `process(...)` and `begin`. The critical difference from a `signal`: variable assignment uses `:=` and takes effect **immediately**, so the very next statement that reads it sees the updated value, exactly like an assignment in C. There is no scheduling, none of the wait-until-the-process-suspends delay that §3.6 spelled out for signals, and no "last assignment wins".

	`signal` (`<=`)	**`variable` (`:=`)**
Declared	Architecture's declarative part	Process's declarative part
Visible from	Anywhere in the architecture	Only within its own process
Update timing	Scheduled: applied when the process next suspends	Immediate: applied before the next statement
Repeated assignment in one pass	Only the last one survives; earlier ones are discarded	Every one takes effect, in order

5.2.1 Why this actually matters: an XOR-parity generator

An entity computing the XOR-parity of an 8-bit input, `'1'` if `bits` has an odd number of set bits:

```
entity parity_gen is
    port(bits  : in  std_logic_vector(7 downto 0);
         parity: out std_logic);
end entity;
```

Coming from software, the first instinct is to accumulate the running XOR in a loop, using a signal as the accumulator:

```
-- Don't do this: shown to demonstrate why it doesn't work.
architecture broken of parity_gen is
signal parity_s: std_logic;
begin
    parity <= parity_s;

    process(bits) is
    begin
        parity_s <= '0';
        for i in bits'range loop
            parity_s <= parity_s xor bits(i);
        end loop;
    end process;
end architecture;
```

Trace one pass, using the scheduling rule from §3.6: every read of `parity_s` returns whatever it held *before this pass started* (call it `old_parity`), and only the last assignment survives.

- `parity_s <= '0';` schedules `'0'`. Not yet applied.
- `i = 7` (`bits'range` on `(7 downto 0)` runs high-to-low): reads `parity_s`, gets `old_parity`, because the `'0'` has not applied. Schedules `old_parity xor bits(7)`, discarding the `'0'`.
- `i = 6`: reads `parity_s`, again `old_parity`. Schedules `old_parity xor bits(6)`, discarding the previous one.
- ...and so on for every iteration ...
- `i = 0`, the last statement in the pass: still reads `old_parity`, schedules `old_parity xor bits(0)`, discarding everything before it.
- The process suspends and `parity_s` becomes `old_parity xor bits(0)`, the only one of nine scheduled assignments that survived.

The result depends only on `bits(0)` and on whatever `parity_s` happened to already hold. Seven iterations and the initial `'0'` were computed and thrown away. This is not a corner

case: it is what a signal-as-accumulator *always* does, because every iteration reads the same frozen value and only the textually last assignment is kept.

There is a second thing wrong with it, which the trace does not show. The surviving assignment is `parity_s <= parity_s xor bits(0);`, and `parity_s` is not in the sensitivity list: as hardware that is a **combinational feedback loop**, an output fed back to its own input through a gate with no flip-flop in the path. GHDL analyzes and simulates it happily, which is how the bug reaches a simulation at all; Quartus reports the loop rather than building it. The lesson is the same one either way, and the fix below removes both problems at once.

The same computation with a `variable`:

```
architecture behaviour of parity_gen is
signal parity_s: std_logic;
begin
    parity <= parity_s;

    PARITY_PROCESS: process(bits) is
        variable acc: std_logic;
    begin
        acc := '0';
        for i in bits'range loop
            acc := acc xor bits(i);
        end loop;
        parity_s <= acc;
    end process;
end architecture;
```

Now `acc := '0'` applies immediately; `i = 7` reads `'0'`, XORs in `bits(7)` and updates `acc` at once; `i = 6` reads that just-updated value, and so on through every iteration. The loop ends with `acc` holding the true XOR of all eight bits, and `parity_s <= acc;` schedules that one correct value, the only assignment to `parity_s` in the pass.

This is the on-disk worked example, `lectures/L05/parity_gen/parity_gen.vhd`, in full:

lectures/L05/parity_gen/parity_gen.vhd

```
--------------------------------------------------------------------------------
-- 8-bit XOR-parity generator.
--
-- Inputs:
--     - bits: The eight bits to compute parity over.
-- Outputs:
--     - parity: '1' if 'bits' holds an odd number of set bits, '0' otherwise.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
```

```
entity parity_gen is
    port(bits  : in std_logic_vector(7 downto 0);
         parity: out std_logic);
end entity;

architecture behaviour of parity_gen is
-- Internal representation of 'parity'.
signal parity_s: std_logic;
begin
    parity <= parity_s;

    PARITY_PROCESS: process(bits) is
        variable acc: std_logic;
    begin
        acc := '0';
        for i in bits'range loop
            acc := acc xor bits(i);
        end loop;
        parity_s <= acc;
    end process;
end architecture;
```

5.2.2 What that loop is not

It is not eight steps the hardware performs one after another. This is where a C background misleads hardest, so it is worth stating outright: a `for` loop whose bounds are fixed at elaboration is **unrolled** by the synthesis tool. Those eight iterations become eight XOR gates in a chain, all settling within one propagation delay, in the same clock cycle. There is no counter for `i`, no branch, no iteration in time. The loop writes eight gates in three lines; it does not do eight things in sequence.

That is also why `acc`, which reads like a running total, costs no storage at all. It never has to survive a clock edge: each of its values is just the wire between two of those XOR gates. A `variable` is the right tool precisely because it never outlives one pass.

The same goes for `bits'range`. The `'range` **attribute** gives the index range of `bits`, here `7 downto 0`, so the loop covers every bit without writing the bounds twice. It is resolved at elaboration, like everything else in the loop header, which is exactly why the tool can unroll it. `for i in 0 to 1 loop` in Chapter 3's `led_toggle.vhd` means the same thing: both bits get their own gates and both update in the same cycle.

5.2.3 The rule of thumb

- Use a `variable` for a scratch computation consumed entirely within one pass: loop accumulators, temporary swaps, intermediate values handed off to a signal at the end, exactly like `acc`.
- Use a `signal` for anything another process or the outside world reads, and anything that must hold its value across separate passes (a counter, or a shift register's stored bits, both in Chapter 6).

A variable *can* retain its value between passes through the same process, behaving like static local storage rather than a fresh local each time, so it is possible to build state with one. But because its updates are immediate rather than scheduled, it behaves differently from a signal used the same way, particularly for anything modelling a chain of registers stepping one stage per clock edge. If state genuinely needs to persist and be read by other logic, reach for a `signal`: the scheduled-update semantics is what makes Chapter 6's counters and shift registers work.

5.3 What the FPGA Actually Builds

5.3.1 The two things the fabric is made of

An FPGA is not a blank slate onto which gates are etched. It is a fixed grid of identical small blocks, manufactured before anybody wrote your VHDL, and "synthesis" is working out how to configure them so they behave like the circuit you described. Two matter here:

- A **look-up table**, or **LUT**: a tiny memory, typically four to six inputs and one output. Load it with the truth table of any Boolean function of those inputs and it computes that function. This is the honest answer to "what does a gate become on an FPGA": nothing becomes a gate. A four-input LUT computes `a and b`, `a xor b or (c and not d)`, or any other function of up to four inputs at exactly the same cost, because in every case it is the same block holding a different table.
- A **flip-flop**: one bit of clocked storage, sitting next to each LUT.

On the DE0-CV's Cyclone V these come packaged together as an **ALM**, holding a fracturable six-input LUT, usable as one six-input function or two smaller ones, plus four flip-flops. That is the unit the Quartus fitter report counts, which is why it talks about ALMs rather than gates.

That is nearly the whole story. `register4` becomes four flip-flops, `mux_8to1` a handful of LUTs, and Chapter 8's state machine LUTs feeding the flip-flops that hold the state.

It also explains something from Chapter 2 that would otherwise look like a broken promise: the book had you minimize expressions with Karnaugh maps to save gates, then never mentioned the saving again. On an FPGA a four-input function costs one LUT whether you minimized it or not. Minimization still matters: it is how you understand a circuit, it is exactly right for the discrete-logic and ASIC flows K-maps were invented for, and reducing a

function below the LUT's input width is what stops it needing a second level of LUTs. But on this board it usually buys you nothing, and it is better to know that than to wonder.

5.3.2 Why a clock has a speed limit

Signals take time to get through a LUT and along the wires between blocks. That **propagation delay** is why a clock cannot go arbitrarily fast.

Picture the shape every design in this book has, a flip-flop, some combinational logic, another flip-flop:

- On a rising edge the first presents a new value, which has to travel through every LUT and wire between the two and *settle* before the next edge arrives.
- If it has not, the second flip-flop captures a value that is still changing, and the design does the wrong thing reliably and mysteriously.

So the clock period has to cover three things:

- The first flip-flop's **clock-to-Q** delay: the time between the edge and its output actually changing.
- The **combinational delay** through the LUTs and wires between the two.
- The second flip-flop's **setup time**.

The slowest such path is the **critical path**, and one over that total is the **Fmax**.

Quartus computes Fmax for you, in the timing analyzer stage of the Quartus workflow (Appendix B), but only if you tell it what the clock is. That means an `.sdc` constraints file with a `create_clock` on the `50 MHz` input. Without one the analyzer has no period of yours to compare anything against, so it warns that no base clocks were constrained and reports Fmax against a default clock instead: a number, but not one you can act on. Separately, a design that "fails timing" is one where some path turned out slower than the period you did declare.

None of that is part of this book. You will not write an `.sdc`, read a timing report, or meet a design that fails timing, because every design here fits inside 20 ns with room to spare. It is worth knowing the step exists and what it needs, so that you recognize it the first time a design of yours gets close to the limit.

Two consequences worth carrying forward:

- **Deep combinational logic costs you speed, not the number of flip-flops.** Chaining a long expression between two registers lengthens the critical path; splitting it so part of the work happens on one cycle and the rest on the next shortens it. That trade, more registers for a faster clock, is the single most common move in high-speed digital design.
- **`50 MHz` is a 20 ns budget.** That is the number every path in your design is racing against, and it is a lot of time at these speeds.

5.3.3 The latch you get by accident

Chapter 3 built a D latch, showed it was transparent while enabled, then set it aside in favour of the flip-flop. Here it comes back uninvited.

§2.5 required a `when others` on every `case`, and gave the language's reason: a `case` has to cover every value of its selector's type. The general rule behind it is that a combinational `process` must assign its output on *every* path through it, and that reason is now sayable:

- VHDL requires a signal to keep its old value if nothing assigns a new one.
- If some path through your process leaves `x` unassigned, you have said `x` must *remember* its previous value on that path, and remembering is not something combinational logic can do.
- So the tool builds the only thing that can: a latch, exactly the one from §3.2.

That is an **inferred latch**, and it is almost never what anybody meant. It is a warning rather than an error, the design will often appear to work, and the message is worth recognizing:

```
Warning: Inferred latch(es) for signal "x"
```

The fix is never to add a latch on purpose. It is to assign the signal on every path, which is the rule you were already following.

5.4 Looking Ahead

Nothing in Chapters 6 to 8 introduces a new idiom. Counters, shift registers and timers all hold state across clock edges using §3.6's clocked process and the scheduling rule above, and Chapter 8's state machines add only an enumerated type, a `case` on it, and one new concurrent form, the conditional signal assignment. What changes is what the state is used for.

Two things to watch for. Chapter 6's `serial_rx8` is the first design whose correctness you cannot check by eye on an LED, which is where the provided testbenches stop being a formality. And §4.8's generic goes to work immediately: Chapter 7's `timer` uses one for its tick count, and `walking_led` instantiates that timer alongside `reset_sync` and `button_sync`, writing almost no logic of its own.

5.5 Review

What you should be able to do now

After this chapter, you should be able to:

- Say what separates a `signal` from a `variable`, predict the different result each produces from the same code, and choose between them deliberately.
- Explain what an FPGA is physically made of, why two different-looking descriptions of the same circuit produce the same hardware, and why a Karnaugh map buys less here than on discrete logic.
- Explain what limits a design's clock frequency, and why adding registers can make a design faster rather than slower.
- Recognize an inferred latch from its warning, say what caused it and fix it, and say why "it compiled with no errors" promises less for an FPGA design than for a C program.

Questions to test yourself

You should be able to explain:

1. A `signal` is assigned twice, to different values, in one pass through a `process`. What ends up on it, and when does that update actually happen?
2. Same question for a `variable`.
3. Why does accumulating across a loop with a `signal` give a different, usually wrong, result?
4. Nothing on an FPGA becomes a gate. What does a Boolean function become instead, and why does a four-input function cost the same whether or not you minimized it first?
5. What is the critical path, and why can adding flip-flops make a design run faster?
6. A combinational `case` assigns its output in three branches out of four. What does the synthesis tool build for the fourth, and why is that the only thing it *can* build?
7. Why is an inferred latch a warning rather than an error, and why does that make it more dangerous rather than less?

Looking ahead

Chapter 6 is about counters and shift registers:

- The first designs whose state advances on its own, rather than tracking an input the way Chapter 3's and Chapter 4's registers do.
- Built from Chapter 3's clocked-process idiom, with this chapter's scheduling rule doing the real work: a shift register is a chain of signals that all update together, which is exactly why it shifts instead of collapsing.

5.6 Exercises

How to work these. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
How to check your work. Every exercise that asks you to write a VHDL module ships a self-checking testbench under `lectures/L05/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
No FPGA board is needed for any exercise. Your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L05/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

`variable` Versus `signal`

Exercise 5.1 Tracing the signal accumulator PAPER

The broken `parity_gen` architecture from §5.2 is reproduced below:

```
architecture broken of parity_gen is
signal parity_s: std_logic;
begin
    parity <= parity_s;

    process(bits) is
    begin
        parity_s <= '0';
        for i in bits'range loop
            parity_s <= parity_s xor bits(i);
        end loop;
    end process;
end architecture;
```

Assume:

- `bits` has the range `7 downto 0`.
- `bits = "10110110"`.
- `parity_s` holds `'0'` before the process begins.

a) Trace the process by hand. For the initial assignment and every loop iteration, record:

- The current index `i`.
- The value of `bits(i)`.

- The value read from `parity_s`.
- The value scheduled for `parity_s`.
- Whether that scheduled assignment survives until the process suspends.

Remember:

- A signal assignment does not update the signal immediately.
- Assignments to the same signal during one process activation replace earlier scheduled assignments for the same simulation time.

b) Determine the final value assigned to `parity_s` after the process suspends. Then:

- Calculate the correct XOR parity of `"10110110"` by hand.
- Compare it with the result from the broken architecture.
- Explain why repeatedly assigning to the signal does not create an accumulator.

c) Trace the variable-based `behaviour` architecture from the chapter's `parity_gen.vhd` using the same input. For every loop iteration, record:

- The value of `bits(i)`.
- The value of `acc` before the XOR operation.
- The value of `acc` immediately after the XOR operation.

Confirm that:

- `acc` retains the result of each iteration for use by the next iteration.
- The final value assigned to `parity` is the correct XOR parity.

Exercise 5.2 The smaller of two values

VHDL

Write an entity named `min_of_two`.
The entity must have:

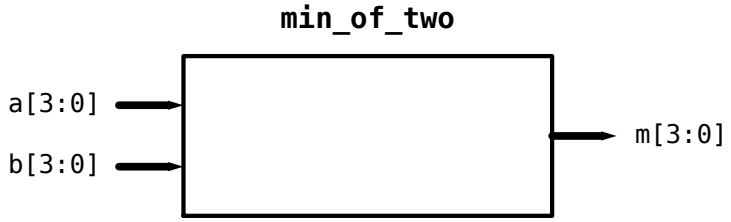
Port	Direction	Type	Description
`a`	in	`std_logic_vector(3 downto 0)`	First value.
`b`	in	`std_logic_vector(3 downto 0)`	Second value.
`m`	out	`std_logic_vector(3 downto 0)`	The smaller of the two.

Treat `a` and `b` as unsigned four-bit values.
Implement the circuit using a process:

- Include `a` and `b` in the sensitivity list.
- Declare a process-local variable to hold the smaller value.
- Compare `a` and `b` as unsigned numbers.
- Assign the variable exactly once during each process activation.
- Assign the variable to `m` after the comparison.

Use the facilities from `ieee.numeric_std` when performing the unsigned comparison, following §2.3.

This exercise does not require a loop. Its purpose is to use a variable as temporary process-local storage for something *other than* an accumulator: §5.2's worked example builds a value up across the iterations of a loop, which is the case a variable is usually introduced for, and it is easy to come away thinking that is the only thing they are good for. Here the variable simply holds an intermediate result for the few lines between computing it and using it, which is the more common use of the two in real code.



Self-check: name your entity `min_of_two`, with ports `a`, `b` (in) and `m` (out), all `std_logic_vector(3 downto 0)`, declared in that order; its testbench is in `exercises/min_of_two`.

Exercise 5.3 Counting set bits

VHDL PAPER

Write an entity named `ones_count8` that counts how many bits of an 8-bit input are set.

Exercise 5.1 had you trace the accumulator bug on paper and Exercise 5.2 used a variable that was not an accumulator at all. This one is the case §5.2 is actually about: a value built up across the iterations of a loop. It is the only exercise in the book where you write one. The entity must have:

Port	Direction	Type	Description
`bits`	in	`std_logic_vector(7 downto 0)`	The value to count over.
`ones`	out	`natural range 0 to 8`	The number of bits of `bits` that are `'1'`.

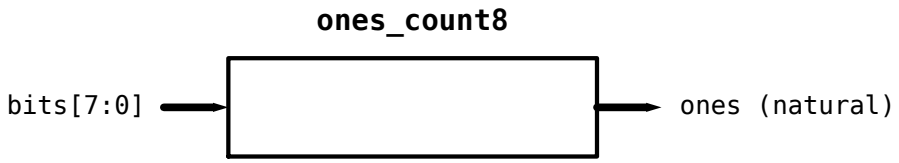
Implement it using a single combinational process:

- Put `bits` in the sensitivity list.
- Declare a process-local `variable` for the running count, and set it to `0` at the top of every activation. A variable keeps its value between activations, so a count that is never cleared keeps growing.
- Loop over `bits'range`, adding `1` for each bit that is set.
- Assign the variable to `ones` once, after the loop.

Then answer, in a sentence each:

- Write the same thing with a `signal` accumulator instead, run the testbench, and record what it reports for the input `00000010`. Explain the number you get using the scheduling rule, not by guessing.

- The loop runs eight times. How many adders does this design put on the FPGA, and how many clock cycles does it take to produce an answer? If those two answers surprise you, reread §5.2.2.
- `ones` is a `natural range 0 to 8` rather than a `std_logic_vector(3 downto 0)`. Both can hold the answer. What does the range buy you, and what would GHDL do if your count ever left it?



Self-check: name your entity `ones_count8`, with input `bits` (`std_logic_vector(7 downto 0)`) and output `ones` (`natural range 0 to 8`), declared in that order; its testbench is in `exercises/ones_count8` and sweeps all 256 input values.

EXERCISE SET 2

A Module Worth Having on Its Own

Exercise 5.4 A synchronizer module

VHDL PAPER

Write the double-flop synchronizer as a module of its own, named `sync`.

You have already built this circuit. It is the first two flip-flops of Chapter 4's `button_sync` (Exercise 4.9), written inline there because at that point it was the *idea* that mattered: an asynchronous input needs two flip-flops before anything else may look at it. Here you pull those two out into a module, and the exercise is not the logic, which you know, but everything around it.

Two things make it worth the second pass. The first is that a bare synchronizer is what most asynchronous inputs actually need: `button_sync` adds an edge detector because a button press is an event, but a serial line, a status flag from another clock domain, or a switch you only ever read the level of needs the two flip-flops and nothing more. Chapter 8's serial receiver instantiates exactly this on its `rx` pin. The second is that it takes **two generics**, and one of them is not a number.

Generic	Type	Default	Description
`SIZE`	`natural range 1 to 15`	`1`	How many independent signals to synchronize, and therefore the width of both vector ports.
`PRESET`	`std_logic`	`'0'`	The value both stages take on reset.

and these ports:

Port	Direction	Type	Description
`clock`	in	`std_logic`	50 MHz system clock.
`reset_s2_n`	in	`std_logic`	Active-low, **already synchronized** reset. A synchronizer does not synchronize its own reset; Chapter 4's `reset_sync` does that.
`async_in`	in	`std_logic_vector(SIZE - 1 downto 0)`	The asynchronous input or inputs, straight from outside the clock domain.
`sync_out`	out	`std_logic_vector(SIZE - 1 downto 0)`	The same signals, safe to use, two rising edges later. Each bit is synchronized independently.

a) Implement it:

- Two internal `std_logic_vector(SIZE - 1 downto 0)` signals, one per stage.
- One process, sensitive to `clock` and `reset_s2_n`.
- On reset, drive both stages to `PRESET`, outside the clocked branch.
- On a rising clock edge, shift `async_in` through the two stages.
- Drive `sync_out` from the second stage with a concurrent assignment.

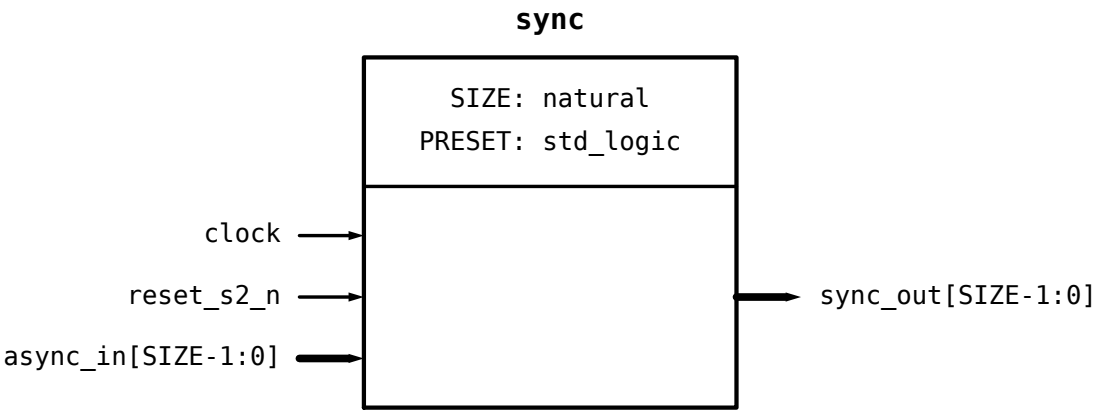
b) `PRESET` is a `std_logic` generic, which is the first one in this book that is not a number, and it exists because the safe reset value depends on what the signal *means*. Only the instantiating design knows that. Answer:

- `button_sync` synchronizes active-low buttons, so it would pass `'1'`. What would `'0'` claim about the buttons for the two cycles after every reset?
- A serial line idles high, so Chapter 8's receiver passes `'1'` too. What would `'0'` look like to a receiver watching for a start bit?
- Given both, why is the *default* `'0'` rather than `'1'`, and what does that say about who is responsible for getting it right?

c) This is also a question about what a generic costs. `SIZE` is `natural range 1 to 15` and `PRESET` is `std_logic`, and both are fixed before synthesis runs. Using §5.3's picture of what the FPGA is actually made of, say how many flip-flops an instance at `SIZE = 4` uses, how many an instance at `SIZE = 1` uses, and where in the built circuit `PRESET` ends up. Neither generic appears as logic anywhere; say what happened to them instead.

d) Verify it with its testbench. It instantiates your module twice, once one bit wide with `PRESET = '1'` and once three bits wide with `PRESET = '0'`, so a design that hard-codes either generic fails. It checks that `sync_out` follows `async_in` after **exactly** two rising edges rather than one or eventually, that each bit of a vector is synchronized on its own, and that reset loads `PRESET` with no clock edge involved.

Tip: write the architecture for `SIZE = 1` in your head first, then check that every line is already correct for a vector. The aggregate `(others => PRESET)` and a plain vector assignment both work element-wise, so the generalization should cost no extra code.



Self-check: name your entity `sync`, with generics `SIZE` (`natural range 1 to 15`, default `1`) and `PRESET` (`std_logic`, default `'0'`), inputs `clock`, `reset_s2_n`, `async_in` (`std_logic_vector(SIZE - 1 downto 0)`) and output `sync_out` (`std_logic_vector(SIZE - 1 downto 0)`), declared in that order; its testbench is in `exercises/sync`.

Keep this file. Chapter 8's serial receiver capstone instantiates it on the `rx` pin, and asks you to copy it in.

EXERCISE SET 3

From VHDL to Hardware

The course demonstrates the Quartus workflow on a real DE0-CV board, first with the braking assistant back in Chapter 1, and Appendix B writes it up for reference. You don't need Quartus or a board yourself; the exercises below are about understanding what that workflow does to the VHDL you write.

Exercise 5.5 What Quartus does to your VHDL

PAPER

Answer each of the following, referring to Appendix B. One or two sentences each.

a) A Quartus project needs a **top-level entity** selected before it will compile:

- What does the tool do differently with the top-level entity than with any other entity in the project?
- `min_of_two` from Exercise 5.2 and its testbench are both valid VHDL. Why can only one of them ever be the top-level entity of an FPGA project?

b) Pin assignment connects a port name to a physical pin:

- Why must pins be assigned *before* compiling, rather than afterwards?
- What would `m` physically be connected to if you compiled `min_of_two` having assigned no pins at all?

c) Synthesis turns your architecture into gates on the chip:

- Chapter 2's `combo_logic` exercise (Exercise 2.10) had you write `x = ab + c'd` twice: once as one expression, and once with `c'd` named by an internal signal. What should synthesis produce for each, and why is your answer the same for both?
- Which of the two would you rather read six months from now?

d) Compilation produces warnings as well as errors:

- Why is "it compiled with no errors" a much weaker statement about an FPGA design than "it compiled with no errors" about a C program?
- Name one class of warning you would never ignore.

Exercise 5.6 Predicting parity

PAPER VHDL

`parity_gen` from §5.2 is this chapter's worked example, and it ships a reference testbench at `lectures/L05/parity_gen/parity_gen_tb.vhd`.

a) Before running anything, predict `parity` by hand for each of these inputs:

- `"00000000"`
- `"10110110"`
- `"11111111"`
- `"10000000"`

b) Run the reference testbench over the worked example and confirm your four predictions:

```
cd lectures/L05/parity_gen
ghdl -a --std=93 parity_gen.vhd parity_gen_tb.vhd
ghdl -e --std=93 parity_gen_tb
ghdl -r --std=93 parity_gen_tb --assert-level=error --stop-time=10ms
```

c) Take any one of those inputs and flip exactly one bit, leaving the other seven unchanged:

- What happens to `parity`?
- Explain why changing exactly one input bit must *always* invert the XOR-parity result, whichever bit you pick and whatever the starting value.
- This is the property that makes a parity bit able to detect a single-bit transmission error. What kind of error does it fail to detect, and why?

Exercise 5.7 Reviewing a latched decoder

PAPER

A colleague sends you this combinational decoder for review. It compiles, and Quartus reports no errors.

```
architecture behaviour of level_decode is
begin
    process(sel) is
    begin
        case sel is
            when "00"   => leds <= "0001";
            when "01"   => leds <= "0010";
            when "10"   => leds <= "0100";
            when others => null;
        end case;
    end process;
end architecture;
```

a) Quartus emits `Warning: Inferred latch(es) for signal "leds"`. Using §5.3, explain what the tool built and why. What did the VHDL ask for that combinational logic cannot do?

b) The author's defence is that `sel` is two bits, so `"11"` is the only value `when others` can match, and the design never generates it. Give two separate reasons that argument does not save them:

- one about what `sel` can actually hold, from §1.4.
- one that would still apply even if `sel` really could only ever be `"00"`, `"01"` or `"10"`.

c) Fix it, in two different ways:

- by changing only the `when others` branch.
- by adding a single line *before* the `case` statement, leaving every branch untouched.

Which of the two would you rather maintain, and why? The second scales to a `case` with twenty branches; the first does not.

d) This is a warning, not an error, and the design will often appear to work on the board. Why does that make it more dangerous than an error rather than less?

CHAPTER 6

Counters and Shift Registers

Building blocks for anything that needs to count or move bits.

IN THIS CHAPTER

- Counters: a register, an adder and a constant `1`, and why wraparound comes for free.
- Shift registers: moving data one bit per clock edge, in one direction used consistently.
- Serial-in/parallel-out and parallel-in/serial-out, and which fits which job.
- An 8-bit serial receiver: a shift register plus a bit counter, knowing when a byte has arrived.
- Two demonstrations on the DE0-CV: a row of LEDs counting and wrapping, and a byte shifted in one button press at a time.

This chapter builds four things, in this order. A 4-bit counter in CircuitVerse: a register, an adder, and a constant `1`, run past `1111` to watch it wrap. The same counter in VHDL. A shift register in VHDL, traced one clock edge at a time on paper before any of it is written. And `serial_rx8`, the shift register just written plus the bit counter that tells it when a byte is done: the two halves of the chapter meeting in one module. Then both go onto the board. First the counter, its top bits on a row of LEDs, slowed to something the eye can follow, running up to all-ones and wrapping back to zero on its own; then `serial_rx8` with `shift_enable` driven by a push button instead of held high, so one press shifts one bit and the byte assembles across eight LEDs, most significant bit first.

Before the counter in §6.1 reaches `1111`, predict what it will show next, and then find the element in the drawing that makes it happen. There isn't one, which is the point: nothing in the circuit performs the wrap, and that is a consequence of the register's width rather than logic anybody added.

`serial_rx8` (`lectures/L06/serial_rx8/serial_rx8.vhd`) is the first design in the book whose correctness you cannot check by eye at full speed: enabled continuously it swallows a byte in 160 ns. That is why it ships a reference testbench, and why the board demonstration drives `shift_enable` from a button instead. Slowing the *clock* would be the software instinct and the wrong move; the module already has the control that lets you take one bit at a time. The board demonstration in §6.7 wraps it in Chapter 4's `reset_sync` and `button_sync`, both

unchanged, so it is a composition exercise as much as a receiver. Read the sections through before you start on the exercises; running their testbenches is covered in §2.8.

6.1 From Registers to Counters

A register (Chapter 3) is a bank of D flip-flops storing whatever value is fed in on the next rising edge. A **counter** is the same idea with one twist: instead of a new value from outside, you feed the register its *own current value plus one*. Each edge it captures `current_value + 1`, so the stored number increments once per clock cycle with no further input.

6.1.1 Build it by hand first

As with every circuit in this book, build it before you write it. A 4-bit counter is three elements, and watching it run is the point:

- A **4-bit register**: four D flip-flops sharing one clock, exactly the register from §3.5.
- An **adder** and a **constant 1**: the register's four outputs into one adder input, the constant into the other.
- The adder's sum wired back to the register's four `D` inputs. That feedback path is the whole circuit: the register holds a number, the adder computes that number plus one, the next edge stores it. It is §3.1's feedback loop again, with an adder in the path instead of a plain wire.

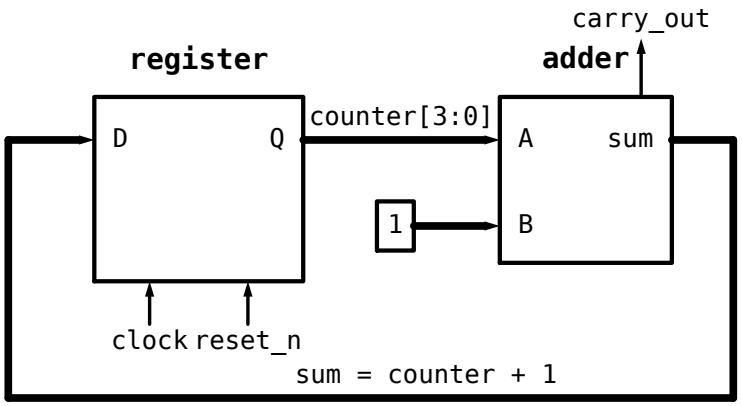


Figure 6.1 A 4-bit counter: a register whose own output, plus one, feeds back to its input. The adder's `carry_out` is connected to nothing.

Then confirm, one clock edge at a time, with a period of `1000 ms`:

- Watch the outputs step `0000, 0001, 0010, 0011, ...`, one count per rising edge.

- **Let it run past 1111.** It returns to `0000` on its own, and nothing in your circuit says to do that: there is no comparison, no reset logic, no "if the count is at maximum" anywhere. The adder produced `10000`, the fifth bit had nowhere to go, and the four bits that survived are `0000`. That is **overflow**, and in a counter it is not a bug to work around but the feature the rest of this chapter and all of Chapter 7 are built on.
- **Watch `carry_out` on the adder while it happens.** That is the fifth bit, and the drawing shows exactly where it ends up: high throughout the single count at which the register rolls over, connected to nothing, leaving the adder and stopping there. A wider counter would feed it into a fifth stage; here it is discarded, and discarding it *is* the wraparound. So "the count wraps because it runs out of bits" is not a figure of speech. You can point at the bit it ran out of.

Save the project. Chapter 7 starts from this exact circuit and adds two things to it, and reopening it beats redrawing it.

6.1.2 The same counter in VHDL

A signal and a process, exactly like Chapter 3's registers, except the right-hand side refers to the signal itself:

```
signal counter: natural range 0 to 15;
...
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        counter <= 0;
    elsif (rising_edge(clock)) then
        counter <= counter + 1;
    end if;
end process;
```

`natural` is a predefined VHDL integer subtype: a non-negative whole number, as opposed to the `std_logic` types that model bits directly. VHDL's arithmetic operators work on it the way ordinary integer maths does, which is why `counter + 1` needs no extra machinery. Synthesis still turns it into a register of flip-flops, exactly as a `std_logic_vector` would; the difference is purely how you write and reason about the value.

The range constraint `0 to 15` is not just documentation. It tells the synthesis tool this counter needs four bits, so incrementing past `15` rolls back to `0` for free, purely as a consequence of running out of bits, with no explicit "if counter = 15" logic in the drawing or the VHDL.

That free wrap has a precondition worth stating: it only happens when the upper bound is `2^N - 1`. `0 to 15` wraps for free; `0 to 9` does not, because `10` is not a power-of-two boundary,

and a modulo-10 counter needs an explicit comparison. That is exactly what Exercise 6.4 asks you to write.

One place the simulator disagrees with the hardware. The free wrap is a property of the *synthesized* circuit, and the drawing shows it is real. A VHDL *simulator* is stricter than a wire:

- `natural range 0 to 15` is range-checked, so GHDL treats `15 + 1 = 16` as an error and aborts rather than wrapping.
- The synthesized hardware has no such notion. There is no sixteenth value for the bits to take, so they land on `0000`, exactly as your four LEDs did.

The gate network and the chip both wrap; only the simulator objects, because a range-constrained integer is a promise you made and `16` breaks it. The rule that follows is not "trust the hardware": it is that anything you intend to verify should say what it means. Write the comparison out, the way `timer.vhd` and `serial_rx8.vhd` do, and the drawing, the simulator and the chip all agree.

This wraparound-by-bit-width is the single most important property of a counter, and it is what Chapter 7 builds a timer out of: a counter that watches for one particular count and raises a flag when it arrives.

6.2 Shift Registers

A **shift register** is, again, N D flip-flops, wired so each flip-flop's output feeds the *next* one's input rather than its own. On every clock edge every bit moves one position down the chain simultaneously:

```
serial_in -> [D Q]-->[D Q]-->[D Q]-->[D Q] -> serial_out
              FF0     FF1     FF2     FF3
             bit 0   bit 1   bit 2   bit 3
               ^clock  ^clock  ^clock  ^clock  (all share the same clock)
```

After one edge, `FF0`'s contents are in `FF1`, `FF1`'s in `FF2`, and so on; a new bit enters at `FF0` from `serial_in`, and whatever was in the last flip-flop falls off the end as `serial_out`. Run it for N edges and N serial bits have moved all the way through.

One direction, used everywhere in this book. A shift register can move bits either way, and both are equally valid; mixing them within one design is what causes confusion. So every shift register here follows the diagram above:

- the chain maps onto a `std_logic_vector` with `FF0` as **bit 0** and `FF(N-1)` as bit `N-1`.
- serial data **enters at bit 0** and **leaves from bit `N-1`**.
- every bit moves one position toward the **most significant** bit per edge.

In VHDL that direction is a single expression, identical in every example below:

```
shift_reg <= shift_reg(N-2 downto 0) & serial_in;
```

`shift_reg(N-2 downto 0)` is everything except the top bit, moved up one position; the top bit has nowhere to go and falls off as `serial_out`; `serial_in` fills the vacated bit 0.

Whenever you meet a shift register outside this book, check its direction before assuming. The mirror-image form, `serial_in & shift_reg(N-1 downto 1)`, is just as common and reads almost identically at a glance.

Shift registers are named by how data enters and leaves:

Configuration	Data in	Data out	Typical use
SISO (serial-in/ serial-out)	one bit per clock	one bit per clock	a pure delay line
SIPO (serial-in/ parallel-out)	one bit per clock	all N bits at once	receiving a stream of serial bits, e.g. an incoming byte on a single wire
PISO (parallel-in/ serial-out)	all N bits at once (loaded)	one bit per clock	sending N bits out over a single wire, e.g. driving a chain of LEDs or an external shift-register chip
PIPO (parallel-in/ parallel-out)	all N bits at once	all N bits at once	effectively a plain register (§3.5)

This chapter covers the two practically useful ones, SIPO and PISO.

6.3 Serial-In/Parallel-Out (SIPO)

A SIPO register is what you reach for when bits arrive one at a time on a single wire and you need the accumulated value all at once. The classic case is deserializing an incoming byte, which is what the receive path of any serial protocol does at heart.

Which end of the byte arrives first is each protocol's own decision. SPI and CAN send the most significant bit first by default, which is what §6.2's direction assumes: the first bit to arrive ends up in the MSB once all N have shifted in. (SPI bit order is configurable on most controllers, so check the part.) A protocol sending the least significant bit first, such as UART, shifts the mirror-image way, and its idiom is the mirror image too.

The idiom is one line inside a clocked process, §6.2's shift expression verbatim:

```
signal shift_reg: std_logic_vector(7 downto 0);
...
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_reg <= (others => '0');
    elsif (rising_edge(clock)) then
```

```
        shift_reg <= shift_reg(6 downto 0) & serial_in;
    end if;
end process;
parallel_out <= shift_reg;
```

After 8 edges the received bits have fully replaced the register's original contents, and `parallel_out` exposes all 8 at once.

6.4 Parallel-In/Serial-Out (PISO)

A PISO register reverses SIPO's job: load N bits at once, then send them out one at a time. It is what you reach for when a fixed pattern has to go out over a single wire, such as a chain of LEDs or a display driven by an external shift-register chip.

It needs one more control signal than SIPO: something to distinguish *loading* a new parallel value from *shifting* the existing one out.

```
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        shift_reg <= (others => '0');
    elsif (rising_edge(clock)) then
        if (load = '1') then
            shift_reg <= parallel_in;                          -- Parallel load.
        elsif (shift = '1') then
            shift_reg <= shift_reg(N-2 downto 0) & serial_in;  -- Shift toward the↩
    ↪  MSB.
        end if;
    end if;
end process;
serial_out <= shift_reg(N-1);
```

Bit `N-1` is always the next bit to leave, so a loaded value goes out most-significant bit first: load `"1010"` into a 4-bit register and `serial_out` presents `1`, `0`, `1`, `0` over the following cycles.

Notice how little separates this from §6.3's SIPO. The **shift expression is identical**, same direction, same line of VHDL. Only two things differ: PISO adds a `load` branch, and it taps `shift_reg(N-1)` as a serial output where SIPO exposes the whole vector. "SIPO" and "PISO" name how you *wire up* a shift register, not two kinds of hardware, which is why §6.2 could describe the shift once before either name appeared.

Whether a design needs a genuinely new `serial_in` on every shift, or is happy to recirculate whatever falls off the end, is up to how it is wired. That recirculation is exactly the trick the walking-LED example in Chapter 7 uses, once it has a timer to pace the shifting.

6.5 Worked Example: An 8-Bit Serial Receiver

The two halves of this chapter meet in one module. A shift register can accept serial bits forever but cannot tell you *when* a complete byte has arrived, because it has no notion of "how many bits so far". That is a counting problem, and §6.1 already solved it. `lectures/L06/serial_rx8/serial_rx8.vhd` combines the two, plus a one-cycle `data_ready` pulse when the eighth bit lands.

This is not a toy: it is the front half of every serial receiver there is. SPI, CAN and UART receive paths all start with a shift register and a bit counter, differing only in what decides when a bit is valid.

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_s2_n`	in	`std_logic`	Active-low, **already-synchronized** reset; clears the register and the bit count. Asynchronous, so it clears them with no clock edge involved.
`shift_enable`	in	`std_logic`	High for exactly one clock cycle per incoming bit.
`serial_in`	in	`std_logic`	The incoming bit, sampled on each enabled rising edge.
`data_out`	out	`std_logic_vector(7 downto 0)`	The byte received so far, most significant bit first.
`data_ready`	out	`std_logic`	One-cycle pulse as the eighth bit lands.

The reset port is the synchronized one, and the `_s2` says so. It is named `reset_s2_n` rather than `reset_n` because what belongs on it is the output of a `reset_sync` (§4.5), the module you wrote in Chapter 4's Exercise 4.8: assert asynchronously, release synchronously. **Instantiate one in whatever design encloses this receiver and wire its `reset_s2_n` here**, which is exactly what §6.7's board wrapper does. Wire a push button straight to this port instead and you get the race Chapter 4 exists to prevent: the shift register and the bit counter are a dozen flip-flops, and an asynchronously *released* reset lets them leave reset on different edges, so the receiver can start counting a byte a cycle before it starts shifting one. The rule is the same one §7.3 states for `timer`, and it holds for every subcomponent in this book: a module composed inside a larger design never touches a raw asynchronous input itself.

`counter`, `sipo8` and `piso8` in §6.9 take `reset_s2_n` for the same reason. None of them is a design you put on a board by itself: a counter, a shift register and a receiver are parts, and a part is entitled to assume the design around it has done the synchronizing. So the whole of this chapter takes the synchronized reset, and only the wrappers in §6.6 and §6.7, which own the pins, ever see a raw `reset_n`.

A **top level** is named the other way round: it takes `reset_n` and instantiates `reset_sync` itself, because it is the thing holding the pin. `blinker` and `walking_led` in Chapter 7, and both Chapter 8 capstones, are written that way.

Get into the habit of reading the `_s2` as a requirement on the caller, because from here on most of what you write is a subcomponent.

On each rising edge where `shift_enable = '1'`, `serial_in` is shifted into bit 0 by §6.3's idiom unchanged, so the first bit to arrive has walked up to bit 7 by the time the eighth lands, and an internal `natural range 0 to 7` bit counter (§6.1) advances. On the edge where that counter is already at `7`, `data_ready` pulses high for one cycle and the counter is cleared back to `0`, ready for the next byte. `data_out` is driven concurrently from the shift register, so the complete byte is on it during the cycle `data_ready` is high.

Note that the counter is *cleared* here rather than left to wrap: `0 to 7` is a power-of-two range so it would wrap for free, but writing it out keeps the simulator in agreement with the hardware, per §6.1.

Two details are worth dwelling on, because both are patterns you have met more than once:

- **`data_ready` is a pulse, not a level.** It is driven low unconditionally at the top of the clocked branch and raised only on the edge completing a byte: the same single-cycle shape as §3.7's edge detector. A consumer that misses it has missed the byte, which is exactly why it is paired with a `data_out` that *holds* its value afterwards. A real peripheral would put a holding register or a small FIFO here so a slow consumer cannot lose data.
- **`shift_enable` gates the counter, not just the shift.** Both advance together or neither does, so a gap in the incoming stream pauses the receiver mid-byte rather than corrupting it, and it resumes on exactly the bit it was waiting for.

This is the whole of `serial_rx8.vhd`:

lectures/L06/serial_rx8/serial_rx8.vhd

```
--------------------------------------------------------------------------------
-- 8-bit serial receiver: a SIPO shift register paired with a bit counter, so it
-- reports a complete byte rather than shifting forever. Bits arrive most
-- significant first, one per enabled clock edge. See Appendix A.5.
--
-- Inputs:
--     - clock       : 50 MHz system clock.
--     - reset_s2_n  : Active-low, already-synchronized reset: drive it from a
--                     reset_sync (L04 A.5), never straight from a pin. ↩
    ↪ Asynchronous
--                     here, so it clears the register and the bit count with no
--                     clock edge involved.
--     - shift_enable: High for one clock cycle per incoming bit; holds both the
--                     shift register and the bit count when low.
--     - serial_in   : Serial input bit, sampled on each enabled rising edge.
-- Outputs:
--     - data_out  : Received byte, valid while data_ready is high.
```

```
--     - data_ready: One-cycle pulse as the eighth bit lands.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity serial_rx8 is
    port(clock, reset_s2_n, shift_enable, serial_in: in std_logic;
         data_out                                  : out std_logic_vector(7 downto
    ↪  0);
         data_ready                                : out std_logic);
end entity;

architecture behaviour of serial_rx8 is

-- Bit width.
constant BIT_WIDTH: natural := 8;

-- Most significant bit.
constant MSB: natural := BIT_WIDTH - 1;

-- Shift register.
signal shift_reg: std_logic_vector(MSB downto 0);

-- Bit counter.
signal bit_count: natural range 0 to MSB;

begin
    data_out <= shift_reg;

    RX_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            shift_reg  <= (others => '0');
            bit_count  <= 0;
            data_ready <= '0';
        elsif (rising_edge(clock)) then
            data_ready <= '0';
            if (shift_enable = '1') then
                shift_reg <= shift_reg(MSB - 1 downto 0) & serial_in;
                if (bit_count >= BIT_WIDTH - 1) then
                    data_ready <= '1';
                    bit_count  <= 0;
                else
                    bit_count <= bit_count + 1;
```

```
                end if;
            end if;
        end if;
    end process;
end architecture;
```

Checking it: the example ships `serial_rx8_tb.vhd`, which shifts two bytes in with a pause between them and checks that `data_ready` pulses once per byte, never early:

```
cd lectures/L06/serial_rx8
ghdl -a --std=93 serial_rx8.vhd serial_rx8_tb.vhd
ghdl -e --std=93 serial_rx8_tb
ghdl -r --std=93 serial_rx8_tb --assert-level=error --stop-time=10ms
```

Clocked at `50 MHz` and left enabled, this receiver consumes a byte in 160 ns, which no LED can show you. That is why the testbench matters here in a way it did not for a blinking LED, and it is the case for most real digital design. But it is not a reason to skip the board, because `shift_enable` is exactly the way out: see §6.7.

6.6 FPGA Demonstration: Watching a Counter Wrap

The course demonstrates the counter on the DE0-CV, built the same way as every design since Chapter 1: a Quartus Prime Lite project targeting `5CEBA4F23C7N`, the `.vhd` file added, compile, assign each port to a physical pin in the Pin Planner, then recompile and program the board (Appendix B).

One thing has to change first, and it is worth seeing why. Clocked at `50 MHz`, a 4-bit counter runs through all sixteen states and wraps every `320 ns`, roughly three million times a second. An LED driven straight from it is not blinking, it is simply on at half brightness. To watch a wrap you have to look at a much slower bit: count in a wider register, say 28 bits, and assign the **top** four bits to the LEDs.

The demonstration is therefore a two-module wrapper rather than the exercise entity on its own: a `reset_sync` taking the raw `reset_n` from the pin, and the widened counter taking its `reset_s2_n` from that. The counter's port is `reset_s2_n` precisely so that this wiring is the only one that type-checks, and §6.7 repeats the pattern with a third module.

- `clock` to the 50 MHz oscillator, `reset_n` to a push button (into the `reset_sync`, never straight into the counter), and `count[27]` down to `count[24]` — the top four bits of the bus Quartus builds for the widened integer port, since `count` is a `natural` and not a vector you could slice in VHDL — to four LEDs.
- Bit 24 changes once every `2^24` clock cycles, about a third of a second, so the four LEDs count visibly from `0000` up to `1111` and then, with nothing in the circuit doing it, back to `0000`. A full lap takes about five seconds.

Dividing a clock by watching a high bit is the crudest possible timer, and its limitation is worth naming now: the period is fixed at a power of two, and the only way to change it is to pick a different bit. Chapter 7 replaces it with a counter compared against a target you choose, which is all a timer really is.

6.7 FPGA Demonstration: Shifting a Byte In by Hand

§6.5 said a byte arrives too fast to watch. The fix is not to slow the clock down, which would be the instinct from software and is the wrong move in hardware: the clock is the one thing in a synchronous design you do not fiddle with. `serial_rx8` already has the right control, and it is `shift_enable`, which "gates the counter, not just the shift". Hold it low and the receiver stops dead, mid-byte, for as long as you like. Raise it for exactly one cycle and precisely one bit goes in.

So drive `shift_enable` from a **button** instead of holding it high. One press, one bit.

The demonstration is a small wrapper around three modules you already have, and no new logic beyond a single flip-flop. The course builds it live:

- `reset_sync` (§4.5), turning the raw `reset_n` into `reset_s2_n` for everything else.
- `button_sync` (§4.6) at `COUNT = 1`, turning a press into the one-cycle pulse that `shift_enable` wants. This is not optional decoration: a raw button wired to `shift_enable` would shift a random handful of bits per press, once per clock cycle for as long as the contacts bounce.
- `serial_rx8` itself, unchanged, with its `reset_s2_n` taken from the `reset_sync` above rather than from the button. That is the whole reason the port is named `reset_s2_n`: the wrapper owns the raw `reset_n`, and the receiver only ever sees the synchronized one.
- One flip-flop latching `data_ready`, set when the pulse arrives and cleared by reset. Without it there is nothing to see: `data_ready` is high for 20 ns, and an LED cannot show you that.

On the board:

- `clock` to the 50 MHz oscillator, `reset_n` to a push button.
- `serial_in` to one slide switch: this is the bit you are about to send.
- The shift button to a second push button.
- `data_out(7 downto 0)` to eight LEDs, and the latched `data_ready` to a ninth.

Now set the switch, press the button, and watch one bit enter at the right-hand LED and the whole pattern step left. Send `10110010` one bit at a time and the byte assembles across the LEDs in front of you, most significant bit first, exactly as §6.3's idiom says it must. On the eighth press the ninth LED lights and the bit counter starts over.

Three things this makes visible that the testbench can only assert:

- **The bit ordering.** The first bit you send ends up in bit 7. Reading that off a waveform is a chore; watching it walk across a row of LEDs is not.

- **What `shift_enable` is for.** Let go of the button for a minute in the middle of a byte and nothing moves, nothing is lost, and the eighth bit still completes the byte. That is the "pauses mid-byte rather than corrupting it" property, demonstrated rather than claimed.
- **Pulse versus level, one more time.** The reason a flip-flop had to be added for `data_ready` is the whole distinction, and it is the same reason Chapter 3's edge detector existed.

6.8 Review

What you should be able to do now

After this chapter, you should be able to:

- Build a 4-bit counter as a gate network, and explain, pointing at your own drawing, why nothing in it makes the count return to zero.
- Implement the same counter in VHDL, explain wraparound as a consequence of the register's bit width, and say why a simulator objects to a wrap the hardware performs happily.
- Implement a shift register in both SIPO and PISO configurations, know which fits which job, and know which direction this book shifts in and why sticking to one matters.
- Combine a counter with a shift register to detect that a complete word has arrived, and verify a clocked design by running its testbench.

Questions to test yourself

You should be able to explain:

1. Why does a `natural range 0 to 15` counter return to `0` after `15` with no explicit "if at maximum, clear it" logic? What has to be true of the range? Which element performs the wrap?
2. A VHDL simulator aborts on the wrap your CircuitVerse drawing performs without complaint. Which of the two is modelling the hardware, and what does that tell you about trusting either alone?
3. What is the practical difference between a SIPO and a PISO shift register? Which would you reach for to receive a stream of serial bits, and which to drive a chain of LEDs from a fixed pattern?
4. A shift register and a counter are both "N flip-flops sharing a clock". What differs is what feeds each flip-flop's input. Describe that difference for each.
5. In `serial_rx8`, why is `data_ready` a one-cycle pulse rather than a level that stays high once a byte has arrived?
6. Why does `shift_enable` have to gate the bit counter as well as the shift itself?

7. The board demonstration drives `shift_enable` from a button rather than slowing the clock down. Why is slowing the clock the wrong answer, and what would break if the button were wired to `shift_enable` without passing through `button_sync` first?
8. `data_ready` needs a flip-flop of its own before an LED can show it. Which earlier chapter's problem is that, in different clothes?

Looking ahead

Chapter 7 is about timers:

- Timers: a counter with a target count, which is what turns clock ticks into real time.
- Built by adding exactly two things to the counter you drew here, a comparator and a clear, so keep that CircuitVerse project.
- The walking LED: this chapter's shift register, paced by that timer and started by a synchronized button, built live in the course and run on the DE0-CV board.

6.9 Exercises

How to work these. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
How to check your work. Every exercise below that asks you to write a VHDL module ships a self-checking testbench under `lectures/L06/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
No FPGA board is needed for any exercise. The Quartus synthesis and board-programming steps are demonstrated in the course, and Appendix B shows them for a reader with a board; your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L06/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

Counters

Exercise 6.1 Sizing a counter register PAPER

A signal is declared as:

```
signal counter: natural range 0 to 1023;
```

a) How many bits wide is the register this synthesizes to?

b) What is the largest value `counter` can hold before it wraps back around to `0` on the next increment?

c) Rewrite the declaration for a counter that instead needs to count from `0` up to `63`.

Follow the counter pattern in §6.1.

Exercise 6.2 A counter traced by hand PAPER

Trace a `natural range 0 to 3` counter by hand (no VHDL needed).

Assume:

- The counter starts at `2`.
- It is incremented once per clock edge, for five consecutive edges.
- There is no reset in between.

Record, clock edge by clock edge:

- The value held before the edge.
- The value captured on the edge.

Explain why the counter returns to `0` without any explicit "if the count reaches its maximum, clear it" logic.

Note: this is the *synthesized* behaviour, the one that matters for hardware. §6.1 flags the simulation caveat that goes with it: GHDL range-checks a `natural range 0 to 3` signal, so a simulation of this counter aborts on the wrap rather than rolling over. Trace it on paper as hardware, and reach for a full-width `unsigned` if you ever need a counter that wraps in simulation too.

Exercise 6.3 Overflow in CircuitVerse CIRCUITVERSE PAPER

Build the 4-bit counter from §6.1 by hand in CircuitVerse, and use it to see overflow rather than take it on trust.

a) Build it:

- A 4-bit register: four D flip-flops sharing one clock.
- An adder, with the register's outputs on one input and a constant `1` on the other.
- The adder's sum wired back to the register's `D` inputs.
- Four output elements or LEDs, one per bit, so you can read the count.

b) Run it at a clock period of `1000 ms` and record what you see:

- Write down the sequence of counts over sixteen consecutive clock edges, starting from `0000`.
- Watch `carry_out` on the top adder as the count rolls from `1111` to `0000`:
  - On how many of the sixteen edges is it high?
  - What is connected to it?
- Explain, in one sentence, where that fifth bit goes, and why the register stores `0000` rather than `10000`.

c) Now reason about what you did *not* build:

- Point at the part of your drawing that decides the counter should return to `0000`.
- There isn't one. Say what is doing the work instead.
- What would you have to add to make it count `0` to `9` and then wrap? (Exercise 6.4 asks you to write exactly that in VHDL, so keep your answer.)

d) Change the width to three bits by removing the top flip-flop and narrowing the adder:

- Predict the new wrap point before you run it.
- Confirm it.
- State the rule connecting a counter's width to the value it wraps at.

Save this circuit. Chapter 7 builds its timer by adding two things to it — a comparator and a clear — and starting from a counter you have already watched work is the whole point of the exercise there.

Exercise 6.4 A counter of any radix

VHDL

Write an entity named `counter` that counts from `0` up to one below its radix, then wraps back to `0`. With the default radix that is a decimal counter, `0` through `9`.

The entity has one generic:

Generic	Type	Default	Description
`RADIX`	`natural range 1 to 2**16`	`10`	How many values the counter passes through before repeating. It counts `0` to `RADIX-1` inclusive, so `RADIX` is the length of the cycle, not the largest count.

and these ports:

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_s2_n`	in	`std_logic`	Active-low, **already-synchronized**: it comes from your `reset_sync` (Chapter 4's Exercise 4.8), never straight from a pin. Asynchronous, so it clears with no clock edge involved.
`count`	out	`natural range 0 to RADIX-1`	The running count. `0` is a value it takes, so the range starts there: reset drives it to `0`, and it returns there on every wrap.
`tick`	out	`std_logic`	A one-cycle pulse on the edge where `count` wraps from `RADIX-1` back to `0`.

Implement it using a single synchronous process:

- Include `clock` and `reset_s2_n` in the sensitivity list.
- On reset, set `count` to `0` and `tick` to `'0'`.
- On each rising clock edge:
  - Drive `tick` low by default.
  - When `count` has reached `RADIX-1`, wrap `count` to `0` and pulse `tick` high.
  - Otherwise, increment `count`.

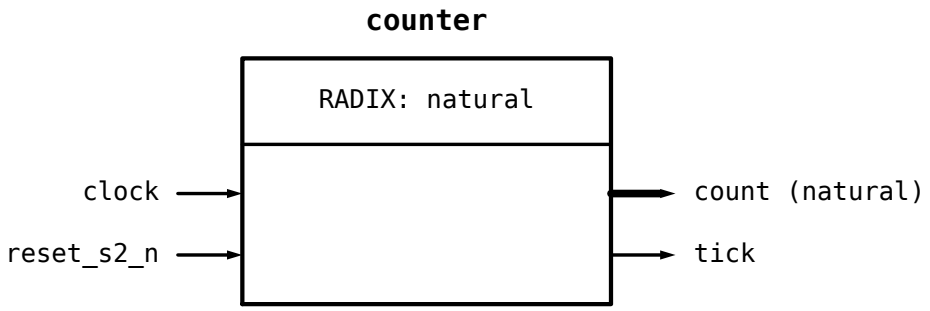
Write the comparison against `RADIX-1`, never against `9`. The generic is the whole difference between a module and a module you can use twice: `RADIX` is fixed before synthesis runs, so the comparator it builds is no larger than a hard-coded one, and the counter's register is sized to the radix you ask for. This is the same argument §4.8 makes for `button_sync`'s `COUNT`, and the reason Chapter 7's `timer` takes its target as a generic rather than as a constant.

Unlike the free wraparound of a `natural range 0 to 15` counter (§6.1), a counter of arbitrary radix needs an explicit comparison, because `RADIX-1` is not in general a power-of-two boundary. Notice what happens when it is: with `RADIX = 16` the comparison is still written, but the hardware underneath no longer needs it.

Tip: The process needs to read the running count (to compare it and to increment it), so hold the count in an internal `signal count_s: natural range 0 to RADIX-1;`, run the

logic on `count_s`, and drive the output with `count <= count_s;` outside the process, exactly as `d_flip_flop.vhd` uses `q_s` in Chapter 3. This is not a style choice: every `ghdl` command in this book passes `--std=93`, and under VHDL-93 reading an `out` port does not compile at all. Later standards relax that, which is why you may have seen it done elsewhere.

Follow the counter pattern in §6.1, and the drive-low-then-pulse shape of `data_ready` in §6.5.



Self-check: name your entity `counter`, with generic `RADIX` (`natural range 1 to 2**16`, default `10`), inputs `clock`, `reset_s2_n` and outputs `count` (`natural range 0 to RADIX-1`) and `tick` (`std_logic`), declared in that order; its testbench is in `exercises/counter`. It elaborates **two** instances, one at `RADIX = 10` and one at `RADIX = 4`, and runs each through two full cycles, checking that `tick` is a one-cycle pulse rather than a level and that it never fires part way through. An architecture that quietly hard-codes `10` passes the first instance and fails the second, which is the point of checking two.

EXERCISE SET 2

Shift Registers

Exercise 6.5 Tracing a SIPO register

PAPER

An 8-bit SIPO shift register starts at `00000000` and uses the idiom from §6.3:

```
shift_reg <= shift_reg(6 downto 0) & serial_in;
```

It receives the serial bit stream `1, 0, 1, 1` (one bit per clock edge, in that order) over four consecutive rising clock edges.

a) Write out the full 8-bit contents of `shift_reg` after each of the four clock edges.

b) After those four edges:

- Which four bits of `shift_reg` hold the received data?
- In what order do they appear, relative to the order the bits arrived in?

Exercise 6.6 Tracing a PISO register

PAPER

A 4-bit PISO register, following the idiom from §6.4, is:

- Parallel-loaded with `parallel_in = "1010"` (`load = '1'` for one clock edge).
- Then shifted with `shift = '1'` for the next four clock edges.
- With `serial_in` held at `'0'` throughout.

Remember the direction from §6.2:

- bits move toward the MSB.
- `serial_out` is bit 3, the value about to leave.
- `serial_in` enters at bit 0.

a) Write out the register's contents and `serial_out` at each step, one line per clock edge:

- The line for the load edge, before any shifting.
- Then one line for each of the four shift edges.

b) In what order do the four loaded bits appear on `serial_out`, relative to their positions in `parallel_in`?

c) What are the final contents of the register after all four shifts, and why?

Tip: You don't need real hardware to check Exercises 6.5 and 6.6 by hand. Walk through them exactly as you would trace a truth table: one clock edge, one line, at a time.

Exercise 6.7 SIPO and PISO in VHDL

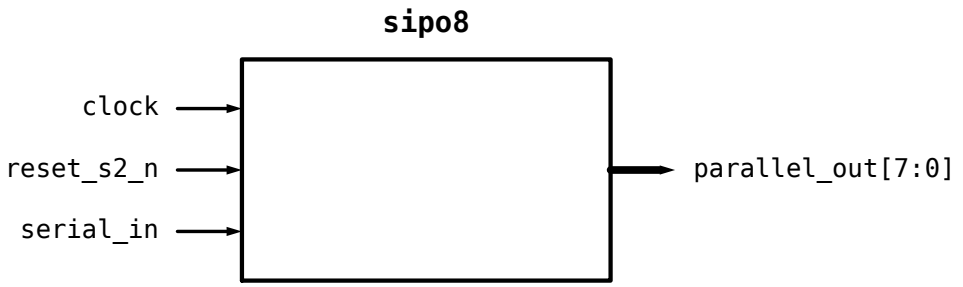
VHDL PAPER

Implement the two useful shift-register configurations from this chapter in VHDL.

a) Write an entity named `sipo8`, a serial-in/parallel-out register.
The entity must have:

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_s2_n`	in	`std_logic`	Active-low, **already-synchronized**: it comes from your `reset_sync` (Chapter 4's Exercise 4.8), never straight from a pin. Asynchronous, so it clears with no clock edge involved. Clears the whole register, so `parallel_out` reads `"00000000"` while it is held low.
`serial_in`	in	`std_logic`	The incoming bit.
`parallel_out`	out	`std_logic_vector(7 downto 0)`	The register's contents.

Follow the SIPO idiom from §6.3.



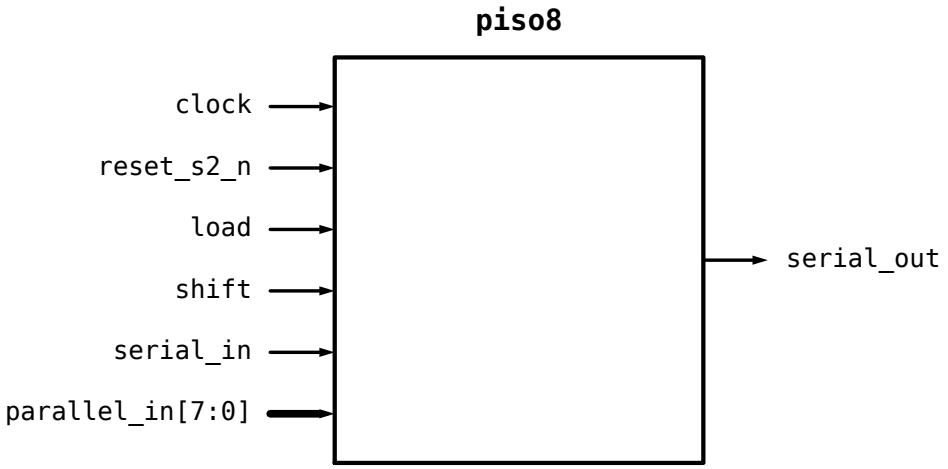
Self-check: name your entity `sipo8`, with ports `clock`, `reset_s2_n`, `serial_in` (in) and `parallel_out` (`std_logic_vector(7 downto 0)`, out), declared in that order; its testbench is in `exercises/sipo8`.

b) Write an entity named `piso8`, a parallel-in/serial-out register.
The entity must have:

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_s2_n`	in	`std_logic`	Active-low, **already-synchronized**: it comes from your `reset_sync` (Chapter 4's Exercise 4.8), never straight from a pin. Asynchronous, so it clears with no clock edge involved. Clears the whole register, so `serial_out` reads `'0'` while it is held low.
`load`	in	`std_logic`	Capture `parallel_in` on the next rising edge.
`shift`	in	`std_logic`	Shift one position on the next rising edge.
`serial_in`	in	`std_logic`	The bit fed in at bit 0 on a shift.
`parallel_in`	in	`std_logic_vector(7 downto 0)`	The value captured on a load.
`serial_out`	out	`std_logic`	The register's top bit.

Follow the PISO idiom from §6.4:

- On a load edge (`load = '1'`), capture `parallel_in`.
- On a shift edge (`shift = '1'`), move every bit one position toward the MSB, feeding `serial_in` in at bit 0.
- Drive `serial_out` from bit 7, so a loaded byte leaves most-significant bit first.
- `load` takes priority over `shift` when both are high.
- With both low, the register **holds**: nothing moves and nothing is captured. Its testbench checks this, so a design that shifts unconditionally fails.



Self-check: name your entity `piso8`, with ports `clock`, `reset_s2_n`, `load`, `shift`, `serial_in`, `parallel_in` (`std_logic_vector(7 downto 0)`) and `serial_out` (out), declared in that order; its testbench is in `exercises/piso8`. Keep this module: Chapter 8's transmitter capstone (Exercise 8.7) does not instantiate it, but it is worth rereading there for the idiom, since that machine shifts the byte out inline.

Re-derive each idiom from the chapter rather than copying `serial_rx8`'s shift line.

c) For each of the four shift-register configurations, name one task it is the natural choice for:

- SISO (serial-in/serial-out).
- SIPO (serial-in/parallel-out).
- PISO (parallel-in/serial-out).
- PIPO (parallel-in/parallel-out).

Follow the summary table in §6.2.

EXERCISE SET 3

Reading the Serial Receiver

The worked example, `serial_rx8` (`lectures/L06/serial_rx8/serial_rx8.vhd`), ships a reference testbench. Run it from the example's own directory before starting:

```
cd lectures/L06/serial_rx8
ghdl -a --std=93 serial_rx8.vhd serial_rx8_tb.vhd
ghdl -e --std=93 serial_rx8_tb
ghdl -r --std=93 serial_rx8_tb --assert-level=error --stop-time=10ms
```

Exercise 6.8 Accounting for the receiver

PAPER VHDL

Account for the receiver's behaviour, referring to §6.5.

a) The bit stream `1, 0, 1, 1, 0, 0, 1, 0` arrives, one bit per enabled clock edge:

- What is on `data_out` in the cycle where `data_ready` pulses?
- Which arriving bit ends up in `data_out(7)`, and which in `data_out(0)`?

b) `shift_enable` goes low for three clock cycles after the fourth bit, then high again for the remaining four:

- Does the receiver still produce the same byte? Explain what the bit counter does during the gap.
- What would go wrong instead if `shift_enable` gated only the shift register and not the counter?

c) Make each of the following changes to your own copy of `serial_rx8.vhd`, one at a time, and **predict** what the reference testbench will report before you run it. Then run it and account for the result:

- Change the shift line to the mirror-image direction, `shift_reg <= serial_in & shift_reg(7 downto 1);`.
- Remove the unconditional `data_ready <= '0';` from the top of the clocked branch.
- Change the bit-counter comparison from `BIT_WIDTH - 1` to `BIT_WIDTH - 2`, that is from 7 to 6.

Each of these is a real bug someone has shipped. For each one, say which of the testbench's checks catches it, and what the failure would have looked like on a real serial link if nothing had caught it.

CHAPTER 7

Timers

A counter with a target count, drawn by hand before it is written.

IN THIS CHAPTER

- Timers: a counter that raises a flag once it reaches a target count.
- Turning clock ticks into real time, and the equality comparator that does it.
- The same timer in VHDL, made reusable by a `generic` rather than by rewiring.
- Composing a design out of pre-built modules instead of rewriting their logic.
- `walking_led` demonstrated on the DE0-CV: a single lit bit walking a row of LEDs.

This chapter builds a timer twice and then puts it to work. The first is a 4-bit timer in CircuitVerse: Chapter 6's counter, plus an XNOR equality comparator and the clear that makes it repeat, watched as `timeout` fires once every eleven edges, for exactly one edge. The second is the same timer in VHDL, as a reusable module with its target count as a `generic`. Then comes `walking_led`: the timer just written, plus Chapter 4's `reset_sync` and `button_sync` unchanged, plus a loop to rotate the pattern, and onto the board. Almost no new logic: that part is about composing what already exists.

Before the timer in §7.2 runs, predict which count `timeout` fires on, and why it is every eleven edges rather than every ten. The off-by-one is the whole reason the module's generic is named `TICK_COUNT` and not `PERIOD`.

`walking_led` is the payoff of the last four chapters: three modules, two of them carried unchanged from Chapter 4 and one written in this chapter, composed into something you can watch from the back of the room. A timer is a counter plus one comparison, and this chapter assumes the counter, so be comfortable with §6.1 before you start. The synchronizers are not re-explained here beyond a short recap. Running the exercises' testbenches, including the multi-file `ghdl -a` line `blinker` and `walking_led` need, is covered in §2.8, and the Quartus project setup, pin assignment, compilation and programming used for the board demonstration are in Appendix B.

7.1 Timers: A Counter with a Target

A **timer** answers a different question than the counter from §6.1. A counter answers "what is the current count?"; a timer answers "has a certain amount of time passed?"

It does that with three parts: an internal counter invisible from outside, a target value `TICK_COUNT` fixed per instance, and a comparison that when true clears the counter and pulses an output flag `timeout` for one clock cycle.

Because the system clock ticks at a known, fixed frequency, counting ticks is the same as measuring elapsed time. On the DE0-CV that is 50,000,000 ticks per second:

```
TICK_COUNT = seconds x 50,000,000 - 1
```

The `- 1` is there because the counter runs `0` through `TICK_COUNT` inclusive, so a full period is `TICK_COUNT + 1` cycles, not `TICK_COUNT`. Exercise 7.3 turns on exactly that, so it is worth reading off this formula rather than the round number beside it:

- `TICK_COUNT = 49,999,999`: timeout once per second.
- `TICK_COUNT = 24,999,999`: twice per second.
- `TICK_COUNT = 4,999,999`: ten times per second.

The rest of this chapter writes the round `50,000,000` and `5,000,000` where the exact cycle count does not matter: one tick in fifty million is between 0.02 and 0.2 ppm at these values, far below the board oscillator's own tolerance.

This is the inverse of a blocking `_delay_ms()`. A blocking delay stops the whole program to wait; a timer runs permanently in parallel with everything else and simply raises a flag, and nothing in the design is ever blocked waiting for it.

7.2 Building a Timer by Hand in CircuitVerse

Before writing any VHDL, build a timer as a gate network, exactly as you did for the combinational circuits of Chapters 1 and 2 and the flip-flops of Chapter 3. A timer is small enough to draw in full, and drawing it is what makes §7.3's VHDL read as a description of something you already understand rather than a new idiom to memorize.

Build it at a size you can watch: a **4-bit counter** with `TICK_COUNT = 10`, which is `1010`. The real design counts to 50,000,000; nothing about the structure changes, only the width.

Start from the counter you already built in §6.1: a 4-bit register `Q0` to `Q3` sharing one clock, an adder and a constant `1` with the sum fed back to the `D` inputs, and nothing else. If you did not keep it, it is three elements and about five minutes to redraw. Confirm it still counts and wraps before adding anything, so that when the timer misbehaves later you know the counter underneath is sound.

Everything here adds **two things on top of that**: a comparison saying "the count has arrived", and a clear that makes it repeat. That is the entire difference between a counter and a timer.

Add the comparison. You want `timeout` high exactly when the four bits spell `1010`, and the clean way to build "are these two numbers equal?" is one **XNOR gate per bit**. An XNOR outputs `1` when its inputs are the *same*, which is §1.1's XNOR column read as an equality test. Wire four, each comparing a counter output against the corresponding target bit (`Q3` against `1`, `Q2` against `0`, `Q1` against `1`, `Q0` against `0`), driving the target side from a CircuitVerse constant rather than a wire you might later mistake for a signal. Then AND the four outputs together; that single AND gate is `timeout`.

Writing $\odot$ for XNOR, which is the one operator §1.2 gave no symbol to:

$$timeout = (Q3 \odot 1) \cdot (Q2 \odot 0) \cdot (Q1 \odot 1) \cdot (Q0 \odot 0)$$

All four bits must match at once, which is what "the counter has reached `1010`" means. This is a **general equality comparator**, worth recognizing as its own building block: the same four XNORs plus an AND compare against any 4-bit value, and only the constants change.

Add the clear, so the timer restarts rather than running on: feed `timeout` back to force the counter to `0000` on the next edge, gating each flip-flop's `D` input so `timeout = 1` overrides the adder's output. Without it the counter keeps going to `1111` and wraps on its own, so `timeout` fires every sixteen edges rather than every eleven. Both are periodic; only one has the period you asked for.

Then confirm, one clock edge at a time:

- The counter advances `0000` through `1010`, and `timeout` is low for every count except `1010`. Watch the four XNOR outputs as it counts and notice how often three are high at once. Three matching bits is not a match; the AND gate is what insists on all four.
- On the edge after `timeout` fires the counter is back at `0000` and `timeout` is low, so it is high for exactly one clock cycle, never two.
- The full cycle is therefore eleven edges, counts `0` through `10` inclusive, repeating indefinitely. That is §7.1's `TICK_COUNT + 1` off-by-one, and this is the easiest place in the book to see it, because you can count the edges by eye.

The drawing in Figure 7.1 is cropped to the part this section adds. It labels the counter's bits `counter[3]` down to `counter[0]` where the text above calls them `Q3` to `Q0`, and the increment feeding them is drawn as gate-level carry logic rather than the single adder block §6.1 used, because that is what CircuitVerse gives you when you build it a bit at a time. The four XNORs, their constants `1`, `0`, `1`, `0`, and the AND that combines them are the whole of what is new here.

Two things are worth noticing before the VHDL, because both are what the language buys you:

- **The comparator is hard-wired to one target.** Changing `TICK_COUNT` from `10` to `12` means going back into the drawing and flipping two constants. It is a small edit, but it is an edit to the *circuit*, and every instance needs its own copy. In §7.3 the target becomes a generic.

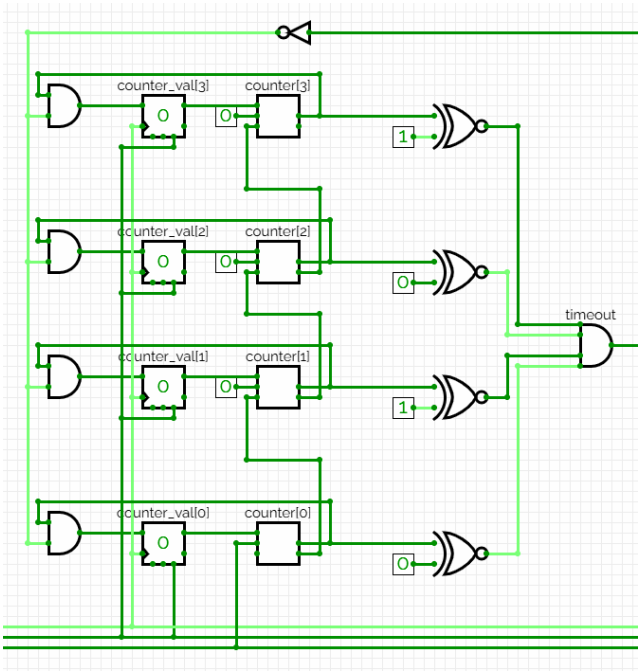


Figure 7.1 The comparison logic: four XNORs against the constants `1`, `0`, `1`, `0`, ANDed into `timeout`.

- **The width is hard-wired too.** Counting to 50,000,000 needs 26 flip-flops and a 26-bit comparator, 26 XNORs across 52 inputs: the same circuit, an unreasonable drawing. In §7.3 it is the same six lines of VHDL either way.

That is the trade this book makes constantly: draw the circuit once, at a size you can see, to know what the tool is building for you, then let the tool build it at a size you could not have drawn.

7.3 The `timer` Module in VHDL

The same circuit as a reusable module. The course builds it live, and Exercise 7.4 asks you to write it yourself afterwards, so it is printed here in full rather than shipped as a file:

```
entity timer is
    generic(TICK_COUNT: natural := 50_000_000);
    port(clock, reset_s2_n, enable: in std_logic;
         timeout                  : out std_logic);
end entity;
```

```
architecture behaviour of timer is
-- Internal tick counter.
signal counter: natural range 0 to TICK_COUNT;
begin
    process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            timeout <= '0';
            counter <= 0;
        elsif (rising_edge(clock)) then
            timeout <= '0';
            if (enable = '1') then
                if (TICK_COUNT > counter) then
                    counter <= counter + 1;
                else
                    timeout <= '1';
                    counter <= 0;
                end if;
            end if;
        end if;
    end process;
end architecture;
```

- `TICK_COUNT` is a **generic**, not a constant baked into the architecture, so the same entity serves any frequency by instantiating it with a different `generic map`. The internal counter's range is written in terms of it, so the register is sized to the target rather than to whatever an unconstrained `natural` would give, the same point §6.1 makes about `natural range 0 to 15`. A generic can size a signal precisely because it is fixed before synthesis runs.
- `reset_s2_n` is the already-*synchronized* reset (§7.4), never the raw asynchronous `reset_n`. A timer never touches a raw asynchronous input directly.
- `enable` **freezes** the timer rather than resetting it. While it is `'0'`, `counter` holds its value and `timeout` stays low, and a disabled timer resumes from where it left off rather than restarting. Chapter 8's `fsm_led` depends on this.
- `timeout` is a single-cycle pulse. The `else` branch that sets it is reached on only one edge: the edge at which `counter` is already sitting at `TICK_COUNT` and is cleared back to `0`. On the following edge the unconditional `timeout <= '0';` runs with nothing after it to override, so the pulse is exactly one cycle wide.

Compare this against §7.2's gate network. The logic is the same, and §7.2 already named what the generic buys. One difference it did not: §7.2's `timeout` is a combinational decode, high during the cycle the count reads `1010`, while this one is registered, high one cycle

later, during the cycle `counter` reads `0`. Same period, different phase, and different glitch behaviour, which is the registered-versus-combinational point §8.5 returns to.

7.4 Composing a Full Timer Circuit

A timer alone only pulses `timeout`; something has to react to that pulse, and a button has to control the timer. Reusing Chapter 4's double-flop synchronizer, a complete button-controlled, timer-driven LED circuit looks like Figure 7.2.

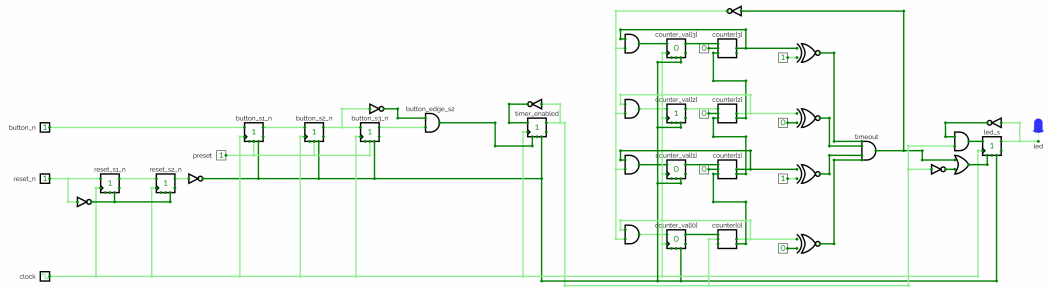


Figure 7.2 The full circuit at a glance: synchronized button, timer, and toggled LED, with the detail readable in the two closer views of Figures 7.3 and 7.4.

Read that one as a floor plan rather than as wiring. It is a wide circuit, so at page size its labels are small; what it is for is showing how few pieces there are and roughly where each one sits. The two closer views below are where the signal names are legible, and between them they cover everything in it.

Left to right:

- **Synchronization and edge detection** (Figure 7.3). `button_n` passes through two flip-flops for metastability protection, then a third lets the circuit compare "now" against "previous" and detect a falling edge, producing a one-cycle `button_edge_s2` pulse.
- **The timer** (§7.3), enabled or disabled by `timer_enabled`, which is itself toggled by `button_edge_s2`.
- **Toggling the LED** (Figure 7.4). The timer's `timeout` pulse flips the LED's state each time it fires, and the LED is forced off whenever the timer is disabled or the circuit is reset.

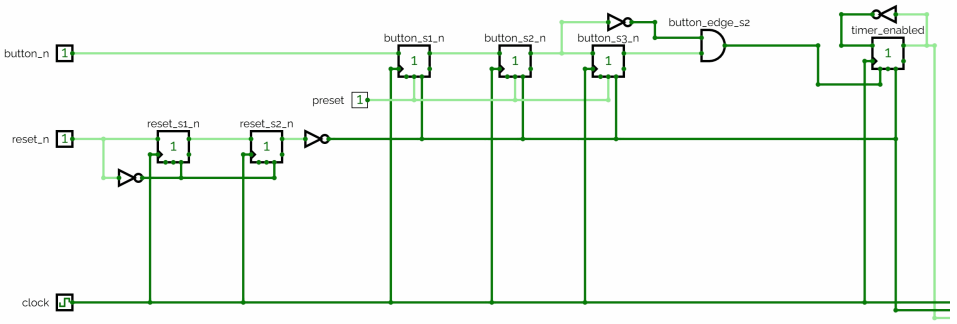


Figure 7.3 The synchronizer and falling-edge detector: `button_n` through two flip-flops and a third for the previous value, ANDed into `button_edge_s2`, which toggles `timer_enabled`.

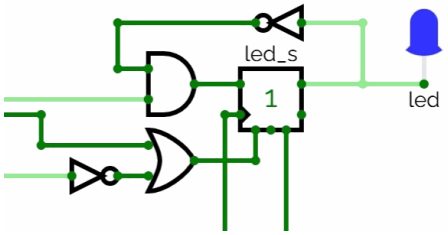


Figure 7.4 The LED toggle logic, driven by the timer's `timeout` pulse and forced off while the timer is disabled.

Two patterns matter here, independent of this circuit:

- **Every asynchronous input is synchronized before it touches any other logic**, and every enable/toggle signal derived from a button is a synchronous signal from that point on. You reuse this unchanged in §7.5 and again for Chapter 8's state machines.
- **One clock, and everything else is an enable.** `timeout` paces the LED by being a one-cycle *enable* on a flip-flop that runs on the system clock, not by being wired into a clock input. It is tempting to route a counter or timer output into a clock port to "slow things down"; do not. A design with one clock is a design whose timing the tool can analyze, and it is the single most common mistake a programmer makes writing a baud generator.

7.5 FPGA Implementation: A Walking-LED Shift Register

`walking_led` is the first design that composes three chapters' building blocks into one hardware-demonstrable circuit: a single lit LED walking along a row, one position per timer tick, started and stopped with a button. The course builds it live, and Exercise 7.6 asks you to write it yourself afterwards, so it is printed here in full rather than shipped as a file.

It writes almost no new logic, reusing unchanged `reset_sync` and `button_sync` (§4.5 and §4.6) to synchronize and edge-detect the button, `timer` (§7.3) to pace the rate, and the shift register from §§6.2–6.4 for the walking itself. That is what the last four chapters have been accumulating: a small set of modules that compose.

You write all three yourself: `reset_sync` and `button_sync` in Chapter 4's Exercises 4.8 and 4.9, `timer` in Exercise 7.4 of this chapter. Copy your own into the exercise directory before building this. Reusing a module means having one, and this is the first design that asks you for the ones you already built.

```
entity walking_led is
    generic(LED_COUNT : natural := 8;
            TICK_COUNT: natural := 25_000_000);
    port(clock, reset_n, button_n: in std_logic;
         led                     : out std_logic_vector(LED_COUNT-1 downto 0));
end entity;

architecture behaviour of walking_led is
signal reset_s2_n, button_edge_s2: std_logic;
signal button_n_v, button_edge_s2_v: std_logic_vector(0 downto 0);
signal shift_enable, shift_tick  : std_logic;
signal shift_reg: std_logic_vector(LED_COUNT-1 downto 0);

begin
    led <= shift_reg;

    button_n_v(0)     <= button_n;
    button_edge_s2 <= button_edge_s2_v(0);

    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    button_sync1: entity work.button_sync
        generic map(1)
        port map(clock, reset_s2_n, button_n_v, button_edge_s2_v);

    timer1: entity work.timer
        generic map(TICK_COUNT)
```

```
        port map(clock, reset_s2_n, shift_enable, shift_tick);

    SHIFT_ENABLE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            shift_enable <= '0';
        elsif (rising_edge(clock)) then
            if (button_edge_s2 = '1') then
                shift_enable <= not shift_enable;
            end if;
        end if;
    end process;

    SHIFT_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            shift_reg <= (0 => '1', others => '0');
        elsif (rising_edge(clock)) then
            if (shift_tick = '1') then
                shift_reg <= shift_reg(LED_COUNT-2 downto 0) & shift_reg(LED_COUNT↩
    ↪ -1);
            end if;
        end if;
    end process;
end architecture;
```

Design choices worth noting:

- `button_n_v`/`button_edge_s2_v` bridge a width difference only: `walking_led`'s button signals are single bits while `button_sync`'s ports are `COUNT`-wide vectors. The two concurrent assignments wire the scalar to element `0` and back. They cost no hardware.
- This is a PISO register (§6.4) in its simplest form: parallel-loaded exactly once, on reset, with a single lit bit at position 0, and from then on only shifting. There is no runtime `load`, because this design never loads a new pattern.
- It is a **circular** shift register. The shift is §6.2's expression unchanged, so the lit bit walks toward the MSB; the only difference from a plain PISO is what feeds bit 0, which is the bit falling off the top rather than a `serial_in` from elsewhere. A one-shot "shift a pattern out" register becomes an endlessly repeating pattern generator for free, just by recirculating its own output.
- `shift_enable` is §7.4's toggle flip-flop reused verbatim: the button's synchronized edge pulse flips a single bit, and that bit becomes the timer's `enable`.

- `shift_tick` is the timer's `timeout` pulse, reused as the shift register's clock enable rather than driving an LED toggle. Same building block, different consumer, and exactly the "one clock, everything else is an enable" rule from §7.4.
- Both `LED_COUNT` and `TICK_COUNT` are generics, so `generic map(4, 25_000_000)` gives a 4-LED row and `generic map(8, 5_000_000)` a faster 100 ms step, without touching the module.

On the board, as the course demonstrates it, the ports are assigned in the Pin Planner (Appendix B): `clock` to the 50 MHz oscillator, `reset_n` and `button_n` to two push buttons, and `led` to a row of onboard LEDs. Press the button once to start the bit walking, again to freeze it.

You can watch the same behaviour without a board by running the reference testbench, which overrides both generics so a full lap takes a handful of clock cycles:

```
cd lectures/L07/exercises/walking_led
cp ../../../L04/exercises/reset_sync/reset_sync.vhd .   # the three you wrote ↩
    ↪ yourself
cp ../../../L04/exercises/button_sync/button_sync.vhd .
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
                 walking_led.vhd walking_led_tb.vhd
ghdl -e --std=93 walking_led_tb
ghdl -r --std=93 walking_led_tb --assert-level=error --stop-time=10ms
```

7.6 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what separates a timer from a plain counter, and why the difference is one comparison.
- Build a timer as a gate network: an equality comparator from one XNOR per bit, and the clear that makes it repeat rather than wrap.
- Implement a timer in VHDL as a reusable module, converting a period in seconds into a `TICK_COUNT`, and say why `timeout` is a one-cycle pulse rather than a level.
- Compose a design out of pre-built modules rather than rewriting them, and say why a button controlling a timer still has to pass through a synchronizer first.

Questions to test yourself

You should be able to explain:

1. In what sense is a timer "just a counter with a comparison"?
2. Why must the internal counter be cleared back to `0` on reaching its target, rather than left to keep counting?
3. Why is `timeout` a single-cycle pulse rather than a level that stays high?
4. In the CircuitVerse version, changing the target count means rewiring the comparator. What replaces that rewiring in the VHDL version, and when is its value fixed?
5. Why is it still necessary to run a button through a synchronizer before using its edge to enable a timer, even though the button has nothing to do with counting?
6. `walking_led` writes almost no logic of its own. Name the three modules it composes and which chapter each came from.

Looking ahead

Chapter 8 is about state machines:

- Finite state machines, the last building block in the book, designed by hand first: state diagram, state table, Karnaugh-derived logic, and a gate network in CircuitVerse, exactly the way this chapter's timer was drawn before it was written.
- Then in VHDL, where an enumerated type and a `case` statement replace the state table and the Karnaugh maps entirely.
- A timer is itself a state machine with two states and a single purpose. Chapter 8 generalizes that to circuits with many states and many transitions between them.

7.7 Exercises

How to work these. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.
How to check your work. Every exercise below that asks you to write a VHDL module ships a self-checking testbench under `lectures/L07/exercises` in the companion repository. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.
From this chapter on, designs reuse modules you wrote earlier, and those are not shipped a second time: each exercise says which of your own files to copy in and prints the `ghdl -a` line that needs them.
No FPGA board is needed for any exercise. The Quartus synthesis and board-programming steps are demonstrated in the course, and Appendix B shows them for a reader with a board; your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in

the companion repository, under `lectures/L07/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

Timers by Hand

Exercise 7.1 Retargeting the timer by hand

PAPER CIRCUITVERSE

Build a second timer by hand in CircuitVerse, following §7.2, but with a target count of `TICK_COUNT = 12` instead of the chapter's `10`.

a) Retarget the comparator:

- Write `12` in 4-bit binary.
- The four XNOR gates stay exactly where they are. Say which of their constant inputs change, and to what.
- State, in one sentence, why an equality comparator built from XNORs needs no gate added, removed or rewired to compare against a different value.

b) Build it:

- Reuse your 4-bit counter and comparator from §7.2, changing only the constants.
- Set the clock period to `1000 ms` and confirm `timeout` fires once every thirteen clock edges, for exactly one edge each time.

c) Now remove the clear, so that `timeout` no longer forces the counter back to `0000`:

- Predict, before running it, how often `timeout` fires now.
- Run it and confirm.
- Explain the result in terms of the counter's own wraparound (§6.1).

d) Deliberately break the comparator: disconnect one XNOR's output from the AND gate and tie that AND input high instead.

- Predict which counts now assert `timeout`, before simulating.
- Confirm by stepping the counter through a full cycle.
- This is what "three of the four bits match" looks like, and it is the most common way a hand-drawn comparator goes wrong.

Tip: counting `0` through `12` inclusive is thirteen edges, not twelve. §7.1's note on `TICK_COUNT + 1` is the same off-by-one, and it is worth getting straight here, where you can count the edges by eye, rather than later at 50 MHz.

Exercise 7.2 Reading the timer module

PAPER

Answer each of the following in one or two sentences, referring to the `timer` module in §7.3:

a) Why is `timeout` a single-cycle pulse, rather than a signal that stays high once the target count is reached?

b) While a running timer's `enable` input is held low, what happens to its internal counter from one clock edge to the next: does it reset, pause, or keep counting?

c) Why does the `timer` take `reset_s2_n` (the synchronized reset) rather than the raw `reset_n` as its reset input?

d) The CircuitVerse timer in Exercise 7.1 needed rewiring to change its target count, while the VHDL module needs no edit at all. What replaces the rewiring, and at what point is its value fixed?

EXERCISE SET 2

Timers in VHDL

Exercise 7.3 Tick counts at 50 MHz

PAPER

The DE0-CV board's system clock is `50 MHz`. For each target below, calculate the `TICK_COUNT` generic you would pass to the `timer` module from §7.3:

a) A `timeout` pulse once per second.

b) A `timeout` pulse every `250` ms.

c) A `timeout` pulse twenty times per second (`20 Hz`).

Tip: The relationship between a period and its tick count is:

$$TICK_COUNT = seconds \times 50{,}000{,}000 - 1$$

The `- 1` is the same off-by-one Exercise 7.1 turned on: the counter runs `0` through `TICK_COUNT` inclusive, so a period is `TICK_COUNT + 1` clock cycles, not `TICK_COUNT`.

Exercise 7.4 Writing the timer

VHDL PAPER

Write the `timer` module itself, as an entity named `timer`.

The course builds it live and §7.3 prints it in full, so this is not a puzzle. Write it anyway, without copying: every design from here to the end of the book instantiates this module, including both capstones, and it is worth having built the thing you are about to reuse six times.

The entity has one generic:

Generic	Type	Default	Description
`TICK_COUNT`	`natural`	`50_000_000`	Ticks to count before a timeout. One second at the DE0-CV's `50 MHz` clock, rounded: by Exercise 7.3's rule the exact value is `49_999_999`, and 20 ns of error in a second is not worth the less readable constant.

and these ports:

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_s2_n`	in	`std_logic`	Active-low, already-synchronized reset. Asynchronous: it clears the counter and `timeout` with no clock edge involved.
`enable`	in	`std_logic`	Timer enable. While low the counter **holds** its value, and resumes from there rather than restarting.
`timeout`	out	`std_logic`	One-cycle pulse, asserted every `TICK_COUNT + 1` cycles while enabled.

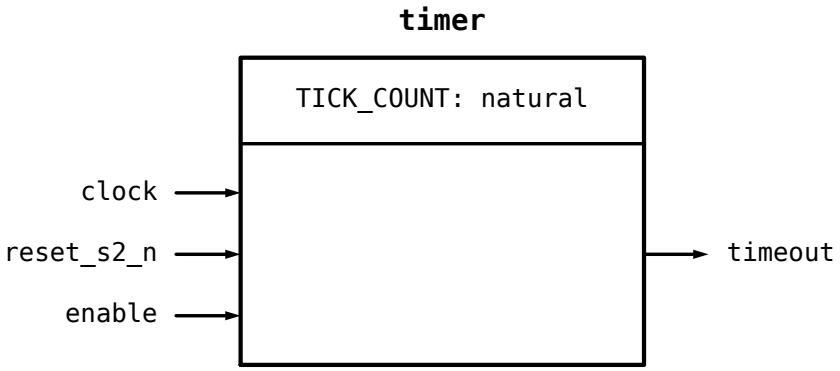
The architecture must:

- Declare the internal counter as `natural range 0 to TICK_COUNT`, so the register is sized by the generic rather than left unconstrained.
- Use one process, sensitive to `clock` and `reset_s2_n`, following the clocked-process template from §3.6.
- Clear both the counter and `timeout` on reset, outside the clocked branch.
- Drive `timeout` low unconditionally at the top of the clocked branch, and high only on the edge that clears the counter. That default-then-override is what makes the pulse exactly one cycle wide rather than two.

Then answer, in a sentence each:

- `enable = '0'` must **pause** the counter, not clear it. Suppose you cleared it instead. Predict what the reference testbench reports, and which of the two properties above it is measuring when it does. Then run it and see. (Chapter 8's `fsm_led` depends on the pause, which is why the testbench bothers.)
- The counter counts `0` through `TICK_COUNT` inclusive, so a period is `TICK_COUNT + 1` cycles. Where in your code is that "+ 1" actually written? It is not a literal anywhere.

- §7.2's hand-drawn `timeout` is a combinational decode of the count; yours is registered. Both have the same period. Which cycle does each one fire on, and why does the difference matter for anything downstream that samples `timeout`?



Self-check: name your entity `timer`, with generic `TICK_COUNT` (`natural`, default `50_000_000`), inputs `clock`, `reset_s2_n`, `enable`, and output `timeout`, declared in that order; its testbench is in `exercises/timer`. It runs three full periods, so a one-shot passes nothing, and pauses mid-count to confirm the count resumed rather than restarted.

Keep this file. Exercise 7.5 below reuses it, so does `walking_led` in Exercise 7.6, and so do both Chapter 8 capstones. Every later exercise that needs a timer asks you to copy this one in.

Exercise 7.5 A one-second blinker

VHDL PAPER

Write an entity named `blinker` that blinks an LED on and off once per second, by reusing the `timer` module you just wrote rather than counting clock cycles itself.

The entity has one generic:

Generic	Type	Default	Description
`TICK_COUNT`	`natural`	`25_000_000`	A half-second at the DE0-CV's `50 MHz` clock. Expose it as a generic rather than hard-coding the count, so a testbench can shorten the blink; the FPGA build just uses the default.

and these ports:

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_n`	in	`std_logic`	Active-low reset, raw from the outside world. Asynchronous: assert it and `led` goes out immediately, without waiting for a clock edge.
`led`	out	`std_logic`	Cleared to `'0'` while the reset is asserted, and left there until the first timeout after the reset is released.

Note that this entity takes the **raw** `reset_n` rather than an already-synchronized `reset_s2_n`. Synchronizing it is part of the exercise, and it is what §4.5 means by "exactly one module in a design is the exception": here that module is `blinker` itself.

The architecture must:

- Instantiate your `reset_sync` from Chapter 4's Exercise 4.8, and use its `reset_s2_n` output as the reset for everything inside.
- Instantiate the `timer` module from §7.3:
  - Pass your own `TICK_COUNT` generic straight through to the timer's `TICK_COUNT`.
  - Hold its `enable` input high.
- Toggle `led` in a synchronous process whenever the timer's `timeout` pulse fires.

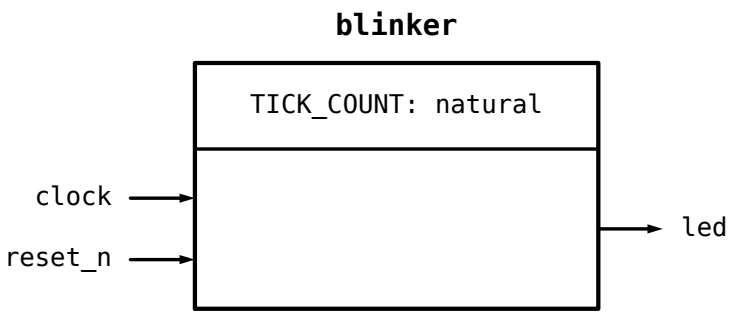
Do not reimplement the counting logic, or the synchronizer, yourself.

Instead:

- Reuse `timer` and `reset_sync` unchanged, via a `generic map` and `port map`, following the composition pattern in §7.4.
- Explain why a **half**-second timer period produces a **one**-second blink cycle (one full on-then-off period).

Then answer, in a sentence each:

- The LED must be off during reset, and the testbench checks that it goes off with no clock edge in between. Which half of the assert-async-release-sync pattern does that check, and which half does it not?
- Your `timer` is reset by `reset_s2_n`, two edges after `reset_n` is released. What would go wrong if you fed it the raw `reset_n` instead? Name the failure, not just the rule.



Self-check: name your entity `blinker`, with generic `TICK_COUNT` (`natural`, default `25_000_000`), inputs `clock`, `reset_n`, and output `led`, declared in that order; its testbench is in `exercises/blinker`. **Copy your own `reset_sync.vhd` and `timer.vhd` into that directory** before you build; neither is shipped there, because you wrote both. It overrides `TICK_COUNT` with a small value and checks that the LED toggles repeatedly, holds for exactly one timer

period between toggles, and that the first toggle arrives late by your synchronizer's two edges (see §2.8):

```
cd lectures/L07/exercises/blinker
cp ../../../L04/exercises/reset_sync/reset_sync.vhd .   # the two you wrote ↪
    ↪ yourself
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd timer.vhd blinker.vhd blinker_tb.vhd
ghdl -e --std=93 blinker_tb
ghdl -r --std=93 blinker_tb --assert-level=error --stop-time=10ms
```

EXERCISE SET 3

The Walking LED

A single lit LED walking along a row, one position per timer tick, started and stopped with a button. The course builds it live, and §7.5 writes it up.

This is the payoff of the last four chapters, and it writes almost no logic of its own: three modules, two of them carried unchanged from Chapter 4 and one written in this chapter, composed. Exercise 7.6 builds it, and Exercises 7.7 and 7.8 modify what you built.

Exercise 7.6 The walking LED

VHDL PAPER

Write an entity named `walking_led`.

Like the `timer` of Exercise 7.4, the course builds it live and §7.5 prints it in full, so this is not a puzzle either. Write it anyway, without copying: composing three modules you wrote yourself is the whole point of the exercise, and the capstones in Chapter 8 ask for it again at twice the size.

The entity has two generics:

Generic	Type	Default	Description
`LED_COUNT`	`natural`	`8`	How many LEDs the bit walks along. The row is this wide, and the walk wraps after this many steps.
`TICK_COUNT`	`natural`	`25_000_000`	Passed straight through to your `timer`, so the bit moves one position per timer period: a half-second step at the DE0-CV's `50 MHz` clock.

and these ports:

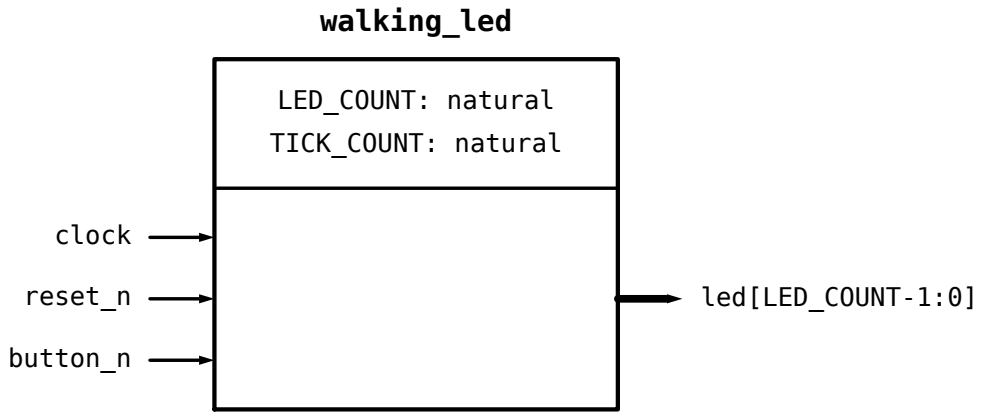
Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_n`	in	`std_logic`	Active-low reset, raw from the outside world. Synchronizing it is your job, as it was in `blinker`.
`button_n`	in	`std_logic`	Active-low push button, raw. Each press starts or stops the walk.
`led`	out	`std_logic_vector( LED_COUNT-1 downto 0)`	The LED row. Exactly one bit is lit at any time; on reset it is bit `0`.

The architecture composes three modules you have already written, and adds two processes:

- Instantiate your **`reset_sync`**, and use its `reset_s2_n` as the reset for everything else in the design, including the two instances below.
- Instantiate your **`button_sync`** with `generic map(1)`, for a one-cycle pulse per press. Its button ports are `COUNT`-bit vectors while `button_n` here is a scalar, so bridge the width with a one-element `std_logic_vector(0 downto 0)` in each direction. §7.5 shows this.
- Instantiate your **`timer`** with your own `TICK_COUNT`, driving its `enable` from the shift-enable flag below and taking its `timeout` as the shift tick.
- A process toggling **`shift_enable`** on each button pulse, cleared to `'0'` on reset. The walk therefore starts stopped.
- A process holding the **shift register** that drives `led`: on reset, load a single lit bit at position `0` and clear the rest; on each timer `timeout`, rotate one step toward the MSB, wrapping the top bit back around into bit `0`. That is a PISO register (§6.4) with its own serial output wired back to its serial input, which is what turns a one-shot shift-out into an endless walk.

Then answer, in a sentence each:

- `shift_enable` drives the timer's `enable`, so stopping the walk stops the timer too. Exercise 7.4 made you show that `enable` **pauses** the count rather than clearing it. What does a reader see on the board, on the press that restarts the walk, if your timer clears instead?
- `button_sync` is instantiated with `COUNT = 1` here and with `COUNT = 2` in Chapter 8's `fsm_led`, from the same file with no edit. What would you have had to write instead if the width were fixed in the module rather than passed as a generic?



Self-check: name your entity `walking_led`, with generics `LED_COUNT` (`natural`, default `8`) and `TICK_COUNT` (`natural`, default `25_000_000`), inputs `clock`, `reset_n`, `button_n`, and output `led`, declared in that order; its testbench is in `exercises/walking_led`. It overrides the generics (`LED_COUNT = 4`, `TICK_COUNT = 3`) so a full lap takes a handful of clock cycles rather than two seconds, and it checks the reset pattern, that nothing moves before the first press, and that the shift count over a fixed window matches the pace the timer sets. Copy in the three modules it composes first:

```
cd lectures/L07/exercises/walking_led
cp ../../../L04/exercises/reset_sync/reset_sync.vhd .   # the three you wrote ↩
    ↪ yourself
cp ../../../L04/exercises/button_sync/button_sync.vhd .
cp ../timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
                 walking_led.vhd walking_led_tb.vhd
ghdl -e --std=93 walking_led_tb
ghdl -r --std=93 walking_led_tb --assert-level=error --stop-time=10ms
```

Exercise 7.7 Changing the walking LED

VHDL PAPER

Make the following changes one at a time, re-running the testbench after each. Parts **a**) and **b**) change only the testbench's constants — that they need no edit to your `walking_led.vhd` at all is the point of them. Part **c**) is the one that touches the module.

a) Change the walking speed:

- Edit the `TICK_COUNT` constant in the **testbench** from `3` to `6`.
- It should still pass, and this time because the testbench adapts with you: its observation window is a fixed `PULSES * 6` cycles, and it derives the shift count it expects from `TICK_COUNT`, one shift per `TICK_COUNT + 1` cycles.
- Now find where that adaptation stops meaning anything. Keep raising `TICK_COUNT` past the window length and work out what the expected count becomes, and what the check is then actually proving. A test that passes for the wrong reason is worth being able to recognize.
- Then explain why changing the tick count cannot break the *design*, only slow it down.

b) Change the width:

- Edit the testbench's `LED_COUNT` constant from `4` to `8`.
- It should still pass, with no change to your `walking_led.vhd` at all. Explain what the module would have needed to look like for this to require an edit.

c) Load two lit bits on opposite sides of the register on reset (e.g. `"10001000"` for `LED_COUNT = 8`), and let them walk in lockstep:

- Identify the single line in your `walking_led.vhd` you need to change.
- Before running anything, **predict** what the reference testbench will report.
- Then run it and see. Explain the result: is the design wrong, or is the testbench asserting something that is no longer the specification?

Tip: Both `LED_COUNT` and `TICK_COUNT` are generics, so parts **a**) and **b**) need no change to the module's body at all. That is the entire point of a generic: one module, many sizes, no edits.

Exercise 7.8 Reversing the walk

VHDL PAPER

The current design only ever walks in one direction (toward the MSB, wrapping the top bit back around to the bottom). Modify your `walking_led.vhd` so that a **second** button reverses the walking direction.

a) Add a second, synchronized button input:

- Route it through the same `button_sync` instance, passing `2` for its `COUNT` generic (`generic map(2)`, positionally, as everywhere else in this book).

b) Add a `direction` signal:

- Toggle it on the new button's edge, exactly the way `shift_enable` is toggled by the first button.

c) Make the rotation in `SHIFT_PROCESS` depend on `direction`:

- When `direction = '0'`, rotate toward the MSB (the book's convention from §6.2, and what you wrote in Exercise 7.6):

```
shift_reg <= shift_reg(LED_COUNT-2 downto 0) & shift_reg(LED_COUNT-1);
```

- When `direction = '1'`, rotate the other way, toward bit 0:

```
shift_reg <= shift_reg(0) & shift_reg(LED_COUNT-1 downto 1);
```

This is the one place in the book where a shift register deliberately runs against the convention: reversing direction is the entire point of the exercise. Note that the second expression is exactly the mirror image §6.2 warns you to watch for in other people's code.

d) Your entity's ports have now changed: `button_n` is a two-bit vector rather than a single bit.

- Explain why `walking_led_tb.vhd` can no longer be used against your version, and what precisely GHDL will complain about when you try.
- This is the ordinary cost of changing an interface, and it is worth feeling once: a testbench is written against a specific entity, and widening a port breaks every instantiation of it, including the ones you did not write.
- Describe, in prose, the sequence of presses you would use to convince yourself the design works, and what you would expect the LEDs to do at each step. That sequence is exactly what a testbench for the new entity would have to drive.

CHAPTER 8

State Machines

Designed by hand first, then expressed in the language.

IN THIS CHAPTER

- Finite state machines: states, transitions, inputs and outputs.
- Hand-designing a Moore machine: state diagram, state table, Karnaugh-derived logic.
- State encoding as a design decision, and why the worked machine uses a Gray code.
- The same machine in VHDL: an enumerated type and the `case` transition pattern.
- Why Moore is the default in this book.
- `fsm_led` on the DE0-CV: buttons cycling the LED off, blinking, on.
- Mealy machines (§8.5), and `seq_detect_mealy` to read and run.
- The course capstones, as exercises: a serial receiver and a serial transmitter, the two ends of one link.

This chapter designs one state machine by hand and then writes another in VHDL. The first is a Moore machine taken from its specification through a state diagram and a state table to Karnaugh-derived next-state and output logic, with no step skipped, and then realized as a gate network in CircuitVerse and stepped through its state cycle by hand: at each step, predict the next state from the diagram before the clock is advanced. The second is `fsm_led`, the three-state LED controller of §8.4, composed from Chapter 4's synchronizer and Chapter 7's timer. An enumerated `state_t` type and a `case`-statement process replace the state table and the Karnaugh maps entirely. It is a different, larger machine than the hand-designed one, deliberately, so the correspondence you are watching is structural rather than a transcription: every row you derived by hand in the state table becomes one branch of a `case`, and seeing that correspondence is worth more than either step on its own. The course then takes `fsm_led` onto the board.

Mealy machines are read rather than presented: §8.5 covers them, with `seq_detect_mealy` to read and run, and Exercise 8.4 has you write one. Reading it matters more than it might look. Everything else in the chapter is Moore, and Moore is the default in this book. §8.5 is where you find out what the alternative costs and buys, which is the only way that default becomes a choice rather than a habit.

The chapter assumes Chapter 2's Karnaugh maps and Chapter 3's flip-flops, and reuses Chapter 7's `timer` unchanged, together with Chapter 4's synchronizer widened to two buttons. Of the exercises, the first two are pen-and-paper, Exercise 8.5 is reading and running the worked machine, and the rest are VHDL. Exercises 8.6 and 8.7 are the **course capstones**: the two halves of one serial link. The receiver composes Chapter 4's `reset_sync`, Chapter 5's `sync` and Chapter 7's `timer` around a state machine that does its own shifting; the transmitter needs only the first and the last of those, mirrored. Budget real time for them, and design each state diagram on paper before writing any VHDL. Do the receiver first: it is the harder of the two, because it has to recover the bit timing it is given rather than define it. The modules they reuse are worth having open alongside them, and so are Chapter 6's `serial_rx8` and `piso8` for the shift idiom, though neither is instantiated: the capstones shift inside their own state machines.

8.1 What Is a Finite State Machine

A finite state machine (FSM) is a circuit whose behaviour depends not only on its current inputs but on a finite amount of *history*, summarized in a single value called its **state**. At any moment the machine is in exactly one state; on each clock edge it may move to a new one, decided by its current state and inputs; and its output is computed from that state.

None of this is new hardware. An FSM combines two things you already know: a **register** (Chapter 3) holding the current state, one flip-flop per state bit, and **combinational logic** computing the next state and the output from the current state and the inputs.

What is new is the discipline. Instead of ad hoc logic feeding an ad hoc set of flip-flops, you work in a fixed order: first draw a **state diagram**, a few named states with labelled transitions between them, and only once that is right derive the state table, then the next-state logic, then the gate network or VHDL.

Every synchronous circuit so far is technically a trivial state machine: a counter's state is its count, a shift register's is its bit pattern. What sets an FSM apart is that the state is not a count or a shifted pattern but an arbitrary label (`STATE_OFF`, `STATE_BLINK`, ...) whose meaning you choose and whose next value can depend on the input in any way you like.

Two flavours exist, and the distinction decides what an output may look at:

- **Moore machines**: the output depends *only* on the current state. That is the kind designed by hand here.
- **Mealy machines**: the output depends on the current state *and* the current input. §8.5 covers these briefly, in VHDL only, so you recognize one when you meet it and know the one-cycle difference.

8.2 Hand-Designing a Moore Machine: State Diagram, State Table, and Karnaugh-Derived Logic

The same process as every gate network in Chapters 1 and 2, applied to a circuit with memory. The worked example is a small, closed, four-state Moore machine: a circular counter through a Gray-code sequence, advanced one step per button press.

8.2.1 Specification

- Four states, each represented by two flip-flops `{Q1, Q2}`:

State	`{Q1, Q2}`
`STATE_0`	`00`
`STATE_1`	`01`
`STATE_2`	`11`
`STATE_3`	`10`

- A system clock `clock`, and an active-low reset `reset_n` returning the machine to `STATE_0`.
- An active-low push button `button_n`, from which a synchronized, edge-detected pulse `X` is derived exactly the way Chapter 4's double-flop synchronizer does. `X` is high for exactly one clock cycle on a falling edge of `button_n`.
- An output `Y`, connected to an LED, high only in `STATE_3`.
- The machine is **closed**: the state after `STATE_3` is `STATE_0`.
- It is a **Moore** machine: `Y` depends only on `{Q1, Q2}`, never on `X`.

The encoding is a Gray code (`00 -> 01 -> 11 -> 10 -> 00`), where consecutive states differ in exactly one bit. That is a deliberate choice, not a coincidence:

- With an ordinary binary count (`00 -> 01 -> 10 -> 11`), a transition like `01 -> 10` flips two bits at once, and the two flip-flops will not switch at *exactly* the same instant.
- That skew is harmless to the machine itself. Both flip-flops update on the same clock edge, and nothing samples the state register in between, so the machine never acts on the intermediate value. This is the same argument as the critical path of §5.3: as long as everything settles before the next edge, what happens during the settling does not matter.
- What the skew *can* disturb is anything reading the state combinationally. An output decoded straight from the state bits, like `Y` below, can glitch briefly while they skew. So can a value sampled by a *different* clock domain, which is the case §4.3 warned about.
- A Gray code removes both, since only one bit ever changes per transition. That is why it matters most on a counter crossing clock domains, and least on a machine whose output is registered.

Encoding is a design decision in general, and this is the one place in the book you make it by hand. Binary uses the fewest flip-flops; Gray minimizes bits changing per transition; **one-hot** spends one flip-flop per state so the next-state logic is a single wide OR per state, which is often fastest on an FPGA where flip-flops are plentiful. Once you write the machine in VHDL as an enumerated type (§8.4), you hand this decision to the synthesis tool, which picks an encoding for you and re-encodes freely unless you tell it otherwise.

8.2.2 State diagram

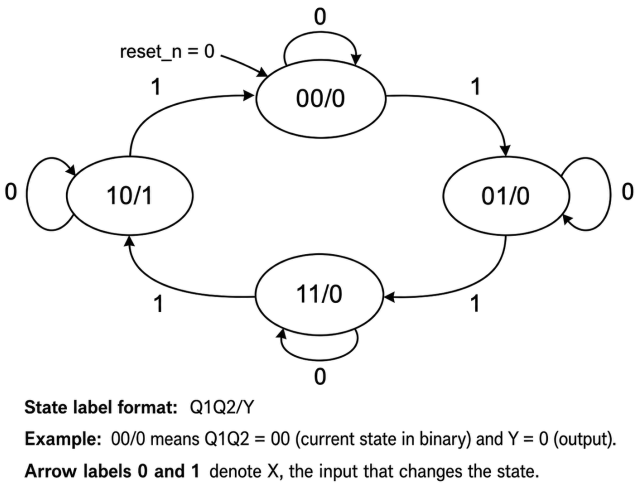


Figure 8.1 The four-state Gray-code Moore machine: each state holds on `X = 0` and advances on `X = 1`, and each bubble reads `Q1Q2/Y`.

Each state bubble shows the current state and the output as `Q1Q2/Y`, and each arrow is labelled with the input `X` that triggers it. Each state loops back to itself while `X=0`, and a synchronized button pulse (`X=1`) advances to the next state, wrapping from `STATE_3` back to `STATE_0`.

8.2.3 State table

Every combination of the current state and input, giving the next state `{Q1+, Q2+}` and output `Y`:

Q1	Q2	X	Q1+	Q2+	Y
0	0	0	0	0	0
0	0	1	0	1	0
0	1	0	0	1	0
0	1	1	1	1	0
1	0	0	1	0	1
1	0	1	0	0	1
1	1	0	1	1	0
1	1	1	1	0	0

8.2.4 Karnaugh-derived logic

Treating `Q1+` and `Q2+` each as a Boolean function of `{Q1, Q2, X}` and running them through a Karnaugh map, the same technique as Chapter 2:

```
Q1+ = Q1X' + Q2X
Q2+ = Q2X' + Q1'X
```

Notice the shape of both:

- each is "hold when `X` is `0`, take something from the other bit when `X` is `1`."
- the hold halves, `Q1X'` and `Q2X'`, really are mirror images of each other.
- the advance halves are `Q2X` and `Q1'X`, and that complement on the second one is not a typo: it is what makes the sequence a Gray code rather than a plain rotation.

The output `Y` reduces further, straight from the state table, with no dependency on `X` at all, confirming this is a valid Moore machine:

```
Y = Q1Q2'
```

8.2.5 Realizing it as a gate network

`{Q1, Q2}` are each a D flip-flop, exactly like Chapter 3's register: `Q1+`/`Q2+` feed the D inputs and `Q1`/`Q2` are read back from the Q outputs. `Y` is a single two-input AND gate with one input inverted, reading the current state. That is the entire machine: two flip-flops and a handful of gates computing `Q1+`, `Q2+` and `Y` from `{Q1, Q2, X}`.

8.3 From Gate Network to CircuitVerse

Before writing any of this in VHDL, realize it by hand in CircuitVerse (`https://circuitverse.org/simulator`), exactly as every gate network in Chapter 1 was: double-flop synchronize `button_n` and `reset_n`, edge-detect the synchronized button to produce `X`, wire `Q1+`/`Q2+` into a pair of D flip-flops, and wire `Y` straight into the LED.

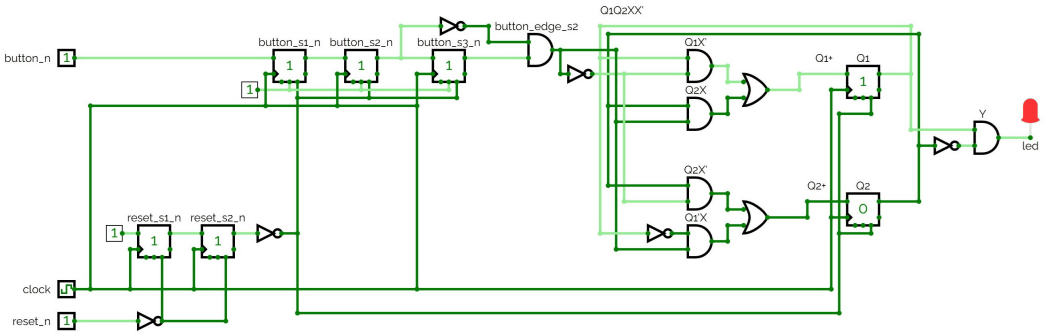


Figure 8.2 The four-state Moore machine as a gate network in CircuitVerse, with the double-flop synchronization and edge detection of `button_n` on the left and `Y` driving the LED on the right.

Reading it left to right:

- `button_n` passes through three synchronizer flip-flops (`button_s1_n`/`button_s2_n`/`button_s3_n`).
- An AND gate derives `button_edge_s2` (this circuit's `X`) from the second and third stages.
- The four vertical rails labelled `Q1Q2XX'` carry the state bits and the edge pulse across to the gates on the right, which implement §8.2's `Q1+`/`Q2+` equations.
- The two labelled flip-flops store `Q1`/`Q2` and feed the final AND/NOT gates producing `Y`, wired to `led`.
- `reset_n` is synchronized by its own pair of flip-flops at bottom left, exactly like every other synchronized reset in this book.

Open the live circuit yourself: `lectures/L08/appendix/images/fsm_gate_network.cv`. Step it by hand, one button press at a time, and confirm it visits `STATE_0 -> STATE_1 -> STATE_2 -> STATE_3 -> STATE_0` in order, with the LED lighting only on `STATE_3`.

8.4 The Same Machine in VHDL

That was the last time in this book you will design a machine by hand. Everything from here writes it directly in VHDL, which is worth having earned rather than started with: the synthesis tool does exactly what you just did on paper, and when a state machine misbehaves you will be debugging the diagram, not the VHDL.

The translation replaces the explicit state table and Karnaugh equations with an **enumerated type** naming each state, a **case statement** describing each state's transitions, and the synthesis tool left to work out the flip-flop encoding and gate-level logic.

8.4.1 The enumerated state type

Suppose a machine with three states controlling an LED: `STATE_OFF`, `STATE_BLINK` (blinking at a fixed rate) and `STATE_ON`. VHDL declares exactly those three values as a new type:

```
type state_t is (STATE_OFF, STATE_BLINK, STATE_ON);
signal state: state_t;
...
state <= STATE_ON;
```

8.4.2 The case-statement pattern

The machine is almost always a synchronous process updating `state` on the rising edge via a `case`, one branch per state, with an asynchronous reset returning it to a known starting state. Assuming `to_next_state` and `to_prev_state` are synchronized, edge-detected pulses produced exactly like `X` in §8.2 and §8.3:

```
process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        -- On reset, go to STATE_OFF.
        state <= STATE_OFF;
    elsif (rising_edge(clock)) then
        case (state) is
            when STATE_OFF =>
                if (to_next_state = '1') then
                    state <= STATE_BLINK;
                elsif (to_prev_state = '1') then
                    state <= STATE_ON;
                end if;
            -- STATE_BLINK and STATE_ON follow the same shape.
            when others =>
                state <= STATE_OFF;
        end case;
    end if;
end process;
```

`case (state) is ... end case;` must cover every value `state_t` can take, and `when others` keeps it exhaustive if you add a state later and forget a branch.

What `when others` is *not* is a safety net against a corrupted state register. Once every state has its own branch, as in `fsm_led.vhd` below (the listing above abbreviates two of them away, so *its* `when others` is reachable and would wrongly reset the machine from `STATE_BLINK` or `STATE_ON`), every value of `state_t` is already named above it, so the branch is unreachable in the source and synthesis normally optimizes it away rather than building recovery logic. Once Quartus re-encodes the machine, most bit patterns are illegal, and recovering from one requires Quartus's Safe State Machine setting rather than anything you can write in VHDL.

8.4.3 Worked example: `fsm_led`

`lectures/L08/fsm_led/fsm_led.vhd` implements exactly that three-state machine, extended to two buttons: `button_n(0)` advances, `button_n(1)` goes back, the states are closed in a loop, and `STATE_BLINK` toggles the LED every 100 ms.

Three subcomponents are reused, all of them modules you wrote yourself:

- `reset_sync`: Chapter 4's reset synchronizer, unchanged (Exercise 4.8).
- `button_sync`: Chapter 4's button synchronizer, also unchanged, but instantiated with `generic map(2)` so it handles both buttons at once, producing a two-bit pulse vector. This is the payoff of the generic: the file is identical to the one Chapters 4 and 7 instantiate with `1` (Exercise 4.9).
- `timer`: Chapter 7's timer, unchanged (Exercise 7.4). `fsm_led` declares its own `TIMER_TICK_COUNT` generic and passes it straight through, defaulting to `5_000_000` for 100 ms at `50 MHz`, and only enables the timer while in `STATE_BLINK`.

None of the three is shipped in `fsm_led/`: copy your own in before building the example. By this point in the book that is four chapters' worth of your own modules composed into one design, which is the thing worth noticing about it.

Two synchronous processes implement the machine:

- `STATE_PROCESS` is the `case` transition logic above, driven by `to_next_state`/`to_prev_state`, derived combinationally from `button_edge_s2`. Each requires the *other* button's pulse to be `'0'` in the same cycle, so pressing both close enough together that their pulses land on one clock cycle moves the machine nowhere rather than picking a winner arbitrarily. An ambiguous input deserves a defined answer, and "do nothing" is the one that cannot surprise you.
- `LED_PROCESS` drives `led_s` purely from the current state: off in `STATE_OFF`, on in `STATE_ON`, toggling on every timer timeout in `STATE_BLINK`. Because it never reads `button_n` or `button_edge_s2` directly, only `state`, this is a genuine Moore machine: the LED reacts to a press only indirectly, through the state it caused.

This is the whole of `lectures/L08/fsm_led/fsm_led.vhd`:

```
lectures/L08/fsm_led/fsm_led.vhd

--------------------------------------------------------------------------------
```

```vhdl
-- Moore FSM cycling an LED through OFF / BLINK / ON via two push buttons.
-- (Moore: the LED depends only on the state, never directly on the buttons;
-- contrast seq_detect_mealy. See Appendix A.5.)
--
-- Generics:
--     - TIMER_TICK_COUNT: The timer's tick count, so the interval between LED
--       toggles in STATE_BLINK (default = 5 000 000, i.e. about 100 ms at
--       50 MHz; override to shorten it in a testbench).
--
-- Inputs:
--     - clock: 50 MHz system clock.
--     - reset_n: Active-low asynchronous reset.
--     - button_n: Active-low buttons; (0) = next state, (1) = previous.
-- Outputs:
--     - led: The controlled LED.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity fsm_led is
    generic(TIMER_TICK_COUNT: natural := 5_000_000);
    port(clock, reset_n: in std_logic;
         button_n      : in std_logic_vector(1 downto 0);
         led           : out std_logic);
end entity;

architecture behaviour of fsm_led is
-- Enumeration of states.
type state_t is (STATE_OFF, STATE_BLINK, STATE_ON);

-- Synchronized input signals after two-flip-flop metastability protection.
signal reset_s2_n    : std_logic;
signal button_edge_s2: std_logic_vector(1 downto 0);

-- Timer signals.
signal timer_enable, timer_timeout: std_logic;

-- Internal LED state.
signal led_s: std_logic;

-- Current state.
signal state: state_t;

-- Signals indicating whether to change state.
```

```
signal to_prev_state, to_next_state: std_logic;
begin
    -- Assign the internal LED state to the output port.
    led <= led_s;

    -- Enable the timer in STATE_BLINK only.
    timer_enable <= '1' when STATE_BLINK = state else '0';

    -- Indicate a change to the previous state on a falling edge of button_n(1).
    -- Each direction also requires the other button's edge to be '0', so two
    -- presses landing on the same clock cycle move the machine nowhere.
    to_prev_state <= '1' when button_edge_s2(1) = '1' and button_edge_s2(0) = '0' ↩
    ↪ else '0';

    -- Indicate a change to the next state on a falling edge of button_n(0).
    to_next_state <= '1' when button_edge_s2(0) = '1' and button_edge_s2(1) = '0' ↩
    ↪ else '0';

    -- Synchronize the asynchronous reset (Appendix A.4, reusing L04's reset_sync)↩
    ↪ .
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    -- Synchronize both buttons and pulse on each press (Appendix A.4, reusing
    -- L04's button_sync at COUNT = 2: the same file, a wider vector).
    button_sync1: entity work.button_sync
        generic map(2)
        port map(clock, reset_s2_n, button_n, button_edge_s2);

    -- Instantiate the 100 ms timer (Appendix A.4, reusing L07's timer unchanged).
    timer1: entity work.timer
        generic map(TIMER_TICK_COUNT)
        port map(clock, reset_s2_n, timer_enable, timer_timeout);

    -- State-transition process:
    --     - On reset: reset the current state to STATE_OFF.
    --     - On rising clock edge: update the current state when a synchronized,
    --       edge-detected button press is seen.
    STATE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            state <= STATE_OFF;
        elsif (rising_edge(clock)) then
            case (state) is
```

```
            when STATE_OFF =>
                if (to_next_state = '1') then
                    state <= STATE_BLINK;
                elsif (to_prev_state = '1') then
                    state <= STATE_ON;
                end if;
            when STATE_BLINK =>
                if (to_next_state = '1') then
                    state <= STATE_ON;
                elsif (to_prev_state = '1') then
                    state <= STATE_OFF;
                end if;
            when STATE_ON =>
                if (to_next_state = '1') then
                    state <= STATE_OFF;
                elsif (to_prev_state = '1') then
                    state <= STATE_BLINK;
                end if;
            when others =>
                state <= STATE_OFF;
        end case;
    end if;
end process;

-- LED (Moore output) process, depending only on the current state:
--    - On reset: disable the LED.
--    - On rising clock edge, based on the current state:
--        - STATE_OFF  : disable the LED.
--        - STATE_BLINK: toggle the LED on every timer timeout.
--        - STATE_ON   : enable the LED.
LED_PROCESS: process(clock, reset_s2_n) is
begin
    if (reset_s2_n = '0') then
        led_s <= '0';
    elsif (rising_edge(clock)) then
        case (state) is
            when STATE_OFF =>
                led_s <= '0';
            when STATE_BLINK =>
                if (timer_timeout = '1') then
                    led_s <= not led_s;
                end if;
            when STATE_ON =>
                led_s <= '1';
```

```
                when others =>
                    led_s <= '0';
            end case;
        end if;
    end process;
end architecture;
```

8.5 Mealy Machines: The One-Cycle Difference

Everything so far has been a **Moore** machine, deliberately: Moore is the safer default and the form you design by hand. A **Mealy** machine relaxes the rule that defines a Moore machine: the output may depend on the current state *and* the current input, computed combinationally. It is not stored in a register, and it changes within the same clock cycle the input does.

8.5.1 The behaviour: detect two consecutive 1 bits

A serial input `din` carries one bit per clock cycle, and `y` should indicate that the last two bits were both `1`, with overlapping matches counting, so `111` reports a match twice.

As a Mealy machine this needs only two states, because the output can react to the input *before* it is reflected in the next state: `STATE_IDLE` (the previous bit was `0`, or this is the first) and `STATE_ONE` (the previous bit was `1`).

State	`din`	Next state	`y`
`STATE_IDLE`	0	`STATE_IDLE`	0
`STATE_IDLE`	1	`STATE_ONE`	0
`STATE_ONE`	0	`STATE_IDLE`	0
`STATE_ONE`	1	`STATE_ONE`	**1**

The state process is §8.4's `case` pattern; only the output differs. This is `lectures/L08/seq_detect_mealy/seq_detect_mealy.vhd`, and its output assignment is a single concurrent statement, entirely outside any clocked process:

```
y <= '1' when (state = STATE_ONE and din = '1') else '0';
```

That is a **conditional signal assignment**, and it is the last piece of VHDL syntax this book introduces. It is a concurrent statement, so it lives in the architecture body rather than inside a process, and it describes a multiplexer: pick the first value whose condition holds, otherwise the value after `else`. It is the same hardware you would get from an `if/else` inside a combinational process, written in one line instead of five, and it is worth having because a one-line output assignment makes it obvious at a glance what the output depends on.

```
-- These two describe exactly the same multiplexer.
x <= a when sel = '1' else b;

process(sel, a, b) is
begin
    if (sel = '1') then
        x <= a;
    else
        x <= b;
    end if;
end process;
```

Always give it an `else`. Leave one out and you have described a signal that keeps its old value when no condition holds, which is a latch, exactly as in §5.3. `fsm_led` uses this form three times, and Exercise 8.4 asks you to use it.

That one line is the whole distinction. It reads `din`, so it is Mealy. Move the same condition inside the clocked process, or drop `din` from it, and you have a Moore machine instead.

This is the whole of `lectures/L08/seq_detect_mealy/seq_detect_mealy.vhd`:

lectures/L08/seq_detect_mealy/seq_detect_mealy.vhd

```
--------------------------------------------------------------------------------
-- Mealy FSM detecting two consecutive '1's on serial input din (overlapping
-- matches allowed, e.g. "111" reports two). y is combinational in (state, din),
-- so it asserts the same cycle the second '1' arrives - one cycle earlier than
-- the Moore equivalent. See Appendix A.5.
--
-- Inputs:
--     - clock: 50 MHz system clock.
--     - reset_n: Active-low asynchronous reset, from a push button.
--     - din: Synchronous serial data (already clocked by clock, so it is NOT run
--       through a synchronizer - doing so would shift the sequence being detected
    ↪ ).
-- Outputs:
--     - y: Asserted during the cycle the second '1' of a "11" is seen on din.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;

entity seq_detect_mealy is
    port(clock, reset_n: in std_logic;
         din           : in std_logic;
         y             : out std_logic);
```

```
end entity;

architecture behaviour of seq_detect_mealy is
-- Enumeration of states. Only two are needed: this is the state count the
-- Mealy design saves relative to the three-state Moore equivalent.
type state_t is (STATE_IDLE, STATE_ONE);

-- Synchronized reset signal after two-flip-flop metastability protection.
signal reset_s2_n: std_logic;

-- Current state.
signal state: state_t;
begin
    -- Instantiate reset_sync (Appendix A.5 / L04) to synchronize reset_n. This
    -- example has no push button of its own, so the button synchronizer is
    -- simply not instantiated.
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    -- State-transition process:
    --    - On reset: reset the current state to STATE_IDLE.
    --    - On rising clock edge: move to STATE_ONE whenever din = '1', back to
    --      STATE_IDLE whenever din = '0'. Note this process only ever reads din
    --      to decide the *next* state; it never assigns y.
    STATE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            state <= STATE_IDLE;
        elsif (rising_edge(clock)) then
            case (state) is
                when STATE_IDLE =>
                    if (din = '1') then
                        state <= STATE_ONE;
                    else
                        state <= STATE_IDLE;
                    end if;
                when STATE_ONE =>
                    if (din = '1') then
                        state <= STATE_ONE;
                    else
                        state <= STATE_IDLE;
                    end if;
                when others =>
                    state <= STATE_IDLE;
```

```
            end case;
        end if;
    end process;

    -- Mealy output: a combinational (unregistered) function of the current
    -- state AND the current input, assigned outside any clocked process.
    -- Asserted the same cycle the second '1' arrives, one cycle earlier than
    -- a Moore machine could report the same event.
    y <= '1' when (state = STATE_ONE and din = '1') else '0';
end architecture;
```

8.5.2 What it costs, and what it buys

The Moore version of this detector needs a **third** state, `M_TWO`, existing purely to remember for one cycle that the match happened, so the output has something to read that does not depend on the input. Feed both machines `1, 1, 0, 1, 1, 1`:

Cycle	`din`	Mealy state (before)	Mealy `y`	Moore state (before)	Moore `y`
1	1	`STATE_IDLE`	0	`M_IDLE`	0
2	1	`STATE_ONE`	**1**	`M_ONE`	0
3	0	`STATE_ONE`	0	`M_TWO`	**1**
4	1	`STATE_IDLE`	0	`M_IDLE`	0
5	1	`STATE_ONE`	**1**	`M_ONE`	0
6	1	`STATE_ONE`	**1**	`M_TWO`	**1**

The Moore `y` column is exactly the Mealy `y` column shifted down one row. That is the entire trade: Mealy reports the match the *same* cycle the second `1` arrives where Moore takes one more, and Mealy needs 2 states where Moore needs 3.

8.5.3 Which to reach for

Moore, almost always, and it is what the rest of this book uses:

- its output depends on no primary input, so it cannot follow a noisy one.
- a Mealy output can in principle glitch combinationally if its inputs aren't already clean.
- note that "Moore" on its own does not mean glitch-free. An output decoded combinationally from several state bits, like §8.2's `Y = Q1Q2'`, can still glitch while those bits skew on a transition. What makes an output glitch-free is *registering* it, the way `fsm_led`'s `LED_PROCESS` does, and that is what you want on anything driving a wire off the chip. A serial `tx` line decoded straight from the state bits will emit spurious edges.
- a machine whose output depends on fewer things is a machine with fewer ways to be wrong.

Reach for Mealy only when you specifically need the lower latency or the smaller state count, and know that you are trading a synchronous output for a combinational one when you do.

8.6 FPGA Demonstration

`fsm_led` synthesizes and runs exactly the way every VHDL design has since Chapter 1 (Appendix B): create a Quartus Prime Lite project targeting the DE0-CV's `5CEBA4F23C7N`, add the `.vhd` files for the example and its subcomponents, compile, assign each port to a physical pin in the Pin Planner, then recompile and program the board.

For `fsm_led`, assign `clock` to the 50 MHz oscillator pin, `reset_n` and `button_n(1 downto 0)` to three push buttons, and `led` to an onboard LED. Then confirm that pressing "next" repeatedly cycles the LED `off -> blinking -> on -> off`, that "previous" runs the other way, and that reset returns it to off.

`seq_detect_mealy` is deliberately **not** demonstrated on the board. It consumes one bit per clock cycle, and a switch flipped by hand holds its value for millions of cycles, so the only thing an LED can show is the degenerate case: `din` held high, `y` high from the second cycle onward. The one-cycle distinction the whole of §8.5 is about happens far too fast to see. Run its testbench instead, which is the general lesson rather than an exception:

```
cd lectures/L08/seq_detect_mealy
cp ../../L04/exercises/reset_sync/reset_sync.vhd .     # the one you wrote in L04
ghdl -a --std=93 reset_sync.vhd seq_detect_mealy.vhd seq_detect_mealy_tb.vhd
ghdl -e --std=93 seq_detect_mealy_tb
ghdl -r --std=93 seq_detect_mealy_tb --assert-level=error --stop-time=10ms
```

Exact pin numbers depend on which DE0-CV buttons and LEDs you choose, assigned the same way as for every design since Chapter 1. Refer to the DE0-CV's own documentation for its physical pinout if you need to confirm which header pin corresponds to which labelled button, switch or LED.

8.7 Review

What you should be able to do now

After this chapter, you should be able to:

- Explain what a finite state machine is, and identify its states, transitions, inputs and outputs in a given circuit.
- Design a Moore machine by hand, from specification through state diagram and state table to Karnaugh-derived equations, and build the result in CircuitVerse.
- Model the same machine in VHDL with an enumerated `state_t` type and a `case` process.

- Recognize a Mealy machine on sight and say what its output costs and buys, one clock cycle of latency against one extra state, and why Moore is the default in this book.
- Compose a state machine with previously built subcomponents rather than rewriting them. By the capstone that means three earlier chapters' modules around a machine that is the only new logic, drawing on the shift and counter idioms of one more.

Questions to test yourself

You should be able to explain:

1. What makes a circuit a state machine, rather than just a register with some logic around it?
2. In the hand-designed machine, what exactly is `X`, and why must it be a one-cycle pulse rather than the button's level?
3. Why is that machine's state encoding a Gray code? What could go wrong with an ordinary binary count?
4. A counter and a shift register are technically state machines too. What distinguishes them from the machine designed here?
5. Which signals is a Moore output allowed to depend on, and a Mealy output? Given a single line of VHDL driving an output, how do you tell which you are looking at?
6. In §8.5's comparison, why does the Moore version need an extra state, and why does its output lag the Mealy version's by exactly one clock cycle?
7. In `fsm_led.vhd`, why does `LED_PROCESS` only ever read `state`, never `button_edge_s2` or `button_n` directly? What would change if it did?
8. Why is `din` in `seq_detect_mealy` not passed through a synchronizer, unlike `reset_n`?
9. What does `when others` buy you in a state-transition `case`, given that `state_t` has only the values you declared?
10. In the capstone receiver, why does the timer run at sixteen times the baud rate rather than once per bit, and why does sampling in the middle of a bit make the design tolerate a transmitter whose clock is slightly different from yours?
11. The capstone transmitter uses a whole-bit timer and needs no mid-bit anything. What does it have that the receiver doesn't, that makes the difference?

Where this leaves you

This is the final chapter of *Digital Design with VHDL*. Over eight chapters the book moved from gates and Karnaugh maps, through registers, metastability, and the VHDL language, to counters, shift registers and timers, and now finite state machines, at every step reasoning about a circuit by hand before writing it in synthesizable VHDL and watching it run on a real FPGA. Every pattern built along the way, the double-flop synchronizer, the `process/case` coding style, and now the Moore state machine, was reused again in this chapter's own examples, and carries forward unchanged into whatever you build next.

8.8 Exercises

How to work these. Exercises 8.1 and 8.2 are pen-and-paper: they are about getting a state machine right *before* it becomes code, and nothing in them needs VHDL, GHDL, or a board. Each exercise is tagged with the kinds of work it asks for: PAPER by hand, CIRCUITVERSE in the browser, VHDL in a module of your own.

How to check your work. From Exercise 8.4 on, every exercise that asks you to write a VHDL module ships a self-checking testbench under `lectures/L08/exercises` in the companion repository, and nothing else; Exercise 8.3 modifies a module you already have and is checked by hand, as it says. Assembling the rest of the directory is part of the work: the modules you wrote in earlier chapters (`reset_sync`, `button_sync`, `sync`, `timer`) are copied in from wherever you wrote them. Each exercise prints the exact commands it needs. Write your module in its `exercises/<module>` directory, using the entity name and the **port order** the exercise specifies, then run it with GHDL; §2.8 has the three commands.

Exercises 8.6 and 8.7 are the course capstones: the two ends of one serial link, a receiver and a transmitter. Each composes modules from four different chapters, so each needs more than one `ghdl -a` argument, and so do Exercises 8.4 and 8.5. Where that is the case, the exercise prints the command it needs.

No FPGA board is needed for any exercise. The Quartus synthesis and board-programming steps are demonstrated in the course, and shown in Appendix B; your job is to get the VHDL right, and the testbench is how you confirm it. Short answers to the parts you work on paper are in Appendix A; reference solutions to the VHDL are published in the companion repository, under `lectures/L08/solutions`. Try each exercise yourself before looking at either.

EXERCISE SET 1

State Diagrams and State Tables

Exercise 8.1 A binary-count Moore machine

PAPER CIRCUITVERSE

Design a new four-state Moore machine, `{Q1, Q2}`, controlling an LED via a single synchronized, edge-detected button pulse `X` (produced exactly like `X` in the worked example of §8.2).

It differs from that worked example in two ways:

- The state order is the ordinary binary count `00 -> 01 -> 10 -> 11 -> 00`, **not** the Gray code used in §8.2.
- The LED (`Y`) lights in **two** states instead of one:
  - Whenever `Q1 = 1` (i.e. in both `10` and `11`).

a) Draw the state diagram:

- Four states.
- One transition arrow per state for `X = 1`.
- One self-loop per state for `X = 0`.

b) Fill in the full state table (all eight rows of `{Q1, Q2, X}`).

c) Derive `Q1+`, `Q2+`, and `Y` via Karnaugh maps, the same way §8.2 derives them for the Gray-code version.

d) Realize the gate network by hand:

- Two D flip-flops for `{Q1, Q2}`.
- Gates computing `Q1+`, `Q2+`, and `Y`.
- Build and simulate it in CircuitVerse, double-flop synchronizing and edge-detecting the button input exactly as in §8.3.

e) Verify it in CircuitVerse, against your own state table. There is no testbench for this machine, and there is not meant to be: the state table you derived in **b)** *is* the specification, and checking a circuit against a specification you worked out yourself is the whole point of the exercise.

Step the clock by hand and check, in this order:

- **The advance path.** Pulse `X` once per step and walk a full lap, `00 -> 01 -> 10 -> 11 -> 00`. Before each edge, read the expected next state off your table. After the edge, read `{Q1, Q2}` off the flip-flop outputs. All four transitions must agree.
- **The hold path.** In each of the four states, step the clock with `X = 0` and confirm the machine stays put. That is four more checks, and it is the half people skip; a machine that advances correctly but does not hold is a machine whose `X'` terms are wrong.
- **The output.** In every state, check `Y` against your table: high in `10` and `11`, low in `00` and `01`. Do this as its own pass, not while you are watching the state bits.

Where they disagree tells you which gate to look at. A wrong `Q1+` shows up only on the transitions where `Q1` was supposed to change. A wrong `Y` shows up with the state sequence still perfectly correct, which is why it is worth a separate pass.

Tip: Since the state order changed from Gray code to binary, expect **two** transitions where both state bits flip at once: `01 -> 10` and `11 -> 00`. Per §8.2 that is harmless to the machine itself, since nothing samples the state register between edges, and CircuitVerse's ideal simulation will not show you anything either way.

It is harmless to `Y` here too, and for a reason worth naming: `Y` is high exactly when `Q1 = 1`, so it is one flip-flop's output taken straight to the LED, with no gate between them to produce a glitch while the two bits skew. Contrast §8.2's machine, whose `Y = Q1 * Q2'` **is** decoded from both bits and can therefore glitch briefly on real hardware. Ask yourself what would change here if the LED lit in only one state, say `10`, and what you would do about it if `Y` were driving something that cared.

EXERCISE SET 2

Sequence Detection

Exercise 8.2 A Moore `1, 0, 1` detector

PAPER

Design a **Moore** machine that detects the sequence `1, 0, 1` arriving one bit per clock cycle on an input `din`, raising its output `y` once the third bit of a match has been seen.

- Design it **non-overlapping**: after a full match, start looking for a fresh `1, 0, 1` from scratch, rather than reusing the trailing bits. Exercise 8.4 turns this machine into VHDL and its shipped testbench checks for it, so an overlapping design fails there.

a) Draw the state diagram:

- One state per amount of the pattern matched so far.
- An arrow out of every state for `din = 0` *and* for `din = 1`; a sequence detector is never idle, it is always either advancing or falling back.
- Mark which state drives `y = 1`.

b) Fill in the state table, one row per (state, `din`) combination.

c) Trace it. Pick a six-bit sequence for `din` containing at least one match, and record, cycle by cycle:

- The state before the edge.
- The value of `din`.
- The output `y`.

d) The trap in this machine is the fall-back arrows, not the forward ones. For each state, say what happens on the input that does *not* advance the match:

- Which of them return to the start?

- Is there any input that should send you back to a *partially* matched state rather than all the way to the beginning? Explain your answer for this pattern.

Tip: count your states before you draw them. "Nothing matched yet", "saw `1`", "saw `10`", "saw `101`" is the natural decomposition, and the last of those is the one that drives `y`.

Keep your answer. Exercise 8.4 below implements this same detector in VHDL, as a Mealy machine, which, per §8.5, needs one state fewer than what you just drew. Working out which of your states disappears, and why, is part of that exercise.

EXERCISE SET 3

Finite State Machines in VHDL

Exercise 8.3 A fast-blink state

VHDL

Extend `fsm_led` with a fourth state, `STATE_FAST_BLINK`, blinking at 20 ms instead of `STATE_BLINK`'s 100 ms. Follow the worked example in §8.4.

Work on a **copy** of the module, not on `lectures/L08/fsm_led/fsm_led.vhd` itself. Copy the whole directory somewhere of your own first, along with the three modules of your own that it instantiates:

```
cp -r lectures/L08/fsm_led /tmp/fsm_led_fast
cp lectures/L04/exercises/reset_sync/reset_sync.vhd /tmp/fsm_led_fast     # the ↩
    ↪ three you wrote
cp lectures/L04/exercises/button_sync/button_sync.vhd /tmp/fsm_led_fast   # ↩
    ↪ yourself
cp lectures/L07/exercises/timer/timer.vhd /tmp/fsm_led_fast
cd /tmp/fsm_led_fast
```

There is no testbench to run here, but the four-state machine should still analyze, and that is worth doing after every edit: it catches a missing `case` branch, or a `state_t` value you added in one process and not the other.

```
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd fsm_led.vhd
```

The reason is worth knowing, because it is a real hazard and not busywork. `fsm_led` ships a reference testbench, `fsm_led_tb.vhd`, and Exercise 8.5 asks you to run it. That testbench asserts the **three**-state cycle: it presses the button once from `STATE_BLINK` and checks that the LED goes solid, because in the machine as written the next state is `STATE_ON`. In your four-state version that press lands in `STATE_FAST_BLINK`, the LED is still blinking, and the assertion fails. Nothing about the failure tells you which of the two exercises it belongs to. Keep the reference machine intact and that question never comes up.

There is no testbench for the four-state machine. Check it the way §8.2 had you check a state diagram: predict the sequence of states for a run of button presses, then read your `case` branches back and confirm they agree.

You'll need to:

- Add the new value to `state_t`.
- Add a branch to each `case` statement, in both `STATE_PROCESS` and `LED_PROCESS`.
- Handle the two blink rates, by either:
  - Adding a second timer instance, or
  - Making the existing timer's tick count switch based on the current state.

Exercise 8.4 A Mealy `1, 0, 1` detector

VHDL

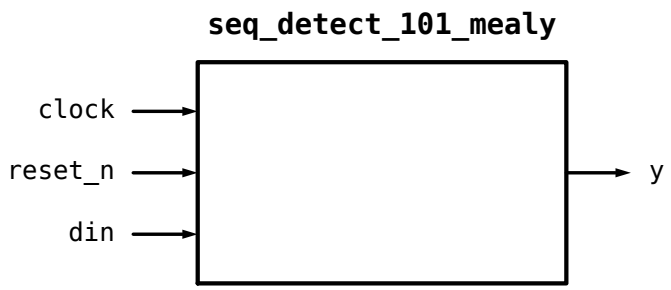
Implement the `1, 0, 1` detector from Exercise 8.2 in VHDL, in a new directory `seq_detect_101_mealy`, this time as a **Mealy** machine.

Start from your Moore state diagram and convert it:

- Your final "saw `101`" state exists only to hold the output high for one cycle.
- A Mealy machine doesn't need it: the output condition can read `din` directly in the state that has already matched `1, 0`.
- Drop that state, and move its output into the condition on `y`.

Then follow the `type state_t is (...)` / `case` pattern from §8.4 and §8.5:

- Reuse `reset_sync` for `reset_n`, exactly as `seq_detect_mealy.vhd` does. There is no button in this design, so `button_sync` is simply not instantiated.
- Make sure the output assignment for `y` is a single concurrent statement, outside any clocked process, depending on both `state` and `din`, not just `state`.
  - Otherwise you've quietly built a Moore machine instead, and its output will arrive one cycle late, which is exactly what the testbench checks.



Self-check: name your entity `seq_detect_101_mealy`, with ports `clock`, `reset_n`, `din` (in) and `y` (out), declared in that order to match `seq_detect_mealy`. A testbench is in `exercises/seq_detect_101_mealy`; it checks a non-overlapping "101" detector. Your module instantiates

`reset_sync`, which is not shipped there because you wrote it in Chapter 4's Exercise 4.8: copy your own in. This is the first exercise that needs more than one file on the `ghdl -a` line, with the subcomponent first so GHDL sees it before the module that instantiates it:

```
cd lectures/L08/exercises/seq_detect_101_mealy
cp ../../../L04/exercises/reset_sync/reset_sync.vhd .     # the one you wrote in L04
ghdl -a --std=93 reset_sync.vhd seq_detect_101_mealy.vhd seq_detect_101_mealy_tb.vhd
ghdl -e --std=93 seq_detect_101_mealy_tb
ghdl -r --std=93 seq_detect_101_mealy_tb --assert-level=error --stop-time=10ms
```

EXERCISE SET 4

Reading the Worked Machine

Exercise 8.5 Accounting for `fsm_led`

PAPER

`fsm_led` is demonstrated on the DE0-CV in the course (see §8.6). This exercise checks that you can account for what the demonstration shows, and it ships a reference testbench you can run yourself. It reuses your two Chapter 4 synchronizers and your Chapter 7 `timer`, none of which it ships, so copy them in first:

```
cd lectures/L08/fsm_led
cp ../../L04/exercises/reset_sync/reset_sync.vhd .     # the three you wrote yourself
cp ../../L04/exercises/button_sync/button_sync.vhd .
cp ../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd button_sync.vhd timer.vhd \
                 fsm_led.vhd fsm_led_tb.vhd
ghdl -e --std=93 fsm_led_tb
ghdl -r --std=93 fsm_led_tb --assert-level=error --stop-time=10ms
```

a) Work out the blink rate from the source, not from the demonstration:

- `fsm_led` instantiates `timer` with `TIMER_TICK_COUNT` defaulting to `5_000_000`, on a `50 MHz` clock.
- How long is one timeout period?
- `STATE_BLINK` toggles the LED on each timeout. What is the resulting **blink** frequency, and why is it half what you might first write down?

b) The testbench overrides `TIMER_TICK_COUNT` with a much smaller value than the default:

- Find that override in `lectures/L08/fsm_led/fsm_led_tb.vhd`.

- Explain why a testbench cannot reasonably use the real value, and roughly how long a simulation of one real blink period would have to run.

c) Account for the state machine's behaviour:

- All three states are reachable in both directions. Trace the presses of `button_n(0)` and `button_n(1)` that visit every state and return to `STATE_OFF`.
- Press both buttons on the same clock cycle. What does `fsm_led` do, and which two lines of the architecture decide that?
- `timer_enable` is driven from `state`, so the timer only runs in `STATE_BLINK`. Note that the `timer` **pauses** rather than clears when disabled (§7.3). What does that mean for the moment you re-enter `STATE_BLINK` after leaving it, and would holding `timer_enable` high permanently be better or worse?

EXERCISE SET 5

Capstone: A Serial Receiver

Exercise 8.6 The serial receiver

PAPER VHDL

This is the first half of the course capstone, and it composes four chapters into one design. Exercise 8.7 builds the other end of the same serial link, so do not stop here. Nothing in it is a new idea; all of it is wiring together modules you already have, around a state machine you design yourself:

- Chapter 4's `reset_sync`, synchronizing the reset.
- Chapter 5's `sync`, synchronizing the `rx` line, which arrives from somebody else's crystal and is asynchronous in exactly the sense Chapter 4 means.
- Chapter 7's `timer`, generating the oversample tick.
- This chapter's state machine, framing the whole thing, and doing its own shifting and bit counting. You built that as a module in Chapter 6; here it is four lines inside the machine that already has the counters it needs, which is the more usual way a receiver is written.

Write an entity named `uart_rx8`: a receiver for one byte of asynchronous serial data, in a frame format modelled on a UART's, with one deliberate difference set out below.

The line format

The `rx` line carries frames, and between frames it sits idle:

- **Idle**: the line is held high.
- **Start bit**: the line goes low for exactly one bit period. This is what marks the beginning of a frame; there is no separate clock line, which is what "asynchronous serial" means.
- **Eight data bits**, most significant first, one bit period each. This is the deliberate difference: see the note below the figure.
- **Stop bit**: the line returns high for one bit period.

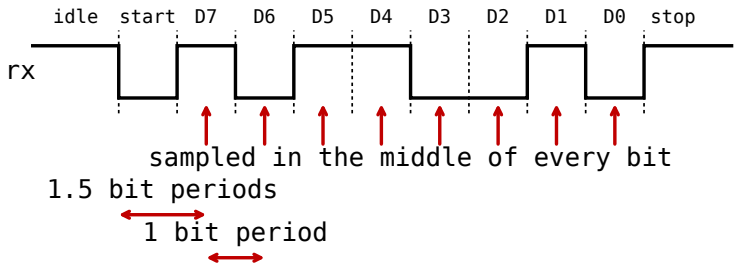


Figure 8.3 A frame on the `rx` line: idle high, a start bit, eight data bits most significant first, and a stop bit, with the receiver sampling in the middle of every bit.

The frame in Figure 8.3 carries `10110010`, which is the first byte the testbench sends. The arrows are the only eight moments the receiver actually looks at the line: one and a half bit periods after the falling edge, then once per bit period after that.

The receiver has to recover the bit timing from the falling edge of the start bit alone. That is the entire problem this exercise solves, and it is why the timer is here.

One deliberate deviation from real UART: a real UART sends its data bits **least** significant first, as §6.3 notes. This exercise sends them most significant first:

- the framing and the timing, which are what these two exercises are actually about, are identical either way.
- it is the direction §6.3's SIPO idiom shifts, so the receiver's shift is one line, and the transmitter in Exercise 8.7 is its mirror.
- the provided testbenches transmit and expect MSB-first to match, so the pair is self-consistent.
- to speak to a real UART you would flip the shift direction to §6.2's mirror-image form and change nothing else. Worth knowing before you wire this to a PC's serial port and wonder why every byte arrives bit-reversed.

Oversampling: sixteen ticks per bit

The receiver has to find the middle of a bit whose start it can only infer. The technique is **oversampling**: run the `timer` at **sixteen times** the baud rate, so there are sixteen ticks per bit period, and count them.

- The falling edge of the start bit resets the tick count to `0`. The timer runs continuously, before, during and after a frame; it is the *count* that is reset, never the timer.
- **Tick 8** is half a bit later, the middle of the start bit. Check the line is *still low* there. If it has gone high, that was a glitch and not a start bit, and the receiver returns to idle. That re-check is what stops noise on an idle line from being read as data.
- From there, every **sixteenth** tick is the middle of the next bit: sample the line and shift it in. Eight of those are the data bits, and the ninth is the stop bit.

Sampling mid-bit rather than at the edges is what gives the receiver its timing margin. The transmitter's clock is a different physical crystal from yours, so the two drift apart over a frame; a receiver that sampled near the boundaries would read the wrong bit as soon as they did. Sixteen ticks is the conventional choice.

The `uart_rx8` entity

Generic	Type	Default	Description
`OVERSAMPLE_TICK_COUNT`	`natural`	`325`	The `timer` tick count for **one sixteenth** of a bit period. 9600 baud at `50 MHz`; part **b**) asks you to derive it.

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_n`	in	`std_logic`	Asynchronous, active-low; goes through `reset_sync`, as always.
`rx`	in	`std_logic`	The serial line, straight off a pin and therefore **asynchronous**. Synchronize it to `rx_s2` before anything else reads it.
`data_out`	out	`std_logic_vector(7 downto 0)`	The received byte, valid in the cycle `byte_valid` is high.
`byte_valid`	out	`std_logic`	A one-cycle pulse: a complete, well-framed byte is on `data_out`.
`frame_err`	out	`std_logic`	A one-cycle pulse: the stop bit was not high, so the frame is broken and no byte is handed over.

What to build: the receiver

a) Design the state machine **on paper first**, exactly as §8.2 had you do. Four states:

- `STATE_IDLE`, waiting for the line to go low.
- `STATE_START`, checking at tick 8 that the line is still low.
- `STATE_DATA`, sampling one bit every sixteen ticks, eight times.
- `STATE_STOP`, sampling the stop bit sixteen ticks after the last data bit and deciding between `byte_valid` and `frame_err`.

Draw it before writing any VHDL, and on the diagram:

- Give every state its **self-loop**: the arrow for "the tick I am waiting for has not arrived yet, stay put". That is the arrow that goes missing, and a state without it falls through to wherever the `else` happens to point.
- Write beside each state **how many ticks it waits for**, so the tick accounting is on the paper rather than in your head.

- Mark where `ticks` is **cleared**. The timer itself never stops, so those clears are the only places the receiver decides what "the start of a bit" means.
- Label the arrow out of `STATE_IDLE` with an **edge**, not a level: `rx_s2` low *and* high on the previous tick. That needs one bit of history, so note where you are going to keep it.

b) Work out the timing before you implement it:

- The DE0-CV clock is `50 MHz` and 9600 baud means 9600 bits per second. How many clock cycles is one bit period? How many is a sixteenth of one?
- `timer` pulses every `TICK_COUNT + 1` clock cycles (§7.1). What `OVERSAMPLE_TICK_COUNT` does that give you? The exact answer is not a whole number, so there are two candidates either side of it: work out the error each one leaves, and say which the default above is and why.
- That error is a fraction of a percent, and it accumulates across a frame. The stop bit is sampled 9.5 bit periods after the start edge: half a bit period to reach the middle of the start bit, then nine more bit periods. How far has your sample point drifted from the middle of the stop bit by then, measured in oversample ticks? How many ticks of margin did it have to begin with?
- Now the reason the answer is comfortable: you resynchronize on every start bit. What would have to be true of the frame length for that to stop being enough?

c) Implement `uart_rx8`. It instantiates three modules and does the rest itself:

- `reset_sync` for the reset. Everything else in the design takes its `reset_s2_n`.
- **`sync`**, the module you wrote in Chapter 5's Exercise 5.4, on the `rx` pin, giving the `rx_s2` the state machine reads. Instantiate it at `SIZE = 1` and `PRESET = '1'`: an idle line is high, and a synchronizer coming out of reset reading `'0'` would look to this machine like a start bit. `rx` arrives from a pin driven by a transmitter with its own crystal, so it is asynchronous in exactly the sense Chapter 4 means, and every rule from that chapter applies unchanged. Note it takes a one-element `std_logic_vector` either side, so you declare a pair of those and pull `rx_s2` out of the output.
- `timer`, with `OVERSAMPLE_TICK_COUNT` passed straight through, `enable` tied to `'1'`, and its `timeout` as your only sense of elapsed time. Call it `baud_tick`. The timer free-runs: it is never stopped, never cleared, and no state ever touches it. What the machine re-phases instead is its own `ticks` counter.
- No shift register module. The machine already needs a bit counter, so it shifts the line straight into a `frame` signal itself, one line of VHDL per bit sampled.

The state machine, state by state. Two counters: `ticks` counting `0` to `15` within one bit, and `bit_count` counting the eight data bits. Everything below happens **only when `baud_tick = '1'`**; on every other clock edge the machine sits still. Since the timer free-runs, a tick is always on its way, in every state including idle.

State	Waits for	Then	To
`STATE_IDLE`	falling edge on `rx_s2`	clear `ticks`	`STATE_START`
`STATE_START`	`ticks` = 8	`rx_s2` still low? clear `ticks`	`STATE_DATA`
		`rx_s2` high? it was a glitch	`STATE_IDLE`
`STATE_DATA`	`ticks` = 15	shift `rx_s2` into `frame`, clear `ticks`	`STATE_DATA` until 8 bits, then `STATE_STOP`
`STATE_STOP`	`ticks` = 15	`rx_s2` high? `data_out <= frame`, pulse `byte_valid`	`STATE_IDLE`
		`rx_s2` low? pulse `frame_err`, drop the byte	`STATE_IDLE`

There is no "done" state. The stop-bit sample is where the frame is judged, so that is where the byte is handed over, and the machine goes straight back to idle.

Why idle waits for an edge rather than a level. Keep one tick of history for `rx_s2`, and leave idle only on the transition from high to low. Testing the level instead, "if `rx_s2` is low, start a frame", looks equivalent and passes everything except the one case that matters.

Consider a **break**: the line held low far longer than a frame, which is what a transmitter losing power looks like. A level test re-arms the instant the previous frame's stop bit is judged, so the receiver spends the break marching through back-to-back all-zero frames. Each ends on a low stop bit and is correctly rejected, and that part is fine. The problem is the frame in flight when the break *ends*: its stop bit is sampled after the line has returned high, so it is a perfectly well-formed frame as far as the receiver can tell, and a byte made of nothing is handed over as real data. The edge test does not re-arm during the break at all, because after the first falling edge there is not another one until the line has recovered.

That is the whole difference, it costs one flip-flop, and it is why a receiver that "works" on clean data can still hand you garbage the first time a cable is unplugged.

The shift. Eight data bits arrive most significant first, so the SIPO idiom from §6.3 does it with no index arithmetic at all:

```
frame <= frame(6 downto 0) & rx_s2;
```

`bit_count` then only counts *how many* bits have arrived, never *where* they go. Storing with `frame(bit_count) <= rx_s2` instead puts the first-arriving bit in bit 0, which reverses the byte: that is the least-significant-first order a real UART uses. Here the line is MSB first, so the shift is the one that matches.

Four things that decide whether this works:

- **Everything is gated on `baud_tick`.** A state that acts on a plain clock edge runs sixteen times per tick and roughly five thousand times per bit.

- **Every state needs its "not yet" path.** If the tick you are waiting for has not arrived, or the count has not reached its target, the machine must **stay where it is**. Sending it anywhere else in the `else` is the single most common way this design fails, and it fails completely rather than subtly: the machine ends up ping-ponging between two states and never reaches the third.
- **Watch the off-by-one on the counter.** With `ticks` starting at `0` and incrementing on each tick, the nth tick is `ticks = n - 1` at the moment you test it. Whether you compare before or after incrementing decides which tick you actually sample on, and being one or two ticks late is survivable while being eight is not.
- **The timer free-runs, so your phase comes from `ticks`, never from the timer.** The start edge arrives whenever the transmitter sends it, which is somewhere inside a tick period the receiver did not choose, so the first tick after that edge can be anywhere from a full tick to almost no time later. Clearing `ticks` on the way out of idle is what re-phases sampling onto the start bit. That leftover fraction of one tick is the price of never stopping the timer, and it is cheap: one sixteenth of a bit, against the half-bit of margin part **b**) asks you to work out.

d) Run the testbench. It needs four design files, subcomponents first, and none of the three subcomponents is shipped in the exercise directory: all three are modules you wrote, `reset_sync` in Chapter 4, `sync` in Chapter 5 and `timer` in Chapter 7.

```
cd lectures/L08/exercises/uart_rx8
cp ../../../L04/exercises/reset_sync/reset_sync.vhd .     # the three you ↩
    ↪ wrote yourself
cp ../../../L05/exercises/sync/sync.vhd .
cp ../../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 sync.vhd reset_sync.vhd timer.vhd uart_rx8.vhd uart_rx8_tb.↩
    ↪ vhd
ghdl -e --std=93 uart_rx8_tb
ghdl -r --std=93 uart_rx8_tb --assert-level=error --stop-time=10ms
```

It acts as the transmitter, and checks:

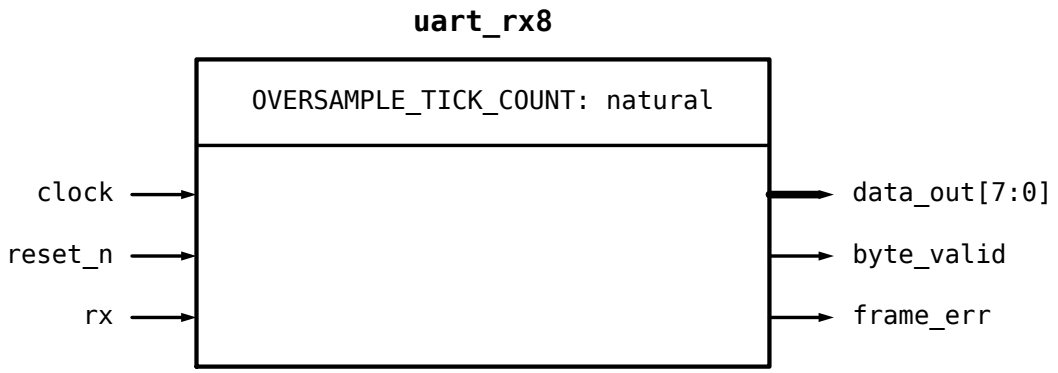
- two clean frames, arriving as the right bytes, with no `frame_err`.
- an idle line, before, between and after them, producing nothing.
- a frame whose stop bit is **low**: `frame_err` must pulse and no byte may be handed over.
- the line held low far longer than a frame, which is a **break**: every frame the receiver thinks it sees there ends in a low stop bit, so none of them is a byte.
- a frame in which each data bit carries its value only in the **middle** of the bit period and the opposite level either side. A receiver sampling near tick 8 reads it perfectly; one sampling near a boundary reads the neighbouring bit.
- that `byte_valid` and `frame_err` are each never high for two cycles in a row.

Tip: build it in stages and run the testbench after each, rather than writing all four states and debugging them at once:

- First get the machine to leave idle on a start bit and reach `STATE_DATA`. If it never gets there, the "not yet" path is what to look at.
- Then sample one data bit and check it is the right one before worrying about the other seven.
- Then the stop bit and the two strobes.

e) Now break it deliberately, and think about what the result does and doesn't prove:

- Remove the stop-bit state, handing the byte over as soon as the eighth data bit is in.
- Predict what should go wrong, then run it. Two checks catch this, and it is worth knowing which fires first: the frame with a low stop bit no longer raises `frame_err`, and it hands over a byte that should have been dropped.
- Now the other half of the lesson. Passing is the weaker signal, so find a break the testbench does **not** catch. Here is one: **delete the mid-start-bit re-check**, so `STATE_START` moves to `STATE_DATA` at tick 8 without confirming the line is still low. The testbench never puts a glitch on an idle line, so nothing notices, and on real hardware a single noise spike starts a phantom frame. Read `uart_rx8_tb.vhd` and say what you would add to catch it.
- Here is a second: **sample at tick 3 instead of tick 8.** Work out why the testbench still passes, and what you have given away. The answer is in part **b**)'s margin calculation.
- And a third, which is the most important of the three: **delete the `rx` synchronizer** and read the pin directly. Every check still passes, and it always will, no matter what is added to the testbench. Say why, in terms of what a simulator models and what it does not, then reread §4.4. This is the one class of bug in the book that testing cannot reach, which is exactly why Chapter 4 gives you a rule to follow rather than a symptom to look for.
- In a few sentences: what does "passes its testbench" actually promise, and what does it not?



Self-check: name your entity `uart_rx8`, with generic `OVERSAMPLE_TICK_COUNT` (`natural`, default `325`), inputs `clock`, `reset_n`, `rx`, and outputs `data_out` (`std_logic_vector(7 downto 0)`), `byte_valid` and `frame_err`, declared in that order. Its testbench is in `exercises/uart_rx8`, and nothing else is: **copy in your own `reset_sync`, `sync` and `timer`**, as part **d**) shows. A project directory holds everything it builds from, and assembling it is part of the exercise. Chapter 7's `walking_led` is put together the same way.

Exercise 8.7 The serial transmitter

PAPER VHDL

The other half of the link: write `uart_tx8`, a transmitter that sends one byte in the same format Exercise 8.6 receives.

A transmitter is the easier of the two, and the reason is worth understanding before you start:

- The receiver had to *recover* the bit timing from a start edge it didn't control, which is why it sampled mid-bit and why its timer ran at a sixteenth of a bit period.
- The transmitter **defines** the timing. Nothing has to be recovered, so the timer runs at a full bit period and the machine simply holds each bit on the line for one timeout.

It sends most significant bit first, matching Exercise 8.6's receiver rather than a real UART, for the reason given there.

The `uart_tx8` entity

Generic	Type	Default	Description
`BIT_TICK_COUNT`	`natural`	`5207`	The `timer` tick count for one **whole** bit period, 9600 baud at `50 MHz`. Sixteen of Exercise 8.6's oversample ticks, to within the rounding its part **b**) asks you to work out, so a `uart_tx8` and a `uart_rx8` built with the matching defaults speak at the same rate.

Port	Direction	Type	Description
`clock`	in	`std_logic`	System clock.
`reset_n`	in	`std_logic`	Asynchronous, active-low; through `reset_sync`, as always.
`data_in`	in	`std_logic_vector(7 downto 0)`	The byte to send.
`send`	in	`std_logic`	A request to transmit `data_in`. Ignored while the transmitter is busy.
`tx`	out	`std_logic`	The serial line. **Idles high**, including during reset.
`busy`	out	`std_logic`	High from the moment a frame starts until the stop bit is over.

What to build: the transmitter

a) Draw the state machine first. Four states is the natural decomposition: idle, start bit, data bits, stop bit, and each one holds the line at a defined level for one bit period:

- Mark what drives `tx` in each state. Notice that `tx` never depends on `send` or any other primary input: it is the state, plus the shift register's current output. That makes this a Moore machine, like everything else in this book.
- Mark where `busy` comes from. It is one comparison against the state.
- Decide where the byte is captured. It has to be latched when the frame starts, not read continuously from `data_in`: the producer is free to change `data_in` the moment `busy` goes high, and a real one will.

b) Shift the byte out most significant bit first, in the machine itself, the mirror of what the receiver does:

- Latch `data_in` into a `frame` signal when the frame starts.
- Drive `tx` from the top bit of `frame` while in the data state.
- Shift `frame` up one place per bit period, and count the eight bits out.

You wrote this as a module in Chapter 6's Exercise 6.7 b), and `piso8` is worth rereading for the idiom. Instantiating it here would work, but the machine already owns the bit counter that a PISO register needs, so the shift is one line inside the state it belongs to. That is how both halves of this link are built.

c) Implement it, instantiating `reset_sync` for the reset and `timer` with `BIT_TICK_COUNT` passed straight through.
Here the timer's `enable` **is** driven by the state, high only while a frame is in flight, and that is the one place this design deliberately differs from Exercise 8.6. The receiver had to leave its timer free-running because it does not control when a frame starts: it can only re-phase its own tick count once the start edge has already arrived. The transmitter has the opposite problem. It *decides* when the frame starts, so it can start the timer at that moment and get a first bit period exactly as long as every other one. Free-run it

here instead and the start bit is short by however much of a tick period happened to be left when `send` arrived, which is a defect in the one thing a transmitter is responsible for. Two requirements the testbench is strict about:

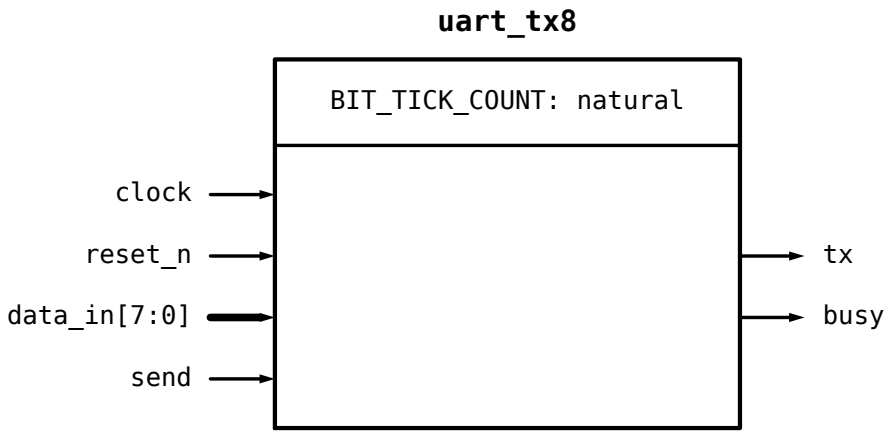
- **`send` is ignored while `busy`.** Reloading mid-frame would restart the byte and corrupt what is already on the wire. A request that arrives during a frame is dropped, not queued.
- **`busy` stays high through the whole stop bit**, not just the data bits. The frame is not over until the line has been held high for that last bit period.

d) Run the testbench. It acts as the receiver: it waits for your start bit, samples the middle of each bit period, and checks the framing, the byte, the `busy` handshake, and that a `send` injected two bit periods into a frame is ignored.

```
cd lectures/L08/exercises/uart_tx8
cp ../../../L04/exercises/reset_sync/reset_sync.vhd .     # the two you wrote ↪
    ↪ yourself
cp ../../../L07/exercises/timer/timer.vhd .
ghdl -a --std=93 reset_sync.vhd timer.vhd uart_tx8.vhd uart_tx8_tb.vhd
ghdl -e --std=93 uart_tx8_tb
ghdl -r --std=93 uart_tx8_tb --assert-level=error --stop-time=10ms
```

e) Answer, in prose:

- A transmitter that skips the stop bit entirely still leaves the line high afterwards, because idle and stop are the same level. What actually goes wrong, and when does it first show up?
  - The testbench catches this one. Which of its checks does it, and why could no amount of watching `tx` alone have caught it?
- Your receiver in Exercise 8.6 samples in the middle of each bit; your transmitter changes the line at the edges. If both ran off crystals differing by 1%, roughly how many bits into a frame would the receiver's sampling point drift into the wrong bit? What does that tell you about why serial formats have a start bit per byte, rather than one at the beginning of a long message?



Self-check: name your entity `uart_tx8`, with generic `BIT_TICK_COUNT` (`natural`, default `5207`), inputs `clock`, `reset_n`, `data_in` (`std_logic_vector(7 downto 0)`), `send`, and outputs `tx`, `busy`, declared in that order. Its testbench is in `exercises/uart_tx8`, and nothing else is: copy in your own `reset_sync` and `timer`, as part **d**) shows.

APPENDIX A

Short Answers

This appendix answers the parts of the exercises that you work on paper: the truth tables, the equations, the traces, the numbers, and the questions whose answer is a sentence or two. It gives the answer and no more, so that an answer worked on paper can be checked on paper; how to reach it is in the chapter the exercise follows.

Two kinds of part are not here. **The VHDL** you are asked to write is checked by the testbench the exercise ships, and its reference solutions are published in the companion repository, under `lectures/LNN/solutions`. **The CircuitVerse builds** are checked by simulating them against a truth table or a trace you already have; where a part asks you to predict what the simulation will show, the prediction is answered here.

An entry is headed by the exercise it answers, with the page the exercise is on; the letters are its parts. A part that is not listed has nothing to answer on paper.

CHAPTER 1

Combinational Logic and First VHDL

Exercise 1.1 A function that ignores an input

page 11

a) `X = A'B' + AB'.`

b) `X = (A' + A)B' = 1 · B' = B'.`

c) `X` does not depend on `A` at all. The gate is a single `NOT` on `B`, and the `A` input is left unconnected.

Exercise 1.3 Three-input XOR

page 13

a)

abc	000	001	010	011	100	101	110	111
x	0	1	1	0	1	0	0	1

x is 1 in exactly the four rows with one or three 1s.

Exercise 1.4 Majority gate

page 14

a)

abc	000	001	010	011	100	101	110	111
x	0	0	0	1	0	1	1	1

b) `x = a'bc + ab'c + abc' + abc`.

Looking ahead: Four 3-input ANDs, one 4-input OR and three `NOT`s, eight gates; `ab + ac + bc` is three 2-input ANDs and one 3-input OR, four gates.

Exercise 1.6 OR from NAND

page 16

a) `(A'B')' = A'' + B'' = A + B`. Read as a circuit, `A'B'` with an inverted output is a `NAND` of `A'` and `B'`, and each inversion is a `NAND` with its inputs tied together: three `NAND` gates.

b) `NOT`: one `NAND`; `AND`: two; `OR`: three.

d) A process that offers one gate type has one gate to design, characterize and lay out, and because `NAND` can build every function, a library of it alone is complete. `NAND` is the cheapest CMOS gate, four transistors with no output inverter, where `AND` is a `NAND` plus an inverter, so building from it is the cheapest way to build anything.

Exercise 1.7 The braking assistant

page 17

b)

- It fails first at `driver_brake = 1`, `sensor = 0`, `radar = 0`, `error = 1`, the testbench's case `i = 9`: the driver is on the pedal, the assistance system is reporting a fault, and the car does not brake.
- Four rows, `i = 9`, `11`, `13` and `15`: all of them have `driver_brake = 1` and `error = 1`.
- A truth table derived from the requirement is an independent check that catches the mistake; one read off the circuit you built would contain the same mistake and agree with it.

CHAPTER 2
Larger Networks, Multiplexers and Submodules

Exercise 2.1 A three-variable map

page 38

Equation: `X = B'`: one group of four, the rows `AB = 00` and `AB = 10`, which are adjacent through the top/bottom wraparound. The network is a single `NOT` on `B`.

Exercise 2.2 A four-variable map

page 38

Equation: `X = B'D + ABD'`. `B'D` is the 2 × 2 block at `CD = 01, 11` in rows `AB = 00, 10`, wrapping top to bottom; `ABD'` is the pair at `CD = 00, 10` in row `AB = 11`, wrapping left to right.

Exercise 2.3 A three-output network

page 39

a) With `AB` down the rows and `CD` across the columns:

- `X = A'B + AB'` (`A xor B`): the rows `AB = 01` and `AB = 10`, four cells each.
- `Y = D`: the eight cells in the columns `CD = 01` and `CD = 11`.
- `Z = B'D' + CD`: the four corners, wrapping both ways, and the column `CD = 11`. A group `B'C` is also possible, but every cell of it is already covered.

Exercise 2.4 The 4-to-1 mux by hand

page 40

a)

`S[1:0]`	00	01	10	11
`X`	A	B	C	D

b) `X = AS1'S0' + BS1'S0 + CS1S0' + DS1S0`.

Exercise 2.7 One 7-segment digit

page 44

A to F: `A "0001000"`, `b "0000011"`, `C "1000110"`, `d "0100001"`, `E "0000110"`, `F "0001110"`.

Exercise 2.9 A digit as gates

page 46

Segments: With `number` as `ABCD`, `A` the most significant bit, the minimized equations for each output bit `hex(i)`, which is `1` where the segment is off, and for its complement, where the segment is lit:

Segment	**`hex(i) = 1`, off**	**Lit**
`a, hex(0)`	`A'B'C'D + A'BC'D' + AB'CD + ABC'D`	`A'C + BC + AD' + B'D' + A'BD + AB'C'`
`b, hex(1)`	`A'BC'D + BCD' + ABD' + ACD`	`A'B' + B'D' + A'C'D' + A'CD + AC'D`
`c, hex(2)`	`A'B'CD' + ABD' + ABC`	`A'B + AB' + A'C' + A'D + C'D`
`d, hex(3)`	`A'B'C'D + A'BC'D' + BCD + AB'CD'`	`A'B'D' + B'CD + BC'D + BCD' + AC'`
`e, hex(4)`	`A'D + B'C'D + A'BC'`	`B'D' + CD' + AB + AC`
`f, hex(5)`	`A'B'D + A'B'C + A'CD + ABC'D`	`C'D' + BD' + A'BC' + AB' + AC`
`g, hex(6)`	`A'B'C' + A'BCD + ABC'D'`	`B'C + CD' + A'BC' + AB' + AD`

Segment `a`'s off-set, digits `1`, `4`, `b` and `d`, has no two adjacent cells, so it does not minimize at all. The lit forms of `b` and `c` are not unique: other covers with the same number of terms are as small.

Exercise 2.10 Naming a sub-expression

page 47

Same circuit: Both architectures give this `x` in every row, so they describe the same circuit:

`abcd`	0000	0001	0010	0011	0100	0101	0110	0111
`x`	0	1	0	0	0	1	0	0
`abcd`	1000	1001	1010	1011	1100	1101	1110	1111
`x`	0	1	0	0	1	1	1	1

CHAPTER 3
Sequential Logic

Exercise 3.1 A D latch

page 63

b) With `enable = 1`, `Q` follows `D` at once and `Qn = D'`: the latch is transparent. With `enable = 0`, `Q` and `Qn` hold whatever `D` does: both AND terms are `0`, so each NOR only inverts the other's output, and the loop keeps its value.

c) `Q` is the value `D` had at the instant `enable` fell, held until `enable` rises again. Taking `enable` low first wherever both switches change:

`D enable`	00	01	10	11	00	01	10	11
`Q`	–	0	0	1	1	0	0	1

`10` holds the `0` from `01`, and `00` holds the `1` from `11`; the first `00` shows whatever the latch held before.

Exercise 3.2 A D flip-flop from two latches

page 64

b) With `enable = 1`, `Q` and `Qn` change only at a rising clock edge, to the value `D` had at that edge; a change of `D` in either phase shows at the next rising edge. While the clock is low the master is transparent and follows `D` and the slave is closed; at the rising edge the master closes, freezing `D`, and the slave opens and passes that value to `Q`; while the clock is high the closed master keeps `D` away from `Q`.

c) `Q` goes to `0` immediately, without waiting for a clock edge, and releasing `reset_n` changes nothing until the next rising edge. The reset is asynchronous: it acts on both latches' NOR gates directly, whatever the clock is doing, where a synchronous signal is only acted on at a clock edge.

Exercise 3.3 A four-bit register

page 65

c) Each register flip-flop's next value is its own bit of `d`, so the four are independent, identical copies and widening the vector is all it takes. A counter's bit `i` depends on its own value and the carry from every bit below it, so its flip-flops are coupled through an adder; four one-bit counters side by side would all toggle together, `0000`, `1111`, `0000`, rather than count.

Exercise 3.4 An edge-detected toggle

page 67

b) The flip-flop holding the previous sampled value of `button`. Once an edge has sampled the button high it holds `1`, so `edge = button · previous'` stays `0` for as long as the button is held, and `led` toggles only once.

CHAPTER 4

Metastability and Synchronization

Exercise 4.1 Metastability in your own words

page 84

An input changing inside the setup/hold window can leave the flip-flop's cross-coupled feedback loop balanced between its two stable states, so its output sits at a voltage that is neither a valid `0` nor a valid `1`. It resolves on its own, but how long that takes depends on how nearly balanced it was left, which nobody controls and which differs every time. Until it resolves, each downstream gate reads the voltage against its own threshold, so one can see `0` and another `1` at the same instant, and the parts of the circuit they feed act on different values: a counter increments in one place and not another, a state machine lands in an impossible state.

Exercise 4.2 The setup-and-hold window

page 85

A synchronous signal is launched by an edge of the same clock, so it arrives a fixed, known delay after that edge and can be designed to settle before the next edge's window. A pushbutton changes whenever a person presses it, with no relation to the clock's phase, so over enough presses some changes will fall inside the window however the logic around it is built. The risk therefore cannot be designed away, only contained: by giving the unresolved value time to settle before anything uses it.

Exercise 4.3 Drawing a double-flop synchronizer

page 85

`FF1` samples the asynchronous input directly, so it is the stage whose input can change inside its window and go metastable. Its output has a whole clock period to resolve before `FF2` samples it, and the chance of still being unresolved shrinks exponentially with that time, so `FF2` goes wrong only in the rare case where `FF1` stayed unresolved for the whole period; that is why only `FF2`'s output is used. The residual probability is not zero, but the resulting mean time between failures is so long (far beyond the product's life, at these speeds) that the risk is acceptable.

Exercise 4.4 A third synchronizer stage

page 86

a) It turns on the required mean time between failures against the failure rate this input produces. A third stage wins when two stages do not give an MTBF comfortably beyond what is required: a fast-changing input, a flip-flop with poor metastability characteristics, or a failure that must essentially never happen. Two stages are enough when their MTBF already exceeds the requirement by orders of magnitude, and then the third only costs a cycle of latency and a flip-flop per input.

b)

- A frequently changing input (high `f_data`) lowers MTBF in proportion and pushes towards a third stage; a button pressed once a second pushes away from it.

- A family with a slow decay (large `tau`) or a wide window (`T_w`) lowers MTBF and pushes towards a third stage; `tau` sits in the exponent, so it matters most.
- A costly failure (safety, money) raises the MTBF you need, pushing towards a third stage; a demo LED does not.

c) Two stages: a button changing a few times a second already gives an astronomically long MTBF, and a third stage buys nothing but latency. A third stage is worth arguing for on, say, an asynchronous line toggling at around 10 MHz in a braking controller, where `f_data` and the cost of a failure do the work.

Exercise 4.5 Assert async, release sync

page 86

Assertion: Both stages go low the instant `reset_n` does, without waiting for an edge. That matters when the clock is stopped (the edge never comes), unstable (its edges cannot be trusted, as at power-up), or when the system must reach its safe state at once.

Recovery and removal: Releasing an asynchronous reset is itself an asynchronous change: if the release lands inside the window around a clock edge, the flip-flop may leave reset on that edge or the next, or go metastable.

Synchronous release: The design sees the release only through the two-flop chain, so `reset_s2_n` rises just after a clock edge, a full period clear of the next; every flip-flop sees the same clean release and leaves reset on the same edge, so no two of them can disagree for a cycle about the state.

Exercise 4.6 `led_toggle_sync` by hand

page 87

e)

- No. The first edge only loads the press into `button_s1_n`.
- Two edges: on the second, `button_s2_n` goes to `0` while `button_s3_n` still holds `1`, so `button_edge_s2` is `1` for one cycle. The LED toggles on the third edge, the same edge that ends the pulse.
- Yes: a small, fixed latency, two edges through the synchronizer and one more for the toggle flip-flop, as §4.3 and §4.9 predict.

Exercise 4.7 Synchronized is not debounced

page 88

a) Synchronization and edge detection; not debouncing.

b) No more than one sampling edge fell inside the bounce, so the samples went from released to pressed exactly once and the chain saw one clean transition. The design controls only its sampling (the clock period), not the bounce, which the switch decides; with a 1 s period a 1 ms bounce almost always fits between two edges, but nothing in the circuit makes it.

c)

- At most once, and usually not at all: about one press in a thousand has an edge inside the bounce.
- 1 ms / 20 ns = 50 000 times.
- Every sampled bounce is a fresh falling edge, so the LED toggles several times for one press and ends on or off unpredictably. In CircuitVerse the whole bounce fell between two edges a second apart and was never seen.

d) Hardware: an RC low-pass filter smooths the bounce into one slow edge, and a Schmitt-trigger input turns that into a single clean transition. Digital: after the synchronizer, accept a new level only once it has held for a few milliseconds' worth of cycles, restarting the count on any change; or, with Chapter 7's timer, ignore further edges for a fixed interval after a press.

e) Debouncing removes the extra transitions but not the asynchrony: the debounced signal still changes with no relation to the clock and can still land inside a flip-flop's setup/hold window, so metastability still needs the double-flop chain.

Exercise 4.8 The reset synchronizer

page 90

b) The reset branch forces the in-reset value, `'0'`, at once, while the clocked branch shifts in the released value, `'1'`, which is how the release reaches the output: one branch asserts the reset and the other releases it.

d)

- `s2` says the signal is the output of the second flip-flop of a two-flop synchronizer; the name would become wrong if the number of stages changed, say to three.
- Some module has to take the raw asynchronous reset, because a synchronized one does not exist until it has made it. Exactly one does, so that every flip-flop in the design is released by the same `reset_s2_n` on the same edge; two reset synchronizers could release on different edges.

Exercise 4.9 The button synchronizer

page 92

b)

- No. From all-zeros, the released level moves through as `s1 s2 s3` = `000`, `100`, `110`, `111`, and `(not s2) and s3` is `0` at every step: the chain sees a release, and a release does not pulse.
- All-ones: one pulse, two edges after the reset releases. All-zeros: no pulse at all, because the chain already reads "pressed" and never sees a falling edge. All-ones is the one you want: the press is real, and all-zeros silently loses it.

d)

- An out-of-range width, above all `COUNT = 0` with its null-range ports, is rejected when the design is elaborated instead of being built into something meaningless, and the interface states the widths the module is meant for.

- Chapter 8's `seq_detect_mealy`, which has no button but still needs its reset synchronized. Merged, it would have had to instantiate the whole module and disable the button half it does not use.

Exercise 4.10 Two synchronized LED toggles page 94

c) Two synchronizer flip-flops bring the press to `button_s2_n` on the second edge, where the edge pulse goes high, and the LED flip-flop takes it on the third. An extra pipeline stage would move the toggle to the fourth edge, and the testbench, which expects the toggle on the third, fails it.

d)

- It makes each press safe from metastability, but does nothing about bounce: at 50 MHz every bounce is still its own pulse, so one press can still toggle the LED several times.
- A physical button is asynchronous to the 50 MHz clock, and the raw one reached two flip-flops on the same edge, `button_prev_n` directly and `led_s` through the edge logic; a press inside the setup/hold window could leave either metastable, or be read as pressed by one and released by the other, losing the toggle or doubling it.
- Metastability: a simulated flip-flop always captures a clean `'0'` or `'1'`. Bounce, by contrast, a testbench could at least imitate by driving a burst of transitions.

CHAPTER 5

Variables and the Hardware Underneath

Exercise 5.1 Tracing the signal accumulator page 106

a)

`i`	`<= '0'`	7	6	5	4	3	2	1	0
`bits(i)`	–	1	0	1	1	0	1	1	0
read	–	0	0	0	0	0	0	0	0
scheduled	0	1	0	1	1	0	1	1	0
survives	no	no	no	no	no	no	no	no	yes

b) `parity_s` ends at `'0'`. The correct parity is `'1'`: `"10110110"` has five 1s. Every read in the pass returns the same frozen pre-pass value, and each assignment replaces the one scheduled before it, so no iteration ever sees another's result and only the last, `old_parity xor bits(0)`, survives.

c)

`i`	7	6	5	4	3	2	1	0
`bits(i)`	1	0	1	1	0	1	1	0
`acc` before	0	1	1	0	1	1	0	1
`acc` after	1	1	0	1	1	0	1	1

Each iteration reads the value the previous one wrote, and `parity` ends at the correct `'1'`.

Exercise 5.3 Counting set bits

page 108

Signal accumulator: The testbench stops at that input, `expected 1, got 2`. Every read of the signal in the pass returns `1`, the count left by the previous input `00000001`; the last assignment that runs, at `i = 1`, schedules `1 + 1`, and it is the one that survives.

Adders and cycles: Eight adders, one per unrolled iteration, chained (and then packed into LUTs), and no clock cycles at all: the process is combinational and the count settles within one propagation delay.

The range: It tells synthesis the value needs only four bits, and it is a promise the simulator checks: if the count ever left `0 to 8`, GHDL would stop with a bound-check failure at the offending line, where a `std_logic_vector(3 downto 0)` would hold `9` to `15` without a word.

Exercise 5.4 A synchronizer module

page 109

b)

- That every button is being held down, until the real released level arrives.
- The line falling low straight after reset: a start bit, so the receiver would start receiving a frame that was never sent.
- `'0'` is a flip-flop's ordinary reset value, as in every other reset in the book, but no default can know what a signal means: both real users override it, so getting `PRESET` right is the instantiating design's job, and it should always pass it explicitly.

c) Eight flip-flops at `SIZE = 4`, two at `SIZE = 1`. `PRESET` ends up as each flip-flop's reset value, whether its asynchronous reset clears or presets it. Both generics were used up before synthesis placed anything: `SIZE` decided how many flip-flops to build and how wide the ports are, `PRESET` how each one resets.

Exercise 5.5 What Quartus does to your VHDL

page 111

a)

- It is the root of the design: its ports become the chip's pins, and every other entity is built only where it is instantiated beneath it.
- The testbench has no ports to put on pins, and its body (timed waits, assertions, reports) describes a simulation, not hardware, so it cannot be synthesized; only `min_of_two` can.

b)

- The pins are an input to fitting: placement, routing and the programming file are all made for them, and a `.sof` compiled before a change is not patched afterwards.
- To whatever pins the fitter picked on its own, not the LEDs you meant; some project settings refuse to fit unassigned pins at all.

c)

- The same circuit for both, one four-input LUT holding the truth table of `x = ab + c'd`: the internal signal only names a wire, and synthesis builds the function, not the text.
- At two terms, honestly either; the named version pulls ahead as soon as the expression outgrows one line, and the habit is worth having before you need it.

d)

- A clean C build does what the code says; legal VHDL can still describe hardware you did not mean, such as a latch, a combinational loop, an unassigned pin or an incomplete sensitivity list, and Quartus reports all of those as warnings.
- An inferred latch.

Exercise 5.6 Predicting parity

page 112

a) `"00000000": '0'; "10110110": '1'; "11111111": '0'; "10000000": '1'.`

b) It sweeps all 256 inputs, these four among them, and prints only `parity_gen: all checks passed!`.

c)

- It inverts.
- Parity is the XOR of all eight bits, and XORing in a flipped bit complements the result: the number of 1s changes by exactly one, so odd becomes even or even odd.
- An even number of flipped bits, two for instance: each inverts the parity, so the second undoes the first and the parity still matches.

Exercise 5.7 Reviewing a latched decoder

page 113

a) A latch on each of the four bits of `leds`. The `when others` branch assigns nothing, so on that path VHDL requires `leds` to keep its old value; remembering is something combinational logic cannot do, so a latch is the only thing the tool can build.

b)

- `sel` is a `std_logic_vector`, each bit with nine values, so `when others` matches 78 of the 81 values it can hold (`'U'` before it is driven, `'X'`, `'Z'` and the rest), not just `"11"`.
- The latch is inferred from the code, not from the values that occur: the process still has a path that leaves `leds` unassigned, so the tool builds the latch whatever `sel` does, and the design contains storage nobody asked for.

c) In the branch, `when others => leds <= "0000";`. Before the `case`, the default `leds <= "0000";`. The second is easier to maintain: the default covers every path, including branches added later, where the first has to be got right again each time the `case` grows. Both give the same combinational logic.

d) An error stops you; a warning lets the design through, and because the latch usually holds the right value the design appears to work, so the bug ships and surfaces later as an intermittent fault in a design that "compiled cleanly".

CHAPTER 6

Counters and Shift Registers

Exercise 6.1 Sizing a counter register

page 128

a) 10 bits.

b) 1023.

c) A 6-bit counter: `signal counter: natural range 0 to 63;`

Exercise 6.2 A counter traced by hand

page 128

Trace:

Edge	1	2	3	4	5
Before	2	3	0	1	2
Captured	3	0	1	2	3

Why it wraps: The range needs only two bits. On the edge after `3` the adder produces `100`; the carry has no flip-flop to go into, and the two bits that remain are `00`.

Exercise 6.3 Overflow in CircuitVerse

page 128

b) One count per edge, `0000` up to `1111`, and back to `0000` on the sixteenth edge. `carry_out` is high on one of the sixteen, while the count is `1111` and the register rolls over, and nothing is connected to it. The fifth bit leaves the adder and goes nowhere: the register has only four `D` inputs, so it stores the low four bits of `10000`, which are `0000`.

c) No part of the drawing decides it: the register's four-bit width does the work, by discarding the adder's carry-out. To count `0` to `9`, add an equality comparator that detects `1001` and a clear that forces the register to `0000` on the next edge.

d) It wraps from `111` to `000`, after `7`. An N-bit counter counts `0` to `2^N - 1` and wraps every 2^N edges.

Exercise 6.5 Tracing a SIPO register

page 131

a)

Edge	`serial_in`	`shift_reg` after
1	1	`00000001`
2	0	`00000010`
3	1	`00000101`
4	1	`00001011`

b) Bits 3 down to 0, holding `1011`. Read from bit 3 down to bit 0 they are in arrival order: the first bit to arrive is in bit 3, the most recent in bit 0.

Exercise 6.6 Tracing a PISO register

page 132

a)

Edge	Register	`serial_out`
Load	`1010`	1
Shift 1	`0100`	0
Shift 2	`1000`	1
Shift 3	`0000`	0
Shift 4	`0000`	0

b) Most significant bit first: bit 3, then 2, 1 and 0 of `parallel_in`, that is `1`, `0`, `1`, `0`.

c) `0000`. All four loaded bits have been shifted out of the top, and each shift filled bit 0 with `serial_in = '0'`.

Exercise 6.7 SIPO and PISO in VHDL

page 132

c) SISO: a delay line, holding a signal back a fixed number of clock cycles. SIPO: receiving a serial bit stream, such as an incoming byte on a single wire. PISO: sending a word out over a single wire, such as driving a chain of LEDs or an external shift-register chip. PIPO: a plain register, holding a word.

Exercise 6.8 Accounting for the receiver

page 135

a) `10110010`. The first bit to arrive, `1`, is in `data_out(7)`; the last, `0`, is in `data_out(0)`.

b) Yes: `10110010`, with `data_ready` pulsing on the eighth bit. During the gap the bit counter holds at `4`, and the register at `00001011`; both resume on the fifth bit. If only the shift were gated, the counter would count the three idle edges too, so `data_ready` would pulse as the fifth bit lands, with the partial byte `00010110` on `data_out`, not pulse when the byte really completes, and stay out of step with the byte boundaries from then on.

c)

- Mirror-image shift: `data_ready` still pulses on time, but the byte-check fails, "`serial_rx8: wrong byte on data_out for BYTE1!`", because the byte arrives bit-reversed, `01001101`. On a real link every byte would be read reversed, with correct framing, so nothing but the data would look wrong.
- No unconditional `data_ready <= '0';`: `data_ready` rises on the first byte and never falls, caught on the first idle cycle by "`serial_rx8: data_ready must be a single-cycle pulse!`". On a real link the consumer would see a byte ready on every cycle after the first, and could never tell when a later byte completed.
- Comparison against 6: `data_ready` pulses on the seventh bit of the first byte, caught by "`serial_rx8: data_ready fired before eight bits arrived!`". On a real link the receiver would report 7-bit words, each missing its last bit, with the framing slipping one bit further from the real byte boundaries on every byte.

CHAPTER 7
Timers

Exercise 7.1 Retargeting the timer by hand

page 148

a) `12` is `1100`. Two constants change: `Q2`'s from `0` to `1` and `Q1`'s from `1` to `0`; `Q3`'s stays `1` and `Q0`'s stays `0`. Each XNOR only tests one bit for equality with its constant, so the target lives entirely in the constants and any 4-bit value is the same four XNORs and one AND.

c) Every 16 edges. Without the clear the counter runs on past `1100` through `1111` and wraps to `0000` on its own, so it reads `1100` once per sixteen-state lap.

d) Two counts: `1100` and the count that differs from it only in the disconnected bit (`0100`, `1000`, `1110` or `1101` for `Q3`, `Q2`, `Q1` or `Q0`). With the clear in place the counter is cleared at whichever it reaches first, so a broken `Q3` or `Q2` fires at 4 or 8 instead of 12, while a broken `Q1` or `Q0` goes unseen, because 14 and 13 are never reached; without the clear both counts fire on every lap.

Exercise 7.2 Reading the timer module

page 149

a) It is set only on the edge where the counter is already at `TICK_COUNT` and is cleared, and the unconditional `timeout <= '0';` at the top of the clocked branch takes it low again on the next edge. One pulse per period is one event, usable downstream as a one-cycle enable; a level would act on every cycle it stayed high.

b) It pauses: the counter holds its value, `timeout` stays low, and counting resumes from there when `enable` returns high.

c) The timer is a part inside a larger design, and its flip-flops must leave reset on the same clock edge as everything else. The raw `reset_n` is released asynchronously, which could let them leave reset on different edges or go metastable; the design around it synchronizes the reset once, with `reset_sync`.

d) The `TICK_COUNT` generic, set per instance in its `generic map`. Its value is fixed when the design is elaborated, before synthesis runs, and cannot change while the circuit runs.

Exercise 7.3 Tick counts at 50 MHz

page 149

a) `49_999_999`.

b) `12_499_999`.

c) `2_499_999`.

Exercise 7.4 Writing the timer

page 150

A clearing enable: Case 5 fails, with "`timer: disabling must pause the count, not clear it...`": a re-enabled timer that was cleared needs a full `TICK_COUNT + 1` edges and is still low when the interrupted period should end. It is measuring that `enable` holds the count and resumes it, not the period or the pulse width.

The "+ 1": In the comparison: the counter increments while `TICK_COUNT > counter` and is cleared only on the edge where it already equals `TICK_COUNT`, so the value `TICK_COUNT` itself takes a cycle of its own.

Decoded versus registered: The decode is high during the cycle the count reads `TICK_COUNT`; the registered `timeout` is high one cycle later, while `counter` reads `0`. Downstream logic sees the same period a cycle apart, and only the registered pulse is clean: it comes straight off a flip-flop, where the decode can glitch while the counter bits change.

Exercise 7.5 A one-second blinker

page 151

Half a second, one blink: Each timeout toggles the LED once, and one blink is two toggles, on then off, so a full cycle takes two half-second timer periods.

LED off during reset: It checks the asynchronous assertion. It does not check the synchronous release, which the testbench checks separately through when the first toggle arrives.

Raw reset into the timer: A reset-release race: the timer would leave reset two edges before the LED flip-flop, and a release asynchronous to the clock could let its own flip-flops leave reset on different edges or go metastable, starting the count from a corrupt value. The testbench sees the first toggle 5 edges after `reset_n` is released instead of 7.

Exercise 7.6 The walking LED

page 154

A clearing timer: Every restart waits a full timer period, half a second, before the first step, however much of the interrupted step had already passed, instead of finishing that step; a walk stopped and restarted within half a second at a time would never move.

A fixed width: Two copies of the same module, a one-button and a two-button `button_sync` (or the file edited for each design), to be kept identical by hand.

Exercise 7.7 Changing the walking LED

page 156

a) At `TICK_COUNT = 6` it expects `60 / 7 = 8` shifts, give or take one, in its 60-cycle window. The expected count is $\lfloor 60/(\mathit{TICK_COUNT}+1) \rfloor$: from `TICK_COUNT = 30` it is 1, and the tolerance already accepts a walk that never moves; past the window, from `TICK_COUNT = 60`, it is 0, and the check proves only that the LED moved at most once. `TICK_COUNT` only sets how many clock cycles pass between timeout pulses, and everything else does the same on each pulse, so a larger value only lengthens the step.

b) Its widths would have been written as literals rather than in terms of `LED_COUNT`: a port `std_logic_vector(3 downto 0)`, a rotate `shift_reg(2 downto 0) & shift_reg(3)`, a reset value `"0001"`.

c) The line that loads the reset pattern in `SHIFT_PROCESS`'s reset branch, `shift_reg <= (0 => '1', others => '0');`. The testbench fails its very first check, with "`walking_led: reset should load a single lit bit at position 0!`". The design is right for the new specification; the testbench's `RESET_PATTERN` still asserts the old one, and with it changed to the new pattern every check passes.

Exercise 7.8 Reversing the walk

page 157

d)

- The testbench's port map connects its one-bit `std_logic` signal `button_n` to the now two-bit port, so the testbench no longer analyses. GHDL reports `can't associate "button_n" with port "button_n"`, giving the actual's type as `STD_LOGIC` and the port's as a subtype of `STD_LOGIC_VECTOR`.
- After reset, bit 0 is lit and still. The first button starts a walk toward the MSB, wrapping to bit 0; the second button reverses it, the bit now walking toward bit 0 and wrapping to the top; the second button again restores the original direction; the first button stops it, and a second-button press while stopped moves nothing but reverses the next walk; reset puts bit 0 back, stopped.

CHAPTER 8

State Machines

Exercise 8.1 A binary-count Moore machine

page 177

a) Four bubbles, `00/0`, `01/0`, `10/1` and `11/1` as `Q1Q2/Y`, in a ring `00 -> 01 -> 10 -> 11 -> 00` with one arrow per state labelled `X = 1`, a self-loop labelled `X = 0` on each, and reset entering `00`.

b)

`Q1Q2X`	000	001	010	011	100	101	110	111
`Q1+`	0	0	0	1	1	1	1	0
`Q2+`	0	1	1	0	0	1	1	0
`Y`	0	0	0	0	1	1	1	1

c) `Q1+ = Q1X' + Q1Q2' + Q1'Q2X`: two pairs and the lone cell `011`, all three essential (it is `Q1` XOR `Q2X`). `Q2+ = Q2X' + Q2'X`, two pairs, which is `Q2 XOR X`. `Y = Q1`, one group of four.

Tip: With the LED lit only in `10`, `Y = Q1Q2'` is decoded from both bits, and on the `11 -> 00` transition, if `Q2` falls before `Q1`, the state reads `10` for an instant and `Y` glitches high. If `Y` drove something that cared, register it: a flip-flop loaded with `Q1+Q2+'`, so `Y` changes only on the clock edge.

Exercise 8.2 A Moore `1, 0, 1` detector

page 178

a) Four states: `S0` nothing matched, `S1` saw `1`, `S2` saw `10`, and `S3` saw `101`, the only state with `y = 1`; reset enters `S0`. Its arrows are the rows of the table in **b**).

b)

state, `din`	`S0`,0	`S0`,1	`S1`,0	`S1`,1	`S2`,0	`S2`,1	`S3`,0	`S3`,1
next state	`S0`	`S1`	`S2`	`S1`	`S0`	`S3`	`S0`	`S1`
`y`	0	0	0	0	0	0	1	1

c) For example, `din = 1, 1, 0, 1, 0, 1`:

cycle	1	2	3	4	5	6
state before	S0	S1	S1	S2	S3	S0
din	1	1	0	1	0	1
y	0	0	0	0	1	0

y is high in cycle 5, the cycle after the match's last bit; the `1, 0, 1` in bits 4 to 6 overlaps the match and is correctly not reported.

d) A `0` in `S0`, `S2` or `S3` returns to the start. A `1` in `S1` must stay in `S1`, not fall back to `S0`, because that `1` is itself the first bit of a possible match (`1101` ends in `101`); likewise a `1` in `S3` goes to `S1`, since non-overlapping discards the match's bits but not a new one. A `0` in `S2` goes all the way back, because `100` ends in no prefix of `101`.

Exercise 8.5 Accounting for `fsm_led`

page 181

a) A timeout comes every `TICK_COUNT + 1` = 5,000,001 cycles of 20 ns, 100 ms. The blink frequency is 5 Hz: the LED toggles once per timeout, so one on-and-off cycle takes two timeouts, 200 ms, and the 10 Hz you might first write down is the toggle rate.

b) The override is `constant TIMER_TICK_COUNT: natural := 3;`, passed in by `generic map(TIMER_TICK_COUNT)`. One real blink period is 2 × 5,000,001, about ten million clock cycles: 200 ms of simulated time at 50 MHz, far past the 10 ms `--stop-time`, and tens of thousands of times longer than the whole testbench takes with the override.

c)

- From `STATE_OFF`, three presses of `button_n(0)` visit `STATE_BLINK`, `STATE_ON` and `STATE_OFF` in turn; three presses of `button_n(1)` visit `STATE_ON`, `STATE_BLINK` and `STATE_OFF`.
- The machine stays where it is. The two lines are the assignments to `to_prev_state` and `to_next_state`, each of which requires the other button's edge to be `'0'`.
- The timer resumes from wherever it was frozen, so the first toggle after re-entry comes anywhere from one cycle to a full period later, and the first blink interval is short. Holding `timer_enable` high is neither better nor worse for correctness, since re-entry would still catch the timer at an arbitrary point in its period; it only keeps a counter switching, and spending power, when nothing reads it.

Exercise 8.6 The serial receiver

page 182

a)

- `STATE_IDLE`: leaves on a tick with `rx_s2 = '0'` and the previous tick's `rx_s2 = '1'`, clearing `ticks`; the history is one flip-flop loaded with `rx_s2` on every tick.
- `STATE_START`: waits 8 ticks; then `rx_s2` low goes to `STATE_DATA`, clearing `ticks`, and high goes back to `STATE_IDLE`.
- `STATE_DATA`: waits 16 ticks per bit, eight times; each sample shifts `rx_s2` into `frame` and clears `ticks`, and the eighth moves on to `STATE_STOP`.

- `STATE_STOP`: waits 16 ticks; then goes to `STATE_IDLE`, pulsing `byte_valid` with `data_out <= frame` if `rx_s2` is high, and `frame_err` if it is low.
- Every state has a self-loop for "no tick yet, or not the right count yet".

b)

- 50,000,000 / 9600 = 5208.33 cycles per bit, and 325.52 per sixteenth.
- `TICK_COUNT` = 324.52. 324 gives 325-cycle ticks, 0.52 cycles short (−0.16%); 325 gives 326, 0.48 cycles long (+0.15%). The default is 325, the closer of the two.
- 9.5 bit periods is 152 ticks: 152 × 326 = 49,552 cycles against 49,479, about 73 cycles late, 0.22 of a tick. The margin was 8 ticks, half a bit.
- The drift within one frame would have to reach the margin: at 0.024 of a tick per bit, that is a frame of some 340 bit periods (less once the crystals' own tolerance adds to it).

e)

- The low-stop-bit frame is handed over as a byte and `frame_err` never pulses. The first check to fire is "a frame with a low stop bit must raise `frame_err`!", which the testbench makes before its byte-count check.
- Add a case that pulls `rx` low on an idle line for a few ticks, well under half a bit, then holds it high for longer than a frame, and asserts that neither `valid_count` nor `error_count` changes. Without the re-check the glitch starts a frame of all ones with a high stop bit, and `x"FF"` is handed over as a byte.
- The testbench's transmitter runs off the receiver's own clock at exactly sixteen ticks a bit, so nothing drifts, and even its noisy frame holds each bit clean from tick 3 to tick 13. What is given away is the margin on the early side, down from 8 ticks to 3 or less, so a slightly slow transmitter or a slow edge reads the previous bit.
- A simulator models every signal as a clean `'0'` or `'1'` changing at an exact instant, and every flip-flop as sampling perfectly; it has no setup and hold window, so it cannot produce metastability (and the testbench changes `rx` just after a clock edge anyway). The failure is physical and probabilistic, so only the rule prevents it.
- It promises that for the stimuli the testbench applies, in an ideal simulation, the outputs are the ones it checks for. It does not promise anything about inputs it never applies, such as glitches or drifting clocks, about timing margin, or about what the simulator does not model: metastability, glitches and timing on the real chip.

Exercise 8.7 The serial transmitter

page 189

a) Idle: `tx = '1';` on `send = '1'`, latch `data_in` into `frame` (the byte is captured here) and start the timer. Start bit: `tx = '0'` for one timeout. Data: `tx = frame(7)`, shifting `frame` up one place per timeout, eight times. Stop: `tx = '1'` for one timeout, then idle. `busy` is `'1'` whenever the state is not idle.

e)

- `busy` drops at the end of the eighth data bit, so a producer that sends the next byte straight away puts its start bit right after the last data bit, with no stop bit. The receiver samples its stop bit inside that start bit and rejects the frame with `frame_err`, and if the last data bit was `0` the new start bit has no falling edge for it to find. It first shows up on the first pair of bytes sent back to back; one byte followed by idle looks perfect.
- The check "`busy` must stay asserted for the whole stop bit!", made in the middle of the first frame's stop bit. On `tx` a skipped stop bit is idle, which is the same high level, so the line is identical until another frame follows; only `busy` shows that the transmitter thinks the frame is over.
- The sample point moves 1% of a bit per bit and starts half a bit from either edge, so it reaches the wrong bit about 50 bits in (nearer 44 counting the tick of start-edge uncertainty). A ten-bit frame drifts only a tenth of a bit, so a start bit per byte resynchronizes long before the drift matters, where a single start bit for a long message would lose it within a few bytes.

APPENDIX B

Quartus Prime Lite and the DE0-CV

This appendix is the toolchain reference for the book's board work. **Nobody needs a board to follow the book**: every exercise is checked on your own laptop with a testbench, and no chapter asks you to run Quartus. In the course this book comes from, the Quartus and DE0-CV work is demonstrated from the front, starting with the braking assistant in Chapter 1, and no participant needs Quartus or a board of their own. This appendix is for a reader who does have a DE0-CV and wants to put the book's designs on it, the way the course demonstrates them. It lives here rather than in a chapter because nobody following the book has to execute it.

It covers:

- installing Quartus Prime Lite (§B.2).
- once installed, taking a VHDL file from source to a running FPGA:
  - new project (§B.3).
  - adding sources (§B.4).
  - assigning pins (§B.5).
  - compiling (§B.6).
  - programming the board (§B.7).

Everything described here ends on real FPGA hardware, verified by hand. That is a separate layer from the exercises, which you verify on your own laptop by running a self-checking testbench under GHDL (see §2.8).

B.1 What You Need

- **Quartus Prime Lite** (free edition): Intel/Altera's FPGA design software, covering:
  - synthesis.
  - placement and routing ("fitting").
  - programming file generation.

- **Cyclone V device support**: a separate add-on package.
  - The DE0-CV's FPGA, `5CEBA4F23C7N`, is a Cyclone V part.
  - Quartus doesn't include every device family's support by default.
- **USB-Blaster driver**: the DE0-CV programs over USB using Altera's USB-Blaster protocol.
  - the driver usually installs alongside Quartus.
  - it occasionally needs a manual nudge (§B.8).
- A **Terasic DE0-CV board** and a USB cable.

Quartus Prime Lite is available for both Windows and Linux; the steps below are the same regardless of host OS except where noted (the Windows 11-specific gotcha in §B.2 doesn't apply on Linux).

B.2 Installing Quartus Prime Lite

1. Download Quartus Prime Lite from Intel's Quartus Prime Lite download page, `https://www.intel.com/content/www/us/en/software-kit/711791/intel-quartus-prime-lite-edition-design-software-version-20-1-for-windows.html`.
   - That link is the Windows kit for one particular release. Any recent Lite release works, on either host OS; this book doesn't depend on a specific point version, so pick the edition matching your platform from the same download centre.
2. During installation, make sure **Cyclone V** is selected under the device family options.
   - If Quartus is already installed without it, add Cyclone V support afterwards by either:
     - re-running the installer.
     - using Quartus's own "Additional Software" / device installer.
3. On first connecting the DE0-CV board over USB, Windows should detect the USB-Blaster and either:
   - install its driver automatically.
   - prompt you to point it at Quartus's own driver files (typically under the Quartus installation directory).
   - If Windows can't find a driver automatically, see §B.8.

Windows 11 gotcha: a couple of Windows 11's security defaults can interfere with this install or with programming the board later:

- If the installer itself won't open:
  - check Windows Security's `Smart App Control` (search for it in the Start menu).
  - turn it off, then restart.
- If everything installs cleanly but the board still won't program (§B.7):
  - go to `Windows Security → Device security → Core isolation`.

– disable **Memory integrity**:

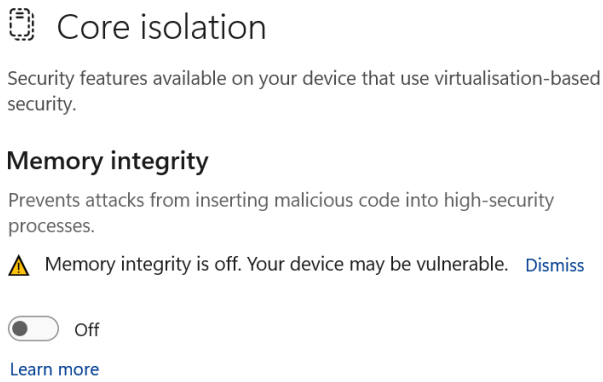


Figure B.1 The `Memory integrity` option under Windows 11's Core isolation settings, switched off.

Neither setting affects Quartus itself; both are general Windows 11 hardening defaults that happen to be strict enough to interfere with driver-level USB access.

B.3 Creating a New Project

1. Open Quartus and start the **New Project Wizard** (`File → New Project Wizard`).
2. Choose a working directory and a project name.
   - Quartus uses the project name as the default top-level entity name too.
   - Keep them matching your VHDL entity's name to avoid having to override it later (for `or_gate.vhd`, name the project `or_gate`).
3. Choose **Empty Project** as the project type.
   - This book doesn't use Quartus's IP catalog or any project templates.
4. Skip adding files at this step (§B.4 covers it) unless you already know the exact file; either is fine.
5. On the device selection page, select the exact part on the DE0-CV board:
   - filter by family **Cyclone V**.
   - then locate and select the exact device `5CEBA4F23C7N`.
6. Skip the EDA tool settings page (leave it on "none"); this book runs its simulations directly in GHDL rather than through Quartus.
7. Finish the wizard.

B.4 Adding Your VHDL Source

- Add your source file:
  - `Project → Add/Remove Files in Project...`, then browse to your `.vhd` file and add it (for example, Chapter 1's `lectures/L01/or_gate/or_gate.vhd` or Chapter 5's `lectures/L05/parity_gen/parity_gen.vhd`). `or_gate` stands in for Chapter 1's braking assistant throughout this appendix because the assistant is built live in the course and deliberately not committed; the steps are identical either way.
- If your design has more than one file (a top-level entity plus one or more subcomponents):
  - add all of them.
  - Quartus resolves the instantiation hierarchy automatically as long as every file is in the project.
- Confirm the top-level entity is set correctly:
  - `Project → Set as Top-Level Entity`, with your main file selected.
  - For a single-file design like `or_gate` or `parity_gen`, this is automatic.

B.5 Assigning Pins with the Pin Planner

Every port in your entity needs a physical pin on the FPGA before Quartus can produce a working programming file, otherwise there's no way to connect your design's `a`/`b`/`x` (or `bits`/`parity`) to an actual switch or LED on the board.

1. Run `Processing → Start → Start Analysis & Elaboration` first, then open the Pin Planner:
   - open it via `Assignments → Pin Planner`.
   - on a brand-new project the port list is empty until Quartus has elaborated the design and knows its ports.
2. Each of your entity's ports appears as a row.
   - for each one, fill in the **Location** column with the pin identifier for the physical switch or LED you want to use.
3. This book doesn't reproduce the DE0-CV's specific pin numbers here.
   - Look them up yourself, either:
     - directly in the Pin Planner (it lists the device's available pins).
     - in the DE0-CV user manual's pinout tables.
   - Use whichever switches/LEDs are convenient on your board.
4. Set the **I/O Standard** column if Quartus doesn't infer a sensible default for the pin you chose.

- the DE0-CV's general-purpose I/O is 3.3 V LVTTL/LVCMOS.
- consult the pin's entry in the Pin Planner if in doubt.

5. Save the assignments (they're written into the project's `.qsf` file).
   - any change here requires recompiling (§B.6) before it takes effect.
   - Quartus won't retroactively patch a `.sof` that was compiled before the pin assignment changed.

B.6 Compiling the Design

- `Processing → Start Compilation` (or the ▶ toolbar button) runs the full flow:
  - analysis and synthesis.
  - placement and routing ("fit").
  - timing analysis.
  - assembly into a programming file.
- Read the compilation report once it finishes, especially the **Warnings**:
  - an unconnected port, an incomplete sensitivity list, or a latch inferred where you meant combinational logic all show up here as warnings rather than errors.
  - they're easy to miss if you only check for a green checkmark.
- A successful compile produces a `.sof` file (SRAM Object File) in the project's `output_files` directory.
  - this is what §B.7 loads onto the board.
  - it's volatile: it configures the FPGA's SRAM directly and is lost on power-down; there's no persistent flashing step in this book.

B.7 Programming the DE0-CV Board

1. Connect the DE0-CV to your computer via USB and power it on.
2. Open the Programmer: `Tools → Programmer`.
3. Under **Hardware Setup**, select the USB-Blaster.
   - it should be auto-detected if the driver installed correctly (see §B.8 if not).
4. Make sure the `.sof` file produced in §B.6 is listed, and its **Program/Configure** checkbox is ticked.
   - if it's not listed, use `Auto Detect` or add the file manually.
5. Click **Start**.
   - the progress bar should reach 100% and report success.
6. Test the design directly on the board:

- flip the switches you assigned in §B.5 and confirm the LED matches your entity's truth table:
  - for `or_gate`: lit whenever either switch is on.
  - for `parity_gen`: toggles every time exactly one switch changes.

B.8 Troubleshooting

- **USB-Blaster not detected in the Programmer**:
  - check Windows Device Manager for an unknown device or one flagged with a driver error.
  - point it manually at the USB-Blaster driver under the Quartus installation directory.
- **Board won't program, no obvious error**:
  - on Windows 11, disable `Memory integrity` under `Core isolation` (§B.2).
  - this is the single most common cause once the driver itself is installed.
- **Installer won't run or open**: check `Smart App Control` under Windows Security (§B.2).
- **Compilation succeeds but the design doesn't behave correctly on the board**:
  - re-check the Pin Planner assignments (§B.5) before suspecting the VHDL.
  - a swapped or missing pin assignment produces a board that compiles cleanly but behaves nonsensically, since it isn't wired the way you think it is.
- **Fitter reports unassigned pins as errors**:
  - some project settings treat this as fatal rather than a warning.
  - either assign every port in the Pin Planner or, for a stray unused port during early development, mark it explicitly.
  - Don't leave ports silently unassigned and expect Quartus to guess.

APPENDIX C

The Self-Assessment Papers

This appendix and the four after it hold two complete three-hour papers for *Digital Design with VHDL*, with worked solutions. Appendices D and E are the papers, questions only; Appendices F and G are their model answers, with marks.

What these are for

The book has **no exam and nothing is marked**. Assessment is the exercises after every chapter, the self-checking testbench nearly every one of them ships, and the circuit you predict and then simulate by hand in CircuitVerse before any VHDL exists — the first two of the three verification layers the preface describes.

These papers do not replace that. **They are here to test your own skills and knowledge, nothing more.** They gate nothing, they are not a qualification, and no part of the book requires them. No design is built from them, and nothing else in the companion repository depends on them. They are useful where a written result is wanted anyway, for a certifying employer or a formal course credit, and useful on their own for finding out what you can reconstruct with nothing in front of you.

Take one after the last chapter. Every paper draws on all eight chapters and on both capstones, so sitting one partway through examines material you have not read yet, and the result says more about how far you have read than about what you have understood. The intended point is after Chapter 8 and the serial receiver and transmitter.

The two papers

Both cover the whole book, Chapters 1 to 8, at the same weighting. They share no question. Either can be used alone; use both as a main sitting and a resit, or in alternate years.

Question	Topic	Chapter	Marks
1	Gates, Boolean algebra, and the first module	1	12
2	Karnaugh maps, multiplexers and submodules	2	14
3	Flip-flops, registers, and when <= happens	3	13
4	Metastability, synchronizers and generics	4	13
5	Variables, and what the toolchain builds	5	13
6	Counters and shift registers	6	12
7	Timers, and composing modules	7	10
8	State machines, Moore and Mealy	8	13
			100

Paper A leans towards reading and tracing. Most of its questions hand you something — a truth table, a clocked process, a listing, a waveform in words — and ask what it does: trace six clock edges through a two-assignment process, trace a signal accumulator that throws away eight of its nine assignments, work out how often a timer fires with its clear removed, and design one state machine from a specification by hand.

Paper B leans towards writing and reviewing. Most of its questions ask you to produce a module — `button_sync` with its generic, `register4`, `piso8`, a modulo-10 counter, `blinker`, a Mealy detector — or to say what is wrong with one you are handed. Its arithmetic is the capstone's: oversample ticks, rounding error, and how far a sample point drifts across a frame.

Both papers include at least one design that compiles cleanly and is wrong, because that is the failure mode this subject actually has.

Why these papers ask for VHDL, when the exercises already do

Because **VHDL is the subject**. A paper that examined the state diagram and stopped short of the `case` statement would be examining digital design, which this book refreshes rather than teaches.

What a written paper cannot do is run GHDL, and it should not pretend to. So the rubric says outright that syntax is not what is being marked: a missing semicolon costs nothing, and a process that describes a latch where the candidate meant a wire costs that part in full. What is being examined is whether the module describes the right hardware, which is the thing a testbench cannot tell you and the thing the chapters spend their length on.

Conventions the papers assume

Both papers state these in their own rubric, so a candidate never has to have read this appendix.

- **VHDL-93**, with `ieee.std_logic_1164`, and `ieee.numeric_std` where a conversion needs it.
- **The book's conventions are the expected ones**: vectors declared `downto`, `port map` positional, an active-low signal named `_n`, a two-flop-synchronized signal named `_s2`, a subcomponent taking `reset_s2_n` and a top level taking `reset_n`, and shift registers shifting toward the most significant bit.
- **The DE0-CV clock is 50 MHz**, a 20 ns period, unless a question says otherwise.
- **Hardware answers, not code answers**, where a question asks what the toolchain builds. "It assigns `x`" is not an answer to "what does synthesis produce".
- **Tables in full.** A Karnaugh map group has to be written down as a term.

Marking

Every solution is written to be marked by somebody who has read the chapters and does not otherwise write VHDL daily, so each carries the reasoning rather than the answer alone. Marks are shown per part.

Three conventions worth agreeing before a paper is marked:

- **Method carries the marks.** A correct derivation with a slip in it is worth more than a correct answer with no working, and both papers are built so that later parts consume earlier ones. Follow through an error rather than penalising it twice: an architecture that faithfully implements a wrong equation from the part above loses nothing.
- **Mark the hardware, not the syntax.** This is the one that decides whether the paper examines the right thing. Where a listing is asked for, the marks are in the sensitivity list, the reset branch, the scheduling, the direction of the shift, and whether every path assigns its output — not in the punctuation.
- **The named traps are worth full marks on their own.** Several parts exist entirely to see whether a candidate avoids one specific mistake. In Paper A: reading a scheduled signal as if it had already updated, synchronizing a bus one bit at a time, calling metastability a wrong answer rather than a late one, a `case` branch that assigns nothing, a state machine driven by a level instead of a pulse, and a timer output wired into a clock port. In Paper B: keeping a redundant Karnaugh group, expecting one line of a clocked process to see the next line's assignment, a synchronizer chain reset to the wrong idle level, and a process whose sensitivity list omits an input it reads. Where a solution flags one of these, a candidate who walks into it loses those marks and no others.

There is a fourth, particular to this book. Two questions ask what a design does in **simulation** and what it does in **hardware**, and the answer is that they differ. An answer that gives one of the two and calls it the answer has missed the point of the question, however correct that half is.

APPENDIX D

Self-Assessment Paper A

Time: 3 hours. **Closed book.** A basic calculator is permitted. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all eight chapters, so **it is meant to be taken once you have finished the book**, after Chapter 8 and the two capstones. The model answers are in Appendix F.

Rubric

VHDL is the subject, so you write it here. Every listing you produce is VHDL-93 using `ieee.std_logic_1164`, and `ieee.numeric_std` where a conversion needs it. Write the `library` and `use` clauses out at least once; after that they are assumed.

Syntax is not what is being marked. Hardware is. A missing semicolon or a forgotten `is` costs nothing. A process that describes a latch where you meant a wire, or a chain of flip-flops where you meant one, costs that part in full, because that is a different circuit.

The book's conventions are the ones expected, and a design that breaks one should say why:

- Vectors are declared `downto`, and `port map` is positional.
- An active-low signal is named with an `_n` suffix; a signal that has been through a two-flop synchronizer is named with an `_s2` suffix.
- A **subcomponent** takes `reset_s2_n` and never touches a raw asynchronous input. A **top level** takes `reset_n` and instantiates `reset_sync` itself.
- Shift registers shift toward the most significant bit.

Where a question asks what the toolchain builds, answer in hardware, not in code. "It assigns `x`" is not an answer to "what does synthesis produce".

Where a question asks for a truth table or a state table, give every row. A Karnaugh map group has to be written down as a term; circling it in the air scores nothing.

Supplied constants and formulas

Quantity	Value
DE0-CV system clock	50 MHz
DE0-CV system clock period	20 ns
Synchronizer mean time to failure	`MTBF = e^(t_r / tau) / (T_w * f_clock * f_data)`

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	14	13	13	13	12	10	13

Question 1 A table, a network, and a module

12 MARKS

(a) A function `X` of three inputs `A`, `B`, `C` is specified by this table:

A	B	C	X
0	0	0	0
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	1
1	1	1	1

Read a sum-of-products equation straight off the table. Then simplify it by Boolean algebra alone, naming the law used at each step, and count the gates the minimized equation needs against the gates the unsimplified one needs. (*4 marks*)

(b) Every button and every reset in this book is active-low. A design carries `button_n` and `enable_n`, both active-low, and must drive an active-high `go`, true exactly when both are asserted.

Give the Boolean expression for `go`, apply De Morgan to it, and name the **single** gate that implements the result. Then give both VHDL forms of the assignment, and state what synthesis produces for each. (*3 marks*)

(c) Write the complete, synthesizable VHDL module for your minimized `X` from (a): entity `gate_net`, inputs `a`, `b`, `c`, output `x`, all `std_logic`, declared in that order. Include the `library` and `use` clauses, and state what they buy you. (*3 marks*)

(d) `x <= a or b;` is a concurrent assignment. State what it describes and why it is not a statement that runs once. Then state what a signal sitting at `'U'` and a signal sitting at `'X'` each tell you, and why neither is a value. *(2 marks)*

Question 2 The map, the multiplexer, and the module used twice

14 MARKS

(a) A function `X` of four inputs `A`, `B`, `C`, `D`:

A	B	C	D	X	A	B	C	D	X
0	0	0	0	1	1	0	0	0	1
0	0	0	1	0	1	0	0	1	0
0	0	1	0	1	1	0	1	0	1
0	0	1	1	0	1	0	1	1	0
0	1	0	0	0	1	1	0	0	0
0	1	0	1	1	1	1	0	1	1
0	1	1	0	0	1	1	1	0	0
0	1	1	1	1	1	1	1	1	1

Draw the Karnaugh map with `AB` down the rows and `CD` across the columns, both in Gray-code order. Mark your groups, give the minimized sum-of-products equation, and state for each group which cells it covers and which variables survive.

Then say how many AND-terms a direct Chapter 1-style reading of the table would have given, name the two inputs that do not appear in your answer at all, and state which **single gate** realizes the whole function. *(5 marks)*

(b) Write the architecture of a 4-to-1 multiplexer:

```
entity mux4 is
    port(inputs: in  std_logic_vector(3 downto 0);
         sel   : in  std_logic_vector(1 downto 0);
         x     : out std_logic);
end entity;
```

Selector `"00"` routes `inputs(3)`, `"01"` routes `inputs(2)`, `"10"` routes `inputs(1)` and `"11"` routes `inputs(0)`, matching this book's `mux_8to1`.

Then answer: why can the `case` statement not appear directly in the architecture body, and what does `when others` protect you against given that a two-bit selector "can only" hold four values? *(4 marks)*

(c) The same entity again, built structurally. A 2-to-1 multiplexer is available:

```
entity mux2 is
    port(a, b, sel: in  std_logic;
         x        : out std_logic);
end entity;
```

with `x = a` when `sel = '1'` and `x = b` when `sel = '0'`.

Write an architecture `structure` for `mux4` that uses **three** instances of `mux2` and no logic of its own. Then state how many copies of `mux2`'s logic the FPGA ends up with, and whether your architecture and part (b)'s describe the same hardware. *(3 marks)*

(d) This book writes every `port map` positionally. State what happens when an entity declares its ports in a different order from the one its testbench expects, distinguishing the case where the misplaced ports have different types from the case where they have the same type, and say **when** you find out in each case. *(2 marks)*

Question 3 What a clock edge does, and when an assignment happens

13 MARKS

(a) Distinguish the D latch from the D flip-flop: state the condition under which each changes its output, and why synchronous designs are built almost entirely from the flip-flop.

Then: `D` rises and falls again entirely between two rising clock edges. State what `Q` does in each of the two, and why. *(3 marks)*

(b) A clocked process:

```
process(clock) is
begin
    if (rising_edge(clock)) then
        c <= b;
        b <= a;
    end if;
end process;
```

`b` and `c` both hold `'0'` before the first edge. `a` carries these values at the six rising edges:

Edge	1	2	3	4	5	6
a	1	0	1	1	0	0

Give `b` and `c` after each of the six edges. State how much hardware this process describes, what happens if the two lines are swapped, and the rule that decides all of it. *(4 marks)*

(c) Write the VHDL for a rising-edge detector on an active-high input `req`, producing `req_edge`, using one flip-flop and one gate. Include the asynchronous active-low reset.

Then state exactly how long `req_edge` is high and why, and say why its stored previous value resets to `'0'` here where `led_toggle`'s resets to `"11"`. (*3 marks*)

(d) State the rule for a synchronous process's sensitivity list, and give the reason behind it. Then say what changes, in simulation and in the built hardware, if a designer:

- adds `d` to the list of a flip-flop process, and
- leaves `reset_n` out of the list of a process with an asynchronous reset.

(*3 marks*)

Question 4 The input you do not control

13 MARKS

(a) State what a flip-flop's setup time and hold time are, what happens inside the flip-flop when an input changes inside that window, and why an asynchronous input can never be promised to avoid it.

Then a sharper point: sampling a signal in the middle of a transition and getting either level back is **not** the failure. Say what the actual failure is, and why. (*3 marks*)

(b) Draw the double-flop synchronizer and state which flip-flop's output the rest of the design may use, and why the other one's must not be.

Then read the supplied `MTBF` expression: name what each of `t_r`, `tau`, `T_w`, `f_clock` and `f_data` is. A design has two asynchronous inputs, a push button pressed about once a second and a line from another clock domain toggling at 10 MHz, and both go through a two-flop chain. State which of the two makes the case for a third stage, by how many orders of magnitude, and what the third stage costs. (*4 marks*)

(c) Write `reset_sync`: the reset synchronizer, entity and architecture, with ports `clock`, `reset_n` (in) and `reset_s2_n` (out), declared in that order.

Name the pattern it implements, state what each half of that pattern does, and say what goes wrong in a design of a dozen flip-flops if the second half is left out. (*3 marks*)

(d) A colleague has a 4-bit counter crossing from another clock domain. They put each bit through its own double-flop synchronizer and use the four synchronized bits as a number.

State what is safe about this and what is not. Give a concrete value the receiving domain can read that the counter never held, naming the transition that produces it. Then name the three standard fixes and say which one removes the problem by construction rather than by handshaking around it. (*3 marks*)

Question 5 The accumulator that isn't, and the latch nobody asked for

13 MARKS

(a) A colleague writes an entity `any_set`, driving `found` high when any bit of `bits` is `'1'`. It compiles, and it is wrong:

```
architecture broken of any_set is
signal any_s: std_logic;
begin
    found <= any_s;

    process(bits) is
    begin
        any_s <= '0';
        for i in bits'range loop
            any_s <= any_s or bits(i);
        end loop;
    end process;
end architecture;
```

Assume `bits` has the range `7 downto 0`, `bits = "01000000"`, and `any_s` holds `'0'` before the process runs.

Trace the pass. For the initial assignment and every iteration, give the index, the value read from `any_s`, the value scheduled, and whether that scheduled assignment survives. Give the value `found` ends up with, give the correct answer, and state why repeatedly assigning a signal does not build an accumulator.

Then, the part that makes this bug dangerous: characterize **every** input for which this architecture happens to produce the right answer. (*4 marks*)

(b) Rewrite the architecture correctly, and answer: the loop runs eight times, so how many clock cycles does the fixed design take to produce an answer, and how much storage does your accumulator cost? Justify both. (*3 marks*)

(c) Nothing on an FPGA becomes a gate. State what a Boolean function becomes instead, name the two things the fabric is made of, and name the unit Quartus counts on the DE0-CV's Cyclone V.

Then use that to say what your Karnaugh map in Question 2 actually bought you on this board, and what it would still buy you anywhere. (*3 marks*)

(d) A combinational decoder, which compiles with no errors:

```
process(sel) is
begin
    case sel is
```

```
        when "00"   => leds <= "0001";
        when "01"   => leds <= "0010";
        when "10"   => leds <= "0100";
        when others => null;
    end case;
end process;
```

Quartus reports `Warning: Inferred latch(es) for signal "leds"`. State what the tool built and the two-step reason it had no choice. Give two different fixes, one changing only the `when others` branch and one adding a single line before the `case`. Then say why a warning here is more dangerous than an error would be. (*3 marks*)

Question 6 Counting, shifting, and knowing when a byte has arrived

12 MARKS

(a) A counter is declared `signal counter: natural range 0 to 15;` and incremented once per clock edge with no comparison anywhere in the design.

State what happens after `15` and why. Name the element in the gate-level drawing that performs the wrap, state what has to be true of the range for the wrap to be free, and say what changes at `natural range 0 to 9`.

Then: GHDL aborts on the wrap that the drawing and the FPGA both perform happily. State which of the two is modelling the hardware, and give the rule that follows for anything you intend to verify. (*4 marks*)

(b) An 8-bit shift register, reset to `"00000000"`, is clocked with this book's shift:

```
shift_reg <= shift_reg(6 downto 0) & serial_in;
```

`serial_in` carries `1`, `0`, `1`, `1` on four consecutive enabled edges. Give `shift_reg` after each of the four.

Then state which bit position holds the first bit you sent, where it will be after eight edges, and give the mirror-image shift expression together with what it changes. (*3 marks*)

(c) Distinguish SIPO from PISO: what enters and leaves each, which you would reach for to receive a stream of serial bits and which to drive a chain of LEDs from a fixed pattern, and the **two** things that differ between the two architectures. (*2 marks*)

(d) In `serial_rx8`, `data_ready` is a one-cycle pulse rather than a level that stays high once a byte has arrived.

State why it is a pulse, what that requires of `data_out`, and what a real peripheral would add. Then state why the board demonstration had to add a flip-flop before an LED could show `data_ready` at all, and name the earlier chapter whose problem that is in different clothes. (*3 marks*)

Question 7 The counter that watches for a number — 10 MARKS

(a) State what separates a timer from a plain counter.

Then build the comparison as gates: name the gate used per bit and say what it computes, name the gate that combines them, and give the total gate count for a 4-bit target and for the 26-bit target that counting to 50,000,000 needs.

Finally, watching a 4-bit timer count toward `1010`, three of the four per-bit outputs are high on plenty of counts that are not `1010`. State what insists on all four. *(3 marks)*

(b) A 4-bit timer has `TICK_COUNT = 10`, and its designer leaves out the clear that returns the counter to `0000` when the target is reached.

State how often `timeout` fires with the clear in place and how often without it, in clock edges, and show the arithmetic for both. State which of the two periods is the one that was asked for, and give the general form of the off-by-one that makes the first number what it is. *(3 marks)*

(c) The CircuitVerse timer's `timeout` is a combinational decode of the count; the VHDL `timer`'s is registered inside the clocked process. State the difference this makes to *when* `timeout` is high and to what it can do between edges, and name the discussion later in the book that turns on exactly this distinction. *(2 marks)*

(d) A colleague wants an LED to blink once a second and wires the `timeout` output of a timer into the clock input of the flip-flop holding the LED state, "so it only ticks once a second".

State the rule this breaks, what it costs in a design of any size, and what they should have done instead. *(2 marks)*

Question 8 A machine designed by hand, then written down — 13 MARKS

A **Moore** machine watches a serial input `din`, one bit per clock cycle, and raises `y` once it has seen the sequence `1, 1, 0`. Matches do **not** overlap: after a match the machine starts looking for a fresh `1, 1, 0` from scratch. It has four states, encoded in two flip-flops `{Q1, Q2}`:

State	Meaning	`{Q1, Q2}`
`S0`	nothing matched yet	`00`
`S1`	saw `1`	`01`
`S2`	saw `11`	`11`
`S3`	saw `110`	`10`

(a) Draw the state diagram, with an arrow out of every state for `din = 0` **and** for `din = 1`, and mark which state drives `y = 1`. Then give the complete state table: all eight rows of `{Q1, Q2, din}`, with `{Q1+, Q2+}` and `y`.

State what happens in `S2` on `din = 1` and justify it in one sentence. *(5 marks)*

(b) Derive `Q1+`, `Q2+` and `y` by Karnaugh map, treating each as a function of `{Q1, Q2, din}`. Give the equations and say what gates each needs. *(3 marks)*

(c) Write the same machine in VHDL: an enumerated state type and a `case`-statement process, with `reset_n` synchronized the way every top level in this book synchronizes it.

Then state what your VHDL does **not** contain that parts (a) and (b) produced, and who decides it instead. *(3 marks)*

(d) In `fsm_led`, a colleague replaces the edge-detected pulse driving `to_next_state` with the button's **level**, `not button_n(0)`, on the grounds that it is high exactly when the button is pressed. (`button_sync` exposes only the pulse, so there is no synchronized level to take instead without adding a `sync` of your own.)

State what the machine does while the button is held down, with the arithmetic, and say which state it ends up in. Name the fix. *(2 marks)*

APPENDIX E

Self-Assessment Paper B

Time: 3 hours. **Closed book.** A basic calculator is permitted. **Total: 100 marks.**

This paper exists to test your own skills and knowledge. It is not a qualification and it gates nothing. It draws on all eight chapters, so **it is meant to be taken once you have finished the book**, after Chapter 8 and the two capstones. The model answers are in Appendix G.

Rubric

VHDL is the subject, so you write it here. Every listing you produce is VHDL-93 using `ieee.std_logic_1164`, and `ieee.numeric_std` where a conversion needs it. Write the `library` and `use` clauses out at least once; after that they are assumed.

Syntax is not what is being marked. Hardware is. A missing semicolon or a forgotten `is` costs nothing. A process that describes a latch where you meant a wire, or a chain of flip-flops where you meant one, costs that part in full, because that is a different circuit.

The book's conventions are the ones expected, and a design that breaks one should say why:

- Vectors are declared `downto`, and `port map` is positional.
- An active-low signal is named with an `_n` suffix; a signal that has been through a two-flop synchronizer is named with an `_s2` suffix.
- A **subcomponent** takes `reset_s2_n` and never touches a raw asynchronous input. A **top level** takes `reset_n` and instantiates `reset_sync` itself.
- Shift registers shift toward the most significant bit.

Where a question asks what the toolchain builds, answer in hardware, not in code. "It assigns `x`" is not an answer to "what does synthesis produce".

Where a question asks for a truth table or a state table, give every row. A Karnaugh map group has to be written down as a term; circling it in the air scores nothing.

Supplied constants

Quantity	Value
DE0-CV system clock	50 MHz
DE0-CV system clock period	20 ns
Baud rate used by the capstone	9600 baud

Marks

Question	1	2	3	4	5	6	7	8
Marks	12	14	13	13	13	12	10	13

Question 1 A requirement, and the shape of the network it asks for

12 MARKS

A two-hand press control. The operator must keep both hands on buttons while the press cycles, so that neither hand can be under it.

Port	Direction	Meaning
`left`	in	The left hand button is pressed (active high).
`right`	in	The right hand button is pressed (active high).
`curtain_clear`	in	The light curtain sees nothing in the way (active high).
`estop_n`	in	The emergency stop, **active low**: `0` when it has been pulled.
`maint`	in	Maintenance mode is selected (active high).
`pedal`	in	The maintenance foot pedal is pressed (active high).
`run`	out	Cycle the press.

(a) Ignore `maint` and `pedal` for now. The press cycles when **both** hand buttons are pressed and the curtain is clear, and never when the emergency stop has been pulled.

Give the Boolean equation for `run`, state how many gates it needs, and say how many rows of the full four-input truth table are `1`. (*2 marks*)

(b) Now the whole requirement. In maintenance mode the press cycles from the foot pedal alone, ignoring the hand buttons and the curtain entirely — but the emergency stop is never ignored, in any mode.

Give the Boolean equation for `run`. State which single input your equation factors outside everything else, and what safety property that factoring expresses.

Then contrast it with the braking assistant of Chapter 1: there the override sits outside the fault logic and forces the output **high**, here it sits outside the mode logic and forces it **low**. State what each of the two guarantees, and say what feeding the override *through* the logic it is supposed to sit outside would cost in each case.

Finally: the full table has 64 rows. How many are `1`? Use that number to say what deriving this circuit from its table rather than from its statement would have cost you. (*4 marks*)

(c) Write the complete, synthesizable VHDL module for (b): entity `press_control`, with the ports above declared in the order the table lists them, all `std_logic`.

Then say where a `not` appears in your architecture and where an **active-low** input needed none, and give the rule that decides it. (*3 marks*)

(d) VHDL splits every module into an `entity` and an `architecture`, and the book calls that split a contract rather than ceremony.

State what each part holds, name the two things the split makes possible, and say what is different about VHDL's version of it compared with a header file beside a source file in C. (*3 marks*)

Question 2 The map, the vector, and the process that lies

14 MARKS

(a) A function `X` of four inputs `A`, `B`, `C`, `D`:

A	B	C	D	X	A	B	C	D	X
0	0	0	0	1	1	0	0	0	0
0	0	0	1	1	1	0	0	1	0
0	0	1	0	0	1	0	1	0	1
0	0	1	1	0	1	0	1	1	1
0	1	0	0	1	1	1	0	0	0
0	1	0	1	1	1	1	0	1	0
0	1	1	0	1	1	1	1	0	1
0	1	1	1	1	1	1	1	1	1

Draw the Karnaugh map with `AB` down the rows and `CD` across the columns, both in Gray-code order, and derive a minimal sum-of-products equation. Give each group's cells and its term. (*4 marks*)

(b) A colleague hands in this answer for the same function:

```
X = A'C' + AC + A'B + BC
```

Every one of their four groups is a legal group. State whether the equation is **correct**, name which group could be dropped, and give the rule from the Karnaugh-map procedure that drops it.

Then state how many different minimal answers this function has, and what that means for marking somebody else's. (*2 marks*)

(c) `std_logic_vector` questions, one or two sentences each:

- `input` is declared `std_logic_vector(7 downto 0)` and passed as `input(7 downto 4)` to a port declared `std_logic_vector(3 downto 0)`. State which bit of the port `input(7)` drives, and the rule that decides it. What has to match, and what does not?
- `count` is a `std_logic_vector(3 downto 0)`. Say why `count + 1` does not compile, and why that is the type doing its job rather than a gap in the language.
- Give what this book uses instead for anything that is counted, and the two conversions that cross between the two worlds. *(4 marks)*

(d) Write two architectures for an entity computing `x = ab + c'd`: one as a single concurrent expression, and one naming the intermediate term `c'd` in an internal signal.

State what synthesis produces for each, why your answer is the same for both, and which of the two you would rather be handed six months from now. *(2 marks)*

(e) A colleague's 4-to-1 multiplexer. It compiles, its testbench passes, and it is wrong:

Its entity has four `std_logic` data inputs `a`, `b`, `c`, `d`, a `std_logic_vector(1 downto 0)` selector `sel`, and a `std_logic` output `x`.

```
architecture behaviour of word_mux is
begin
    process(sel) is
    begin
        case sel is
            when "00"   => x <= a;
            when "01"   => x <= b;
            when "10"   => x <= c;
            when others => x <= d;
        end case;
    end process;
end architecture;
```

State what a **simulator** does with this and what **synthesis** does with it, say why the two answers differ, and name the general class of bug that puts this design in. *(2 marks)*

Question 3 Storage, and the two ways a value updates

13 MARKS

(a) An edge-triggered D flip-flop is built from two D latches in series, a master-slave pair.

State how the two latches are driven relative to each other, and derive from that why the output can change only at the instant the clock goes low to high, and never while it sits at either level.

Then state the latch's fundamental weakness on its own, and why that weakness is much harder to live with in a large synchronous system than in a small one. *(3 marks)*

(b) Write a 4-bit register: entity `register4`, ports `clock`, `reset_n`, `enable` (in), `d` (`std_logic_vector(3 downto 0)`, in) and `q` (`std_logic_vector(3 downto 0)`, out), in that

order. The reset is asynchronous and active-low, and `enable` gates whether a rising edge captures `d`.

Then state how many flip-flops the design puts on the FPGA and what decides that number, and say whether writing the reset branch before or after the `rising_edge` branch is a matter of style or of meaning. *(4 marks)*

(c) A colleague wants `q` to follow `din` after exactly **one** clock cycle. It compiles, and `q` follows `din` after **two**:

```
DELAY_PROCESS: process(clock, reset_n) is
begin
    if (reset_n = '0') then
        stage2_s <= '0';
        stage1_s <= '0';
    elsif (rising_edge(clock)) then
        stage2_s <= stage1_s;
        stage1_s <= din;
    end if;
end process;

q <= stage2_s;
```

Both are architecture signals. The author's reasoning was that the first line reads `stage1_s` after the second line has already put `din` into it, so `stage2_s` would land one edge behind `din`.

Trace `stage1_s`, `stage2_s` and `q` through four rising edges with `din` carrying `0`, `1`, `0`, `0`, starting from reset. Name the rule that makes the author's reasoning wrong.

Then state what changes if the two assignments are swapped, and why. Finally, give the change that really does make `q` follow `din` after one cycle, and say how many flip-flops synthesis puts on the FPGA for the listing as written and how many after your change. *(4 marks)*

(d) Synchronous designs pick one clock edge and update every flip-flop on that edge and only that edge. State why, and what that buys somebody trying to reason about a circuit too large to see all at once. *(2 marks)*

Question 4 One module, several widths

13 MARKS

(a) Write `button_sync`: the three-flip-flop chain that synchronizes a set of active-low push buttons and turns each press into a one-cycle pulse.

```
entity button_sync is
    generic(COUNT: natural range 1 to 3 := 1);
```

```
    port(clock, reset_s2_n: in  std_logic;
         button_n         : in  std_logic_vector(COUNT-1 downto 0);
         button_edge_s2   : out std_logic_vector(COUNT-1 downto 0));
end entity;
```

Write the architecture in full. Then state which flip-flops are the synchronizer and which is the edge detector, what value the chain resets to and why the other value would be wrong, and why this module's reset port is `reset_s2_n` rather than `reset_n`. *(5 marks)*

(b) `button_sync` carries a generic and `reset_sync` carries none.

State the rule that decides it, and what would be wrong with adding a `WIDTH` generic to `reset_sync` for symmetry. Then say where in the built circuit `COUNT` ends up, how many flip-flops an instance at `COUNT = 2` uses against one at `COUNT = 1`, and what a `generic map` costs in hardware. *(3 marks)*

(c) In Chapter 4's CircuitVerse exercises the chain above appears to debounce the button perfectly. On the DE0-CV at 50 MHz it does not, and one physical press can toggle an LED several times.

Explain both observations with one mechanism, name what a real debounce circuit adds, and say why it sits **on top of** the synchronizer rather than instead of it. *(2 marks)*

(d) The rule is that every asynchronous input gets its own double-flop chain before anything else looks at it. Two limits on that rule matter:

- `seq_detect_mealy` synchronizes `reset_n` but sends `din` straight into the machine. State why, and what synchronizing `din` would cost.
- `serial_rx8`, `timer` and `counter` all declare their reset port as `reset_s2_n`, and `walking_led` declares its as `reset_n`. State the rule that separates the two, and what goes wrong if a subcomponent is wired straight to a push button. *(3 marks)*

Question 5 Two kinds of update, and a speed limit — 13 MARKS

(a) Give the four-row comparison between a `signal` and a `variable`: where each is declared, where each is visible, when an assignment to each takes effect, and what happens when each is assigned twice in one pass through a process.

Then give the rule of thumb for choosing between them. *(3 marks)*

(b) Write `ones_count8`: a combinational entity counting how many bits of an 8-bit input are set.

Port	Direction	Type
`bits`	in	`std_logic_vector(7 downto 0)`
`ones`	out	`natural range 0 to 8`

Use a single process and a loop.

Then answer: `ones` is a `natural range 0 to 8` rather than a `std_logic_vector(3 downto 0)`, and both can hold the answer. State what the range buys you, and what GHDL would do if your count ever left it. (*4 marks*)

(c) Signals take time to get through a LUT and along the wires between blocks, so a clock cannot go arbitrarily fast.

Name the three delays a clock period has to cover for the standard flip-flop / logic / flip-flop path, define the critical path and Fmax in terms of them, and state what happens in the second flip-flop if the period is too short.

Then: adding registers to a design adds hardware, and can make it **faster**. Explain how, and state what it trades away. Finally, say what Quartus needs before it will report an Fmax at all, and why a design with no constraint gets no number rather than a good one. (*4 marks*)

(d) A `variable` retains its value between activations of its process, behaving like static local storage rather than a fresh local each time.

State what that means for a loop accumulator that is not cleared at the top of every activation, and give the value such an accumulator would report over a long run. Then say why the book still tells you to reach for a `signal` when state genuinely has to persist and be read by other logic. (*2 marks*)

Question 6 Counting to something that is not a power of two 12 MARKS

(a) Write `counter10`: a modulo-10 counter, counting `0` to `9` and returning to `0`. This is a deliberately fixed-radix variant of Chapter 6's `counter`, which takes its radix as a generic and is written to compare against `RADIX-1`; here the radix is wired in, so the comparison is against a literal and the entity carries an `enable` instead of a `tick`.

Port	Direction	Type
`clock`	in	`std_logic`
`reset_s2_n`	in	`std_logic`
`enable`	in	`std_logic`
`count`	out	`natural range 0 to 9`

Then state which lines of your architecture would be unnecessary if the range were `0 to 15` instead, and why exactly those. (*4 marks*)

(b) Write the architecture of `piso8`, a parallel-in/serial-out shift register:

```
entity piso8 is
    port(clock, reset_s2_n, load, shift, serial_in: in  std_logic;
         parallel_in                              : in  std_logic_vector(7 downto ↩
    ↪ 0);
```

```
        serial_out                              : out std_logic);
end entity;
```

`load` takes precedence over `shift`. Then state which bit of a loaded value leaves first, and name the **two** things that separate this architecture from a serial-in/parallel-out one — the shift expression is not one of them. *(4 marks)*

(c) A colleague modifies `serial_rx8` so that `shift_enable` gates the shift register but not the bit counter, which now advances on every clock edge.

The transmitter sends three bits, pauses for five clock cycles, then sends the remaining five. State what the receiver does, on which edge `data_ready` first pulses, and what is on `data_out` when it does. *(2 marks)*

(d) A counter and a shift register are both "N flip-flops sharing one clock". State, for each, what feeds flip-flop `i`'s `D` input, and use the two answers to say what the difference between the two circuits actually is. *(2 marks)*

Question 7 Turning ticks into time

10 MARKS

(a) A design needs an LED that blinks at a visible rate, driven by this book's `timer`, whose `timeout` toggles the LED.

State how many clock cycles a 100 ms timeout takes at 50 MHz, and give the `TICK_COUNT` for it, both as the round number the book writes and as the cycle-exact value.

Then give the resulting **blink** frequency in Hz, and say why it is half the number somebody would first write down. *(3 marks)*

(b) `timer`'s `enable` freezes the timer rather than clearing it: while it is low, the counter holds its value and `timeout` stays low.

State what that means for the first timeout after a design re-enters a state that uses the timer, compared with what a clearing enable would give. Then say whether holding `enable` high permanently would be better or worse, and for what. *(2 marks)*

(c) Write `blinker`: a top-level design blinking an LED with no button at all.

```
entity blinker is
    generic(TICK_COUNT: natural := 25_000_000);
    port(clock, reset_n: in  std_logic;
         led           : out std_logic);
end entity;
```

Write the architecture. It instantiates two modules and contains one process of its own. *(3 marks)*

(d) A button controlling a timer has nothing to do with counting, and a designer proposes wiring it straight to the timer's `enable`.

State why it must go through a synchronizer first, and what specifically the design would do on a real press without one. *(2 marks)*

Question 8 The output that arrives a cycle early

13 MARKS

(a) Write `seq_detect_01_mealy`: a **Mealy** machine watching a serial input `din`, one bit per clock cycle, whose output `y` is high whenever the previous bit was `0` and the current bit is `1` — a rising edge on the data itself.

Port	Direction	Type
`clock`	in	`std_logic`
`reset_n`	in	`std_logic`
`din`	in	`std_logic`
`y`	out	`std_logic`

Write the module in full: enumerated state type, transition process, and output. Two states are enough; say what each one means.

Then name the **one line** that makes this a Mealy machine, and say what `when others` buys you in the transition `case` given that `state_t` has only the values you declared. *(5 marks)*

(b) The Moore machine detecting the same thing needs one state more than yours. State why, what its output does differently, and which of the two you should reach for by default with the reason. *(2 marks)*

(c) The capstone receiver recovers its bit timing from the falling edge of a start bit and nothing else.

- At 50 MHz and 9600 baud, how many clock cycles is one bit period, and how many is a sixteenth of one?
- This book's `timer` pulses every `TICK_COUNT + 1` clock cycles. The exact sixteenth is not a whole number, so there are two candidate `TICK_COUNT` values either side of it. Give both, give the error each leaves, and say which one the capstone's default of `325` is and why.
- State why the receiver samples in the **middle** of each bit rather than near a boundary, in terms of what it is tolerating.
- Ten bit periods after the start edge, how far has the sample point drifted, in oversample ticks, and how many ticks of margin did it start with? *(4 marks)*

(d) The capstone's `STATE_IDLE` leaves on a falling **edge** of the synchronized line, not on the line being low, which costs one extra flip-flop.

Consider a **break**: the line held low for far longer than a frame, which is what a transmitter losing power looks like. State what a level test does during the break and what it does at the moment the break ends, and say why that last frame is the one that matters. *(2 marks)*

APPENDIX F

Model Answers to Paper A

Marks are shown per part. Method carries them: a correct derivation with a slip in it is worth more than a correct answer with no working, and later parts consume earlier ones, so an error should be followed through rather than penalised twice.

Where a part asks for VHDL, mark the **hardware described**, not the syntax. A listing with a missing semicolon and the right circuit in it scores full marks; a listing that compiles and infers a latch does not.

Question 1 A table, a network, and a module — 12 MARKS

(a) — 4 MARKS

Straight off the table. Four rows are `1`, so four AND-terms, each carrying every input directly where it is `1` in that row and primed where it is `0`:

```
X = A'BC' + AB'C' + ABC' + ABC
```

(*1 mark*)

Simplified. The term `ABC'` is needed three times, which the idempotent law `A + A = A` supplies for free:

```
X = A'BC' + AB'C' + ABC' + ABC' + ABC' + ABC        (idempotent, twice)
  = (A' + A)BC' + (B' + B)AC' + AB(C' + C)          (distributive)
  = 1·BC' + 1·AC' + AB·1                            (complement)
  = BC' + AC' + AB                                  (identity)
```

(*2 marks*)

The gate count.

Form	Gates
`A'BC' + AB'C' + ABC' + ABC`	3 NOT, 4 three-input AND, 1 four-input OR = **8**
`BC' + AC' + AB`	1 NOT, 3 two-input AND, 1 three-input OR = **5**

(1 mark)

Worth noticing and worth a comment rather than a mark: `X = AB + AC' + BC'` is the **majority vote** of `A`, `B` and `C'`. Check it against the table row by row and every row agrees. That is the same observation §1.2 makes when `A'B + AB'` turns out to be one XOR gate: reading a table gives you a correct network, and recognizing the function gives you a smaller one.

Accept any correct simplification path. Do not require these particular law names, but do require *some* justification per step: an answer that jumps from four terms to three with no working is a remembered answer, not a derived one.

(b)

3 MARKS

Both signals are active-low, so "asserted" means `0`:

```
go = button_n' · enable_n'
```

(1 mark)

De Morgan, in the form `A'B' = (A + B)'`:

```
go = (button_n + enable_n)'
```

which is a **NOR gate**, one gate for the whole function. *(1 mark)*

```
go <= (not button_n) and (not enable_n);   -- the expression as derived
go <= not (button_n or enable_n);          -- the same thing after De Morgan
```

Synthesis produces **the same circuit for both**, and on the Cyclone V that circuit is not a NOR gate either: it is one two-input LUT loaded with the table (Question 5(c)). The two lines are two spellings of one function, and the tool does not care which you wrote. *(1 mark)*

This is the everyday use of De Morgan the book leans on: an active-low signal meeting logic written for active-high, which in a book where every button and every reset is active-low is most of them.

(c)

3 MARKS

```
library ieee;
use ieee.std_logic_1164.all;

entity gate_net is
    port(a, b, c: in  std_logic;
         x      : out std_logic);
end entity;

architecture behaviour of gate_net is
```

```
begin
    x <= (b and not c) or (a and not c) or (a and b);
end architecture;
```

(*2 marks: 1 for the entity with the ports in the stated order, 1 for an architecture matching the candidate's own answer to* (*a*))

Naming the intermediate terms in internal signals is equally correct and synthesizes identically.

What the clauses buy. `std_logic` is not built into VHDL. It comes from the `std_logic_1164` package in the `ieee` library, and so do the `and`/`or`/`not` operators as they apply to it. Without those two lines the port types do not exist and the file does not analyze. (*1 mark*)

Follow through an incorrect (a): an architecture that faithfully implements a wrong equation loses nothing here.

(d) 2 MARKS

The concurrent assignment. It describes a **wire**, or rather the gate driving one: a standing, continuous relationship that holds at all times, exactly like the piece of hardware it stands for. It is not executed once and it has no position in a sequence; `x` is `a or b` for as long as the circuit exists. Two concurrent assignments in one architecture are two pieces of hardware operating at the same time, not two steps. (*1 mark*)

The two values. `'U'` is uninitialized: nothing has driven this signal at all since the simulation began. `'X'` is unknown: two things are driving it and they disagree. Neither is a level a synthesized wire can carry, so neither is a value your design computed — each is the simulator telling you something is wrong, and the correct reaction is to find out what, not to work around the symbol. (*1 mark*)

Question 2 The map, the multiplexer, and the module used twice 14 MARKS

(a) 5 MARKS

The map, `AB` down the rows and `CD` across the columns, both in Gray-code order `00, 01, 11, 10`:

AB \ CD	00	01	11	10
00	1			1
01		1	1	
11		1	1	
10	1			1

(1 mark for a correctly filled map in Gray-code order)

Two groups of four.

Group	Cells	Constant across the group	Term
The four corners	`0000, 0010, 1000, 1010`	`B = 0, D = 0`	`B'D'`
The centre block	`0101, 0111, 1101, 1111`	`B = 1, D = 1`	`BD`

The corner group is the one that has to be seen: it wraps **both** ways, top-to-bottom and left-to-right, which is legal precisely because the Gray-code ordering makes the last row adjacent to the first and the last column adjacent to the first. A candidate who instead writes four groups of two here has read the map as a grid rather than as a torus, and has lost the mark for this part but nothing later. *(2 marks: 1 per group, including the surviving variables)*

```
X = B'D' + BD
```

(1 mark)

The rest of the question. Eight rows are `1`, so a direct Chapter 1-style reading gives **eight** AND-terms of four literals each. `A` and `C` do not appear in the answer at all: the function does not depend on them. And `X = 1` exactly when `B` and `D` are equal, so the whole thing is a single **XNOR** gate on `B` and `D`. *(1 mark)*

That last step is the same one §1.2 makes with `A'B + AB'`, and it is what the technique is for: the map found a two-term equation, and recognizing the two-term equation found a one-gate circuit.

(b)

4 MARKS

```
architecture behaviour of mux4 is
begin
    MUX_PROCESS: process(inputs, sel) is
    begin
        case (sel) is
            when "00"   => x <= inputs(3);
            when "01"   => x <= inputs(2);
            when "10"   => x <= inputs(1);
            when "11"   => x <= inputs(0);
            when others => x <= '0';
        end case;
    end process;
end architecture;
```

(2 marks: 1 for the four branches mapping the right way round, 1 for a `process` with both `inputs` and `sel` in the sensitivity list. Deduct nothing for a label, or for its absence.)

Why the case needs a process. `case`, like `if` and `for`, is a **sequential** statement, legal only inside a process. An architecture body holds concurrent statements: standing descriptions

of hardware, all active at once. A process is how a block of sequential statements is packaged up so that it can be one of those concurrent statements. *(1 mark)*

What `when others` protects against. It is a **language** requirement, not a defensive one. VHDL insists a `case` cover every value of its selector's *type*, and `sel` is a `std_logic_vector`, whose elements have nine values each rather than two, so `"00"`, `"01"`, `"10"` and `"11"` do not make the `case` exhaustive. It is not a claim about the wire: a synthesized net carries a `0` or a `1` and nothing else, and the other values belong to the simulator, where a signal sitting at `'U'` or `'X'` is a bug rather than a level. *(1 mark)*

An answer arguing that `when others` "catches an illegal selector on the board" has the wrong reason and should get no more than half the mark, even though the line it writes is identical.

(c) 3 MARKS

```vhdl
architecture structure of mux4 is
signal x_high, x_low: std_logic;
begin
    -- sel(0) chooses within each pair.
    mux2_high: entity work.mux2
        port map(inputs(2), inputs(3), sel(0), x_high);

    mux2_low: entity work.mux2
        port map(inputs(0), inputs(1), sel(0), x_low);

    -- sel(1) chooses between the two pairs.
    mux2_out: entity work.mux2
        port map(x_low, x_high, sel(1), x);
end architecture;
```

(2 marks: 1 for three instances with unique labels and two internal signals, 1 for getting the `a`/`b` order right against `mux2`'s `sel = '1'` selecting `a`)

Check it against the specification one selector value at a time, which is the check worth writing down: `"00"` takes `x_high` and within it `inputs(3)`; `"01"` takes `x_high` and `inputs(2)`; `"10"` takes `x_low` and `inputs(1)`; `"11"` takes `x_low` and `inputs(0)`.

The rest. The FPGA ends up with **three** copies of `mux2`'s logic, exactly as writing the logic three times would. Instantiating a module more than once saves source, not silicon.

And yes, this architecture and part (b)'s describe the same hardware, or near enough that the tool will not distinguish them: both are a four-input multiplexer, and a fitter given either one packs it into LUTs the same way. (b) describes the **intent** and lets the tool find the gates; (c) describes the **structure** and hands the tool a wiring diagram. That gap is most of what a hardware description language buys you. *(1 mark)*

(d) 2 MARKS

The testbench connects to the module **by position**, not by name, so the entity's port declaration order is part of its interface. A wrong order fails in one of two ways, and neither says "wrong order":

- **Different types or widths.** Analysis stops with a message like `can't associate "sel" with port "sel"`, pointing at the testbench's `port map` line. You find out at `ghdl -a`, immediately.
- **Same type**, several `std_logic` ports swapped, say. It analyzes and elaborates quite happily, then fails at run time as an ordinary assertion failure that reads exactly like a logic bug in the architecture. You find out at `ghdl -r`, after spending time in the wrong file.

(*1 mark per case, including when it is caught*)

The practical consequence, which is why the exercises spell their ports out in order: a testbench that fails on the very first case it checks is a reason to reread the port order before hunting through the architecture.

Question 3 What a clock edge does, and when an assignment happens 13 MARKS

(a) 3 MARKS

The latch is **level-sensitive**: while `enable` is high it is transparent and `Q` follows `D` continuously; while `enable` is low it holds. It changes its output at any time during the entire enabled window.

The flip-flop is **edge-triggered**: it samples `D` at the single *instant* the clock transitions and holds that value for the rest of the period, whatever `D` does. (*1 mark*)

Why synchronous designs use the flip-flop. Because the latch is transparent for a whole clock phase, any glitch on `D` during that phase passes straight through, and guaranteeing data is stable across an entire phase is far harder than guaranteeing it around one instant. Every flip-flop changing at the same well-defined instants is also what makes a large circuit's timing analyzable: you can reason one clock cycle at a time instead of tracking every signal continuously. (*1 mark*)

`D` rises and falls between two edges. The latch follows it up and back down, so `Q` shows the pulse (during whatever part of it the clock is high). The flip-flop shows **nothing at all**: no edge happened while `D` was high, so `Q` never moves. Both are behaving correctly; they are answering different questions. (*1 mark*)

(b) 4 MARKS

A signal assignment does not take effect when its line runs. It **schedules** a value, applied only once the process suspends, so every read during the pass returns the value the signal held *coming into* the edge. `c <= b` therefore reads the old `b`, not the one being scheduled on the line below.

Edge	1	2	3	4	5	6
a	1	0	1	1	0	0
b after	**1**	**0**	**1**	**1**	**0**	**0**
c after	**0**	**1**	**0**	**1**	**1**	**0**

(*2 marks for the table; 1 if* `b` *is right and* `c` *is shifted the wrong way or not at all*)

The hardware. Two flip-flops in a chain. Every signal assigned inside a clocked process becomes a flip-flop, one bit each, and here `a` reaches `b` after one edge and `c` after two. (*1 mark*)

Swapping the lines changes nothing. Neither assignment can observe the other's result, because both read the pre-edge values and both take effect together when the process suspends. Order does not matter between assignments to *different* signals in one clocked process, which is the opposite of the sequential reading the word "process" invites, and it is why a chain of flip-flops can be written in any order. (*1 mark*)

Two related rules are worth a comment if a candidate volunteers them: assigning the **same** signal twice in one pass keeps only the last assignment, and a `variable` behaves the other way round on both counts, which is Question 5.

(c) 3 MARKS

```
signal req_prev: std_logic;
...
    -- One gate: high for the single cycle where req is high and was not.
    req_edge <= req and not req_prev;

    REQ_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            req_prev <= '0';
        elsif (rising_edge(clock)) then
            req_prev <= req;
        end if;
    end process;
```

(*2 marks: 1 for the flip-flop holding the previous value with an asynchronous active-low reset, 1 for* `current AND NOT previous` *as the rising-edge form. A registered* `req_edge` *inside the process is also correct hardware, one cycle later; accept it if the candidate says so.*)

How long it is high. Exactly **one clock cycle**: the one immediately after the transition, because `req_prev` only catches up at the *next* edge, at which point `req` and `req_prev` agree again and the gate goes low.

Why `'0'` here. The reset value has to be the signal's **idle** level, so that coming out of reset does not look like a transition. `req` is active-high and idles low, so `req_prev` resets to `'0'`. `led_toggle`'s buttons are active-**low** and idle at `1`, so its `button_prev_n` resets to `"11"`; reset it to `"00"` and the circuit comes out of reset claiming both buttons are already held down. (*1 mark*)

(d) 3 MARKS

The rule. `clock` is the only signal in the sensitivity list, unless the design has an asynchronous reset, in which case that reset joins it and nothing else does.

The reason. The process has no business reacting to anything but the events that can change its outputs. A flip-flop's output changes at a clock edge, and an asynchronously reset one also changes the moment the reset is asserted, with no clock edge involved, so the reset has to be able to wake the process on its own. Everything else the process reads is sampled *at* the edge and has no business waking it. (*1 mark*)

Adding `d` to the list. In **simulation**, the process now also runs whenever `d` changes; since `rising_edge(clock)` is false on those runs, nothing is assigned, and the behaviour is unchanged. In **hardware** nothing changes either: the sensitivity list is not synthesized. So this one costs nothing but noise, and it is worth saying so plainly, because the reflex is to assume every sensitivity-list mistake is fatal. (*1 mark*)

Leaving `reset_n` out. Now the two disagree. In **simulation** the process only runs on `clock` events, so the reset is no longer asynchronous: asserting it does nothing until the next clock edge, and a reset asserted and released between two edges is missed entirely. In **hardware** the tool reads the `if (reset_n = '0') then ... elsif (rising_edge(clock))` structure and builds the asynchronous reset anyway, or refuses the process. Either way the simulator and the chip no longer agree, which is worse than being wrong in one place. (*1 mark*)

Question 4 The input you do not control 13 MARKS

(a) 3 MARKS

Setup and hold are the two halves of a window around the active clock edge: the **setup time** is how long `D` must already be stable *before* the edge, and the **hold time** is how long it must stay stable *after* it.

What happens inside. A flip-flop is built from cross-coupled gates that snap to `0` or `1` when the clock samples. An input changing inside the window can leave that feedback loop balanced between the two states, neither fully high nor fully low, for longer than expected: **metastability**. A ball balanced on top of a hill rolls down eventually, but when it commits,

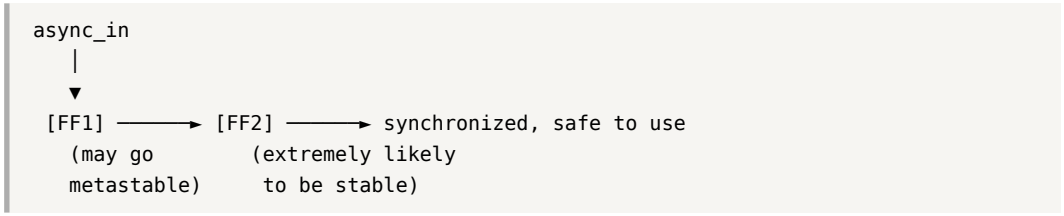
and to which side, is not predictable from the moment it was placed there, and while it is still balanced its position is not a valid digital value at all. (*1 mark*)

Why an asynchronous input cannot be promised to miss the window. Because you do not control when it changes. A push button, an external reset, or a line from another clock domain changes when the outside world decides to, with no fixed relationship to your clock, so no amount of design makes the window unreachable. (*1 mark*)

What the actual failure is. Not a wrong answer. Sampling a signal that genuinely was in the middle of a transition and getting either level back is a legitimate outcome and is harmless: both answers were true at some instant inside the window. The failure is the **delay before there is an answer at all**, and specifically what happens if the unresolved voltage reaches downstream logic before it settles, because then different gates can legitimately read it differently at the same instant. One part of the circuit increments and another does not; a state machine lands in an impossible state. The disagreement is the bug. (*1 mark*)

An answer that says "it gives the wrong value" has missed the whole point of §4.2 and scores nothing for this third mark, however fluent the rest is.

(b) 4 MARKS



Only **FF2's** output may be used elsewhere. FF1 is the flip-flop exposed to the asynchronous input, so it is the one that may go metastable; its output has an entire clock period to settle before FF2 samples it, and by then it almost always has. FF1's output used directly is the unresolved node itself, which is the failure in (a). (*1 mark*)

The terms.

Term	What it is
`t_r`	The resolution time a stage is given, roughly one clock period
`tau`	A property of the flip-flop: how fast a metastable value decays
`T_w`	A property of the flip-flop: the width of the window around the edge in which an input can provoke metastability at all
`f_clock`	The clock rate of the receiving domain
`f_data`	How often the asynchronous input actually changes

(*2 marks, or 1 for naming three or four of the five*)

Which input makes the case for a third stage. The **10 MHz line**. It is `f_data` that separates the two, and `1 Hz` against `10^7 Hz` is **seven orders of magnitude** in that term alone; `t_r`, `tau`, `T_w` and `f_clock` are identical for both, because they are properties of the flip-flop and the

receiving clock, not of the input. So the case for a third stage is a question about the input, not about the clock.

What it costs. One more clock cycle of latency, against a numerator that grows *exponentially* with each added `t_r`. That asymmetry is why extra stages exist at all. (*1 mark*)

(c) 3 MARKS

```
library ieee;
use ieee.std_logic_1164.all;

entity reset_sync is
    port(clock, reset_n: in  std_logic;
         reset_s2_n    : out std_logic);
end entity;

architecture behaviour of reset_sync is
signal reset_s1_n: std_logic;
begin
    process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            reset_s1_n <= '0';
            reset_s2_n <= '0';
        elsif (rising_edge(clock)) then
            reset_s1_n <= '1';
            reset_s2_n <= reset_s1_n;
        end if;
    end process;
end architecture;
```

(*2 marks: 1 for both stages forced to `'0'` in the asynchronous branch, 1 for the clocked branch shifting a constant `'1'` in rather than shifting `reset_n` in*)

Nothing here reads `reset_s2_n` back, so under VHDL-93 it can be driven straight from the process, as §4.5's listing does. A candidate who routes it through an extra internal signal is equally correct, just carrying one declaration that buys nothing.

The pattern: assert asynchronously, release synchronously.

- **Assert.** When `reset_n` drops, both stages are forced to `0` immediately, with no dependency on the clock. A reset that waited for a clock edge would defeat its own purpose: it has to take effect the instant it is needed, clock or no clock.
- **Release.** When `reset_n` returns to `1` the reset condition does not vanish everywhere at once. It shifts out through the two stages like any other asynchronous input, so the signal the design actually uses releases cleanly on a clock edge along with everything else.

(*1 mark for both halves*)

Without the synchronous release, in a design of a dozen flip-flops — worth a comment rather than a mark, but the sentence the other two marks are really testing for: the reset's rising edge arrives at each of them at a slightly different time relative to the clock, so some leave reset on one edge and some on the next, and for one cycle the design holds a combination of state that no correct execution ever produces. `serial_rx8` is the concrete case the book gives: its shift register and its bit counter are a dozen flip-flops between them, and a receiver that starts counting a byte one cycle before it starts shifting one has been broken by its own reset.

(d) 3 MARKS

What is safe. Each bit individually. Every bit gets a full clock period to resolve, so no *single* bit is metastable when the receiving domain reads it. Metastability, per bit, is solved.

What is not. The bits are not safe **together**. Each one independently resolves to the old or the new value on the same edge, so the receiver can sample a combination that was never on the bus. The double-flop answer synchronizes **one bit**, and that word is the whole limit. (*1 mark*)

A concrete value. The counter stepping from `0111` to `1000` changes all four bits at once. Each synchronizer independently lands on the old or the new bit, so the receiver can read `1111`, or `0000`, or any of the other fourteen combinations — values the counter held neither before nor after the step. (*1 mark*)

The three fixes.

- A **handshake**: the sender holds the bus still and raises a single-bit request, which *is* safe to synchronize; the receiver reads the stable bus and acknowledges.
- A **FIFO** with synchronized pointers, which is the same idea industrialized.
- **Gray coding** the counter, so that only one bit ever changes per step and there is no combination to get wrong.

Gray code is the one that removes the problem **by construction**: there is no instant at which two bits are in flight, so a per-bit synchronizer can only return the old value or the new one, both of which are values the counter really held. The other two work by making sure nobody looks while the bus is moving. (*1 mark*)

None of this is needed anywhere in this book, which never crosses a clock domain with a bus. Knowing that the two-flop answer stops at one bit is the examinable part.

Question 5 The accumulator that isn't, and the latch nobody asked for

13 MARKS

(a)

4 MARKS

Two rules do all the work: a signal assignment **schedules** rather than applies, so every read during the pass returns the value held *before the pass started*; and assigning the same signal more than once in one pass keeps only the **last** assignment.

`bits'range` on `(7 downto 0)` runs high to low, and `bits = "01000000"`, so only `bits(6)` is `'1'`.

Step	`bits(i)`	Read from `any_s`	Scheduled	Survives?
`any_s <= '0'`	–	–	`'0'`	no
`i = 7`	`0`	`'0'`	`'0'`	no
`i = 6`	`1`	`'0'`	`'1'`	no
`i = 5`	`0`	`'0'`	`'0'`	no
`i = 4`	`0`	`'0'`	`'0'`	no
`i = 3`	`0`	`'0'`	`'0'`	no
`i = 2`	`0`	`'0'`	`'0'`	no
`i = 1`	`0`	`'0'`	`'0'`	no
`i = 0`	`0`	`'0'`	`'0'`	**yes**

(*2 marks for the trace, of which 1 is for reading `'0'` in **every** row rather than the previous row's scheduled value*)

`found` ends up `'0'`. The correct answer is `'1'`: `bits(6)` is set. Nine assignments were computed and eight of them were thrown away.

Why this is not an accumulator. Every iteration reads the same frozen value, so no iteration can see the previous one's result, and only the textually last assignment survives. The final value depends on exactly two things: whatever `any_s` already held, and `bits(0)`. The loop is doing no accumulating at all — it is computing the same expression eight times with a different last operand and discarding all but one of the answers. (*1 mark*)

When it happens to be right. The architecture computes `old_any_s or bits(0)` where the correct answer is the OR of all eight bits. So:

- **Whenever `bits(0) = '1'` it is right**, unconditionally, because both expressions are then `'1'`. That is **half of all 256 inputs**.
- When `bits(0) = '0'` it is right only by luck: when the value it happened to be carrying already matches the correct answer.

That is what makes this bug dangerous rather than merely wrong. `"11111111"` passes. `"00000001"` passes. Anything ending in a `1` passes. A smoke test of a few "obvious" inputs has a good chance of reporting success, and the design fails on the ones nobody typed. (*1 mark*)

A candidate who adds that in a real simulation `any_s` starts at `'U'` rather than `'0'`, so the first pass propagates `'U'` and Chapter 1's "a signal sitting at `'U'` is a bug, not a value" fires immediately, has understood more than the question asked and should be credited within the marks available.

So has one who points out that the surviving assignment is `any_s <= any_s or bits(0);` with `any_s` absent from the sensitivity list, so what synthesis is being handed is a **combinational feedback loop**: an output fed back to its own input through a gate, with no flip-flop in the path. Quartus reports that rather than building it. "Compiles cleanly" here means GHDL analyzes and simulates it, which is exactly the gap the question is about.

(b) 3 MARKS

```vhdl
architecture behaviour of any_set is
signal any_s: std_logic;
begin
    found <= any_s;

    ANY_PROCESS: process(bits) is
        variable acc: std_logic;
    begin
        acc := '0';
        for i in bits'range loop
            acc := acc or bits(i);
        end loop;
        any_s <= acc;
    end process;
end architecture;
```

(*1 mark: a `variable`, assigned with `:=`, cleared at the top of every activation, handed to the signal once at the end*)

Clock cycles: zero. This is a combinational process. The loop's bounds are fixed at elaboration, so the synthesis tool **unrolls** it: the eight iterations become a chain of OR gates that all settle within one propagation delay, in the same clock cycle. There is no counter for `i`, no branch, and no iteration in time. The loop writes gates in three lines; it does not do eight things in sequence. (*1 mark*)

Storage: none. `acc` reads like a running total but never has to survive a clock edge. Each of its values is simply the wire between two of those OR gates, which is exactly why a `variable` is the right tool: it never outlives one pass. (*1 mark*)

An answer of "eight clock cycles" is the C reading of the loop and scores nothing for the second and third marks, however well the code is written.

(c)

3 MARKS

A Boolean function becomes a **look-up table**: a tiny memory, loaded with the function's truth table, which then computes it. Nothing becomes a gate.

The fabric is made of **LUTs and flip-flops**, one bit of clocked storage sitting beside each LUT. On the DE0-CV's Cyclone V they are packaged as an **ALM**, holding a fracturable six-input LUT plus four flip-flops, and the ALM is what the Quartus fitter report counts — which is why it talks about ALMs rather than gates. (*2 marks*)

What the map bought. On this board, in LUT count, nothing. A four-input function costs one LUT whether you minimized it or not, because in every case it is the same block holding a different table. What Question 2's map actually bought was the *knowledge* that `A` and `C` are irrelevant and the function is one XNOR — which no amount of LUT packing would have told you, and which is the sort of thing you need before you can tell whether a specification is the one you meant.

What minimization still buys anywhere: it is how you understand a circuit; it is exactly right for the discrete-logic and ASIC flows Karnaugh maps were invented for; and reducing a function below the LUT's input width is what stops it needing a second level of LUTs. (*1 mark*)

(d)

3 MARKS

What the tool built: a latch, one per bit of `leds`, exactly the D latch of §3.2.

Why it had no choice, in two steps:

1. VHDL requires a signal to keep its old value if nothing assigns a new one. The `when others` branch assigns nothing, so on that path the design has said `leds` must **remember** its previous value.
2. Remembering is not something combinational logic can do. The only element that can is a latch, so that is what gets built.

(*1 mark*)

Two fixes.

```
        when others => leds <= "0000";      -- assign on every path, in the branch
```

```
    leds <= "0000";                          -- a default before the case; every ↪
    ↪ branch overrides it
    case sel is
        ...
```

(*1 mark for both*)

The second is the one to prefer, and the reason is maintenance rather than hardware: it scales to a `case` with twenty branches and stays correct when somebody adds a twenty-first, whereas the first has to be got right again every time the `case` grows. Both synthesize to the same combinational logic.

Why a warning is more dangerous than an error. An error stops you. A warning does not, and the design will often appear to work on the board, because the latch happens to be holding the right value most of the time — so the bug ships, and it surfaces later as an intermittent fault in a circuit that "compiled cleanly". "It compiled with no errors" promises a great deal less about an FPGA design than the same sentence does about a C program. (*1 mark*)

Question 6 Counting, shifting, and knowing when a byte has arrived

12 MARKS

(a)

4 MARKS

After `15` it returns to `0`, with nothing in the circuit saying to: no comparison, no reset logic, no "if the count is at maximum" anywhere.

The element that performs the wrap is the adder's **carry-out**: the fifth bit. The adder produced `10000`, the fifth bit had nowhere to go, and the four bits that survived are `0000`. Discarding that bit *is* the wraparound — "the count wraps because it runs out of bits" is not a figure of speech, and you can point at the bit it ran out of. (*2 marks*)

What has to be true of the range. The upper bound must be `2^N - 1`. `0 to 15` wraps for free; `0 to 9` does not, because `10` is not a power-of-two boundary, so a modulo-10 counter needs an explicit comparison against `9` and an explicit clear. (*1 mark*)

Which is modelling the hardware. The drawing and the chip. There is no sixteenth value for four bits to take, so they land on `0000`. GHDL objects because `natural range 0 to 15` is a **promise the designer made** and `16` breaks it; a range-constrained integer is a claim about the value, not a description of a wire.

The rule that follows is not "trust the hardware". It is that anything you intend to verify should say what it means: write the comparison out, the way `timer.vhd` and `serial_rx8.vhd` do, and the drawing, the simulator and the chip all agree. (*1 mark*)

(b)

3 MARKS

`shift_reg(6 downto 0)` is everything except the top bit, moved up one place; the top bit falls off; `serial_in` fills the vacated bit 0.

Edge	`serial_in`	`shift_reg` after
–	–	`00000000`
1	`1`	`00000001`
2	`0`	`00000010`
3	`1`	`00000101`
4	`1`	`00001011`

(*2 marks; 1 if the sequence is right but shifted the wrong way*)

The first bit sent is now at **bit 3**, and after eight edges it will be at **bit 7**: the first-arriving bit ends up in the most significant position, which is what "most significant bit first" means for this book's receivers.

The mirror image is

```
shift_reg <= serial_in & shift_reg(7 downto 1);
```

which moves every bit toward the **least** significant end, takes data in at bit 7 and drops it from bit 0, and so leaves the first-arriving bit in bit 0. That is the least-significant-first order a real UART uses. It reads almost identically at a glance, which is why the direction is worth checking before assuming. (*1 mark*)

(c) 2 MARKS

	Data in	Data out
SIPO	one bit per clock	all N bits at once
PISO	all N bits at once, loaded	one bit per clock

SIPO to receive a stream of serial bits: it accumulates them and hands you the whole word. **PISO** to drive a chain of LEDs from a fixed pattern: it takes the word and sends it out one wire. (*1 mark*)

What differs between the architectures: almost nothing. The shift expression is **identical** — same direction, same line of VHDL. PISO adds a `load` branch to distinguish loading a new parallel value from shifting the existing one out, and taps `shift_reg(N-1)` as a serial output where SIPO exposes the whole vector. The two names describe how you *wire up* a shift register, not two kinds of hardware. (*1 mark*)

(d) 3 MARKS

Why a pulse. The event being reported is instantaneous: a byte has just completed. A level would say "a byte completed at some point", which a consumer cannot count. `data_ready` is driven low unconditionally at the top of the clocked branch and raised only on the edge completing a byte, which is the same single-cycle shape as Chapter 3's edge detector. (*1 mark*)

What it requires of `data_out`. That it **holds** its value afterwards. A consumer that misses the pulse has missed the *event*, not the data, and the byte is still there. A real peripheral would add a holding register or a small FIFO here so a slow consumer cannot lose data at all. (*1 mark*)

Why the board demonstration needed a flip-flop. At 50 MHz, `data_ready` is high for 20 ns. No LED can show that, so the demonstration adds one flip-flop that is set by the pulse and cleared by reset, turning the event into a level a human can see.

Whose problem that is. Chapter 3's edge detector, run backwards. There the problem was turning a level into a one-cycle event; here it is turning a one-cycle event back into a

level. Pulse against level is the same distinction both times, and it is why the extra flip-flop is not a hack. (*1 mark*)

Question 7 The counter that watches for a number 10 MARKS

(a) 3 MARKS

Timer against counter. A counter answers "what is the current count?" A timer answers "has a certain amount of time passed?" It is a counter plus two things: a **comparison** against a fixed target, and a **clear** that makes it repeat. (*1 mark*)

As gates. One **XNOR per bit**, each comparing a counter bit against the corresponding bit of the target: an XNOR outputs `1` when its inputs are the same, so each one is a single-bit equality test. Their outputs are combined by a single **AND**, whose output is `timeout`.

Target width	Gates
4 bits	4 XNOR + 1 four-input AND = **5**
26 bits	26 XNOR + an AND combining 26 signals = **27**, or 51 with the AND built as a tree of 25 two-input gates

Twenty-six bits is what counting to 50,000,000 needs, and 26 XNORs across 52 inputs is the drawing the VHDL exists to avoid. (*1 mark*)

What insists on all four. The AND gate. Three matching bits is not a match, and watching the four XNOR outputs as the count runs makes that concrete: three of them are high on plenty of counts that are not `1010`. The comparison is the AND, not the XNORs. (*1 mark*)

(b) 3 MARKS

With the clear. The counter runs `0` through `10` inclusive and is then forced back to `0`, so a full cycle is `11` edges: `10 - 0 + 1 = 11`.

Without it. The comparator still fires when the count reads `1010`, but nothing resets the counter, so it carries on `1011, 1100, 1101, 1110, 1111` and wraps to `0000` on its own — four bits, sixteen states — and does not read `1010` again until sixteen edges have passed. So `timeout` fires every **16** edges. (*2 marks for both numbers with the arithmetic*)

Which was asked for: 11. Both are periodic; only one has the period you specified. And on a 4-bit counter the two are close enough to look like a rounding error, which is exactly why this is worth checking rather than eyeballing.

The off-by-one in general. A counter running `0` through `TICK_COUNT` visits `TICK_COUNT + 1` values, so the period is `TICK_COUNT + 1` clock cycles, and for an exact count you write `TICK_COUNT = seconds * 50,000,000 - 1`. The round numbers the book quotes leave an error of between 0.02 and 0.2 ppm, far below the board oscillator's own tolerance. (*1 mark*)

(c) 2 MARKS

The CircuitVerse `timeout` is a **combinational decode** of the count, so it is high during the cycle the counter reads `1010`, and between edges it can **glitch**: while the counter bits skew on a transition, the XNOR-and-AND network can momentarily see a match that never existed.

The VHDL `timeout` is **registered** inside the clocked process, so it is high one cycle later, during the cycle `counter` reads `0`, and it cannot glitch at all: it comes straight off a flip-flop.

Same period, different phase, different glitch behaviour. (*1 mark*)

The distinction is exactly the one Chapter 8's **Moore-versus-Mealy** discussion turns on: an output decoded combinationally from state bits can glitch while they skew, and what makes an output glitch-free is registering it, the way `fsm_led`'s `LED_PROCESS` does. "Moore" on its own does not mean glitch-free. (*1 mark*)

(d) 2 MARKS

The rule broken: one clock, and everything else is an enable. `timeout` is a signal produced by logic, not a clock, and routing it into a clock port creates a second clock domain out of it. (*1 mark*)

What it costs. A design with one clock is a design whose timing the tool can analyze; a design with a logic-derived clock is not, so no meaningful Fmax exists for it. Everything downstream is now asynchronous to everything else in the design, so every signal crossing back needs Chapter 4's synchronizer machinery. On an FPGA it also either burns a global clock resource or, worse, routes a clock over ordinary logic routing with skew nobody bounded. The book names this the single most common mistake a programmer makes writing a baud generator.

What to do instead. Keep the one 50 MHz clock everywhere and use `timeout` as a one-cycle **enable** on a flip-flop that runs on it — which is what `walking_led` does with the same pulse, and what `fsm_led` does in `STATE_BLINK`. (*1 mark*)

Question 8 A machine designed by hand, then written down 13 MARKS

(a) 5 MARKS

State diagram. Four states in a ring with fall-back arrows; `y = 1` in `S3` only, and marked on the bubble rather than on any arrow, which is what makes it a Moore machine.

Every state has exactly two arrows out of it, one per value of `din`:

From	`din = 0`	`din = 1`
`S0` (y=0)	`S0`, self-loop: a `0` matches nothing	`S1`: the first `1`
`S1` (y=0)	`S0`: `10` is not a prefix of `110`	`S2`: `11` seen
`S2` (y=0)	`S3`: `110` complete	`S2`, self-loop: still `11`
`S3` (**y=1**)	`S0`: fresh start	`S1`: fresh start, on this `1`

Drawn, that is a ring `S0 -> S1 -> S2 -> S3 -> S0` for the advancing path, a self-loop on `S0` and on `S2`, a fall-back arrow from `S1` to `S0`, and a re-entry arrow from `S3` to `S1` on `din = 1`. That is eight arrows, two out of each of the four states. `y = 1` is written **on the `S3` bubble**, not on any arrow, and that is what makes it a Moore machine.

(*2 marks: 1 for two arrows out of every state, 1 for `y` marked on `S3` alone*)

State table, `S0 = 00`, `S1 = 01`, `S2 = 11`, `S3 = 10`:

Q1	Q2	din	Q1+	Q2+	y	Meaning
0	0	0	0	0	0	`S0` stays: a `0` matches nothing
0	0	1	0	1	0	`S0` to `S1`: saw the first `1`
0	1	0	0	0	0	`S1` back to `S0`: `10` is not a prefix
0	1	1	1	1	0	`S1` to `S2`: saw `11`
1	1	0	1	0	0	`S2` to `S3`: `110` complete
1	1	1	1	1	0	`S2` stays
1	0	0	0	0	**1**	`S3` to `S0`, fresh start
1	0	1	0	1	**1**	`S3` to `S1`, fresh start on this `1`

(*2 marks for all eight rows; 1 if the four `S0`/`S1` rows are right and the fall-backs are not*)

`S2` on `din = 1`: stay in `S2`. The last two bits seen are still `1, 1`, which is exactly what `S2` means, so there is nowhere better to go: an input of `1, 1, 1, 0` must still report a match. The fall-back and self-loop arrows are where these machines go wrong, not the forward ones. (*1 mark*)

(b)

3 MARKS

Maps with `{Q1, Q2}` down the rows in Gray-code order and `din` across:

`Q1+`

`Q1Q2` \ `din`	`0`	`1`
`00`		
`01`		**1**
`11`	**1**	**1**
`10`		

Two groups of two: the whole `11` row gives `Q1Q2`, and the `din = 1` column across the adjacent rows `01` and `11` gives `Q2·din`.

```
Q1+ = Q1Q2 + Q2·din  =  Q2(Q1 + din)
```

Q2+

Q1Q2 \ din	0	1
00		**1**
01		**1**
11		**1**
10		**1**

One group of four: the entire `din = 1` column, in which only `din` is constant.

```
Q2+ = din
```

y, which reads only the state, confirming this is a valid Moore machine:

```
y = Q1Q2'
```

(*2 marks for the three equations, of which 1 is for `Q2+ = din` being spotted as a group of four rather than written out as four terms*)

The gates. `Q1+` is one OR and one AND in the factored form, or two ANDs and an OR unfactored. `Q2+` is a **wire**: `din` goes straight to the second flip-flop's `D` input, with no gate at all. `y` is one AND with one input inverted. Two D flip-flops hold `{Q1, Q2}`, and that is the whole machine: two flip-flops and three or four gates. (*1 mark*)

(c) 3 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity seq_detect_110 is
    port(clock, reset_n: in  std_logic;
         din           : in  std_logic;
         y             : out std_logic);
end entity;

architecture behaviour of seq_detect_110 is
type state_t is (S0, S1, S2, S3);
signal reset_s2_n: std_logic;
signal state: state_t;
begin
    -- Moore output: reads the state and nothing else.
    y <= '1' when state = S3 else '0';

    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);
```

```
    STATE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            state <= S0;
        elsif (rising_edge(clock)) then
            case (state) is
                when S0 =>
                    if (din = '1') then state <= S1; else state <= S0; end if;
                when S1 =>
                    if (din = '1') then state <= S2; else state <= S0; end if;
                when S2 =>
                    if (din = '1') then state <= S2; else state <= S3; end if;
                when S3 =>
                    if (din = '1') then state <= S1; else state <= S0; end if;
                when others =>
                    state <= S0;
            end case;
        end if;
    end process;
end architecture;
```

(*2 marks: 1 for an enumerated* `state_t` *with a* `case` *covering every state and a* `when others`*, 1 for the transitions agreeing with the candidate's own state table*)

`reset_n` goes through `reset_sync` because this is a **top level** and owns the pin. `din` does **not** get a synchronizer: it is synchronous serial data, already clocked by this clock, and synchronizing it would only delay it and shift the sequence being detected. This is the same split `seq_detect_mealy` makes.

What the VHDL does not contain. The **encoding** (`S0 = 00`, `S1 = 01`, …) and the **next-state equations** derived in (b). Both are gone. The synthesis tool decides the encoding, and derives the gate-level logic from the `case` itself; it will re-encode freely, often to one-hot, unless told otherwise. That is exactly the trade Chapter 8 is about: you designed it by hand once so that you know what the tool is doing, and then hand the tool the decision. (*1 mark*)

Credit an answer that also notes `y <= '1' when state = S3 else '0';` is a Moore output that is still combinationally decoded, so it can glitch briefly while the state bits skew; registering it inside a second clocked process is what makes it glitch-free.

(d)

2 MARKS

`to_next_state` is now high on **every clock cycle the button is held**, not once per press, and the `case` advances the state on every one of them.

The arithmetic. At 50 MHz a press is 20 ns per cycle. A brisk 100 ms press is `0.1 / 20e-9 = 5,000,000` clock cycles, so the machine takes 5,000,000 transitions around a three-state ring. `5,000,000 mod 3 = 2`, so *that* press lands two states on — but nobody can hold a

button for exactly 100 ms, and one cycle either way changes the answer. The state it lands in is decided by how long you happened to hold the button, measured to 20 ns, which from the user's point of view is a coin toss. (*1 mark*)

The fix. Edge-detect the synchronized level, which is what `button_sync`'s third flip-flop is for: compare the current synchronized value against the previous one, giving a pulse high for exactly one clock cycle per press. That is the `X` of the hand-designed machine and the `button_edge_s2` of `fsm_led`, and it is why the machine's input has to be an event rather than a level. (*1 mark*)

APPENDIX G

Model Answers to Paper B

Marks are shown per part. Method carries them: a correct derivation with a slip in it is worth more than a correct answer with no working, and later parts consume earlier ones, so an error should be followed through rather than penalised twice.

Where a part asks for VHDL, mark the **hardware described**, not the syntax. A listing with a missing semicolon and the right circuit in it scores full marks; a listing that compiles and infers a latch does not. The questions are in Appendix E.

Question 1 A requirement, and the shape of the network it asks for

12 MARKS

(a)

2 MARKS

```
run = left · right · curtain_clear · estop_n
```

(*1 mark*)

Gates. One four-input AND, or three two-input ANDs if the library has nothing wider. **No inverter**, which is worth noticing before part (c) asks about it: the condition wanted is "the stop has *not* been pulled", and that is `estop_n = 1` as it stands.

The table. Four inputs, sixteen rows, and exactly **one** of them is 1: the row where all four are 1. (*1 mark*)

(b)

4 MARKS

```
run = estop_n · ( maint'·left·right·curtain_clear  +  maint·pedal )
```

(*1 mark*)

The factored input: `estop_n`. It multiplies the whole sum rather than appearing inside either branch.

What that expresses. No mode can bypass the emergency stop. It is not an input to the mode logic; it gates the mode logic's result. A third mode added next year cannot

accidentally create a path that runs the press with the stop pulled, because every path goes through the same AND. *(1 mark)*

The contrast.

	Braking assistant	**Press control**
Shape	`engine_brake = driver_brake + adas_brake`	`run = estop_n · (mode logic)`
The override	is ORed in **last**, outside the fault logic	is ANDed in **last**, outside the mode logic
Forces the output	**high**	**low**
Guarantees	the action is always **available**: the driver can always brake, including when the assistance system is faulty and doing nothing	the action can always be **prevented**: the press never cycles with the stop pulled, in any mode

What routing the override through the logic would cost, in each case. The same structural mistake with opposite consequences, which is why the two are worth seeing together. Feed `driver_brake` through the assistance system's fault logic and a fault stops the driver braking — the safety requirement the structure exists to protect. Feed `estop_n` into only one branch of the sum, say the maintenance branch, and pulling the stop no longer stops the press in normal mode. Both are one wire in the wrong place, and neither shows up in a test that only exercises the happy path. *(1 mark)*

The count. Sixty-four rows, of which **10** are 1:

- `estop_n = 1` throughout, `maint = 0`: needs `left = right = curtain_clear = 1`, `pedal` free — 2 rows.
- `estop_n = 1`, `maint = 1`: needs `pedal = 1`, the three others free — 8 rows.

What the table would have cost you. Ten AND-terms of six literals each, from a requirement that is two clauses long, plus the whole job of simplifying them back down. The structure came out of the *statement* in one line. That is the case §1.6 makes with the braking assistant: derive from the requirement, and use the table to check the answer rather than to find it. *(1 mark)*

(c) 3 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity press_control is
    port(left, right, curtain_clear, estop_n, maint, pedal: in  std_logic;
         run                                               : out std_logic);
end entity;

architecture behaviour of press_control is
```

```
-- The two mode branches, before the emergency stop gates either of them.
signal two_hand, maintenance: std_logic;
begin
    two_hand    <= (not maint) and left and right and curtain_clear;
    maintenance <= maint and pedal;

    run <= estop_n and (two_hand or maintenance);
end architecture;
```

(2 marks: 1 for the entity with the ports in order, 1 for an architecture matching the candidate's own (b). The internal signals are optional; a single collapsed expression is equally correct.)

Where the `not` is, and where it is not. The one inversion is on `maint`, an **active-high** signal that has to be *false* for the two-hand branch. `estop_n`, the **active-low** signal, needs no inversion at all.

The rule. An `_n` suffix says what a `0` on that wire means; it does not say that an inversion is needed. Write down the condition you want first, then invert whatever does not already say it. "Active-low, therefore `not`" is the reflex to lose: here it would have produced a press that runs only while the emergency stop is pulled. *(1 mark)*

(d)

3 MARKS

The two parts. The `entity` is the black-box view: the ports, and nothing about what happens inside. The `architecture` is the implementation behind it. *(1 mark)*

What the split makes possible.

- One design can **instantiate** another knowing only its entity, without reading the architecture.
- An implementation can be **rewritten** without touching anything that depends on it, because nothing that depends on it was allowed to see it.

(1 mark)

What is different from a C header. The shape is the same — a header beside a source file, or an interface beside its class — but VHDL **enforces** it, for every module, with no way to skip the interface and no way to leak the implementation through it. A C header is a convention: the compiler does not require one, nothing stops a translation unit from exposing its internals, and the interface and the implementation can disagree until the linker or the run finds out. In VHDL the entity is the only way in. *(1 mark)*

Question 2 The map, the vector, and the process that lies

14 MARKS

(a)

4 MARKS

The map, `AB` down the rows and `CD` across, both in Gray-code order `00, 01, 11, 10`:

AB \ CD	00	01	11	10
00	**1**	**1**		
01	**1**	**1**	**1**	**1**
11			**1**	**1**
10			**1**	**1**

(*1 mark for the filled map*)

A minimal cover, three groups of four:

Group	Cells	Constant	Term
The top-left block	`0000, 0001, 0100, 0101`	`A = 0, C = 0`	`A'C'`
The bottom-right block	`1010, 1011, 1110, 1111`	`A = 1, C = 1`	`AC`
The whole `AB = 01` row	`0100, 0101, 0111, 0110`	`A = 0, B = 1`	`A'B`

```
X = A'C' + AC + A'B
```

(*3 marks: 1 per group with its cells and term. Nothing here wraps: all three groups are rectangles you can draw without leaving the map, which is what makes this one easier to see than the four-corner shapes.* `A'B` *may be replaced by* `BC` *throughout — see* (*b*).)

(b)

2 MARKS

The colleague's answer. `X = A'C' + AC + A'B + BC` is **correct**: it computes the same function, since every group in it is a legal group of cells that are genuinely `1`. It is not minimal.

Which group to drop: either `A'B` or `BC`, but not both. `A'C'` and `AC` are forced — `0000` and `0001` sit in no other legal group of four, and neither do `1010` and `1011` — and between them they leave exactly two cells uncovered, `0110` and `0111`. Both `A'B` and `BC` cover that pair, and each of them has its other two cells already covered by one of the forced groups. So one of the two is redundant and either one may go.

The rule is step 3 of the procedure: **use as few groups as will do it, and drop any group whose cells are every one of them already covered by another.** Nothing in the earlier steps forbids a redundant group, which is exactly why the step exists — a redundant group is a redundant AND-term, correct but not minimal. (*1 mark*)

How many minimal answers. Two.

```
X = A'C' + AC + A'B        and        X = A'C' + AC + BC
```

are both three terms of two literals, and both minimal. What that means for marking: check the **function**, not the string. Two candidates can hand in different equations and both be right, and the only way to tell is to evaluate. What is *not* right is keeping both, which is the colleague's answer. *(1 mark)*

(c)

4 MARKS

Slicing. `input(7)` drives **bit 3** of the port. The two are matched **by position, left to right**, not by index number: `input(7)` drives bit 3, `input(6)` drives bit 2, and so on. The **lengths** have to match; the **numbering** does not. `input(7 downto 4)` really does have the index range `7 downto 4` and not `3 downto 0`, which is the detail that surprises a software engineer. *(2 marks)*

Why `count + 1` does not compile. `std_logic_vector` says nothing about what the bits *mean*, so it offers only the operations that make sense for a bundle of wires: the logical operators applied bit by bit, comparison, and concatenation. Arithmetic is not on that list because nothing has declared whether those four wires are an unsigned number, a signed one, or four unrelated signals. That is the type doing its job: the meaning genuinely is missing, so the operation is refused rather than guessed at. *(1 mark)*

What this book uses instead. A plain integer with a range:

```
signal count: natural range 0 to 9;
```

which adds and compares directly, and which synthesis turns into exactly the flip-flops the range needs. Vectors stay what they are, bundles of wires for things that really are bundles of wires. The two worlds meet only at a boundary, with the conversions from `ieee.numeric_std`:

```
count_v <= std_logic_vector(to_unsigned(count, 4));   -- integer to vector
count   <= to_integer(unsigned(count_v));             -- vector to integer
```

(1 mark)

(d)

2 MARKS

```
-- One expression.
x <= (a and b) or ((not c) and d);
```

```
-- The same circuit, with the second term named.
architecture behaviour of combo_logic is
signal cd: std_logic;
```

```
begin
    cd <= (not c) and d;
    x  <= (a and b) or cd;
end architecture;
```

(*1 mark for both*)

What synthesis produces: the same circuit, twice. An internal signal names a wire that already existed in the first version — the output of the AND gate computing `c'd` — so nothing has been added. On the Cyclone V both are one LUT, because a LUT is a lookup table over its inputs: a function of four or fewer inputs fits in one however it was written, and naming an intermediate term does not add an input. The answer is the same for both because VHDL describes hardware, and the two descriptions describe the same hardware.

Which to be handed in six months. At two terms, honestly, either; the named version's advantage appears as soon as the expression outgrows one line, and the habit is worth having before you need it. An answer that simply asserts "the named one is always better" is worth less than one that says where the line is. (*1 mark*)

(e) 2 MARKS

What a simulator does. The process re-runs only when a signal in its **sensitivity list** changes, and the list holds only `sel`. So a change on `a`, `b`, `c` or `d` while `sel` holds still is invisible: `x` keeps whatever value it took the last time `sel` moved. That is not a multiplexer. It behaves like a multiplexer whose output is captured whenever the selector changes and held in between.

What synthesis does. It builds the multiplexer. The tool reads the logic, works out that `x` depends on `sel`, `a`, `b`, `c` and `d`, and produces a real 4-to-1 mux — usually emitting a warning that the sensitivity list is incomplete, and ignoring it either way. (*1 mark*)

Why they differ. The sensitivity list is a **simulation** construct: it tells the simulator when to re-evaluate the process. It is not hardware, so there is nothing for the synthesis tool to build from it.

The class of bug: the simulator and the chip model different things. That is the one class where a green testbench stops meaning anything, because the thing that passed is not the thing that ships. The book's other example is the opposite direction — GHDL aborting on a counter wrap the drawing and the chip both perform happily. The lesson is the same in both: where the two models disagree, neither one alone is evidence. (*1 mark*)

Worth a comment rather than a mark: the testbench passes here only because it changes `sel` for every case it checks. One that held `sel` still and varied the data would fail immediately, which is a useful thing to know about your own testbenches.

Question 3 Storage, and the two ways a value updates

13 MARKS

(a)

3 MARKS

How the two latches are driven. In opposition. The master's own transparency control is the inverted clock and the slave's is the clock, so exactly one of the two is ever transparent.

Why that gives edge triggering. While the clock is **low** the master is transparent and follows `D`, and the slave is closed, holding `Q`. At the **rising edge** the master closes, freezing whatever `D` was at that instant, and the slave opens and passes that frozen value to `Q`. While the clock is **high** the slave is transparent but its input cannot change, because the master ahead of it is closed; so `D` may do anything it likes and `Q` will not move. At the falling edge the slave closes and the master reopens for the next capture.

The net effect is that exactly one value passes per period, captured at the instant the clock goes low to high, and `Q` never changes while the clock sits at either level. That external behaviour is what `rising_edge()` models. (*2 marks*)

The latch's weakness. It is transparent for the *entire* time `enable` is high, so any glitch on `D` during that window passes straight through to `Q`.

Why that is worse in a large system. In a small circuit you can look at every path feeding `D` and convince yourself it is stable across the phase. In a large one there are thousands, each with its own delay, and guaranteeing that all of them are settled throughout an entire clock phase is far harder than guaranteeing they are settled at one instant — which is all the flip-flop asks for. (*1 mark*)

(b)

4 MARKS

```
library ieee;
use ieee.std_logic_1164.all;

entity register4 is
    port(clock, reset_n, enable: in  std_logic;
         d                     : in  std_logic_vector(3 downto 0);
         q                     : out std_logic_vector(3 downto 0));
end entity;

architecture behaviour of register4 is
begin
    REGISTER_PROCESS: process(clock, reset_n) is
    begin
        if (reset_n = '0') then
            q <= (others => '0');
        elsif (rising_edge(clock)) then
            if (enable = '1') then
```

```
                q <= d;
            end if;
        end if;
    end process;
end architecture;
```

(*2 marks: 1 for the entity and port order, 1 for the process with* `clock` *and* `reset_n` *in the sensitivity list, the reset branch outside the clocked branch, and the enable inside it*)

`q` is only ever assigned here, never read, so no internal signal is needed. A design that toggles or otherwise reads its own output does need one, because an output port cannot be read back.

Flip-flops: four. What decides it is the **width of the signal assigned inside the clocked process** — every signal assigned there becomes flip-flops, one per bit. Four bits, one process, four flip-flops, no different in principle from four `d_flip_flop` instances wired in parallel. (*1 mark*)

Reset branch first: meaning, not style. The `if/elsif` is read in priority order, so the reset branch coming first is what makes the reset take priority and act with no clock edge involved. Put `rising_edge(clock)` first and the reset is only examined at an edge, which is a **synchronous** reset: a different circuit, one that ignores a reset pulse landing entirely between two edges. It is also the shape the synthesis tool pattern-matches to infer an asynchronous reset at all. (*1 mark*)

(c) 4 MARKS

The trace. Both are signals, so both assignments **schedule**: neither takes effect until the process suspends, and every right-hand side in the pass reads the value the signal held *before* the edge. `stage2_s <= stage1_s;` therefore reads the old `stage1_s`, not the `din` the line below is putting there, and the two lines describe two flip-flops in series.

Edge	`din`	`stage1_s` after	`stage2_s` after	`q` after
(after reset)	–	0	0	0
1	0	0	0	0
2	1	1	0	0
3	0	0	1	1
4	0	0	0	0

The `1` on `din` at edge 2 reaches `stage1_s` at edge 2 and `q` at edge 3: **two** edges, not one.

(*2 marks: 1 for the table, 1 for reading each right-hand side against the pre-edge value rather than asserting the answer*)

The rule. A signal assignment inside a process **schedules** a new value, applied only when the process suspends; every read in the same pass sees the old value. It is the same

rule Chapter 3 states for the two-assignment process, and it is what lets a clocked process describe a chain of flip-flops in whatever order the lines happen to be written.

Swapping the two assignments changes nothing. Write `stage1_s <= din;` first and `stage2_s <= stage1_s;` second, and `stage2_s` still reads the pre-edge `stage1_s`, because scheduling does not care which line came first. That is the point worth taking away: **with signals in a clocked process, order does not matter**, which is exactly why the author's line-order reasoning was the wrong tool. A `variable` is the object for which order *would* matter, and Chapter 5 is where the book picks that up. *(1 mark)*

The change that gives one cycle. Delete `stage2_s` and drive the output from the first stage: `stage1_s <= din;` in the process, `q <= stage1_s;` outside it. One assignment, one register.

Flip-flops. The listing as written builds **two** — `stage1_s` and `stage2_s` are each assigned under `rising_edge(clock)` with an asynchronous reset, which is a flip-flop apiece; `q <= stage2_s;` is a wire and costs nothing. After the fix, **one**. The extra edge of latency and the extra flip-flop are the same fact seen from two sides. *(1 mark)*

(d) 2 MARKS

Why one edge. Every flip-flop in the design changes at the same well-defined instants, so the circuit's behaviour is defined at a sequence of points rather than continuously. *(1 mark)*

What that buys. It turns a continuous-time problem into a discrete one. You can reason **one clock cycle at a time** — "given the state at this edge, what is the state at the next?" — instead of tracking every signal continuously and worrying about the order in which things happen in between. Between edges, nothing has to be true except that everything settles before the next one arrives, which reduces the whole timing question for an arbitrarily large design to a single number: the longest path against the period. That is what makes a circuit too large to see all at once analyzable at all, by you or by the tool. *(1 mark)*

Question 4 One module, several widths 13 MARKS

(a) 5 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

architecture behaviour of button_sync is
-- Three stages, each COUNT bits wide: two synchronizing, one remembering.
signal button_s1_n, button_s2_n, button_s3_n: std_logic_vector(COUNT-1 downto 0);
begin
    -- Falling edge: pressed now, released the cycle before. Bit by bit.
    button_edge_s2 <= (not button_s2_n) and button_s3_n;
```

```
    BUTTON_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            button_s1_n <= (others => '1');
            button_s2_n <= (others => '1');
            button_s3_n <= (others => '1');
        elsif (rising_edge(clock)) then
            button_s1_n <= button_n;
            button_s2_n <= button_s1_n;
            button_s3_n <= button_s2_n;
        end if;
    end process;
end architecture;
```

(*3 marks: 1 for three vector signals declared in terms of* `COUNT`*, 1 for the three-stage chain in a clocked process with an asynchronous reset, 1 for the edge expression as a concurrent assignment outside the process. Deduct nothing for writing the edge inside the process as a registered pulse, provided the candidate says it arrives a cycle later.*)

Which flip-flops do what. Stages 1 and 2 are the **double-flop synchronizer**, so `button_s2_n` is the stable, safe-to-use current value. Stage 3 stores last cycle's `button_s2_n` and is the **edge detector's own** flip-flop; comparing stage 2 against stage 3 gives edge detection for free, high for exactly one clock cycle. (*1 mark*)

The reset value: `(others => '1')`. The buttons are active-low, so `1` means released. Reset the chain to `0` and it comes up claiming every button is already held down, which is a lie about the world for the two cycles it takes the real level to propagate — and, concretely, a press that is already being held when the reset is released is **lost**, because the chain never sees a high-to-low transition on it. This is the same reason Chapter 4's CircuitVerse drawing ties `preset` to `1` on these three flip-flops and leaves it alone on the reset synchronizer's two. (*1 mark*)

Why `reset_s2_n` — worth a comment rather than a mark, and the one place a candidate reveals whether they know why the port is named as it is. This is a **subcomponent**. A synchronizer does not synchronize its own reset; `reset_sync` does that, and the design that owns the pin instantiates it. Reading the `_s2` as a requirement on the caller is the habit worth having, because from Chapter 6 on most of what you write is a subcomponent.

(b) 3 MARKS

The rule: add a generic when something genuinely varies between instances, not on principle. `button_sync` is instantiated at `COUNT = 1` by `led_toggle_sync` and `walking_led`, and at `COUNT = 2` by `fsm_led`: two widths, one file, and that second instantiation is what makes the generic worth having. `reset_sync` is one bit everywhere.

What a `WIDTH` on `reset_sync` would be. A parameter nobody ever passes anything but `1` to: pure indirection, and one more thing for a reader to check before they can be sure what an instance is. (*1 mark*)

Where `COUNT` ends up: nowhere. It is fixed before a single gate is placed, so it does not appear in the built circuit as logic at all. What it did was decide how many flip-flops to instantiate and how wide the ports are, and then it was gone — the same thing that happens to a `for` loop's bounds in Chapter 5. (*1 mark*)

Flip-flop counts. Three stages, `COUNT` bits each: `COUNT = 2` gives **6** flip-flops (plus two inverters and two AND gates for the edge), `COUNT = 1` gives **3**. A `generic map` itself costs nothing in hardware; it is an elaboration-time decision, not a runtime one. (*1 mark*)

(c) 2 MARKS

One mechanism explains both. The chain samples the button once per clock edge, and whether that *looks* like debouncing depends entirely on the clock period relative to the bounce time.

- In CircuitVerse the period is a full second. Bounce settles in far less than that, so every bounce transition happens **between** two samples and by the next edge the chain reads one clean level. The circuit is not debouncing; it is sampling too infrequently to see bounce.
- At 50 MHz the period is 20 ns, and bounce lasts from under a millisecond to a few tens of milliseconds — many thousands of cycles. The chain faithfully detects **every** bounce transition as its own edge, so one physical press produces several pulses and the LED toggles several times.

(*1 mark*)

What a real debounce adds, either of:

- an **analog RC low-pass filter**, optionally with a Schmitt-trigger buffer, on the input before it reaches the pin: the RC pair smooths the fast make/break transitions into one gradual edge and the Schmitt trigger's two thresholds turn that back into one clean digital transition; or
- a **digital debounce counter**: require the input to hold a new level for a few milliseconds' worth of clock cycles before accepting it, which builds the artificially slow detection window on-chip without slowing the system clock.

Why on top rather than instead. Even a perfectly debounced signal is still asynchronous to your clock: it can still change inside the setup/hold window and still drive a flip-flop

metastable. Debouncing solves a mechanical problem and synchronizing solves a timing one, and neither solves the other. (*1 mark*)

(d) 3 MARKS

`din` against `reset_n` in `seq_detect_mealy`. `din` is **synchronous serial data**, already clocked by this clock, so it is not an asynchronous input and the rule does not apply to it. Synchronizing it anyway would delay it by two clock cycles and shift its timing relative to everything around it — in a sequence detector that means the machine is looking at a different part of the sequence than everything else believes, so the bug is a wrong answer rather than a slow one. `reset_n` comes from a push button and is asynchronous, so it does get a chain. The rule is about where a signal comes from, not about being cautious. (*1 mark*)

`reset_s2_n` against `reset_n`. The port name says which side of the boundary the module is on:

- A **subcomponent** takes `reset_s2_n` and never touches a raw asynchronous input. It is a part, and a part is entitled to assume the design around it has done the synchronizing. `serial_rx8`, `timer`, `counter`, `sipo8`, `piso8` and `sync` are all written this way.
- A **top level** takes `reset_n`, because it is the thing holding the pin, and it instantiates `reset_sync` itself. `walking_led`, `blinker`, `fsm_led` and both capstones are written that way.

(*1 mark*)

What goes wrong if a subcomponent is wired straight to a push button. The reset is then released asynchronously, so the module's flip-flops leave reset on different clock edges. In `serial_rx8` that is a dozen flip-flops between the shift register and the bit counter, and a receiver that starts counting a byte one cycle before it starts shifting one is broken by its own reset — a fault that appears once in a while, at power-on, and never in a testbench that resets cleanly. (*1 mark*)

Question 5 Two kinds of update, and a speed limit 13 MARKS

(a) 3 MARKS

	`signal` (`<=`)	`variable` (`:=`)
Declared	The architecture's declarative part	The process's declarative part
Visible from	Anywhere in the architecture	Only within its own process
Update timing	**Scheduled**: applied when the process next suspends	**Immediate**: applied before the next statement
Assigned twice in one pass	Only the last survives; earlier ones are discarded	Every one takes effect, in order

(*2 marks, or 1 for two of the four rows*)

The rule of thumb. Use a **variable** for a scratch computation consumed entirely within one pass: loop accumulators, temporary swaps, an intermediate value handed to a signal at the end. Use a **signal** for anything another process or the outside world reads, and for anything that has to hold its value across separate passes — a counter, a shift register's stored bits, the previous value in an edge detector. *(1 mark)*

(b) 4 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity ones_count8 is
    port(bits: in  std_logic_vector(7 downto 0);
         ones: out natural range 0 to 8);
end entity;

architecture behaviour of ones_count8 is
begin
    COUNT_PROCESS: process(bits) is
        variable count: natural range 0 to 8;
    begin
        count := 0;
        for i in bits'range loop
            if (bits(i) = '1') then
                count := count + 1;
            end if;
        end loop;
        ones <= count;
    end process;
end architecture;
```

(3 marks: 1 for a `variable` accumulator assigned with `:=`, 1 for clearing it at the top of every activation, 1 for `bits` in the sensitivity list and a single assignment to `ones` after the loop)

A `signal` accumulator here scores none of the three: the assignments schedule rather than take effect, so only the last of the nine survives and `ones` reports the final bit rather than a count.

What the range buys. Two things, one for each tool. It tells **synthesis** exactly how many bits the value needs — four, for `0` to `8` — so the register or the wires are sized to the value rather than to whatever an unconstrained `natural` would give. And it is a **promise the simulator checks**: `natural range 0 to 8` is range-checked, so GHDL aborts with a range-check failure at the statement that broke it the instant the count leaves the range.

What GHDL would do. Stop, with an error naming the line, rather than carrying on with a wrong number. A `std_logic_vector(3 downto 0)` would hold `9` through `15` quite

happily and say nothing, which is the difference between a bug that reports itself and one that does not. (*1 mark*)

(c) 4 MARKS

The three delays the clock period has to cover on a flip-flop / logic / flip-flop path:

1. the first flip-flop's **clock-to-Q** delay, between the edge and its output actually changing;
2. the **combinational delay** through the LUTs and the wires between the two;
3. the second flip-flop's **setup time**.

The **critical path** is the slowest such path in the design, and **Fmax** is one over that total. (*2 marks*)

If the period is too short, the value has not settled when the second flip-flop samples, so it captures something still changing — and the design does the wrong thing *reliably and mysteriously*, which is worse than doing it intermittently, because it looks like a logic bug.

Why adding registers can make a design faster. Splitting a long combinational expression so that part of the work happens on one cycle and the rest on the next shortens the longest path, and the clock period only has to cover the longest path. It trades **latency** — more cycles before an answer comes out — and some flip-flops, for **throughput**: a faster clock, and once the pipeline is full, one result per cycle. That trade is the single most common move in high-speed digital design. (*1 mark*)

What Quartus needs before it reports an Fmax. An `.sdc` constraints file with a `create_clock` on the 50 MHz input. Without one the timing analyzer has no period to compare anything against, so it reports every path as **unconstrained** and gives no Fmax at all — not an optimistic number, no number. Separately, a design that "fails timing" is one where a path turned out slower than the period you did declare, which is a different message and a different problem.

None of this is part of the book: every design here fits inside 20 ns with room to spare. Knowing the step exists and what it needs is what is examinable. (*1 mark*)

(d) 2 MARKS

An uncleared accumulator keeps growing. The variable is not a fresh local on each activation, so a pass that starts by adding to it is adding to the *previous* pass's total. Over a long run the count climbs without bound: with a range-constrained type it aborts on a range check as soon as it leaves the range, and with an unconstrained one it simply reports a number that means nothing. The answer it reports is the running total of every activation since the simulation began, not a count over `bits`. (*1 mark*)

Why a signal for persistent state anyway. It is possible to build state with a variable, but its updates are immediate rather than scheduled, so it behaves differently from a signal used the same way. Question 3(c) is the mirror image: a chain of registers stepping one stage per clock edge is exactly what scheduled updates give you for free, and a variable there would collapse the chain. A variable is also invisible outside its own process, so no other

logic can read it. Scheduled updates are what make counters and shift registers work, so if state genuinely has to persist *and* be read elsewhere, reach for a signal. *(1 mark)*

Question 6 Counting to something that is not a power of two

12 MARKS

(a)

4 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity counter10 is
    port(clock, reset_s2_n, enable: in  std_logic;
         count                    : out natural range 0 to 9);
end entity;

architecture behaviour of counter10 is
-- Internal count, because an output port cannot be read back.
signal count_s: natural range 0 to 9;
begin
    count <= count_s;

    COUNT_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            count_s <= 0;
        elsif (rising_edge(clock)) then
            if (enable = '1') then
                if (count_s >= 9) then
                    count_s <= 0;
                else
                    count_s <= count_s + 1;
                end if;
            end if;
        end if;
    end process;
end architecture;
```

(3 marks: 1 for the internal signal and the entity, 1 for the clocked process with the asynchronous reset and the enable, 1 for the explicit comparison and clear)

A candidate who writes `if (count_s = 9)` rather than >= loses nothing; >= is the defensive form the book's `timer` uses.

What would be unnecessary at `0 to 15`. The whole comparison: `if (count_s >= 9) then count_s <= 0; else ... end if`, leaving `count_s <= count_s + 1;` on its own.

Why exactly those. `15` is `2^4 - 1`, so incrementing past it produces a carry out of the fourth bit with nowhere to go, and the four bits that survive are `0000`. The wrap is a consequence of the register's width — you can point at the bit it ran out of. `9` is not a power-of-two boundary, so nothing in the hardware returns the count to zero there, and the comparison has to say so.

Worth noting, and worth a comment rather than a mark: the book writes the comparison out even at `0 to 15`, because a VHDL simulator range-checks `natural range 0 to 15` and aborts on the wrap that the gate network and the chip both perform happily. Anything you intend to verify should say what it means. *(1 mark)*

(b)

4 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

architecture behaviour of piso8 is
signal shift_reg: std_logic_vector(7 downto 0);
begin
    -- Bit 7 is always the next bit to leave.
    serial_out <= shift_reg(7);

    SHIFT_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            shift_reg <= (others => '0');
        elsif (rising_edge(clock)) then
            if (load = '1') then
                shift_reg <= parallel_in;                  -- Parallel load.
            elsif (shift = '1') then
                shift_reg <= shift_reg(6 downto 0) & serial_in;  -- Shift toward ↪
    ↪ the MSB.
            end if;
        end if;
    end process;
end architecture;
```

(3 marks: 1 for the load branch taking precedence over the shift branch, 1 for the shift expression in this book's direction, 1 for tapping bit 7 as the serial output)

What fills the vacated bit 0 is a wiring decision rather than a property of a PISO. Here it is `serial_in`, so tying that low shifts zeros in behind the byte; `walking_led` instead wires the bit falling off the top back round to it, which is what turns a one-shot pattern register into an endlessly repeating pattern generator.

Which bit leaves first: bit 7, the most significant. Load `"10110010"` and `serial_out` presents `1`, `0`, `1`, `1`, `0`, `0`, `1`, `0` over the following cycles.

The two things that separate it from a SIPO, the shift expression being identical in both:

1. PISO has a **`load` branch**, distinguishing loading a new parallel value from shifting the existing one out. SIPO has nothing to load.
2. PISO **taps `shift_reg(7)`** as a one-bit serial output, where SIPO drives the whole vector out in parallel.

"SIPO" and "PISO" name how you wire a shift register up, not two kinds of hardware.

(*1 mark*)

(c)

2 MARKS

The bit counter now counts **clock edges** rather than bits, so it loses track of the byte the moment the stream pauses.

Edge	What arrives	`bit_count` after
1	bit 7	1
2	bit 6	2
3	bit 5	3
4	(pause)	4
5	(pause)	5
6	(pause)	6
7	(pause)	7
8	(pause)	count was already 7: **`data_ready` pulses**, count clears to 0

So `data_ready` first pulses on the **eighth clock edge**, which is the fifth and last cycle of the pause, five bits before the byte is complete. (*1 mark*)

`data_out` at that moment holds the shift register's contents: the five reset zeros it started with, followed by the three bits that really did arrive — `"00000"` then bits 7, 6 and 5 of the byte. And it does not recover: the counter is now permanently out of step with the data, so every subsequent "byte" is a window of eight clock edges rather than eight bits.

That is why `shift_enable` has to gate the counter as well as the shift. With both gated, a gap in the stream **pauses** the receiver mid-byte rather than corrupting it, and it resumes on exactly the bit it was waiting for — which is also what makes the board demonstration possible, one press per bit. (*1 mark*)

(d) 2 MARKS

	What feeds flip-flop `i`'s `D`
Counter	The adder's sum bit `i`: a function of the counter's own bit `i` and the carry out of every bit below it. The register's output feeds back into its own input through an adder.
Shift register	Flip-flop `i-1`'s `Q`, straight through, with bit 0's `D` coming from `serial_in`. No arithmetic anywhere.

(*1 mark each*)

The difference between the two circuits is therefore not in the flip-flops, which are identical, and not in the clocking, which is identical. It is entirely in the combinational network between one stage's `Q` and the next stage's `D`: an adder in one case, a plain wire in the other. That is the whole of it, and it is why both are "technically state machines" while neither is what Chapter 8 means by one — the state is a count or a shifted pattern rather than a label whose meaning you chose.

Question 7 Turning ticks into time 10 MARKS

(a) 3 MARKS

Cycles. `0.1 s * 50,000,000 cycles/s = 5,000,000` clock cycles.

`TICK_COUNT`. The book writes `TICK_COUNT = 5_000_000`. The cycle-exact value is `TICK_COUNT = 4_999_999`, because the internal counter runs `0` through `TICK_COUNT` inclusive, so a full period is `TICK_COUNT + 1` cycles. The round number is 1 cycle long in 5,000,000, which is 0.2 ppm — far below the board oscillator's own tolerance, which is why the book quotes it. (*2 marks*)

Blink frequency: 5 Hz. `timeout` fires every 100 ms, so the LED **toggles** ten times a second; but one blink is on *and* off, so a full cycle takes two timeouts, 200 ms, giving 5 Hz. The number somebody first writes down is 10 Hz, which is the timeout rate rather than the blink rate. (*1 mark*)

(b) 2 MARKS

A freezing enable means the timer resumes rather than restarts. Whatever was left of the period when `enable` went low is what is left when it comes back, so the first timeout after re-entering the state arrives anywhere between one clock cycle and a full period later, depending on where in its count the timer was frozen. A **clearing** enable would give a full, predictable period every time, at the cost of never being able to pause anything.

In `fsm_led` the visible consequence is that the first LED toggle after re-entering `STATE_BLINK` can come almost immediately, so the first blink interval is short. (*1 mark*)

Holding `enable` high permanently. Neither better nor worse for correctness, and it does not fix the short first interval: entering `STATE_BLINK` would still catch a free-running timer

at an arbitrary point in its period. What it does buy is a blink phase that is independent of the state machine entirely; what it costs is a 26-bit counter switching continuously when nothing is reading it, which is power spent on nothing. The freezing enable is the more useful default because a design that wants free-running behaviour can always tie `enable` high, while a design that wants to pause cannot un-clear a clearing timer. *(1 mark)*

(c) 3 MARKS

```
library ieee;
use ieee.std_logic_1164.all;

architecture behaviour of blinker is
signal reset_s2_n, timeout: std_logic;
signal enable_always      : std_logic;
signal led_s              : std_logic;
begin
    led           <= led_s;
    enable_always <= '1';

    -- This is a top level: it owns the pin, so it synchronizes the reset itself.
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    timer1: entity work.timer
        generic map(TICK_COUNT)
        port map(clock, reset_s2_n, enable_always, timeout);

    -- The timeout pulse is an enable on a flip-flop running on the system clock,
    -- never a clock of its own.
    LED_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            led_s <= '0';
        elsif (rising_edge(clock)) then
            if (timeout = '1') then
                led_s <= not led_s;
            end if;
        end if;
    end process;
end architecture;
```

(3 marks: 1 for `reset_sync` instantiated and its `reset_s2_n` carried to the timer, 1 for `timer` with the generic passed through and `enable` tied high, 1 for the toggle process using `timeout` as an enable rather than as a clock)

Tying the literal `'1'` directly in the `port map` is accepted by GHDL and Quartus and loses nothing; the named signal is the form that is unambiguously legal in every tool.

`led_s` exists because an output port cannot be read back, and this process reads its own output.

(d) 2 MARKS

Why the synchronizer. The button is **asynchronous** to the 50 MHz clock: a human decides when it changes, with no relationship to the clock edge, so it can change inside the setup/hold window of the flip-flop sampling it and drive that flip-flop metastable. That the button has nothing to do with counting is beside the point — the rule is about where the signal comes from, not what it controls. *(1 mark)*

What the design would do without one. Two things, and the second is the one that matters. The visible symptom is **bounce**: the raw contacts make and break several times over the first millisecond, which at 20 ns per cycle is many thousands of clock cycles, so the enable would flicker on and off many times per press and the timer would start, stop and restart. The real fault is **metastability**: the enable feeds a 26-bit counter and everything downstream of it, and an unresolved value reaching several destinations can be read as `0` by some and `1` by others in the same cycle, so parts of the design disagree about whether the timer is running. *(1 mark)*

Question 8 The output that arrives a cycle early 13 MARKS

(a) 5 MARKS

```vhdl
library ieee;
use ieee.std_logic_1164.all;

entity seq_detect_01_mealy is
    port(clock, reset_n: in  std_logic;
         din           : in  std_logic;
         y             : out std_logic);
end entity;

architecture behaviour of seq_detect_01_mealy is
-- STATE_HIGH: the previous bit was '1'. Also the reset state, since the line ↩
    ↪ idles high.
-- STATE_LOW : the previous bit was '0'.
type state_t is (STATE_HIGH, STATE_LOW);

signal reset_s2_n: std_logic;
signal state     : state_t;
```

```
begin
    -- Top level: it owns the reset pin. din is synchronous serial data and is NOT
    -- synchronized, exactly as in seq_detect_mealy.
    reset_sync1: entity work.reset_sync
        port map(clock, reset_n, reset_s2_n);

    STATE_PROCESS: process(clock, reset_s2_n) is
    begin
        if (reset_s2_n = '0') then
            state <= STATE_HIGH;
        elsif (rising_edge(clock)) then
            case (state) is
                when STATE_HIGH =>
                    if (din = '0') then
                        state <= STATE_LOW;
                    else
                        state <= STATE_HIGH;
                    end if;
                when STATE_LOW =>
                    if (din = '1') then
                        state <= STATE_HIGH;
                    else
                        state <= STATE_LOW;
                    end if;
                when others =>
                    state <= STATE_HIGH;
            end case;
        end if;
    end process;

    -- Mealy output: the current state AND the current input, combinational,
    -- outside any clocked process.
    y <= '1' when (state = STATE_LOW and din = '1') else '0';
end architecture;
```

*(3 marks: 1 for a two-value enumerated `state_t` with a `case` transition process and an asynchronous reset, 1 for the transitions, 1 for the output as a **concurrent** conditional assignment reading both `state` and `din`)*

What the states mean. `STATE_HIGH` is "the previous bit was `1`", `STATE_LOW` is "the previous bit was `0`". That is the entire history a rising-edge detector needs, which is why two states are enough. Either reset state is defensible provided the candidate says which they chose and why; `STATE_HIGH` suits a line that idles high, and means a first `1` after reset is not reported as an edge.

The line that makes it Mealy.

```
y <= '1' when (state = STATE_LOW and din = '1') else '0';
```

It reads `din`. Move that condition inside the clocked process, or drop `din` from it, and the machine becomes Moore. Note also that it must have an `else`: leave one out and you have described a signal that keeps its old value when no condition holds, which is a latch.

(*1 mark*)

What `when others` buys. In behaviour, nothing: every value of `state_t` is already named above it, so the branch is unreachable in the source and synthesis normally optimizes it away rather than building recovery logic. What it buys is that the `case` stays exhaustive if a state is added later and a branch is forgotten — a maintenance property, checked at analysis time.

What it is **not** is a safety net against a corrupted state register. Once Quartus re-encodes the machine, most bit patterns are illegal, and recovering from one needs Quartus's Safe State Machine setting rather than anything you can write in VHDL. (*1 mark*)

(b)

2 MARKS

Why one more state. A Moore output may not read `din`, so "a rising edge just happened" has to be recorded in the state itself before the output can report it. That means a third state, entered on the `0` to `1` transition, existing purely to hold the output high for one cycle — exactly the `M_TWO` of the book's own Moore-versus-Mealy comparison in §8.5.

What its output does differently. It arrives **one clock cycle later**, and it comes off a register rather than out of a gate, so it cannot follow a noisy input and, if the output is registered, cannot glitch. (*1 mark*)

Which to reach for: Moore, by default. Its output depends on no primary input, so it cannot follow one; a Mealy output can in principle glitch combinationally if its inputs are not already clean; and a machine whose output depends on fewer things is a machine with fewer ways to be wrong. It is what the rest of this book uses. Reach for Mealy only when you specifically need the lower latency or the smaller state count, and know you are trading a synchronous output for a combinational one when you do. (*1 mark*)

Credit an answer that adds that "Moore" alone does not mean glitch-free: an output decoded combinationally from several state bits can still glitch while they skew, and what makes an output glitch-free is registering it.

(c)

4 MARKS

Cycles per bit and per tick.

```
one bit period  = 50,000,000 / 9600 = 5208.33 clock cycles
one sixteenth   = 5208.33 / 16      =  325.52 clock cycles
```

(*1 mark*)

The two candidates. The timer pulses every `TICK_COUNT + 1` cycles, so the target is `TICK_COUNT + 1 = 325.52`, that is `TICK_COUNT = 324.52`, which is not a whole number:

`TICK_COUNT`	Cycles per tick	Error against 325.52
324	325	0.52 cycles short, **−0.16%**
325	**326**	0.48 cycles long, **+0.15%**

The default is **325**, and the reason is simply that 326 is the closer of the two: `|326 - 325.52|` is `0.48` against `|325 - 325.52| = 0.52`. Both would work; neither is exact, because 9600 baud does not divide 50 MHz. (*1 mark*)

Why sample mid-bit. The transmitter runs off a different physical crystal, so the two clocks drift apart across a frame, and the receiver's idea of where a bit starts came from one falling edge it inferred rather than from a clock line it was given. Sampling at the middle leaves half a bit of margin on **each** side; sampling near a boundary reads the neighbouring bit as soon as the two drift at all, or as soon as the line's own edges are anything but instantaneous. What is being tolerated is frequency error between the two crystals, plus the fraction of a tick of phase error left over by never stopping the timer. (*1 mark*)

The drift. Ten bit periods is 160 oversample ticks. The receiver's 160 ticks take `160 * 326 = 52,160` clock cycles; ten true bit periods are `10 * 5208.33 = 52,083` cycles. The difference is about 77 cycles, which is `77 / 325.52 = 0.24` of an oversample tick.

The margin it started with is **8 ticks**, half a bit period. So the sample point has drifted about **3% of its margin** by the stop bit, which is why the answer is comfortable and why resynchronizing on every start bit is enough: the error never gets ten bit periods to accumulate over more than once. (*1 mark*)

(d) 2 MARKS

During the break, a level test re-arms the instant the previous frame's stop bit is judged. The line is low, so the receiver reads it as a start bit immediately and marches through back-to-back all-zero frames for as long as the break lasts. Each of those ends on a low stop bit and is correctly rejected as a framing error, and **that part is fine**: nothing is handed over. (*1 mark*)

At the moment the break ends, one frame is already in flight, and its stop bit is sampled after the line has returned high. So it is a perfectly well-formed frame as far as the receiver can tell: correct framing, high stop bit, and a byte made of nothing handed over as real data.

That last frame is the one that matters because it is the only one that **passes**. Every other frame during the break is rejected, so a design that gets this wrong looks correct right up to the moment a cable is unplugged and plugged back in, and then delivers one plausible byte that was never sent.

The **edge** test does not re-arm during the break at all: after the first falling edge there is not another one until the line has recovered. It costs one flip-flop of history, and that is the whole difference. (*1 mark*)